# C++ Introduction

C++ is a programming language that was first released in 1985, making it older than languages like Java and Python.

**Why C++ is Still Relevant:-** Even though Java and Python are more commonly used in the industry today, C++ is still important for certain applications. It is especially popular in competitive programming and is also used for software development. The main reasons for this are its ability to handle low-level programming and its speed of execution, meaning it requires fewer resources to run tasks quickly.

**C++ as an Extension of C:-** C++ was created as an extension of the C programming language, which is one of the oldest high-level languages. C is known for allowing **direct hardware access**, which is possible through **pointers**. Since C++ is based on C, it inherits these features, allowing programmers to directly manipulate memory and hardware.

**Features of C++:-**Some of the ***features & key points*** to note about the programming language are as follows:

- Simple
- Machine Independent but Platform Dependent
- Mid-level language
- Rich library support
- Speed of execution
- Pointer and direct Memory-Access
- Object-Oriented
- Compiled Language

**Other important features include:**

1. **Generic Programming**: C++ allows you to write code that works for multiple types using **templates**, which makes your code more reusable.
2. **Exception Handling**: Unlike C, C++ includes exception handling to manage errors more effectively.
3. **Standard Template Library (STL)**: C++ has a richer library than C, including the STL, which provides commonly used data structures and algorithms like lists, maps, and stacks.

**C++ finds varied usage in applications such as:**

- Operating Systems & Systems Programming. e.g. *Linux-based OS (Ubuntu etc.)*
- Browsers *(Chrome & Firefox)*
- Graphics & Game engines *(Photoshop, Blender, Unreal-Engine)*
- Database Engines *(MySQL, MongoDB, Redis etc.)*
- Cloud/Distributed Systems

## C++ Standards and Implementation

Just like spoken languages such as English and Hindi have grammatical rules, programming languages like C++ also follow rules known as standards. These standards define what is correct or incorrect in the language, ensuring consistency and quality.

**C++ Standards Timeline:-** C++ was first **standardized in 1998**. Before this, it was released in 1985 but didn't have a formal standard. The **first official version** of C++ was **C++98**. Since then, C++ has gone through several updates to improve the language. The major versions include:

- **C++03**
- **C++11**
- **C++14**
- **C++17**
- **C++20**
- The next version, **C++23**, is currently in development.

You can explore detailed information on these standards on the official website: isocpp.org.

**What Are the Rules in C++ Standards?** Each programming language has its own rules to define what is allowed and what isn't. For example:

- Variable names cannot have special characters.
- The multiplication operator (*) is used for multiplication.
- Operators like * and + follow a specific order of precedence in expressions.

These rules are clearly stated in the C++ standards documentation. As a programmer, it's important to follow these rules to write correct programs.

**Implementing the Rules: C++ Compilers:-**To ensure that the rules are followed, **compilers** are used. Compilers are software tools that convert your C++ code into executable machine code. Some popular C++ compilers include:

- **GCC (GNU Compiler Collection)** – commonly used and supports a wide range of C++ standards.
- **MSVC (Microsoft Visual C++)** – developed by Microsoft.
- **IBM**, **Oracle**, and **Intel** also offer C++ compilers.

These compilers implement the rules specified in the C++ standards. They also provide their own extra functionalities or libraries. For example, GCC includes built-in functions for tasks like computing the **GCD** or **counting set bits**. These extra features are not part of the official C++ standard, but they are useful for specific tasks.

**Undefined Behaviour in C++:-**In C++, there are some **expressions that are syntactically correct** but do not have defined behavior according to the standard. This means that different compilers may handle these expressions differently, leading to unexpected results. As a programmer, it is important to avoid writing code that leads to undefined behavior to prevent inconsistencies in output.

**How does a C++ program run:-**When developing software in C++, you write code in a text editor. Since C++ is a high-level language that the CPU cannot directly understand, the code must be converted into machine-readable form through several steps. This process often involves using libraries, which are pre-written code that handle tasks such as displaying output or receiving user input.

**Writing and Compiling Code**

**1. Writing Code**: C++ programs are usually written in a text editor. When you write your program, you save it as a .cpp file. You might also use libraries (external functions) to help with common tasks in your program.

**2. Compilation**: Since your CPU can't directly execute C++ code, you need to compile it. This step generates an **executable file**:

1. On Windows, this file will have a .exe extension.

2. On Linux, it will be an executable file named a.out by default.

**3. Running the Program**: After the code is compiled, you can run the generated executable file. The CPU can now understand the instructions, and your program will run on your machine.

**Using Integrated Development Environments (IDEs):-** A better way to write and run C++ programs is by using an **Integrated Development Environment (IDE)**. Popular IDEs include:

- **Code::Blocks**
- **DevC++**
- **Microsoft Visual Studio**
- **Xcode** (for macOS)
- **Sublime Text** (with extensions)

IDEs make programming much easier by providing a user-friendly editor with features like **syntax highlighting**, which colors different parts of your code to make it easier to read. With a single click, you can **compile** and **run** your code. These IDEs handle all the background work of compiling and running the code.

1. **Writing Source Code**: In an IDE, syntax highlighting helps you spot errors more easily while writing C++ code.
2. **Compiling**: When you click compile, the IDE runs the compiler, which converts C++ source code into machine-readable object code that is not yet executable.
3. **Linking**: If your program uses libraries or external functions, the IDE will also invoke a **linker**. The linker combines your object code with the object code from libraries or other external sources to create the final executable.
4. **Running**: Once the executable file is created, you can run your program directly from the IDE. The CPU can now understand and execute the program.

**Key Steps in the Compilation Process**

- **Preprocessing**: This step handles tasks like including libraries and defining constants.
- **Syntax Checking**: The compiler checks the code for any errors in syntax.
- **Linking**: The linker combines your program's object code with external libraries and functions to form a complete executable.

**C++ vs. Java and Python:-** C++ operates differently from languages like **Java** and **Python**. For Java and Python, the source code is typically first converted into **bytecode**—a machine-independent code. A **virtual machine** then runs the bytecode on different platforms, which is why Java and Python are called **platform-independent**.

On the other hand, **C++ code** is compiled directly into machine-specific **object code**. The compiled executable is designed for a specific platform, meaning the compiler must be tailored for the operating system or hardware you are using (e.g., Windows or Linux). This makes C++ **platform-dependent**.

**Basic Programming Terminology**

Before we begin coding, it's important to understand some key terms that are frequently used in programming. Let's go through them:

**1. Keywords:-** Keywords are reserved words in a programming language that have predefined meanings and cannot be used as variable or function names. For example:

- for: Used to create loops that repeat actions.
- while: Used to execute a block of code repeatedly.
- true and false: Boolean values that represent truth and falsity.

**2. Variables :-** Variables are named storage locations in a program that can hold a value of a particular data type. In C++, variables must be declared with a specific data type before they can be used. Common data types in C++ include "int" for integers, "float" for decimal numbers, and "char" for characters.

**3. Functions:-** A function is a block of code that performs a specific task. For example, max(a, b) is a function that takes two inputs and returns the larger value. Functions like printf() in C/C++ take inputs (like a string) and output them to the screen. You can define your own functions for repeated tasks in your code.

**4. Object-Oriented Programming (OOP) :-** There are two main programming paradigms:

- **Functional Programming**: Everything is done via functions calling other functions.
- **Object-Oriented Programming (OOP)**: Instead of functions calling each other, you create **objects** that interact with each other.

OOP is ideal for larger software projects as it organizes code into classes and objects. In OOP:

- A **class** is a blueprint that defines the properties and behaviors of objects. **For example**: A Student class might have properties like name and enrollment number, and behaviors like enroll() or graduate().
- An **object** is an instance of a class. **For example**: Student1 can be an object created from the Student class.

C++ supports both functional programming (with functions interacting) and object-oriented programming (with objects interacting).

**5. Primitive and Non-Primitive Types:-** C++ and Java support both:

- **Primitive types**: Variables like int, float, and char that store basic data values.
- **Non-Primitive types**: Objects that are instances of classes.

In **Python**, everything is an object, so there are no primitive types like in C++ or Java.

**6. Statically Typed vs Dynamically Typed**

- **Statically Typed Languages** (C++ and Java): Variables must be declared with a specific type before use.
  For example: you must specify int x = 5; before using x.
- **Dynamically Typed Languages** (Python): Variables don't need an explicit type declaration. You can assign a value to a variable without specifying its type, like x = 5, and later change x to store a string.

**7. Header Files:-** In C++, header files are files that contain declarations for functions, variables, and other constructs. These files are included in C++ programs using the preprocessor directive #include. Header files allow the program to use functions and variables without having to know the implementation details. **For example**: #include <iostream> allows the use of input/output operations like cout and cin.

**8. Namespace:-** A namespace is a container for organizing code into logical groups. For example, std is a namespace that contains standard libraries and functions in C++. The advantage of namespaces is that they prevent name collisions. If two different libraries or parts of your code define a variable with the same name,

namespaces allow them to coexist by distinguishing them.

For example: std::cout refers to the cout function in the std namespace.

## First C++ Program

```
#include<iostream>
using namespace std ;
int main()
{
   cout << "GeeksForGeeks"  ;
   return 0 ;
}
```

*Output:*

```
GeeksforGeeks
```

Let us now understand every line and the terminologies of the above program:

***1. #include <iostream>***

- **What it does**:
  This is a **preprocessor directive**. Lines beginning with # are processed by the **preprocessor**, which is the first step in the compilation process. The preprocessor replaces these lines with relevant code before the compiler starts compiling.
- **Why iostream**:
  The iostream header file is included because it contains the declaration of the cout object, which is used to print output to the screen.
  - **iostream stands for Input-Output Stream.**
  - Without including this file, you cannot use input/output functions like cout or cin.

---

***2. using namespace std;***

- **What it does**:
  This line imports the entire **std namespace** into the program's current scope.
  - The std namespace contains all the standard C++ library functions and objects like cout and cin.
  - By including this line, you can directly write cout instead of std::cout.
- **Better Practice**:
  It's considered better practice to avoid using namespace std; in larger programs to prevent conflicts between multiple namespaces. Instead, use std::cout explicitly.

---

***3. int main()***

- **What it does**:
  The main() function is where every C++ program begins execution.
  - The int before main() indicates that the function will return an **integer value**.
  - The **return value** signals the program's exit status:
    - return 0; means the program finished successfully.
- **Curly Braces {}**:
  - The { indicates the start of the function body, and } marks the end. Everything between these braces is part of the function.

---

***4. std::cout << "GeeksforGeeks";***

- **What it does**:
  This statement outputs the string "GeeksforGeeks" to the screen.

- **std::cout**: Stands for "character output" and represents the standard output stream (usually the screen).
- **<< operator**: This is the stream insertion operator used to send data to the output stream.
- **String inside quotes**: Anything enclosed in double quotes will be printed as is.

- **Semicolon ;**:
  - The semicolon indicates the end of the statement. Missing a semicolon results in a **syntax error**.

---

***5. return 0;***

- **What it does**:
  This statement ends the program and returns the value 0 to indicate successful completion.
  - Functions in C++ generally return a value, and since main() is of type int, it should return an integer value

---

***6. Indentation and Readability***

- **What it does**:
  Indentation is not mandatory in C++ but is highly recommended.
  - It helps organize the code and makes it more readable.
  - Typically, **4 spaces** or a **tab** is used for indentation.

---

**Key Points to Remember:** Important Points to Note while Writing a C++ Program:

1. Always include the necessary header files for the smooth execution of functions.
2. Avoid using using namespace std; in large programs. Instead, use std:: before standard library objects and functions.
3. The execution of code begins from the **main()** function.
4. It is a good practice to use **Indentation** and **comments** in programs for easy understanding.
5. A semicolon marks the end of a statement. Omitting it will cause syntax errors.
6. Curly braces define the boundaries of code blocks, such as functions, loops, or conditionals.

**Comments in C++**:- A well-documented program is a good practice as a programmer. It makes a program more readable and error finding become easier. One important part of good documentation is Comments.

- In computer programming, a comment is a programmer-readable explanation or annotation in the source code of a computer program
- Comments are statements that are not executed by the compiler and interpreter.

**In C/C++ there are two types of comments :**

1. Single line comment //….comments…
2. Multi-line comment /*….comment…*/

**Single line Comment**:- Represented as // double forward slash

It is used to denote a single line comment. It applies comment to a single line only. It is referred to as C++-style comments as it is originally part of C++ programming. for example:

```
// C++ program to illustrate
// use of single-line comment
#include <iostream>
int main()
{
   // Single line Welcome user comment
   std::cout << "Welcome to GeeksforGeeks";
   return 0;
```

```
}
```

**Output** Welcome to GeeksforGeeks

**Multi-line comment**:- Represented as **/* any_text */** start with forward slash and asterisk (/*) and end with asterisk and forward slash (*/).

It is used to denote multi-line comment.

```
/* This is
a multi-line
comment */
#include <iostream>
int main()
{
   /* Multi-line Welcome user comment
   written to demonstrate comments
   in C/C++ */
   std::cout << "Welcome to GeeksforGeeks";
   return 0;
}
```

**Output** Welcome to GeeksforGeeks

**Comment at End of Code Line** :- You can also create a comment that displays at the end of a line of code. But generally its a better practice to put the comment before the line of code.

**Example:** int age; // age of the person

**When and Why to use Comments in programming?**

- **Comments Enhance Readability**:
  Comments make the code easier to understand for someone reading it, especially in large or complex programs.
- **Explain Algorithms**:
  Comments can provide detailed explanations of algorithms or logic used in the code.
- **Helpful for Future Reference**:
  Comments are useful for the original developer when revisiting the code after a long time.
- **Improves Collaboration**:
  In collaborative projects, comments help team members understand each other's code more easily.

**Variables in C++**:- Variables in C++ is a name given to a memory location. It is the basic unit of storage in a program.

- The value stored in a variable can be changed during program execution.
- A variable is only a name given to a memory location, all the operations done on the variable effects that memory location.
- In C++, all the variables must be declared before use.

**Rules For Declaring Variable**

- The name of the variable contains letters, digits, and underscores.
- The name of the variable is case sensitive (ex Arr and arr both are different variables).
- The name of the variable does not contain any whitespace and special characters (ex #,$,%,*, etc).
- All the variable names must begin with a letter of the alphabet or an underscore(_).
- In C++, reserved keywords like float, double, and class cannot be used as variable names.

**How to Declare Variables?**

A typical variable declaration is of the form:

```
// Declaring a single variable
type variable_name;

// Declaring multiple variables:
type variable1_name, variable2_name, variable3_name;
```

**A variable name can consist of alphabets (both upper and lower case), numbers, and the underscore '_' character. However, the name must not start with a number.**

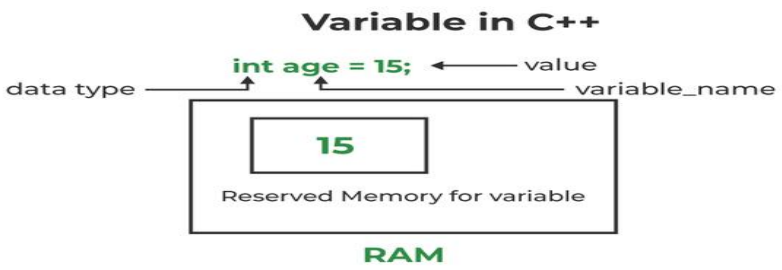
**Variable Naming Rules :-**Variable names in C++ are identifiers used to store data, and there are specific rules and conventions that must be followed to ensure the compiler correctly interprets the code. Let's explore these rules, along with examples of valid and invalid variable names.

**Rules for Naming Variables**

1. **Allowed Characters**:
   - **Lowercase Letters**: a to z
     Example: age, name
   - **Uppercase Letters**: A to Z
     Example: UserName, PlayerScore
   - **Digits**: 0 to 9
     Example: x23, value1
   - **Underscore (_)**:
     Example: _temp, max_value
2. **Cannot Start with a Digit**:
   - Variable names must start with a letter or an underscore.
   - **Valid**: x1, _count
   - **Invalid**: 1a, 2value
3. **Cannot Contain Special Characters**:
   - Only underscore (_) is allowed; other characters like @, #, \$, etc., are invalid.
   - **Invalid**: a#b, total\$amount
4. **Cannot Be a Reserved Keyword**:
   - Reserved keywords like int, else, for, etc., cannot be used as variable names.
   - **Invalid**: else, for

**Conventions for Naming Variables** To make code more readable and maintainable, follow these **naming conventions**:

1. **Camel Case**:
   - Start with a lowercase letter, and capitalize subsequent words.
   - **Example**: currentYear, playerName, maxValue
2. **Snake Case**:
   - Use underscores to separate words.
   - **Example**: current_year, player_name, max_value
3. **For Constants**:
   - Use all uppercase letters with underscores between words.
   - **Example**: MAX_AGE, PI, LIMIT



***datatype: Type of data that can be stored in this variable.***
***variable_name: Name given to the variable.***
***value: It is the initial value stored in the variable.***

```
#include<iostream>
using namespace std ;
int main()
{
   int x = 5 ;
   cout << x << " " ;
   x = 8 ; // we can update the value also
   cout << x << " " ;
   int y = 10 ;
   cout << y ;
   return 0 ;
}
```

**Key Points**

1. **Declaration**: int x = 5; creates an integer variable x with an initial value of 5.
2. **Update**: x = 8; changes the value of x to 8.
3. **New Variable**: int y = 10; declares a second variable y with value 10.
4. **Output**: The program prints 5 8 10.

**other variable declaration Examples**:

```
// Declaring float variable
float simpleInterest;

// Declaring integer variable
int time, speed;

// Declaring character variable
char var;
```

**Data Types in C++ :-**In C++, data types define the type of data a variable can hold. Data types are essential for specifying the type of value stored in variables and ensuring proper memory allocation. They are broadly classified into Primitive (or Fundamental) Types and Non-Primitive Types.

**1. Primitive (Fundamental) Types:-**Primitive data types are predefined by the C++ language and include the following categories:

***a. Integer Types Used*** *to store whole numbers (both positive and negative).*

- **Examples**: short, int, long, long long
- **Unsigned Variants**: unsigned short, unsigned int, unsigned long, unsigned long long (used for positive values only).

***b. Character Types:-****Used to store single characters.*

- **Examples**: char, unsigned char
- **Specialized Types**: wchar_t (wide character), char16_t, char32_t (for Unicode).

***c. Floating-Point Types:-****Used to store decimal or fractional numbers.*

- **Examples**: float, double, long double

***d. Other Types***

- **Boolean (bool)**: Used to store true or false.
- **Void (void)**: Represents the absence of a value, primarily used for functions that do not return any value.

**2. Non-Primitive Types:-**Non-primitive types are derived from primitive types or created by the programmer.

**Examples:**

- **Array**: A collection of elements of the same type.
- **Pointers**: Variables that store the address of another variable.
- **User-Defined Types**: Created by the programmer using classes, structs, or enums.

The following program demonstrates the use of various data types in C++:

```
#include<iostream>
#include<string>
using namespace std ;
int main()
{
   int age = 39 ;
   string name = "Sandeep" ;
   char gender = 'M' ;
   bool isMarried = true ;
   float weight = 68.5 ;
   cout << name      << "\n"
       << age       << "\n"
       << gender    << "\n"
       << isMarried << "\n"
       << weight ;
   return 0 ;
}
```

**1**: Represents true for the boolean value.

**Explanation of Code**

1. **Integer (int)**:
   int age = 39;

   Stores the age as an integer.

2. **String (string)**:
   string name = "Sandeep";

   A non-primitive type used to store a sequence of characters.

3. **Character (char)**:
   char gender = 'M';

   Stores a single character ('M' for Male).

4. **Boolean (bool)**:
   bool isMarried = true;

   Stores a logical value (true or false).

5. **Floating-Point (float)**:
   float weight = 68.5;

   Stores the weight as a decimal number.

**Operator sizeof in C++:-**The sizeof operator is a unary compile-time operator used to determine the size of variables, data types, and constants in bytes at compile time. It can also determine the size of classes, structures, and unions.

## Key Features of sizeof() Operator

1. **Compile-Time Evaluation**:
   - The sizeof() operator is evaluated during the compilation of the program. This means that its results are known before the program runs.
2. **Determines Size in Bytes**:
   - It returns the number of bytes required to store a particular data type or object in memory.

3. **Flexible Usage**:
   - It can be used with basic data types, user-defined types, variables, and even expressions.

**Syntax:** *sizeof (data type)or sizeof (expression)* Here:

- type refers to a built-in or user-defined data type.
- expression evaluates to an object or a data type.

**Example 1: Number of bytes taken by different data types.**

```
#include <iostream>
using namespace std;
int main() {
   cout << "Size of int: " << sizeof(int) << " bytes\n";
   cout << "Size of char: " << sizeof(char) << " bytes\n";
   cout << "Size of long long: " << sizeof(long long) << " bytes\n";
   cout << "Size of float: " << sizeof(float) << " bytes\n";
     return 0;
}
```

**Example 2: Number of bytes taken by variables of different data types.**

```
#include <bits/stdc++.h>
using namespace std;
int main()
{
 int x;
 double d = 10.2;
 cout << sizeof(x) << "\n";
 cout << sizeof(d) << "\n";
 cout << sizeof(10ll) << "\n";
 cout << sizeof(3.4)  << "\n";
 cout << sizeof(3.4f) << "\n" ;
 return 0;
}
```

**Global Variables and Scope:-**In programming , the scope of a variable is defined as the extent of the program code within which the variable can be accessed or declared or worked with. There are mainly two types of variable scopes:

1. **Local Variables**
2. **Global Variables**

**Local Variables** - Variables defined within a function or block are said to be local to those functions.

Anything between '{' and '}' is said to be inside a block.

Local variables do not exist outside the block in which they are declared, i.e. they can not be accessed or used outside that block.

**Declaring local variables:** Local variables are declared inside a block

```
int main()
{
   int x = 10;
   cout<<x;
    return 0;
```

```
}    //Local variable
```
**Output:** 10

**Global Variables -** Global Variables can be accessed from any part of the program.

- They are available throughout the lifetime of a program.
- They are declared at the top of the program outside all of the functions or blocks.

**Declaring global variables:** Global variables are usually declared outside of all of the functions and blocks, at the top of the program. They can be accessed from any portion of the program.

```
#include<iostream>
using namespace std;
int x = 10;
int main()
{
   cout<<x;
   return 0;
}
//global variable
```

*Output:* ***10***

**NOTE** - Whenever there is a local variable defined with same name as that of a global variable then the compiler will give precedence to the local variable

**When a variable is referenced outside its scope:**

- A **local variable** cannot be accessed outside the function/block where it is declared, resulting in a **compiler error**.
- A **global variable** can be accessed anywhere in the program unless explicitly hidden by a local variable.

**Using the extern Keyword:-**If a global variable is defined **after** it is used in the program, the compiler throws an error. To solve this, we use the extern keyword, which tells the compiler that the variable is defined elsewhere in the program.

*Example 1: Without extern (Compiler Error)*

```
#include <iostream>
using namespace std;
int main()
{
 cout << x;  // Error: x is not declared yet
   return 0;
}
int x = 10;
```

**Example 2: Using extern (Correct)**

```
#include <iostream>
using namespace std;
extern int x;
int main()
{
  cout << x;
   return 0;
}
int x = 10;
```

*Output:*

```
10
```

**Local and Global Variables with the Same Name**

- The **local variable** takes precedence within the block where it is declared.
- The **global variable** is overshadowed but still exists and can be accessed using the scope resolution operator (::).

*Example 1: Local Variable Overshadows Global Variable*

```
#include <iostream>
using namespace std;
int x = 20;  // Global variable
int main()
{
   int x = 10;  // Local variable
   cout << x;
   return 0;
}
```

Output: 10

*Example 2: Nested Local Variables*

```
#include <iostream>
using namespace std;
int x = 20;
int main()
{
   int x = 10;
   {
      int x = 30;
      cout << x;
   }
   return 0;
}
```

Output: 30

*Example 3: Redefining a Variable in the Same Scope (Error)*

```
#include <iostream>
using namespace std;
int x = 20;
int main()
{
   int x = 10;
   int x = 20;  // Error: Redefinition of x in the same scope
   cout << x;
   return 0;
}
```

## Scope Rules in C++

1. **Functions, Loops, and Conditional Statements Create a New Scope**:
   - Each function, loop, or conditional block introduces a new scope.
   - Variables declared inside the block are local to that scope.
2. **Outer Scope Variables Are Accessible in Inner Scopes**:
   - Variables from an outer scope can be used inside inner scopes unless overshadowed.
   - However, inner scope variables are not accessible outside their block.
3. **Creating a New Scope with Curly Braces**:
   - Curly braces {} can be used to create a new block scope anywhere in the program.

Ranges of Data Types in C++Data type modifiers available in C++ are:

- Signed

- Unsigned
- Short
- Long

The below table summarizes the modified size and range of built-in datatypes which also depends upon the compiler( i.e 32bits or 64bits ) when combined with the type modifiers:

| Data Type | Size (in bytes) | Range |
|---|---|---|
| short int | 2 | -32,768 to 32,767 |
| unsigned short int | 2 | 0 to 65,535 |
| unsigned int | 4 | 0 to 4,294,967,295 |
| int | 2 or 4 | -32,768 to 32,767 or -2,147,483,648 to 2,147,483,647 |
| long int | 4 | -2,147,483,648 to 2,147,483,647 |
| unsigned long int | 4 | 0 to 4,294,967,295 |
| long long int | 8 | -(2^63) to (2^63)-1 |
| unsigned long long int | 8 | 0 to 18,446,744,073,709,551,615 |
| signed char | 1 | -128 to 127 |
| unsigned char | 1 | 0 to 255 |
| float | 4 | -3.4×10^38 to 3.4×10^38 |
| double | 8 | -1.7×10^308 to1.7×10^308 |
| long double | 12 | -1.1×10^4932 to1.1×10^4932 |
| wchar_t | 2 or 4 | 1 wide character |

**Static Variables in C++** A static variable is a variable that is **allocated memory only once** during the lifetime of a program. Its value is retained across multiple function calls and persists until the program terminates.

**Key Characteristics of Static Variables**

1. **Lifetime**:
   - Static variables exist for the entire lifetime of the program.
2. **Scope**:
   - A static variable declared inside a function has **local scope** but retains its value across multiple calls to the function.

- A static global variable has **file scope** (internal linkage) and is not accessible outside the file in which it is defined.

3. **Initialization**:
   - Static variables are initialized only once, and if not explicitly initialized, they are automatically initialized to **zero**.
4. **Memory Allocation**:
   - Memory for static variables is allocated in the **data segment**, not the stack.

```
// C++ program to demonstrate
// the use of static Static
// variables in a Function
#include <iostream>
using namespace std;
int main()
{
   static int x;
   cout<<x<<endl;
   return 0;
}
```

**Output:** 0

**Const in C++:-**In C++, the const keyword is used to define constants — variables whose values cannot be modified once they are initialized. This is an essential feature when working with values that should remain constant throughout the execution of a program, ensuring immutability and preventing accidental changes.

**What is a const Variable? :-**A const variable is a variable whose value cannot be changed after initialization. It must be initialized at the time of declaration, and any attempt to modify it will result in a compiler error.

```
#include <iostream>
using namespace std;
int main() {
   const int x = 10;
   x = x + 1;        // Compiler error: Cannot modify a const variable
   cout << x;
   return 0;
}
```

If we remove the modification line (x = x + 1), the program will compile successfully and output 10.

**Real-World Example of const** Constants are particularly useful in real-world applications where certain values must remain unchanged. For

```
#include <iostream>
using namespace std;
const double PI = 3.14;
int main() {
   int r;
   cout << "Enter the radius of the circle: ";
   cin >> r;
   cout << "Area of the circle is: " << PI * r * r;
   return 0;
}
```

**Alternative to const: Using #define**Before the introduction of const, the #define preprocessor directive was commonly used to define constants.

```
#define var 5
```

Since Macros are handled by the preprocessor(the pre-processor does text replacement in our source file, replacing all occurrences of ‘var’ with the literal 5), not by the compiler.

Hence it wouldn’t be recommended because Macros don’t carry type-checking information and are also prone to error.

**Constants vs. Macros (#define)**

| **Feature** | **const** | **#define** |
|---|---|---|
| **Type Safety** | Provides type safety | No type checking |
| **Scope** | Obeys scoping rules | Global scope, no scoping |
| **Debugging** | Easier to debug | Harder to trace |

**Best Practices:** There are a certain set of rules for the declaration and initialization of the constant variables -:

- The const variable cannot be left un-initialized at the time of the assignment.
- It cannot be assigned value anywhere in the program.
- Explicit value needed to be provided to the constant variable at the time of declaration of the constant variable.

Ex- const int num = 1;

**Auto Keyword in C++** :- The auto keyword in C++ is a modern and powerful feature introduced in C++11 that simplifies variable declarations. By using auto, the compiler automatically deduces the type of a variable based on the type of the initializer. This feature not only enhances code readability but also reduces the chance of type mismatch errors.

**Note:** The variable declared with auto keyword should be initialized at the time of its declaration only or else there will be a compile-time error.

**How auto Works** The auto keyword allows the compiler to infer the type of a variable at compile time based on its initializer. The programmer doesn't need to explicitly specify the type, making the code cleaner and easier to write.

```
#include <iostream>
#include <typeinfo>
using namespace std;
int main() {
   auto a = 5;      // Type of 'a' is deduced as int
   auto b = 10.5;   // Type of 'b' is deduced as double
   cout << a << " " << b << "\n";
   cout << typeid(a).name() << " " << typeid(b).name();
   return 0;
}
```

**Explanation**:

- The auto keyword automatically deduces the type of a as int and b as double based on the values assigned.
- The typeid It is an operator which is used where the dynamic type of an object needs to be known.
- typeid(x).name() returns the data type of x, for example, it return 'i' for integers, 'd' for doubles, 'Pi' for the pointer to integer etc. But the actual name returned is mostly compiler dependent.

**Advantages**

**1.** No Conversion happens when auto is used float x = 3.4 //double literal converted to float. auto y = 3.4 //type of y is double.

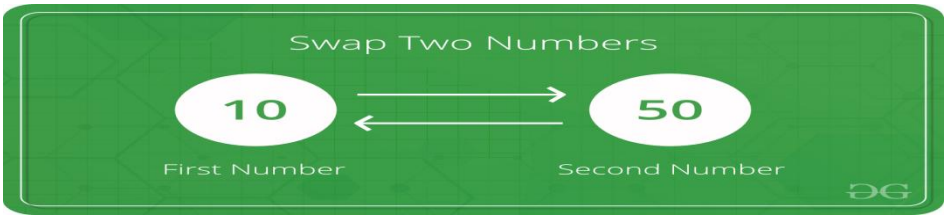
**2.** If a function returns auto, its return type can be changed without worrying about prototype change.

**3.** Can be very helpful for lengthy types, especially STL iterators vector<int>::iterator i; can be replaced with auto i;

**4.** It can be used in Lambda Expressions

**Disadvantages:** The main drawback is that, by using auto, you don't necessarily know the type of object being created. There are also occasions where the programmer might expect the compiler to deduce one type, but the compiler adamantly deduces another. It takes more processing time.

**Swap Two Numbers:-** Swapping numbers is the process of interchanging their values. Here, we will learn algorithms and code to swap two numbers in the C++ programming language.




***Input :*** *a = 10, b = 20*
***Output :*** *a = 20, b = 10*

***Input :*** *a = "geeks", b = "gfg"*
***Output :*** *a = "gfg", b = "geeks"*

**1. Swap Numbers Using a Temporary Variable**

The idea is simple in this approach:

1. Assign a to a temp variable: temp = a
2. Assign b to a: a = b
3. Assign temp to b: b = temp

Below is the C++ program to implement the above approach:

```
#include <iostream>
using namespace std;
int main()
{
   int a = 10, b = 20;
   cout << a << " " << b << "\n";
   int temp = a;
   a = b;
   b = temp;
   cout << a << " " << b << "\n";
   return 0;
}
```

*Output:*

```
10 20
20 10
```

**Literals in C++** Literals are the Constant values that are assigned to the constant variables. Literals represent fixed values that cannot be modified. Literals contain memory but they do not have references as variables. Generally, both terms, constants, and literals are used interchangeably.

For example, “const int variable_name = 5;“, is a constant expression and the value 5 is referred to as a constant integer literal. There are 5 types of literal in C++.

- Integer literal
- Float literal
- Character literal
- String literal
- Boolean literal

**Integer literals:-**Integer literals are used to represent and store the integer values only. Integer literals are expressed in two types i.e.

**A) Prefixes:** The Prefix of the integer literal indicates the base in which it is to be read.

**There are basically represented into 4 types:**

**a. Decimal-literal(base 10):** A **non-zero decimal digit** followed by zero or more decimal digits(0, 1, 2, 3, 4, 5, 6, 7, 8, 9).

**Example :**56, 78

**b. Octal-literal(base 8):** a **0** followed by zero or more octal digits(0, 1, 2, 3, 4, 5, 6, 7).

**Example:** 045, 076, 06210

**c. Hex-literal(base 16): 0x** or **0X** followed by one or more hexadecimal digits(0, 1, 2, 3, 4, 5, 6, 7, 8, 9, a, A, b, B, c, C, d, D, e, E, f, F).

**Example:** 0x23A, 0Xb4C, 0xFEA

**d. Binary-literal(base 2): 0b** or **0B** followed by one or more binary digits(0, 1).

**Example:** 0b101, 0B111

```
#include <iostream>
using namespace std;
int main()
{
   cout << "Prefixes in Integer Literals\n" ;
   {
      int a = 20 ;  //Decimal
      int b = 0x1A; //Hexadecimal
      int c = 016;  //Octal
      int d = 0b11; //Binary
      cout << a << "\n"
          << b << "\n"
          << c << "\n"
          << d << "\n";
   }
   return 0;
}
```

*Output:* ***Prefixes in Integer Literals***

```
20
26
14
3
```

**B) Suffixes:** The Prefix of the integer literal indicates the type in which it is to be read.

```
#include <iostream>
using namespace std;

int main() {
      cout << "Suffixes in Integer Literals\n";
      int a = 124;
      unsigned b = 124u;
      long int c = 124L;
      long long int d = 124LL;
      cout << a << "\n"
          << b << "\n"
          << c << "\n"
          << d << "\n";

   return 0;
}
```

**Floating-Point Literals**:-These are used to represent and store real numbers. The real number has an integer part, real part, fractional part, and exponential part. The floating-point literals can be stored either in decimal form or exponential form. While representing the floating-point decimals one must keep two things in mind to produce valid literal:

- In the decimal form, one must include the decimal point, exponent part, or both, otherwise, it will lead to an error.
- In the exponential form, one must include the integer part, fractional part, or both, otherwise, it will lead to an error.

A few floating-point literal representations are shown below:

```
10.125
1.215E-10
10.5E-3
```

**Invalid Floating Literals:**

```
123E
1250f
0.e879
```

```
#include <iostream>
using namespace std;
int main()
{
   cout << "Floating Point Literals\n";
   {
      float  a = 10.5f;
      double b = 10.515;
      float  c = 2.1e15f;
      double d = 200.1e-60;
      double f = 0x1A.1p2;
      cout << a << "\n"
          << b << "\n"
          << c << "\n"
          << d << "\n"
          << f << "\n";
   }
   return 0;
}
```

**Character Literal:-**This refers to the literal that is used to store a single character within a single quote. To store multiple characters, one needs to use a character array. Storing more than one character within a single quote will throw a warning and display just the last character of the literal. It gives rise to the following two representations:

**A. char type:** This is used to store normal character literal or narrow-character literals.

**Example:**

```
char chr = 'G';
```

**B. wchar_t type:** If the character is followed by L, then the literal needs to be stored in wchar_t. This represents a wide-character literal.

**Example:**

```
wchar_t chr = L'G';
```

**Escape Sequences** : There are various special characters that one can use to perform various operations.

***Character Literals***

Character literals are enclosed in single quotes (') and can include prefixes for different encoding types:

| Prefix | Encoding | Example | Description |
|---|---|---|---|
| **(none)** | ASCII (default) | 'g' | Default 1-byte character literal |
| **u8** | UTF-8 | u8'g' | 1-byte UTF-8 encoded literal |
| **u** | UTF-16 | u'g' | 2-byte UTF-16 encoded literal |
| **U** | UTF-32 | U'g' | 4-byte UTF-32 encoded literal |
| **L** | wchar_t (wide character) | L'g' | Platform-dependent wide literal |

**4) String Literals:**String literals are similar to that character literals, except that they can store multiple characters and uses a double quote to store the same. It can also accommodate the special characters and escape sequences mentioned in the table above. We can break a long line into multiple lines using string literal and can separate them with the help of white spaces.

**Example:**
string stringVal = "GeeksforGeeks"

String literals are enclosed in double quotes (") and may also have prefixes for encoding:

| Prefix | Encoding | Example | Description |
|---|---|---|---|
| **(none)** | ASCII (default) | "hello" | Default string literal |
| **u8** | UTF-8 | u8"hello" | UTF-8 encoded string literal |
| **u** | UTF-16 | u"hello" | UTF-16 encoded string literal |
| **U** | UTF-32 | U"hello" | UTF-32 encoded string literal |
| **L** | wchar_t (wide character) | L"hello" | Wide string literal |

```
#include <iostream>
using namespace std;
   int main()
{
cout << "Character and String Literals\n";
   {
      char c = 'g';
      const char *s1 = "gfg";
      string s2 = "courses";
      cout << c  << "\n"
          << s1 << "\n"
          << s2 << "\n";
   }
   return 0;
```

```
}
```

**Output:**Character and String Literals

```
   g
gfg
courses
```

**Boolean Literals:-**This literal is provided **only in C++** and **not in C**. They are used to represent the boolean datatypes. These can carry two values:

- **true:** To represent True value. This must not be considered equal to int 1.
- **false:** To represent a False value. This must not be considered equal to int 0.

```
#include <iostream>
using namespace std;
int main()
{
   const bool isTrue = true;
   const bool isFalse = false;
   cout << "isTrue? "
      << isTrue << "\n";
   cout << "isFalse? "
      << isFalse << "\n";
   return 0;
}
```

**Output** isTrue? 1

```
isFalse? 0
```

Type Conversion in C++ A type cast is basically a conversion from one type to another.

1. **Implicit Type Conversion** Also known as 'automatic type conversion'.
   - Done by the compiler on its own, without any external trigger from the user.
   - Generally takes place when in an expression more than one data type is present. In such condition type conversion (type promotion) takes place to avoid lose of data.
   - All the data types of the variables are upgraded to the data type of the variable with largest data type.

```
bool -> char -> short int -> int -> unsigned int -> long -> unsigned -> long long -> float -> double -> long double
```

- It is possible for implicit conversions to lose information, signs can be lost (when signed is implicitly converted to unsigned), and overflow can occur (when long long is implicitly converted to float).

**Example of Type Implicit Conversion:**

```
#include<iostream>
using namespace std;
int main()
{
   int x = 10;
   char y = 'A';
   cout << (x+y) << '\n';
   float z = 5.5;
   cout << (x+z) << '\n';
   bool b = z;
   cout << b << '\n';
   return 0;
}
```

**2.Explicit Type Conversion: This process is also called type casting and it is user-defined. Here the user can typecast the result to make it of a particular data type.**

1. **C-Style Casting**
2. **C++ Style Casting**:
   - static_cast
   - dynamic_cast
   - const_cast
   - reinterpret_cast

***1. C-Style Casting:-****This is the traditional way of type casting, inherited from the C programming language.*

**Syntax**:

```
(target_type) variable
```

```
#include <iostream>
using namespace std;
int main() {
  int x=15,y=2;
  double z= double(x)/y;
  cout<<z;
   return 0;
}
```

***2. C++ Style Casting:-****C++ introduces safer and more specialized casting operators to replace the generic C-style cast. These include:*

- **Static Cast**
- **Dynamic Cast**
- **Const Cast**
- **Reinterpret Cast**

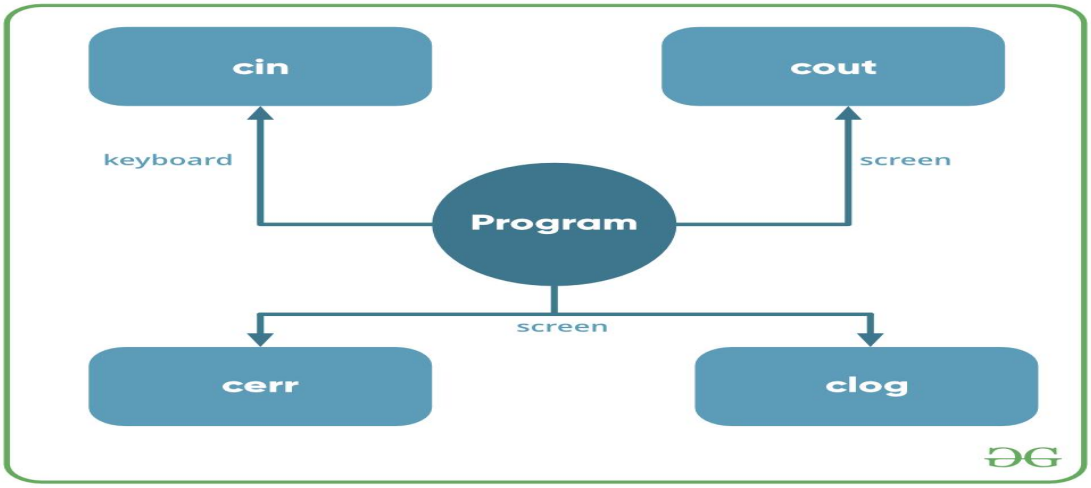
```
#include <iostream>
using namespace std;
int main()
{
   float f = 3.5;
   // using cast operator
   int b = static_cast<int>(f);
   cout << b;
}
```

**Advantages of Type Conversion:**

- This is done to take advantage of certain features of type hierarchies or type representations.
- It helps to compute expressions containing variables of different data types.

**Input & Output in C++: -**C++ comes with libraries that provide us with many ways for performing input and output. In C++ input and output are performed in the form of a sequence of bytes or more commonly known as **streams**.

- **Input Stream:** If the direction of flow of bytes is from the device(for example, Keyboard) to the main memory then this process is called input.
- **Output Stream:** If the direction of flow of bytes is opposite, i.e. from main memory to device( display screen ) then this process is called output.




**Header files available in C++ for Input/Output operations are:**

1. **iostream**: iostream stands for standard input-output stream. This header file contains definitions of objects like cin, cout, cerr, etc.
2. **iomanip**: iomanip stands for input-output manipulators. The methods declared in these files are used for manipulating streams. This file contains definitions of setw, setprecision, etc.
3. **fstream**: This header file mainly describes the file stream. This header file is used to handle the data being read from a file as input or data being written into the file as output.

The two instances **cout in C++** and **cin in C++** of iostream class are used very often for printing outputs and taking inputs respectively. These two are the most basic methods of taking input and printing output in C++. To use cin and cout in C++ one must include the header file *iostream* in the program.

- **Standard output stream (cout)**: Usually the standard output device is the display screen. The C++ **cout** statement is the instance of the ostream class. It is used to produce output on the standard output device which is usually the display screen. The data needed to be displayed on the screen is inserted in the standard output stream (cout) using the insertion operator(<<).

```
#include <iostream>
   using namespace std;
int main()
{
   char sample[] = "GeeksforGeeks";
   cout << sample << " - A computer science portal for geeks";
   return 0;
}
```

**Output** GeeksforGeeks - A computer science portal for geeks

In the above program, the insertion operator(<<) inserts the value of the string variable **sample** followed by the string “A computer science portal for geeks” in the standard output stream **cout** which is then displayed on the screen.

- **standard input stream (cin)**: Usually the input device in a computer is the keyboard. C++ cin statement is the instance of the class **istream** and is used to read input from the standard input device which is usually a keyboard.
  The extraction operator(>>) is used along with the object **cin** for reading inputs. The extraction operator extracts the data from the object **cin** which is entered using the keyboard.

```
#include <iostream>
```

```
using namespace std;
int main()
{
   int age;
   cout << "Enter your age:";
   cin >> age;
   cout << "\nYour age is: " << age;
   return 0;
}
```

**Input :** 18

**Output:** Enter your age:

Your age is: 18

The above program asks the user to input the age. The object cin is connected to the input device. The age entered by the user is extracted from cin using the extraction operator(>>) and the extracted data is then stored in the variable **age** present on the right side of the extraction operator.

- **Un-buffered standard error stream (cerr)**: The C++ cerr is the standard error stream used to output error messages. It is unbuffered, meaning messages sent to cerr are displayed immediately without being stored in a buffer. This ensures errors are displayed promptly. Unlike cout, which is buffered, cerr is suitable for critical messages. Both cout and cerr can be redirected, but they are managed as separate streams.

```
#include <iostream>
using namespace std;
int main()
{
   cerr << "An error occurred";
   return 0;
}
```

**Output:** An error occurred

- **buffered standard error stream (clog)**: The clog stream in C++ is used for logging error messages and is buffered. This means data is first inserted into a buffer and displayed only when the buffer is full or explicitly flushed (e.g., using flush()). In the provided program, clog outputs the message "An error occurred" because the program ends, and the buffer is flushed automatically.

```
#include <iostream>
using namespace std;
int main()
{
   clog << "An error occurred";

   return 0;
}
```

**Output:** An error occurred

**Output in C++ What is cout?**

The cout (short for "character output") is part of the Standard Input/Output Stream Library in C++. It is used to print data to the standard output device, typically the console. To use cout, the iostream header must be included in your program.

**Example 1: Printing a Simple** Message Here is a simple program that demonstrates how to print a message:

```
#include <iostream>
using namespace std;
int main() {
   cout << "Hello Geek!";
   return 0;
}
```

```
Output:
Hello Geek!
```

**Output Operation:** cout << "Hello Geek!"; sends the string "Hello Geek!" to the console.

**Return Statement:** return 0; indicates successful program execution.

**Example 2: Printing Different Data Types and Expressions**

```
#include<iostream>
using namespace std;
int main()
{
   // Printing different data types
   int a = 1;
   float b = 1.1f;
   string s = "string";
   char c = 'c';
   cout << a << " " << b << " " << s << " " << c << endl;

   // Printing Literals and expressions
   cout << 10 << " " << 10*2 << endl;
   return 0;
}
```

```
Output:
1 1.1 string c
10 20
```

**Explanation:** Variable Declarations and Initialisation:

- int a = 1;: Declares an integer variable a and assigns it the value 1.
- float b = 1.1f;: Declares a floating-point variable b and assigns it 1.1. The f suffix specifies it as a float.
- string s = "string";: Declares a string variable s and assigns it the value "string".
- char c = 'c';: Declares a character variable c and assigns it the value 'c'.

**Printing Variables:**

- cout << a << " " << b << " " << s << " " << c << endl; prints the values of a, b, s, and c, separated by spaces. The **endl** moves the cursor to the next line.

**Printing Literals and Expressions:**

- cout << 10 << " " << 10*2 << endl; prints the literal 10 and the result of the expression 10*2 (which is 20).

**Key Takeaways**

**Versatile Output:** The cout object in C++ can handle multiple data types, including integers, floats, strings, and characters.

**Use of Expressions:** You can directly print expressions and their results using cout.

**Combining Outputs:** The << operator allows chaining, enabling multiple items to be printed in a single statement.

**Line Breaks:** Use endl or \n for line breaks in the console output.

**Input in C++** :- The cin object C++ is used to accept the input from the standard input device i.e., keyboard. it is the instance of the class istream. It is associated with the standard C input stream stdin. The extraction operator(>>) is used along with the object cin for reading inputs. The extraction operator extracts the data from the object cin which is entered using the keyboard.

**standard input stream (cin)**: Usually the input device in a computer is the keyboard. C++ cin statement is the instance of the class **istream** and is used to read input from the standard input device which is usually a keyboard.
The extraction operator(**>>**) is used along with the object **cin** for reading inputs. The extraction operator extracts the data from the object **cin** which is entered using the keyboard.

C++ program to implement cin object to take input from the user:

```
#include <iostream>
using namespace std;

int main()
{
   int age;

   cout << "Enter your age:";
   cin >> age;
   cout << "\nYour age is: " << age;

   return 0;
}
```

**Input** 18

**Output** Enter your age:
Your age is: 18

**Note:** Multiple inputs can also be taken using the extraction operators(>>) with cin.

C++ program to implement multiple inputs from the user:

```
// C++ program to demonstrate the taking
// multiple inputs from the user
#include <iostream>
using namespace std;
// Driver Code
int main()
{
   string name;
```

```
    int age;

    // Take multiple input using cin
    cin >> name >> age;

    // Print output
    cout << "Name : " << name << endl;
    cout << "Age : " << age << endl;
    return 0;
}
```

**Input:**ABC
10

**Output:**Name : ABC
Age : 10

*Taking Full String Input Using getline*

In certain cases, the extraction operator (>>) may not be sufficient, such as when handling multi-word inputs. To handle such scenarios, the getline() function is used to read the entire line of input.

```
#include <iostream>
using namespace std;

int main()
{
    string name;
    cout << "Please enter your name \n";
    getline(cin, name); // Reading the full line as input
    // cin >> name; // Wrong approach, try giving a full name
    cout << "Hi " << name << ", \n"
        << "Welcome to GfG";

    return 0;
}
```

*Input: Sandeep jain*
Output: Please enter your name
Hi Sandeep jain,
Welcome to GfG

**Example: Demonstrating Buffering in C++**:-The following program illustrates the concept of buffering by taking two integers as input from the user and displaying their multiplication as the output.

```
#include <iostream>
using namespace std;

int main()
{
    int x, y;

    cout << "Enter the first integer: \n";
    cin >> x; // Input stored in the buffer and then assigned to x

    cout << "Enter the second integer: \n";
    cin >> y; // Input stored in the buffer and then assigned to y
```

```
    cout << "Multiplication is: " << (x * y) << '\n';
    return 0;
}
```

**Input**:*Enter the first integer: 3*

*Enter the second integer: 4*

**Output**: *Multiplication is: 12*

*Input Buffering:*

- When the user enters the first integer (3), it is temporarily stored in the input buffer.
- The cin >> x, statement retrieves the value from the buffer and assigns it to the variable x.
- Similarly, when the user enters the second integer (4), it is stored in the buffer and then assigned to y using cin >> y.

*Output Buffering:*

- The cout statements do not directly print to the screen. Instead, the data is stored in an output buffer.
- The buffer is flushed (i.e., the contents are sent to the screen) when a newline character (\n) or endl is encountered, ensuring efficient output.

**Escape Sequences in C++** : -The **escape sequences** are special non-printing characters that are used to control the printing behavior of the output stream objects (such as 'cout'). These characters are not displayed in the output. An escape sequence is prefixed with a backslash (\) and a coded character is used to control the printing behavior. The backslash (\) is called an escape character. So the escape sequence looks like two characters.

These are written in single or double-quotes. The escape sequence can be inserted in any position of the string such as:

- At the beginning of the string.
- In the middle of the string.
- At the end of the string etc.

For example, the escape sequence '\n' is used to insert a new line. The cursor moves from the current position on the output device to the beginning of the next line. If escape sequence '\n' is inserted at the beginning of the string then the string will be printed after printing a blank line e.g.

**cout<<"\nWelcome";** This statement results in the following output:

```
Welcome
```

**List of Escape Sequences**

| Escape Sequence | Name | Description |
|---|---|---|
| `\\` | Backslash | Represents a literal backslash character. |
| `\'` | Single Quote | Represents a literal single quote (apostrophe). |
| `\"` | Double Quote | Represents a literal double quote. |
| `\n` | Newline | Represents a newline character. |
| `\r` | Carriage Return | Represents a carriage return character. |
| `\t` | Tab | Represents a tab character. |
| `\b` | Backspace | Represents a backspace character. |
| `\f` | Form Feed | Represents a form feed character. |
| `\v` | Vertical Tab | Represents a vertical tab character. |
| `\a` | Alert (bell) | Generates an alert (bell) sound. |
| `\0` | Null Character | Represents the null character (ASCII value 0). |

**Note** - The line continuation sequence (\ followed by a new-line character) is not an escape sequence. It is used in character strings to indicate that the current line of source code continues on the next line.

- The value of an escape sequence represents the member of the character set used at run time. Escape sequences are translated during preprocessing. For example, on a system using the ASCII character codes, the value of the escape sequence \x56 is the letter V.
- Use escape sequences only in character constants or in string literals. An error message is issued if an escape sequence is not recognized.
- In string and character sequences, when you want the backslash to represent itself (rather than the beginning of an escape sequence), you must use a \ backslash escape sequence. For example: **cout << "The escape sequence \\n." << endl;**

This statement results in the following output:

```
The escape sequence \n
```

```
// c++ program to illustrate
// \n escape sequence
#include <iostream>
   using namespace std;
int main()
{
   // Here we are using \n, which
   // is a new line character.
   cout<<"Hello\n"<<endl;
   cout<<"GeeksforGeeks"<<endl;
   return 0;
}
```

**Output**Hello
GeeksforGeeks

**Example -** program that displays square, the cube of a number in table form using Escape Sequences

```
#include <iostream>
using namespace std;
int main()
{
 cout<<" Number\t Square\t Cube "<<endl;
 cout<<" 1\t\t"<<1*1<<"\t\t"<<1*1*1<<endl;
 cout<<" 2\t\t"<<2*2<<"\t\t"<<2*2*2<<endl;
 cout<<" 3\t\t"<<3*3<<"\t\t"<<3*3*3<<endl;
 cout<<" 4\t\t"<<4*4<<"\t\t"<<4*4*4<<endl;

}
```

**Output**

| Number | Square | Cube |
|---|---|---|
| 1 | 1 | 1 |
| 2 | 4 | 8 |
| 3 | 9 | 27 |
| 4 | 16 | 64 |

- The code prints a table with three columns: "Number", "Square", and "Cube".
- The \t escape sequence is used to add tab spaces between the columns, ensuring the data is aligned properly.
- The endl escape sequence is used to print a newline after each row, making the output look organized.

**IO Manipulation: - In C++,** input and output (I/O) are fundamental components of any program. Often, we need to format the output to make it more readable or to meet certain requirements. C++ provides several tools and manipulators to modify how data is displayed on the screen. In this article, we will explore some common I/O manipulations using code examples.

**1. Boolean Output Formatting** In C++, the bool data type is used to store Boolean values (true or false). By default, printing a Boolean value using cout outputs 1 for true and 0 for false. However, we can use manipulators to change this behavior.

```
#include <iostream>
using namespace std;
int main()
{
   bool a = true;
   cout << a << "\n";       // 1
   cout << std::boolalpha;   // Set output to true/false
   cout << a << "\n";       // true
   cout << std::noboolalpha; // Set output back to 0/1
   cout << a;               // 1
   return 0;
}
```

**Explanation: std::boolalpha:** This manipulator causes the Boolean values to be printed as true or false instead of 1 or 0.

**std::noboolalpha:** Reverts the formatting back to printing 1 for true and 0 for false.

**2. Changing Number Base (Decimal, Hexadecimal, Octal)** C++ allows us to display numbers in different numeral systems. By default, integers are displayed in decimal format, but we can use manipulators to change this to hexadecimal (std::hex) or octal (std::oct).

```
#include <iostream>
```

```
using namespace std;
int main()
{
   int a = 26, b = 20;
   cout << a << " " << b << "\n";  // 26 20
   cout << std::hex;
   cout << a << " " << b << "\n";  // 1a 14
   cout << std::oct;
   cout << a << " " << b << "\n";  // 32 24
   cout << std::dec;
   cout << a << " " << b << "\n";  // 26 20
   return 0;
}
```

**Explanation:**std::hex: Prints numbers in hexadecimal format (base 16).

std::oct: Prints numbers in octal format (base 8).

std::dec: Reverts back to the default decimal format (base 10).

**3. Showing Base Prefix and Uppercase Hexadecimal** In certain cases, it is useful to display numbers with a prefix indicating their base (e.g., 0x for hexadecimal). Additionally, you may want to display hexadecimal letters in uppercase.

```
#include <iostream>

using namespace std;
int main()
{
   int a = 26;
   cout << std::showbase;
   cout << std::oct;
   cout << a << "\n";  // 032
   cout << std::hex;
   cout << a << "\n";  // 0x1a
   cout << std::showbase;
   cout << a << "\n";  // +0x1a
   cout << std::uppercase;
   cout << a << "\n";  // +0X1A
   return 0;
}
```

**Explanation: std::showbase:** Displays the base prefix (0x for hexadecimal, 0 for octal).

**std::uppercase:** Converts letters in hexadecimal numbers to uppercase (e.g., a becomes A).

**4. Width, Fill, and Alignment** C++ provides manipulators for controlling the width of the output and for filling empty spaces. You can set the fill character and control the alignment of the printed data.

```
#include <iostream>
#include <iomanip>
using namespace std;

int main()
```

```
{
   int a = 12;
   cout << std::setw(5);          // Set width to 5
   cout << std::setfill('*');     // Set fill character to '*'
   cout << a << "\n";             // ***12
   cout << std::setw(5);
   cout << "Hi" << "\n";          // ***Hi
   cout << std::left;             // Left-align the output
   cout << std::setw(5);
   cout << a << "\n";             // 12***
   return 0;
}
```

**Explanation: std::setw(n):** Sets the width of the output to n characters.

**std::setfill(c):** Specifies the character to be used for padding if the data is smaller than the set width. In this case, it is *.

**std::left:** Aligns the output to the left within the given width. Other alignment options include **std::right** (default) and **std::center** (available in some implementations).

Floating Point Default Print Format Floating-point numbers are used to represent real values with fractional parts in programming. By default, their printed format may produce unexpected precision and rounding results. This article explains the default floating-point output format and its practical implications.

**1. No Trailing Zeros:** By default, floating-point numbers are printed without unnecessary trailing zeros after the decimal point. This behavior ensures a more compact and clean representation. This improves readability but may require manual formatting when consistent, fixed-width output is needed.

```
#include <iostream>
using namespace std;
int main() {
   double a = 3.140000;
   double b = 5.000000;
   cout << a << endl;
   cout << b << endl;
   return 0;
}
```

**Output** 3.14
5

**Explanation:** The output is 3.14 and 5 because C++ omits unnecessary trailing zeros when printing floating-point numbers by default.

**2. Precision Refers to Total Digits:** Precision in floating-point printing defines the **total number of significant digits**, not just the digits after the decimal point. This behavior emphasizes the value's overall accuracy rather than merely focusing on its decimal representation.

```
#include <iostream>
using namespace std;

int main() {
   double a = 123.456789;
```

```
    double b = 0.000123456;
    cout << a << endl;
    cout << b << endl;
    return 0;
}
```

**Output** 123.457
0.000123456

**Explanation:** C++ uses a default precision of 6 significant digits, so 123.456789 is rounded to 123.457 while 0.000123456 is printed fully until six meaningful digits are reached.

**3. Default Precision Value is 6:**When no specific precision is defined, the default is set to **6 significant digits**. This value strikes a balance between readability and precision, ensuring numbers are neither overly truncated nor too detailed. Developers can change the default precision by using formatting functions like setprecision() in C++, which control how many significant digits are displayed in the output.

```
#include <iostream>
using namespace std;

int main() {
    double a = 1.23456789;
    double b = 123456789.0;
    cout << a << endl;
    cout << b << endl;
    return 0;
}
```

**Output**
1.23457
1.23457e+08

**Explanation:** By default, C++ prints floating-point numbers with a precision of 6 significant digits, so the values are rounded and large numbers are shown in scientific notation (discussed further below).

**4. Switching to Power Format:** When the number of digits in the integral part of a floating-point value exceeds the default stream precision (6 significant digits), the output stream automatically switches to scientific notation to preserve significant digits while minimizing output width. This representation expresses the value in normalized exponential form, ensuring numerical clarity and preventing loss of precision. If scientific notation is not desired, stream format flags (such as fixed, setprecision, or scientific) can be explicitly configured to control the output format.

```
#include <iostream>
#include <iomanip>
using namespace std;

int main() {
    double a = 1234567.89;
    double b = 0.000123456;
    cout << a << endl;
    cout << b << endl;
    return 0;
}
```

**Output**1.23457e+06
0.000123456

**Explanation:** When a number exceeds the default precision of 6 significant digits, it is automatically displayed in scientific (power) notation to keep the output compact and precise.

**Floating point**:- Manipulating Default Format Floating-point numbers are printed in C++ using a default output format that controls precision, notation, and rounding. However, this default behavior may not always meet formatting requirements. This article explains the standard stream manipulators used to customize floating-point output format and their practical applications.

**1. Changing Precision with setprecision(n):**setprecision(n) function lets you control the number of significant digits displayed when printing floating-point numbers. By default, the precision is set to 6 digits, but this can be increased or decreased as needed.

```
#include <bits/stdc++.h>
using namespace std;
int main() {
   double x = 123.456789;
   cout << setprecision(4) << x << endl;
   cout << setprecision(7) << x << endl;
   return 0;
}
```

**Output** 123.5
123.4568

**Explanation**: setprecision(4) and setprecision(7) allow 4 and 7 significant digits to be printed respectively.

**2. Showing or Hiding Trailing Zeros with showpoint():** By default, trailing zeros after the decimal point are omitted to keep the output compact. The showpoint() function forces the display of trailing zeros, while noshowpoint() reverts back to the default behavior.

```
#include <bits/stdc++.h>
using namespace std;
int main() {
   double x = 50.0;
   cout << x << endl;
   cout << showpoint << x << endl;
   cout << noshowpoint << x << endl;
   return 0;
}
```

**Output**50
50.0000
50

**Explanation**: showpoint(): forces the display of trailing zeros after the decimal point, so the output becomes 50.0000, whereas noshowpoint(): hides the trailing zeros and restores the default compact format, so the output becomes 50.

***Note**: In the showpoint() output, only 4 zeros appear after the decimal because the default precision is 6 significant digits, so 50 already uses two digits and the remaining four are shown as zeros.*

**3. Displaying Positive Signs with showpos():** By default, only negative numbers are shown with a sign (-). The showpos() function explicitly displays the + sign for positive numbers, while noshowpos() disables it.

```
#include <bits/stdc++.h>
using namespace std;
   int main() {
   int a = 10;
   int b = -10;
   cout << a << " " << b << endl;
   cout << showpos << a << " " << b << endl;
   cout << noshowpos << a << " " << b << endl;
   return 0;
}
```

**Output** 10 -10

```
+10 -10
10 -10
```

**Explanation**: showpos forces the + sign to appear before positive numbers (+10), while noshowpos removes it and restores the normal output.

**4. Changing Exponential Notation Case with uppercase():** When using scientific notation, the e in the output is lowercase by default. The uppercase() function converts it to uppercase (E), while nouppercase() reverts this change.

```
#include <bits/stdc++.h>
using namespace std;
int main() {
   double x = 12345.678;
   cout << scientific << x << endl;
   cout << uppercase << x << endl;
   cout << nouppercase << x << endl;
   return 0;
}
```

**Output**1.234568e+04

```
1.234568E+04
1.234568e+04
```

**Explanation**: scientific prints the number in scientific notation, uppercase changes 'e' to 'E' , and nouppercase changes it back to lowercase 'e'.

Floating Point Fixed and Scientific When working with floating-point numbers in C++, the output format depends on the active formatting rules. By **default**, floating-point values are printed with 6 significant digits. If this limit is exceeded, the value is automatically displayed in scientific (e-notation), and unnecessary trailing zeros are omitted. **For Example**:

```
#include <bits/stdc++.h>
using namespace std;

int main() {
   double a = 123.456789;
   double b = 123456789.0;
```

```
    double c = 5.000000;
    cout << a << endl;
    cout << b << endl;
    cout << c << endl;
    return 0;
}
```

**Output** 123.457

```
1.23457e+08
5
```

**Explanation:** The output shows default precision of 6 significant digits, automatic scientific notation for large numbers, and removal of trailing zeros for compact formatting. **C++ provides two primary formatting modes for floating-**point output - **fixed** and **scientific,** each offering a different way to represent numeric values based on precision and use case.

**1. Fixed Format:-**In fixed format, precision specifies the number of digits after the decimal point. The output always shows trailing zeros to satisfy the given precision, and scientific (e) notation is never used, no matter how large the number is. Values are rounded to the specified decimal places, providing a consistent decimal layout.

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    double x = 1.23;
    double y = 1122456.453;
    cout << fixed << setprecision(6);
    cout << x << endl;
    cout << y << endl;
    cout << fixed << setprecision(2);
    cout << x << endl;
    cout << y << endl;
    return 0;
}
```

**Output**1.230000

```
1122456.453000
1.23
1122456.45
```

**Explanation:** setprecision(6) prints numbers with 6 digits after the decimal point, while setprecision(2) prints numbers with 2 digits after the decimal point.

**2. Scientific Formatln** scientific format, numbers are always displayed using e-notation, where the value is written with exactly one digit before the decimal point followed by the remaining digits and an exponent indicating the power of ten. The precision specifies how many digits are printed after the decimal point, and trailing zeros are automatically added if necessary to satisfy the selected precision.

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    double x = 1.23;
double y = 1122456.453;
    cout << scientific << setprecision(6) << x << endl;
    cout << scientific << setprecision(6) << y << endl;
    cout << scientific << setprecision(2) << x << endl;
    cout << scientific << setprecision(2) << y << endl;
    return 0;
```

}
**Output**1.230000e+00

```
1.122456e+06
1.23e+00
1.12e+06
```

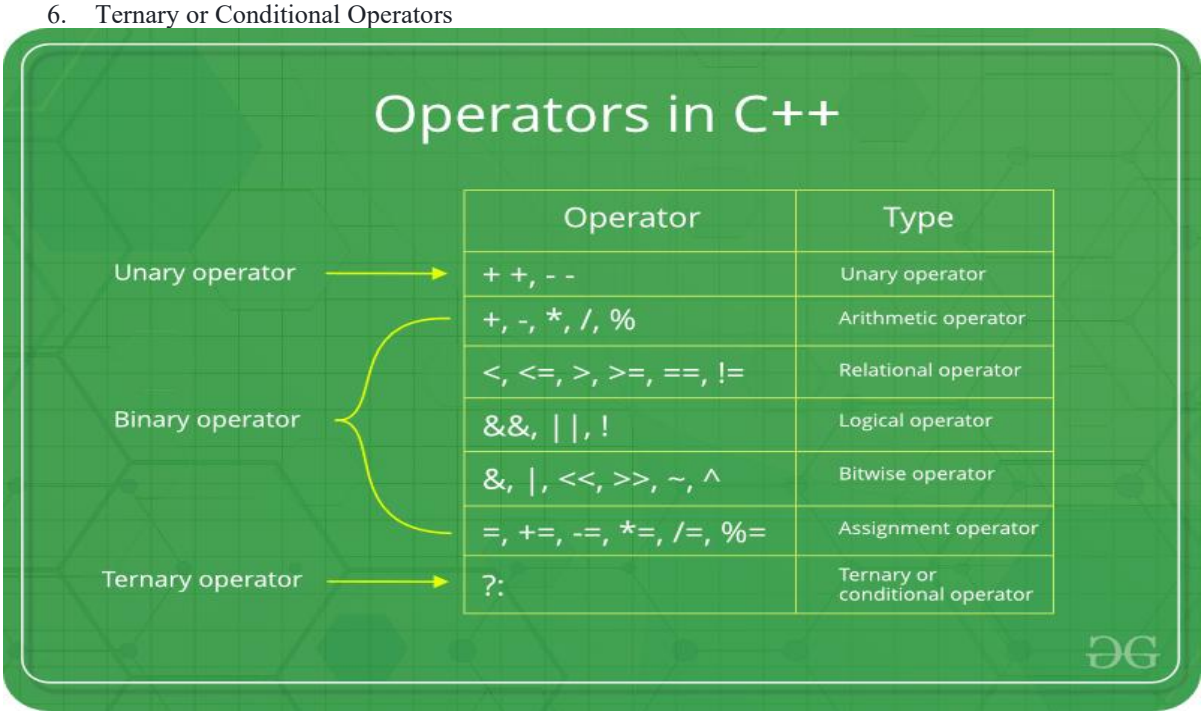
**Explanation:** setprecision(6) prints both x and y in scientific notation with 6 digits after the decimal point, while setprecision(2) prints both values in scientific notation with 2 digits after the decimal point.

**Arithmetic Operators :-**An **operator** is a symbol that operates on a value to perform specific mathematical or logical computations. They form the foundation of any programming language. In C++, we have built-in operators to provide the required functionality.

An operator operates the **operands**. For example, int c = a + b;Here, '+' is the addition operator. 'a' and 'b' are the operands that are being 'added'.

**Operators in C++ can be classified into 6 types:**

1. Arithmetic Operators
2. Relational Operators
3. Logical Operators
4. Bitwise Operators
5. Assignment Operators
6. Ternary or Conditional Operators




**1.Arithmetic Operators:-**These operators are used to perform arithmetic or mathematical operations on the operands. For example, '+' is used for addition, '-' is used for subtraction '*' is used for multiplication, etc.

**Arithmetic Operators can be classified into 2 Types: A) Unary Operators:** These operators operate or work with a single operand. For example: Increment(++) and Decrement(–) Operators.

| Name | Symbol | Description | Example |
|---|---|---|---|
| Increment Operator | ++ | Increases the integer value of the variable by one | int a = 5;<br><br>a++; // returns 6 |
| Decrement Operator | - - | Decreases the integer value of the variable by one | int a = 5;<br><br>a- -; // returns 4 |

```
// CPP Program to demonstrate the increment // and decrement operators

#include <iostream>
using namespace std;
int main()
{
   int a = 10;
   cout << "a++ is " << a++ << endl;
   cout << "++a is " << ++a << endl;
   int b = 15;
   cout << "b-- is " << b-- << endl;
   cout << "--b is " << --b << endl;
   return 0;
}
```

**Output**

```
a++ is 10
++a is 12
b-- is 15
--b is 13
```

***Note:*** *++**a** and **a**++, both are increment operators, however, both are slightly different.In ++**a**, the value of the variable is incremented first and then It is used in the program. In **a**++, the value of the variable is assigned first and then It is incremented. Similarly happens for the decrement operator.*

**B) Binary Operators:** These operators operate or work with two operands. For example: Addition(+), Subtraction(-), etc.

| Name | Symbol | Description | Example |
|---|---|---|---|
| Addition | + | Adds two operands | int a = 3, b = 6;<br><br>int c = a+b; // c = 9 |
| Subtraction | – | Subtracts second operand from the first | int a = 9, b = 6;<br><br>int c = a-b; // c = 3 |
| Multiplication | * | Multiplies two operands | int a = 3, b = 6; |

| Name | Symbol | Description | Example |
|---|---|---|---|
| | | | int c = a*b; // c = 18 |
| Division | / | Divides first operand by the second operand | int a = 12, b = 6;<br>int c = a/b; // c = 2 |
| Modulo Operation | % | Returns the remainder an integer division | int a = 8, b = 6;<br>int c = a%b; // c = 2 |

***Note:*** *The Modulo operator(%) operator should only be used with integers.*

```
// CPP Program to demonstrate the Binary Operators
#include <iostream>
using namespace std;
int main()
{
   int a = 8, b = 3;
   // Addition operator
   cout << "a + b = " << (a + b) << endl;
   // Subtraction operator
   cout << "a - b = " << (a - b) << endl;
   // Multiplication operator
   cout << "a * b = " << (a * b) << endl;
   // Division operator
   cout << "a / b = " << (a / b) << endl;
   // Modulo operator
   cout << "a % b = " << (a % b) << endl;
   return 0;
}
```

**Output**

```
a + b = 11
a - b = 5
a * b = 24
a / b = 2
a % b = 2
```

**2.Logical Operators:-**These operators are used to combine two or more conditions or constraints or to complement the evaluation of the original condition in consideration. The result returns a Boolean value, i.e., **true** or **false**.

| Name | Symbol | Description | Example |
|---|---|---|---|
| Logical AND | && | Returns true only if all the operands are true or non-zero | int a = 3, b = 6;<br>a&&b;// returns true |

| Name | Symbol | Description | Example |
|---|---|---|---|
| Logical OR | \|\| | Returns true if either of the operands is true or non-zero | int a = 3, b = 6;<br>a\|\|b;// returns true |
| Logical NOT | ! | Returns true if the operand is false or zero | int a = 3;<br>!a; // returns false |

```
// CPP Program to demonstrate the Logical Operators
#include <iostream>
using namespace std;
int main()
{
   int a = 6, b = 4;
   // Logical AND operator
   cout << "a && b is " << (a && b) << endl;
   // Logical OR operator
   cout << "a || b is " << (a || b) << endl;
   // Logical NOT operator
   cout << "!b is " << (!b) << endl;
   return 0;
}
```

**Output**

a && b is 1

a ! b is 1

!b is 0 Here, **0** denotes **false** and **1** denotes **true**

**Short-Circuiting in Logical Operators:**

- In the case of **logical AND**, the second operand is not evaluated if the first operand is false. For example, program 1 below doesn't print "GeeksQuiz" as the first operand of logical AND itself is false.

```
#include <iostream>
using namespace std;
int main()
{
   int a = 10, b = 4;
   bool res = ((a == b) && cout << "GeeksQuiz");
   return 0;
}
```

- But the below program prints "GeeksQuiz" as the first operand of logical AND is true.

```
#include <iostream>
using namespace std;

int main()
{
   int a = 10, b = 4;
   bool res = ((a != b) && cout << "GeeksQuiz");
   return 0;
}
```

**Output** GeeksQuiz

- In the case of **logical OR**, the second operand is not evaluated if the first operand is true. For example, program 1 below doesn't print "GeeksQuiz" as the first operand of logical OR itself is true.

```
#include <iostream>
using namespace std;
int main()
{
   int a = 10, b = 4;
   bool res = ((a != b) || cout << "GeeksQuiz");
   return 0;
}
```

- But the below program prints "GeeksQuiz" as the first operand of logical OR is false.

```
#include <iostream>
using namespace std;
int main()
{
   int a = 10, b = 4;
   bool res = ((a == b) || cout << "GeeksQuiz");
   return 0;
}
```

**Output** GeeksQuiz

**3.Assignment Operators**: Assignment operators are used to assigning value to a variable. The left side operand of the assignment operator is a variable and right side operand of the assignment operator is a value. The value on the right side must be of the same data-type of the variable on the left side otherwise the compiler will raise an error.

Different types of assignment operators are shown below:

- **"="**: This is the simplest assignment operator. This operator is used to assign the value on the right to the variable on the left.
  For example:

```
a = 10;
b = 20;
ch = 'y';
```

- **"+="**: This operator is combination of '+' and '=' operators. This operator first adds the current value of the variable on left to the value on the right and then assigns the result to the variable on the left.
  Example:

```
(a += b) can be written as (a = a + b)
```

If initially value stored in a is 5. Then (a += 6) = 11.

- **"-="**This operator is combination of '-' and '=' operators. This operator first subtracts the current value of the variable on left from the value on the right and then assigns the result to the variable on the left.
  Example:

```
(a -= b) can be written as (a = a - b)
```

If initially value stored in a is 8. Then (a -= 6) = 2.

- **"*="**This operator is combination of '*' and '=' operators. This operator first multiplies the current value of the variable on left to the value on the right and then assigns the result to the variable on the left.
  Example:

```
(a *= b) can be written as (a = a * b)
```

If initially value stored in a is 5. Then (a *= 6) = 30.

- **"/="**This operator is combination of '/' and '=' operators. This operator first divides the current value of the variable on left by the value on the right and then assigns the result to the variable on the left.
  Example:

(a /= b) can be written as (a = a / b)

If initially value stored in a is 6. Then (a /= 2) = 3.

Below example illustrates the various Assignment Operators:

```
// C++ program to demonstrate
// working of Assignment operators

#include <iostream>
using namespace std;
int main()
{
   // Assigning value 10 to a
   // using "=" operator
   int a = 10;
   cout << "Value of a is "<<a<<"\n";
   // Assigning value by adding 10 to a
   // using "+=" operator
   a += 10;
   cout << "Value of a is "<<a<<"\n";
   // Assigning value by subtracting 10 from a
   // using "-=" operator
   a -= 10;
   cout << "Value of a is "<<a<<"\n";
   // Assigning value by multiplying 10 to a
   // using "*=" operator
   a *= 10;
   cout << "Value of a is "<<a<<"\n";
   // Assigning value by dividing 10 from a
   // using "/=" operator
   a /= 10;
   cout << "Value of a is "<<a<<"\n";
      return 0;
}
```

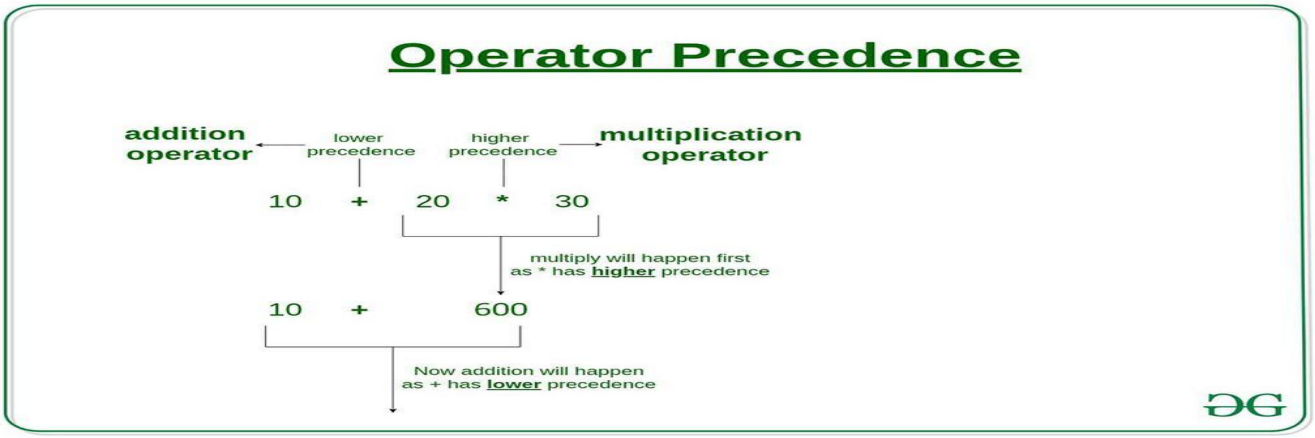
**Output**

```
Value of a is 10
Value of a is 20
Value of a is 10
Value of a is 100
Value of a is 10
```

**Operator Precedence and Associativity** Operator Precedence determines which operation is performed first in an expression with more than one operators with different precedence.

**For example:** Solve

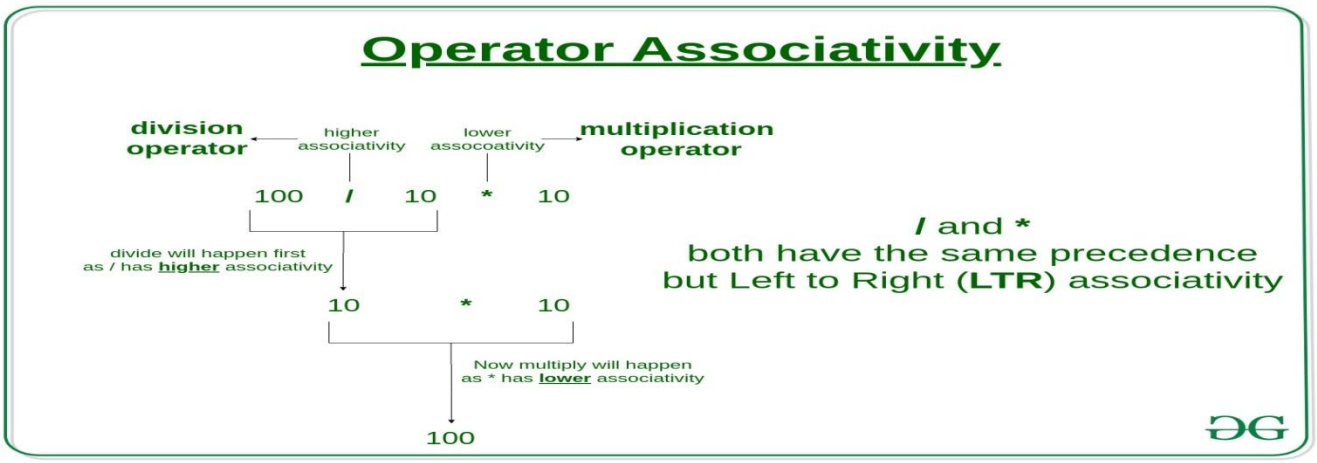
10 + 20 * 30




**10 + 20 * 30 is calculated as 10 + (20 * 30)**
and not as **(10 + 20) * 30**

**Operators Associativity** is used when two operators of same precedence appear in an expression. Associativity can be either Left to Right or Right to Left.

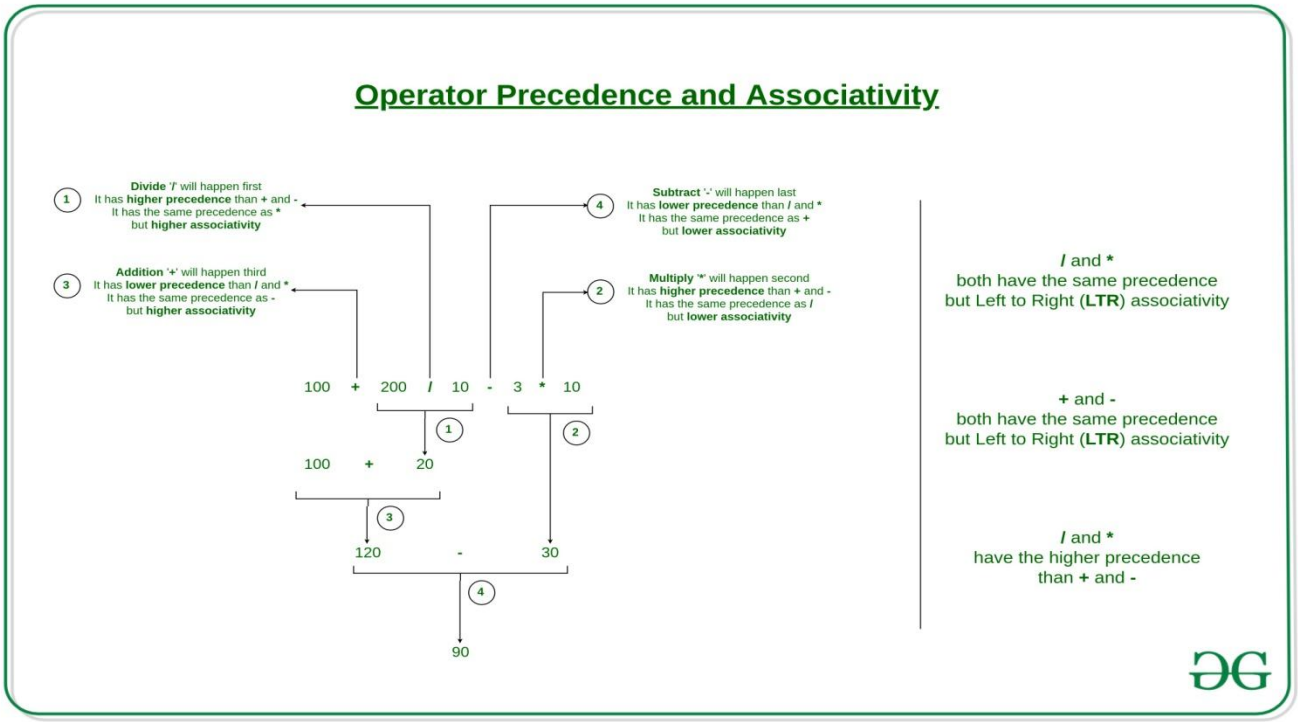
**For example:** '*' and '/' have same precedence and their associativity is Left to Right, so the expression "100 / 10 * 10" is treated as "(100 / 10) * 10".




***Operators Precedence and Associativity are two characteristics of operators that determine the evaluation order of sub-expressions in absence of brackets.***

**For example:** Solve

100 + 200 / 10 - 3 * 10




**1) Associativity is only used when there are two or more operators of same precedence.**

**2) All operators with the same precedence have same associativity**

**3) Precedence and associativity of postfix ++ and prefix ++ are different**

**4) Comma has the least precedence among all operators and should be used carefully**

**5) There is no chaining of comparison operators**

**Example 1: Operators Precedence**

```
#include <iostream>
using namespace std;
int main() {
  // evaluates 17 * 6 first
  int num1 = 5 - 17 * 6;
  // equivalent expression to num1
  int num2 = 5 - (17 * 6);
  // forcing compiler to evaluate 5 - 17 first
  int num3 = (5 - 17) * 6;
  cout << "num1 = " << num1 << endl;
  cout << "num2 = " << num2 << endl;
  cout << "num3 = " << num3 << endl;
  return 0;
}
```

**Output**

```
num1 = -97
num2 = -97
num3 = -72
```

**Example 2: Operators Associativity**

```
#include <iostream>

using namespace std;

  int main() {
 int a = 1;
 int b = 4;
  // a -= 6 is evaluated first
 b += a -= 6;
   cout << "a = " << a << endl; ;
 cout << "b = " << b;
}
```

**Output**a = -5
b = -1

| Operator | Description | Associativity |
|---|---|---|
| ( )<br>[ ]<br>.<br>->  <br>++ – – | **Parentheses (function call) (see Note 1)**<br>**Brackets (array subscript)**<br>**Member selection via object name**<br>**Member selection via pointer**<br>**Postfix increment/decrement (see Note 2)** | **left-to-right** |
| ++ – –<br>+ –<br>! ~<br>(*type*)<br>*<br>&<br>sizeof | **Prefix increment/decrement**<br>**Unary plus/minus**<br>**Logical negation/bitwise complement**<br>**Cast (convert value to temporary value of *type*)**<br>**Dereference**<br>**Address (of operand)**<br>**Determine size in bytes on this implementation** | **right-to-left** |
| * / % | **Multiplication/division/modulus** | **left-to-right** |
| + – | **Addition/subtraction** | **left-to-right** |
| << >> | **Bitwise shift left, Bitwise shift right** | **left-to-right** |
| < <=<br>> >= | **Relational less than/less than or equal to**<br>**Relational greater than/greater than or equal to** | **left-to-right** |
| == != | **Relational is equal to/is not equal to** | **left-to-right** |
| & | **Bitwise AND** | **left-to-right** |
| ^ | **Bitwise exclusive OR** | **left-to-right** |
| \| | **Bitwise inclusive OR** | **left-to-right** |
| && | **Logical AND** | **left-to-right** |

| | | |
|---|---|---|
| **\|\|** | **Logical OR** | **left-to-right** |
| **? :** | **Ternary conditional** | **right-to-left** |
| **=<br>+= -=<br>*= /=<br>%= &=<br>^= \|=<br><<= >>=** | **Assignment<br>Addition/subtraction assignment<br>Multiplication/division assignment<br>Modulus/bitwise AND assignment<br>Bitwise exclusive/inclusive OR assignment<br>Bitwise shift left/right assignment** | **right-to-left** |
| **,** | **Comma (separate expressions)** | **left-to-right** |

It is good to know precedence and associativity rules, but the best thing is to use brackets, especially for less commonly used operators (operators other than +, -, *.. etc). Brackets increase the readability of the code as the reader doesn't have to see the table to find out the order.

**Bitwise Operators:-**These operators are used to perform bit-level operations on the operands. The operators are first converted to bit-level and then the calculation is performed on the operands. The mathematical operations such as addition, subtraction, multiplication, etc. can be performed at the bit-level for faster processing.

| Name | Symbol | Description | Example |
|---|---|---|---|
| Binary AND | & | The bitwise AND operator performs a bit-by-bit comparison of two numbers. It returns 1 for each bit position where both corresponding bits in the operands are 1; otherwise, it returns 0. | int a = 2, b = 3;(a &b); //returns 2 |
| Binary OR | \| | The bitwise OR operator performs a bit-by-bit comparison of two numbers. It returns 1 for each bit position where at least one of the corresponding bits in the operands is 1. | int a = 2, b = 3;(a \| b); //returns 3 |
| Binary XOR | ^ | The bitwise XOR operator performs a bit-by-bit comparison of two numbers. It returns 1 for each bit position where the corresponding bits in the operands are different (i.e., one is 1 and the other is 0); otherwise, it returns 0. | int a = 2, b = 3;(a ^ b); //returns 1 |
| Left Shift | << | The left shift operator (<<) shifts the bits of its left-hand operand to the left by the number of positions specified by its right-hand operand. For each left shift, the most significant bits (leftmost) are discarded, and 0s are inserted into the least significant bits (rightmost). | int a = 2, b = 3;(a << 1); //returns 4 |
| Right Shift | >> | The right shift operator (>>) shifts the bits of its left-hand operand to the right by the number of positions specified by its right-hand operand. For each right shift, the least significant bits (rightmost) are discarded, and the most significant bits (leftmost) are filled with 0 for unsigned numbers or the sign bit for signed numbers | int a = 2, b = 3;(a >> 1); //returns 1 |
| Bitwise NOT | ~ | The bitwise NOT operator (~) inverts the bits of its operand. For every bit in the number, 0 becomes 1, and 1 becomes 0. It essentially flips all bits of the operand. | int b = 3;<br><br>(~b); //returns -4 |

***Note:*** *Only* ***char*** *and* ***int*** *data types can be used with Bitwise Operators.*

**Example 1: Bitwise AND (&)**

```
#include <iostream>
using namespace std;
   int main() {
   int x = 3;  // Binary: 0011
   int y = 6;  // Binary: 0110
    cout << (x & y); // Perform bitwise AND
   return 0;
}
```

**Explanation**:

- 3 in binary: 0011
- 6 in binary: 0110
- AND: 0010
  **Output**: 2

**Example 2: Bitwise OR (|)**

```
#include <iostream>
using namespace std;
   int main() {
   int x = 3;  // Binary: 0011
   int y = 6;  // Binary: 0110
    cout << (x | y); // Perform bitwise OR
   return 0;
}
```

**Explanation**:

- 3 in binary: 0011
- 6 in binary: 0110
- OR: 0111
  **Output**: 7

**Example 3: Bitwise XOR (^)**

```
#include <iostream>
using namespace std;
   int main() {
   int x = 3;  // Binary: 0011
   int y = 6;  // Binary: 0110
    cout << (x ^ y); // Perform bitwise XOR
   return 0;
}
```

**Explanation**:

- 3 in binary: 0011
- 6 in binary: 0110
- XOR: 0101
  **Output**: 5

**Example 4 : Left Shift Operator (<<)**

```
#include <iostream>
using namespace std;

int main() {
   int x = 3;  // Binary: 0000...0011
   cout << (x << 1) << endl;  // Shift left by 1
   cout << (x << 2) << endl;  // Shift left by 2
   int y = 4;  // Binary: 0000...0100
   int z = (x << y);          // Shift left by 4
   cout << z;                 // Output the result
   return 0;
}
```

**Explanation of the Code**

1. **Variable Initialization**:
   - x = 3: Binary representation: 0000...0011.
   - y = 4: Binary representation: 0000...0100.
2. **Shift Left by 1 (x << 1)**:
   - Shifting 3 (0000...0011) left by 1 position: 0000...0110.
   - Result = 6.
3. **Shift Left by 2 (x << 2)**:
   - Shifting 3 (0000...0011) left by 2 positions: 0000...1100.
   - Result = 12.
4. **Shift Left by y (x << y)**:
   - Shifting 3 (0000...0011) left by 4 positions: 0000...110000.
   - Result = 48.
5. **Output**:
   - The program outputs:
     - 6 for x << 1.
     - 12 for x << 2.
     - 48 for x << y

**Example 5 : Right Shift Operator (>>)**

```
#include <iostream>
using namespace std;
   int main() {
   int x = 33;  // Binary: 0000...100001
   cout << (x >> 1) << endl;  // Shift right by 1
   cout << (x >> 2) << endl;  // Shift right by 2
     int y = 4;  // Binary: 0000...0100
   int z = (x >> y);          // Shift right by 4
   cout << z;                 // Output the result
     return 0;
}
```

**Explanation of the Code**

1. **Variable Initialization**:
   - x = 33: Binary representation: 0000...100001.
   - y = 4: Binary representation: 0000...0100.
2. **Right Shift Operations**:
   - x >> 1: Shifts x (33) right by 1 bit (0000...100001 → 0000...010000), result = **16**.
   - x >> 2: Shifts x (33) right by 2 bits (0000...100001 → 0000...001000), result = **8**.
   - x >> y: Shifts x (33) right by 4 bits (0000...100001 → 0000...000010), result = **2**.
3. **Output**:
   - First cout outputs **16**.

- Second cout outputs **8**.
- Third cout outputs **2**.

**Interesting Facts About Left Shift and Right Shift Operators**

***Left Shift Operator (<<)***

1. The left shift operator is equivalent to multiplying a number by $2^y$, where y is the number of positions shifted.
2. Appending a 0 to the trailing bits during a left shift is what makes the operation equivalent to multiplication by 2.
3. It assumes that the leading bits of the number are 0. If the leading bits are 1 (e.g., in very large numbers), the result may not correctly represent $2^y$ multiplication.
4. Using left shift for multiplication is efficient and fits well within the constraints of 32-bit integers. Traditional multiplication may not work correctly with very large numbers due to overflow.

***Right Shift Operator (>>)***

1. The right shift operator is equivalent to dividing a number by $2^y$ and taking the floor of the result (integer division).
2. It discards the least significant bits (trailing bits) during the shift.
3. For example:
   - 33 >> 1 is equivalent to $33/2^1$=16.5, and the floor is **16**.
   - 33 >> 2 is equivalent to $33/2^2$=8.25, and the floor is **8**.
4. The right shift is useful for efficiently performing division by powers of 2 without introducing decimal points or inaccuracies.

**Example 6 : Bitwise NOT Operator (~)**

```
#include <iostream>
using namespace std;
  int main() {
  unsigned int x = 1;  // Binary: 0000...0001
  cout << (~x) << endl; // Perform bitwise NOT on x
  x = 5;  // Binary: 0000...0101
  cout << (~x) << endl; // Perform bitwise NOT on x
  return 0;
}
```

**Explanation of the Code**

1. **Variable Initialization**:
   - x = 1: Binary representation: 0000...0001.
   - x = 5: Binary representation: 0000...0101.
2. **Bitwise NOT Operation**:
   - For x = 1:
     - ~x: Inverts 0000...0001 to 1111...1110.
     - Decimal value: 232−1−1=42949672942^{32} - 1 - 1 = 4294967294232−1−1=4294967294 (for 32-bit unsigned integer).
   - For x = 5:
     - ~x: Inverts 0000...0101 to 1111...1010.
     - Decimal value: 232−1−5=42949672902^{32} - 1 - 5 = 4294967290232−1−5=4294967290.
3. **Output**:
   - First cout outputs 4294967294.
   - Second cout outputs 4294967290.

**Last Digit of a Number:-**This article explains how to find the last digit of a number, including both positive and negative integers. The solution uses the modulus operator %

*to isolate the last digit and ensures the result is always positive, even for negative numbers.*

**Problem Statement**

Given an integer, the task is to find its last digit. For instance:

- For 123, the last digit is 3.
- For −352, the last digit is 2.

**Observations and Approach**

***Key Observations:***

1. When dividing a number by 10, the remainder gives the last digit. For example, 123%10=3, which is the last digit.
2. If the number is negative, using the modulus operator directly will yield a negative last digit. For example, abs(−235%10) = 5.

***Solution Steps:***

1. Calculate the last digit using the modulus operator % 10.
2. If the number is negative, use the abs() function to convert the last digit to a positive value.

**Code Implementation**

```
#include <bits/stdc++.h>
using namespace std;
   int main() {
   int x = -235;  // Input number
   int ans = abs(x % 10);  // Last digit using modulus and convert to positive if needed
   cout << ans;  // Output the last digit
   return 0;
}
```

**Output:***5*

**Comparison Operator** :-Comparison operators are used for comparing two elements. They are commonly used in conditional statements like if-else because they return a boolean value: true (1) if the condition is satisfied, or false (0) if it is not.

**There are mainly 6 Comparison Operators namely:**

**1. Greater than (>) :** Checks if the left operand is greater than the right operand.

**Example:** 5 > 3 // Returns true

**2. Greater than or equal to (>=)**: Checks if the left operand is greater than or equal to the right operand.

**Example:** 5 >= 5 // Returns true

**3. Less than (<)**: Checks if the left operand is less than the right operand.

**Example:** 3 < 5 // Returns true

**4. Less than or equal to (<=)**: Checks if the left operand is less than or equal to the right operand.

**Example:**5 <= 5 // Returns true

**5. Equal to (==)**: Checks if the left operand is equal to the right operand.

**Example:** 5 == 5 // Returns true

**6. Not equal to (!=)**: Checks if the left operand is not equal to the right operand.

**Example:**5 != 3 // Returns true

**Example Code to cover all 6 Comparison Operators:**

```
#include<iostream>
using namespace std;
int main()
{
   int x = 10, y = 20 ;
   cout << (x<y) << "\n"
       << (x>y) << "\n"
       << (x==y) << "\n"
       << (x>=y) << "\n"
       << (x<=y) << "\n"
       << (x!=y) << "\n";
   return 0;
}
```

**Day before N days**:-n this article, we will explore a problem-solving technique in C++ to calculate the day before a given day. This involves leveraging modular arithmetic and conditional checks. The goal is to determine the previous day based on a numerical representation of the week, where 0 corresponds to Sunday, 1 to Monday, 2 to Tuesday, and so on, up to 6 for Saturday.

**Problem Description** You are provided with:

- d: A numerical representation of a current day.
- n: The number of days to go back.

Your task is to calculate the day before N days. For example:

- If d = 1 (Monday) and n = 1, the output is 0 (Sunday).
- If d = 0 (Sunday) and n = 9, the output is 5 (Friday).

**Solution Steps :-**To solve this problem, we use the following steps:

1. **Reduce N to a Cyclic Range (0-6):** Compute N % 7. This ensures N is within a single week's range since the week repeats cyclically every 7 days.
2. **Subtract N from D:** Compute D - (N % 7) to find the preliminary day before N days.
3. **Handle Negative Results:** If the result of the subtraction is negative, add 7 to bring it back within the valid range of 0-6.
4. **Output the Result:** Return the calculated day.

**Code Implementation**

```
#include <bits/stdc++.h>
using namespace std;
   int main() {
   int d = 0;  // The current day (0 = Sunday)
   int n = 9;  // Days to go back
   // Step 1: Calculate the cyclic range of N
   int x = n % 7;
   // Step 2: Subtract the value from D
   int ans = d - x;
   // Step 3: Handle negative results
   if (ans >= 0)
      cout << ans;
   else
  cout << ans + 7;
   return 0;
}
```

**Output:** 5

**Explanation:**

- d=0,n=9
- Calculate n%7:9%7=2.
- Subtract x from d: 0−2=−2
- Check if the result is negative: −2 is negative, so add 7: −2+7=5.
- The output is 5, which corresponds to Friday.

**AP Term** :- In this article, we will explore how to find the Nth term of an **Arithmetic Progression (AP)** series. Arithmetic Progression is a sequence of numbers in which the difference between any two consecutive terms is constant. This difference is referred to as the **common difference** (denoted as D).

**Understanding Arithmetic Progression :-An** Arithmetic Progression (AP) **follows this rule:** $TN = A+(N-1)\times D$

- $TN$ : The Nth term to be calculated.
- A: The first term of the series.
- D: The common difference between consecutive terms.
- N: The term number to find.

*Example: Consider the AP: 2,5,8,11,14.*

- The first term (A) = 2.
- The common difference (D) = 3.

Using the formula:

- The 3rd term ($T3$) = 2+(3-1) x 3=2+6=8.
- The 5th term ($T5$) = 2+(5-1) x 3 = 2+12=14.

**Code Implementation**

```
#include <bits/stdc++.h>
using namespace std;
int main() {
   // Starting number
   int a = 2;
   // Common difference
   int d = 1;
   // Nth term to find
```

```
    int N = 5;
    // Calculate the Nth term
    int ans = (a + (N - 1) * d);
     cout << ans;
    return 0;
}
```

**Output:***6*

**Geometric Progression :-**In this article, we will explore how to find the Nth term of a Geometric Progression (GP) using C++. A geometric progression is a sequence where each term is obtained by multiplying the previous term by a fixed, non-zero number called the common ratio.

Understanding Geometric Progression: **In a** Geometric Progression**, the Nth term can be calculated using the formula:** $TN = A \times R(N-1)$ **Where:**

*TN* : The Nth term to be calculated.

A: The first term of the series.

R: The common ratio between consecutive terms.

N: The term number to find.

*Example:-* ***Consider the GP: 2,4,8,16,32***

- The first term (A) = 2.
- The common ratio (R) = 2.

Using the formula:

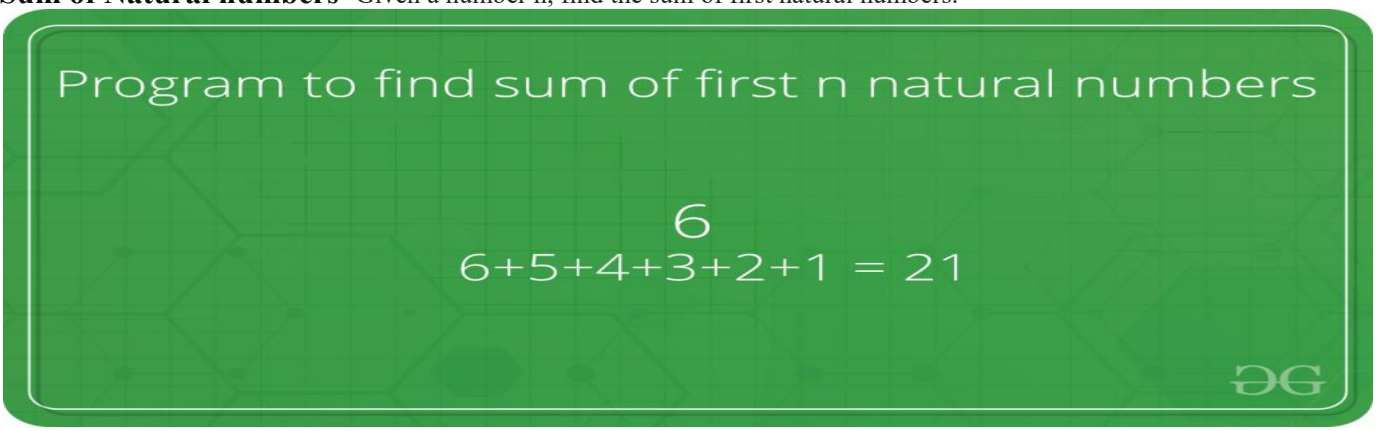
- The 3rd term ($T_3$) = 2 x $2^{(3-1)}$= 2 x 4 = 8
- The 5rd term ($T_5$) = 2 x $2^{(5-1)}$= 2 x 16 = 32

**Code Implementation**

```
#include <bits/stdc++.h>
using namespace std;
// CPP Program to find nth term of geometric progression
int main() {
    // Starting number
    int a = 2;

    // Common ratio
    int r = 3;

    // Nth term to be found
    int N = 3;

    // Calculate the Nth term
    int ans = a * (int)(pow(r, N - 1));

    // Display the output
    cout << ans;

    return 0;
}
```

**Output: 18**

**Sum of Natural numbers** Given a number n, find the sum of first natural numbers.




**Examples :**

```
Input : n = 3
Output : 6
Explanation :
Note that 1 + 2 + 3 = 6

Input  : 5
Output : 15
Explanation :
Note that 1 + 2 + 3 + 4 + 5 = 15
```

A **simple solution** is to do the following.

```
1) Initialize : sum = 0
2) Run a loop from x = 1 to n and
   do following in loop.
    sum = sum + x
```

```
// CPP program to find sum of first
// n natural numbers.
#include <iostream>
using namespace std;

// Driver code
int main()
{
   int n = 5;
   int sum = 0;
   for (int x = 1; x <= n; x++)
      sum = sum + x;
   cout << sum;
   return 0;
}
```

**Output**

```
15
```

***Time Complexity:*** *O(n)*

***Auxiliary Space:*** *O(1)*

An **efficient solution** is to use the below formula.

**Sum of first n natural numbers = (n * (n + 1)) / 2**

**Examples :**
**n = 5**
**Sum = (5 * (5 + 1)) / 2 = (5 * 6) / 2 = 30/2 = 15**

**n = 10**
**Sum = (10 * (10 + 1)) / 2 = (10 * 11) / 2 = 110/2 = 55**

**How does this work?**

**We can prove this formula using induction.**

**It is true for n = 1 and n = 2**
**For n = 1, sum = 1 * (1 + 1)/2 = 1**
**For n = 2, sum = 2 * (2 + 1)/2 = 3**
**Let it be true for k = n-1.**
**Sum of k numbers = (k * (k+1))/2**
**Putting k = n-1, we get**
**Sum of k numbers = ((n-1) * (n-1+1))/2**
**= (n - 1) * n / 2**
**If we add n, we get,**
**Sum of n numbers = n + (n - 1) * n / 2**
**= (2n + $n^{2 - n)/2}$**
**= n * (n + 1)/2**

```
// Efficient CPP program to find sum of first
// n natural numbers.
#include <iostream>
using namespace std;
// Driver code
int main()
{
   int n = 5;
   cout << n * (n + 1) / 2;
   return 0;
}
```

**Outpu t15**

***Time Complexity:*** *O(1)*

***Auxiliary Space:*** *O(1)*

**The above program causes overflow, even if the result is not beyond the integer limit**. We can avoid overflow up to some extent by dividing first.

```
// Efficient CPP program to find sum of first
// n natural numbers that avoids overflow if
// result is going to be within limits.
```

```
#include <iostream>
using namespace std;

// Driver code
int main()
{
   int n = 5;
   long long sum = 0;
   if (n % 2 == 0)
      // Here multiplying by 1LL help to
      // perform calculations in long long,
      // so that answer should not be overflowed
      sum = (n / 2) * 1LL * (n + 1);
   // If n is odd, (n+1) must be even
   else
      // Here multiplying by 1LL help to
      // perform calculations in long long,
      // so that answer should not be overflowed
      sum = ((n + 1) / 2) * 1LL * n;
            cout << sum << endl;
   return 0;
}
```

**Output 15**

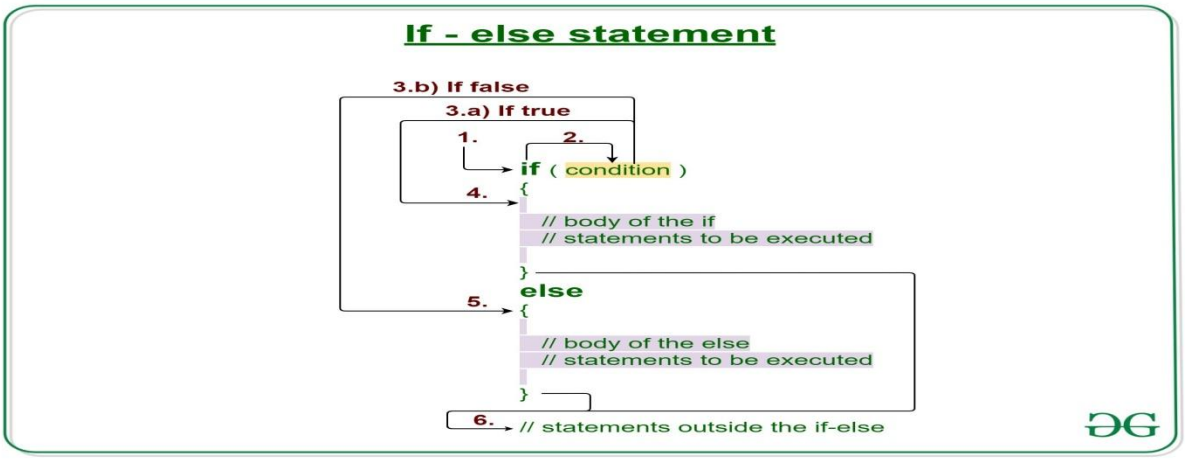
***Time Complexity:*** *O(1)*

***Auxiliary Space:*** *O(1)*

**If Else Syntax in C++:-**Decision Making helps to write decision driven statements and execute a particular set of code based on certain conditions.

The *if* statement alone tells us that if a condition is true it will execute a block of statements and if the condition is false it won't. But what if we want to do something else if the condition is false. Here comes the C/C++ **else statement**. We can use the else statement with if statement to execute a block of code when the condition is

false.



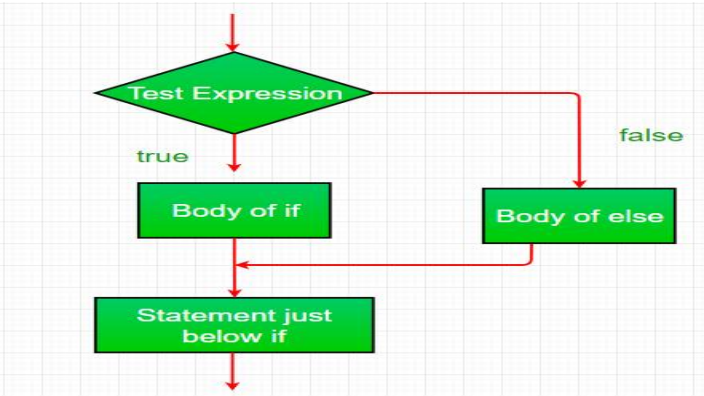

**Syntax:**

```
if (condition)
{
   // Executes this block if
   // condition is true
}
else
{
   // Executes this block if
   // condition is false
}
```

**Working of if-else statements**

1. Control falls into the if block.
2. The flow jumps to Condition.
3. Condition is tested.
    1. If Condition yields true, goto Step 4.
    2. If Condition yields false, goto Step 5.
4. The if-block or the body inside the if is executed.
5. The else block or the body inside the else is executed.
6. Flow exits the if-else block.

**Flowchart if-else:**




**Example: Even or Odd Program:-** Let's look at an example to determine whether a given number is even or odd using the if-else statement.

```
#include <iostream>
using namespace std;
int main() {
   int n;
   cout << "Enter your number:\n";
   cin >> n;
   if (n % 2 == 0) {
      cout << "Even \n";
   } else {
      cout << "Odd \n";
   }
   return 0;
}
```

**How the Program Works**

1. **Input**:
   - The user is prompted to enter a number, which is stored in the variable n.
2. **Condition Check**:
   - The condition n % 2 == 0 is evaluated:
     - If n is divisible by 2 (i.e., the remainder is 0), the condition is true, and the program prints **"Even"**.
     - Otherwise, the condition is false, and the program prints **"Odd"**.
3. **Output**:
   - Based on the evaluation of the condition, the appropriate block of code is executed.

**Input**: Enter your number: 7

**Output**: Odd

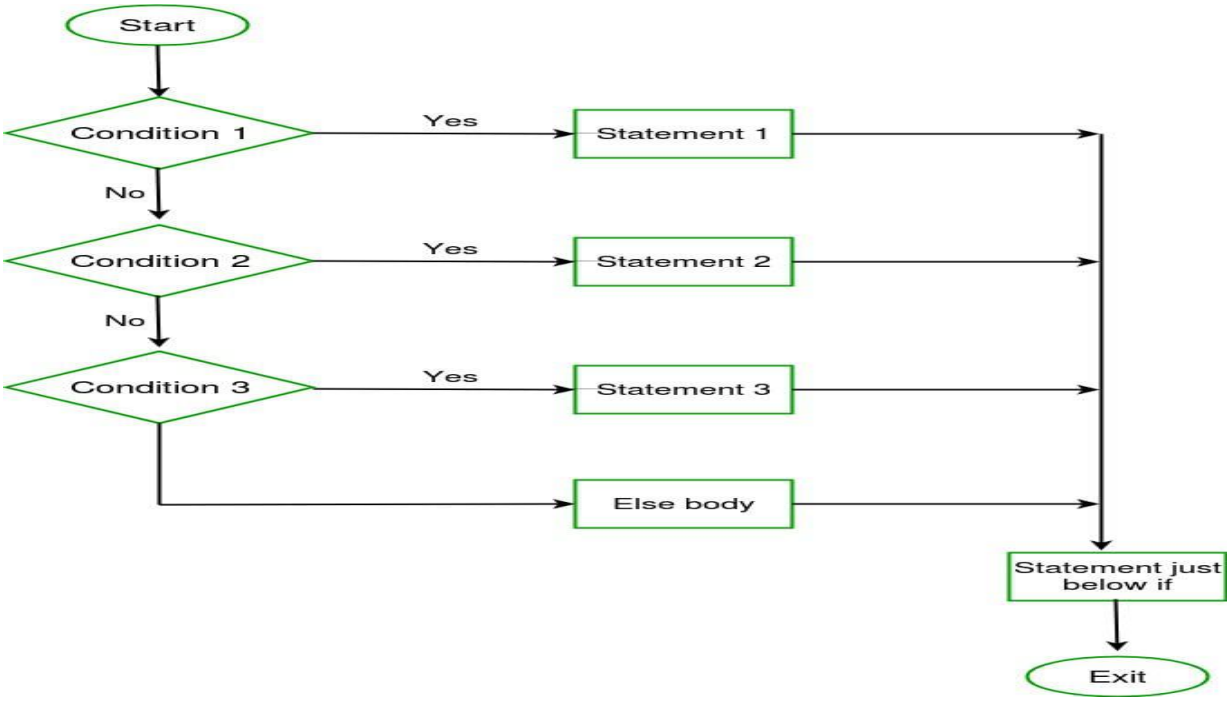
**Else If with Example:-**There come situations in real life when we need to make some decisions and based on these decisions, we decide what should we do next. Similar situations arise in programming also where we need to make some decisions and based on these decisions we will execute the next block of code.
For example, in C++ if x occurs then execute y else execute z. There can also be multiple conditions like in C++ if x occurs then execute p, else if condition y occurs execute q, else execute r. This condition of C++ else-if is one of the many ways of importing multiple conditions.

**if-else-if ladder in C++**

a user can decide among multiple options. The C if statements are executed from the top down. As soon as one of the conditions controlling the if is true, the statement associated with that if is executed, and the rest of the C else-if ladder is bypassed. If none of the conditions is true, then the final else statement will be executed.

**Syntax:** if (condition1)

```
{
 // Execute if condition1 is true
}else if (condition2) {
  // Execute if condition2 is true
 }else{
// Execute if no other condition is true
}
```




**Example: Check if a Number is Positive, Negative, or Zero**

```
#include<iostream>
   using namespace std;
   int main()
{
   int n;
   cout << "Enter a number:\n";
   cin >> n;
   if(n>0)
   {
      cout << "Positive \n";
   } else if(n<0){
      cout << "Negative \n";
   } else{
      cout << "Zero \n";
   }
   return 0;
   }
```

**Conditions**:

- **First if condition**: Checks if n > 0. If true, the program prints **"Positive"**.
- **else if condition**: If the first condition is false, checks if n < 0. If true, the program prints **"Negative"**.
- **else block**: If neither condition is true (i.e., n == 0), the program prints **"Zero"**.

***Input 1***:

```
Enter a number: 5
```

**Output**: Positive

**Nested If Else with Example:-**A nested if is an if statement that is the target of another if statement. Nested if statements mean an if statement inside another if statement. C++ allow us to nested if statements within if statements, i.e, we can place an if statement inside another if statement.

**Syntax:** if (condition1) {

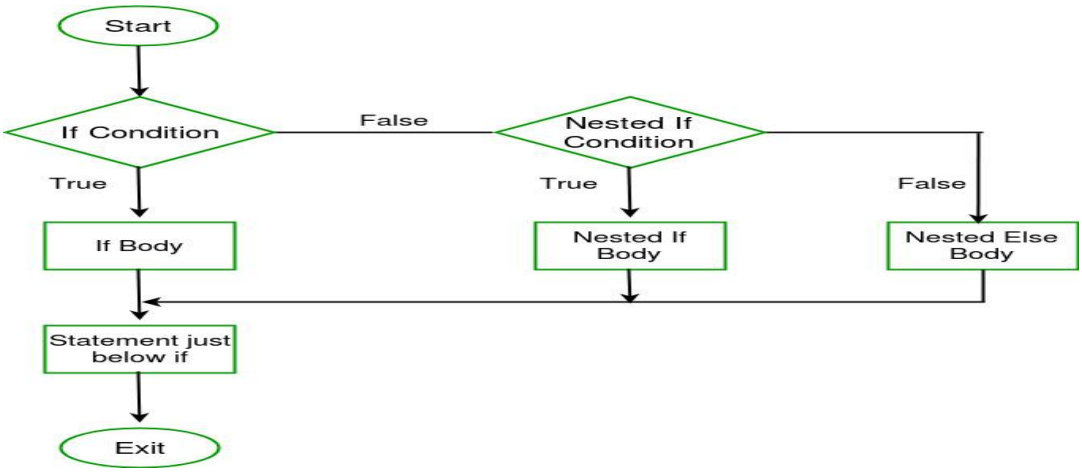
```
// Executes when condition1 is true
  if (condition2)
  {
 // Executes when both condition1 and condition2 are true
  }  else {
 // Executes when condition1 is true and condition2 is false
  }
}else{
//Executes when condition1 is false
}
```

**Example: Check Positive, Negative, Even, Odd, or Zero**

Let's explore a program that uses nested if-else to determine:

- If a number is **positive**, **negative**, or **zero**.
- If it's **even** or **odd** (for positive and negative numbers).

**Flowchart**




```
#include <iostream>
using namespace std;
   int main() {
   int n;
   cout << "Enter a number:\n";
   cin >> n;
   if (n > 0) {
      cout << "Positive ";
      if (n % 2 == 0) {
         cout << "Even\n";
      } else {
         cout << "Odd\n";
      }
   } else if (n < 0) {
      cout << "Negative ";
      if (n % 2 == 0) {
         cout << "Even\n";
```

```
        } else {
            cout << "Odd\n";
        }
    } else {
        cout << "Zero\n";
    }

    return 0;
}
```

**Explanation of the Code**

1. **Outer if-else**:
   - The first if checks if the number is greater than 0. If true, it identifies the number as **positive**.
   - The else if checks if the number is less than 0. If true, it identifies the number as **negative**.
   - The else block handles the case where the number is 0.
2. **Inner if-else**:
   - Inside the positive and negative blocks, an inner if-else checks whether the number is **even** (n % 2 == 0) or **odd** (else block).

***Input 1***:

```
Enter a number:
5
```

**Output**:

```
Positive Odd
```

***Input 2***:

```
Enter a number:
-8
```

**Output**:

```
Negative Even
```

**Switch in C++:-** The switch statement in C++ is a control structure that allows you to execute one block of code out of many based on the value of a variable or expression. It is an alternative to using multiple if-else statements and provides better readability and performance for multi-branch condition.

**Syntax:**

```
switch(expression)
{
case value1:    statement_1; break;
case value2:    statement_2; break;
.....
.....
.....
case value_n:    statement_n; break;
default:     default statement;
 }
```

**Some important keywords:**

**1) Break:** This keyword is used to stop the execution inside a switch block. It helps to terminate the switch block and break out of it.

**2) Default:** This keyword is used to specify the set of statements to execute if there is no case match.

***Note:*** *Sometimes when* ***default*** *is not placed at the end of switch case program, we should use* ***break statement*** *with the default case.*

**Important Points About Switch Case Statements:**

**1)** The expression provided in the switch should result in a **constant value** otherwise it would not be valid. Some valid expressions for switch case will be,

```
// Constant expressions allowed
switch(1+2+23)
switch(1*2+3%4)
// Variable expression are allowed provided
// they are assigned with fixed values
switch(a*b+c*d)
switch(a+b+c)
```

**2) Duplicate case values are not allowed.**

**3)** The **default statement is optional**. Even if the switch case statement do not have a default statement, it would run without any problem.

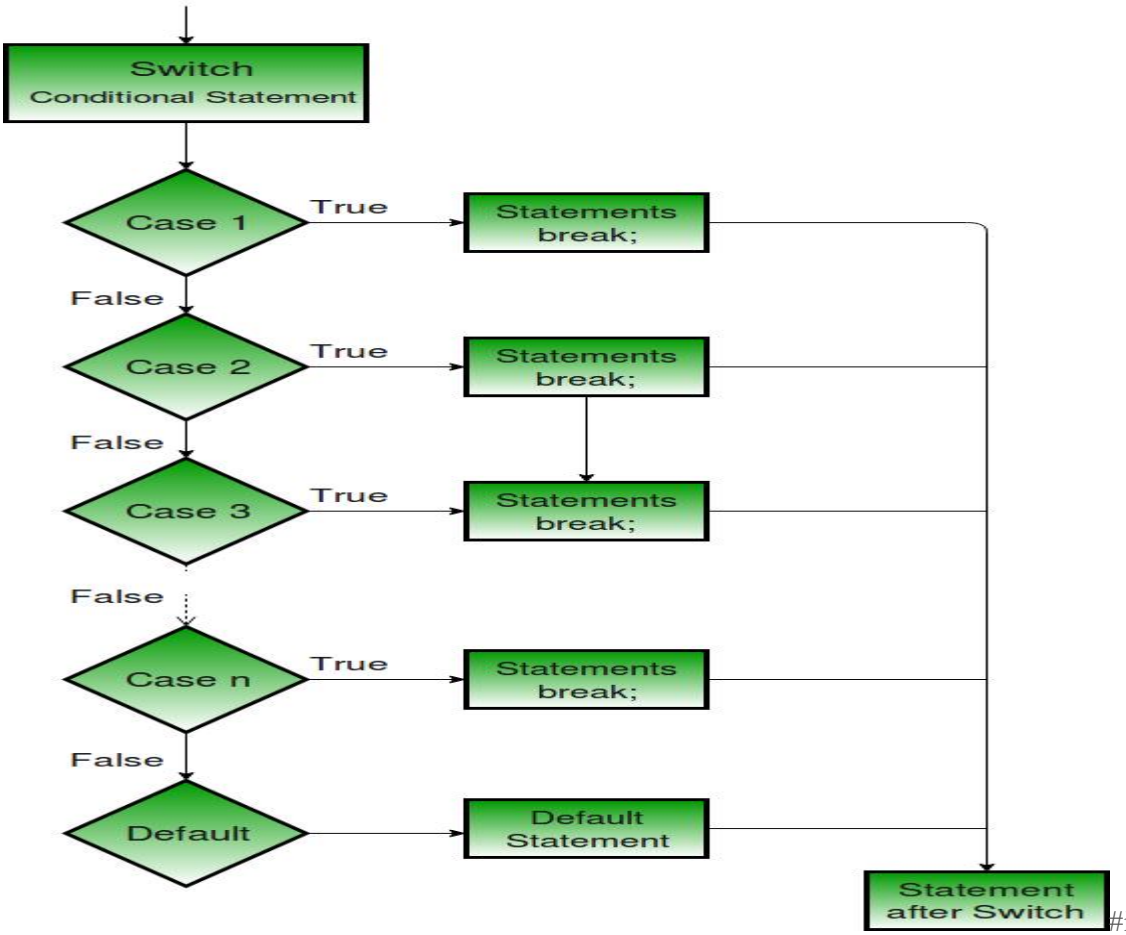
**4)** The **break statement is used inside the switch to terminate a statement** sequence. When a break statement is reached, the switch terminates, and the flow of control jumps to the next line following the switch statement.

**5)** The **break statement is optional**. If omitted, execution will continue on into the next case. The flow of control will fall through to subsequent cases until a break is reached.

**6) Nesting of switch statements is allowed**, which means you can have switch statements inside another switch. However nested switch statements should be avoided as it makes the program more complex and less readable.

**7)** Switch statements are **limited to integer values and characters** only in the check condition i.e., the “case value” can be of “char” and “int” type.

**Flowchart:**




#include <iostream>

```
using namespace std;
   int main() {
   int x = 0, y = 0;
   cout << "Enter a choice (L, R, U, D):\n";
   char move;
   cin >> move;
    switch (move) {
      case 'L':
         x--;  // Move left
         break;
      case 'R':
         x++;  // Move right
         break;
      case 'U':
         y++;  // Move up
         break;
      case 'D':
         y--;  // Move down
         break;
      default:
         cout << "Invalid choice";
   }
    cout << "New Position: " << x << " " << y << '\n';
   return 0;
}
```

*Input 1:*

```
Enter a choice (L, R, U, D):
L
```

**Output**:

```
-1 0
```

*Input 2:*

```
Enter a choice (L, R, U, D):
Z
```

**Output**:Invalid choice

New Position: 0 0

## Even Odd Game:-CUP PROBLEM

There are 'n' cups placed in a line. You and your opponent will take turns picking up cups one at a time. We need to find who win's the game.

**Rules:**

- On each turn, a player must pick up **one** cup from **either end** of the line.
- You cannot skip a turn.
- The game continues until all cups are gone.

The player who collects the **most cups** at the end of the game wins.

```
Note - 1st cup is picked by you.
     If picked last you win.
```

```
#include<iostream>

using namespace std;
   int main()
{
   int n;
   cin >> n;
   if(n%2==0)
   {
      cout << "Opponent Won";
   } else {
      cout << "You Won";
   }
   return 0;
   }
```

Input 3

Output You Won

**Largest of Three Numbers** Given three numbers A, B and C; The task is to find the largest number among the
three.

**Examples**:

***Input***: *A = 2, B = 8, C = 1*

***Output***: *Largest number = 8*

***Input***: *A = 231, B = 4751, C = 75821*

***Output***: *Largest number = 75821*

```
#include<iostream>
   using namespace std;
   int main()
{
  int a, b, c;
   cin >> a >> b >> c;
   if(a>=b and a>=c)
   {
      cout << a;
   } else if(b>=a and b>=c) {
      cout << b;
   } else {
      cout<<c;
   }
   return 0;
}
```

**Comparison**:

- **First if Condition**: Checks if a is greater than or equal to both b and c. If true, a is printed as the largest number.
- **else if Condition**: If the first condition is false, this checks if b is greater than or equal to both a and c. If true, b is printed.
- **else Block**: If neither a nor b is the largest, the program prints c as the largest number.

**Input** 3 2 4

**Output** 4

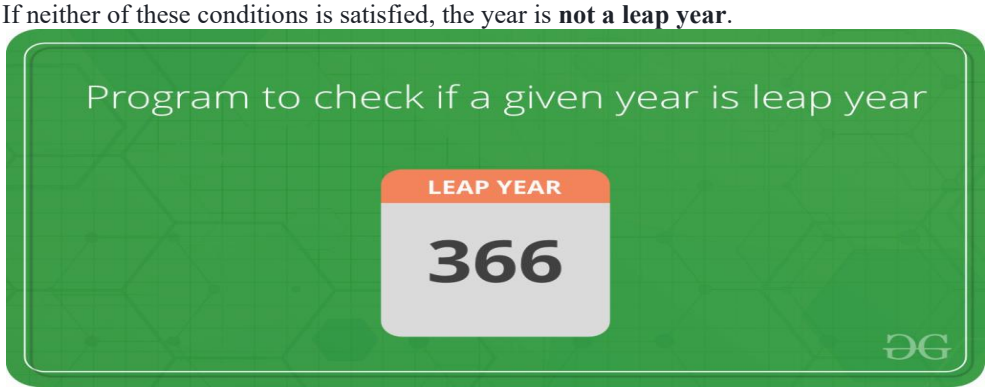
**Leap Year :-**A **leap year** is a year that contains one extra day (February 29) to keep our calendar year synchronized with Earth's revolutions around the Sun. In programming, identifying whether a given year is a leap year is a common problem. Let's explore how to implement this using C++.

**What is a Leap Year?** A year is a **leap year** if it satisfies the following conditions:

1. The year is **divisible by 4** but **not divisible by 100**.
   Example: 1996, 2004, 2008, etc.

2. Alternatively, the year is **divisible by 400**.
   Example: 1600, 2000, 2400, etc.

If neither of these conditions is satisfied, the year is **not a leap year**.

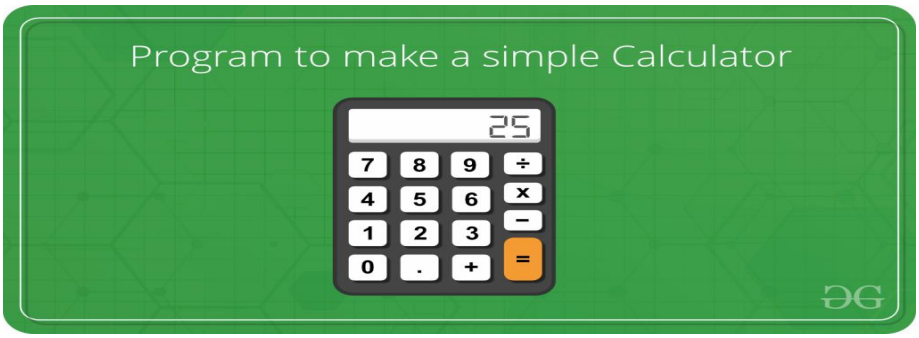



```
#include<iostream>

using namespace std;
   int main()
{
   int year;
   cin >> year;
   if(year % 4 == 0 and year % 100 != 0)
   {
      cout << "Leap Year";
   } else if(year % 400 == 0) {
      cout << "Leap Year";
   } else{
      cout << "Not a Leap Year";
   }
   return 0;
}
```

**INPUT:** 2324

**OUTPUT:**Leap Year

### Simple Calculator




A calculator is one of the simplest and most effective programs to learn the basics of conditional statements in C++. By using if-else if statements, we can build a program that performs basic arithmetic operations like addition, subtraction, and multiplication. Let's look at how to create a simple calculator in C++.

**How the Program Works :-**This program accepts three inputs:

1. **Operation Code (op)**:
   - Specifies the operation to be performed.
   - op==1 means '+'
   - op==2 means '-'
   - op==3 means '*'
2. **Two Numbers (x and y)**:
   - The operands on which the operation is performed.

The program evaluates the operation based on the input op and outputs the result.

**INPUT** 1 2 3

2 10 5

3 7 3

**OUTPUT** 5 3 15

```
#include<iostream>

using namespace std;
  int main()
{
  int op, x, y;
  cin >> op >> x >> y;
  if(op==1)
  {
    cout << x + y;
  }else if(op==2){
    cout << x - y;
  }else if(op==3) {
    cout << x * y;
  } else{
    cout << "Invalid Input";
  }
  return 0;
}
```

**Conditions**:

The if statement checks the value of op:

- If op == 1, the program performs addition (x + y).
- If op == 2, the program performs subtraction (x - y).
- If op == 3, the program performs multiplication (x * y).
- If op doesn't match any of these values, the else block is executed, displaying **"Invalid Input"**.

**INPUT** 2 3 2

**OUTPUT** 1

**Loop:-**In programming, sometimes there is a need to perform some operation **more than once** or (say) **n number** of times. Loops come into use when we need to repeatedly execute a block of statements. **For example**: Suppose we want to print "Hello World" 10 times. This can be done in two ways as shown below:

**Manual Method (Iterative Method) :-** Before understanding loops, let's look at the manual way to solve repetitive tasks. Here is an example program to print the multiplication table of a number: So, *here loops have their role.*

```
#include <iostream>
using namespace std;
   int main() {
   int n;
   cout << "Enter a Number: ";
   cin >> n;
// Manually printing the table
    cout << n * 1 << " "
        << n * 2 << " "
        << n * 3 << " "
        << n * 4 << " "
        << n * 5 << " "
        << n * 6 << " "
        << n * 7 << " "
        << n * 8 << " "
        << n * 9 << " "
        << n * 10 << endl;
   return 0;
}
```

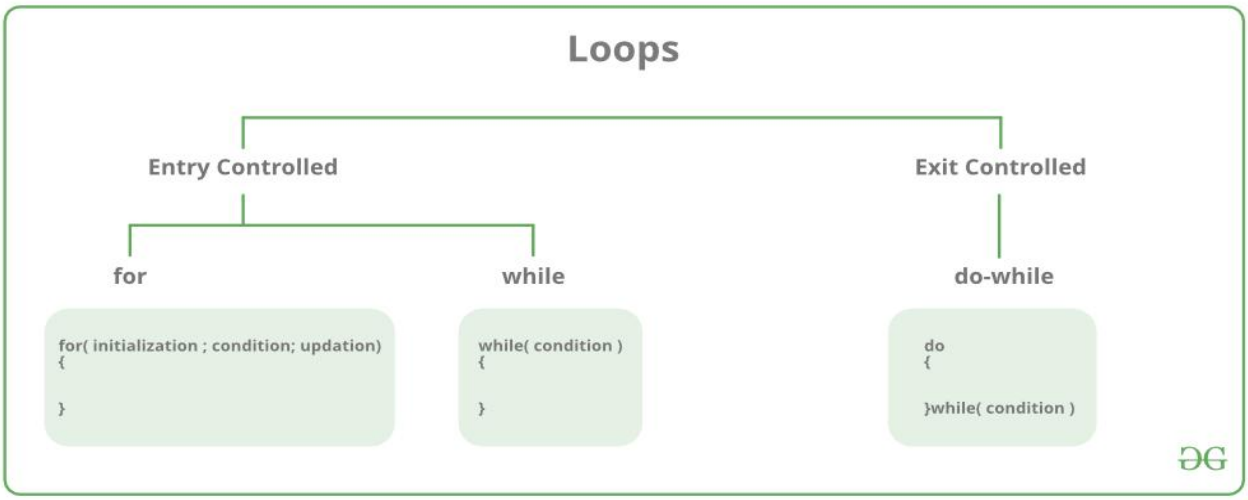
**Example Input and Output:**Input: n = 3

Output: 3 6 9 12 15 18 21 24 27 30

While this works, the manual method becomes tedious and error-prone when scaling up for larger tasks. That's where loops come in.

**Applications of Loops in C++:-** Loops have a wide range of applications in programming. Some common use cases include:

1. **Performing Repeated Tasks:**
   - Loops are used to execute a set of instructions multiple times. For example, calculating factorials, generating patterns, or processing multiple inputs.
2. **Traversing Through Collections:**
   - Loops are essential for traversing arrays, strings, maps, and other data structures. They make it easy to iterate through elements for manipulation or computation.
   - Example: Traversing an array to calculate the sum of its elements.
3. **Running Services in Systems:**
   - Loops can run indefinitely to keep systems and services operational.
   - Examples include:
     - **Web Servers:** An infinite loop listens for and handles user requests.
     - **Memory Managers:** Continuously monitor and allocate memory.
     - **Process Managers:** Handle multiple processes in an operating system.




**While Loop in C++** :-The **while loop** is one of the fundamental loops in C++ that allows repeated execution of a block of statements as long as a specified condition evaluates to true

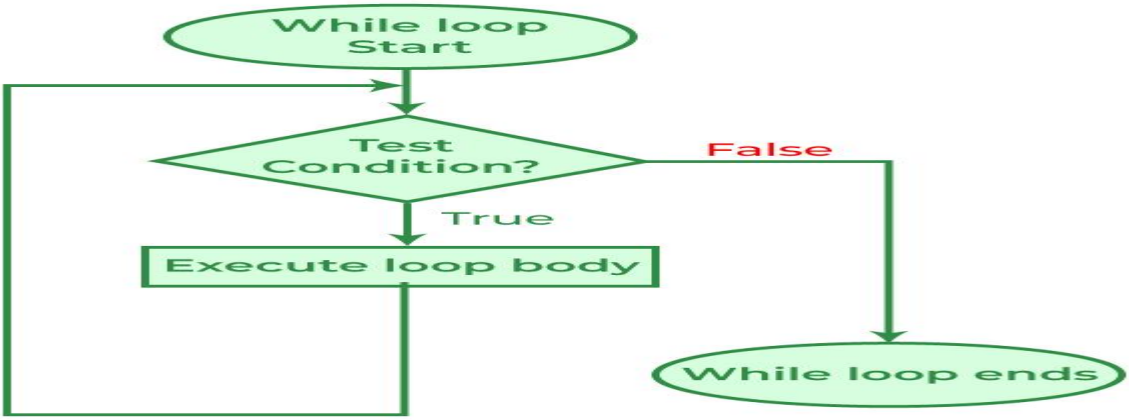
We have already stated that a loop mainly consists of three statements – initialization expression, test expression, and update expression. The syntax of the three loops – For, while, and do while mainly differs in the placement of these three statements.

**Syntax**:

```
initialization expression
while (condition) {
   // Statement 1
   // Statement 2
   // ...
}
```

- **Condition:** The condition is evaluated before each iteration of the loop.
- If the condition is true, the statements inside the loop body are executed.
- If the condition is false, the loop terminates, and the program continues with the next statement after the loop.

**Flow Diagram of while loop:**



```
#include <iostream>
using namespace std;
   int main() {
   int i = 0;
   while (i < 5) {
      cout << "GfG\n";
      i++;
   }
   return 0;
}
```

**Output:**

```
 GfG
GfG
GfG
GfG
GfG
```

*Execution Flow*

Here is the detailed execution flow of the program:

- **Iteration 1:**
  - i = 0
  - Condition i < 5 is true.
  - Output: GfG
  - i is incremented to 1.
- **Iteration 2:**
  - i = 1
  - Condition i < 5 is true.
  - Output: GfG
  - i is incremented to 2.
- **Iteration 3:**
  - i = 2
  - Condition i < 5 is true.
  - Output: GfG
  - i is incremented to 3.
- **Iteration 4:**
  - i = 3
  - Condition i < 5 is true.
  - Output: GfG
  - i is incremented to 4.
- **Iteration 5:**
  - i = 4
  - Condition i < 5 is true.
  - Output: GfG
  - i is incremented to 5.
- **Iteration 6:**
  - i = 5
  - Condition i < 5 is false.
  - The loop terminates.

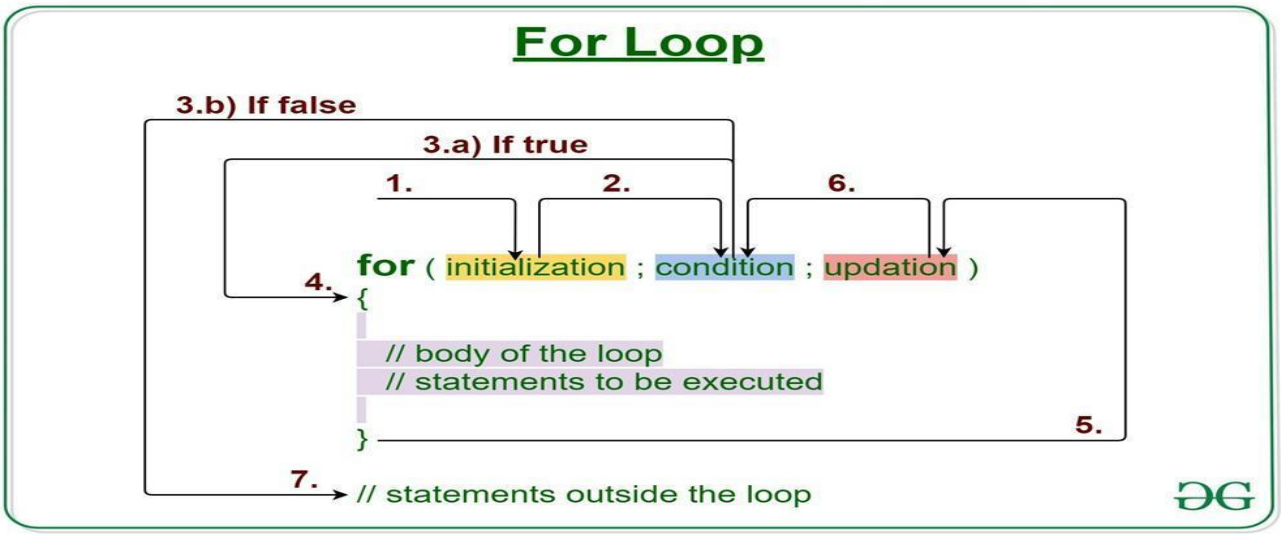
For Loop in C++ he **for loop** is a fundamental looping construct in C++ that allows you to execute a block of code a specific number of times. It is especially useful when the number of iterations is known in advance.

**Syntax:**

```
for (initialization; condition; update) {
   // Statements to be executed inside the loop
}
```

In for loop, a loop variable is used to control the loop. First, initialize this loop variable to some value, then check whether this variable is less than or greater than the counter value. If the statement is true, then the loop body is executed and the loop variable gets updated. Steps are repeated till the exit condition comes.

1. **Initialization:** This part initializes the loop control variable(s). It is executed once at the beginning of the loop.
2. **Condition:** This condition is evaluated before each iteration of the loop. If it evaluates to true, the loop body executes. If it evaluates to false, the loop terminates.
3. **Update:** This part updates the loop control variable(s) after each iteration.




**Example 1: Basic For Loop**

```
#include <iostream>
using namespace std;
   int main() {
   for (int i = 0; i < 3; i++) {
      cout << "GfG\n";
   }
   return 0;
}
```

**Explanation:**

- **Initialization:** int i = 0 sets i to 0.
- **Condition:** i < 3 ensures the loop runs as long as i is less than 3.
- **Update:** i++ increments i after each iteration.

*Output:*

```
GfG
GfG
GfG
```

**Example 2: Printing Variable Values**

```
#include <iostream>
using namespace std;
   int main() {
```

```
    for (int i = 0; i < 3; i++) {
cout << "GfG " << i << endl;
    }
    return 0;
}
```

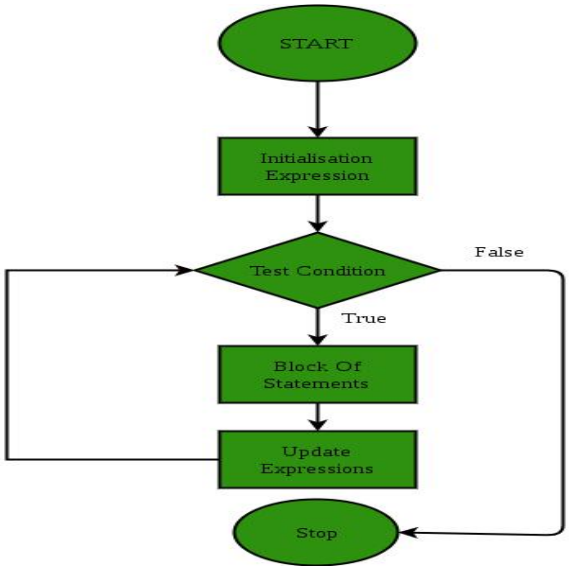
**Explanation:**

- The variable i is printed alongside "GfG" during each iteration.

*Output:*

```
GfG 0
GfG 1
GfG 2
```

**Flow Diagram of for loop:**




**Example 3: Modifying the Start Condition**

```
#include <iostream>

using namespace std;
    int main() {
    for (int i = 3; i < 3; i++) {
        cout << "GfG\n";
    }
    return 0;
}
```

**Explanation:**

- The condition i < 3 is false at the start, so the loop body does not execute.
- **Output:** No output.

**Example 4: Custom Increment**

```
#include <iostream>
using namespace std;
```

```
int main() {
   for (int i = 1; i <= 10; i += 2) {
      cout << "GfG\n";
   }
   return 0;
}
```

**Explanation:**

- The increment step is i += 2, so i increases by 2 after each iteration.

**Output:**

```
GfG
GfG
GfG
GfG
GfG
```

**Example 5: Infinite For Loop**

```
#include <iostream>
using namespace std;

int main() {
   for (;;) {
      cout << "GfG\n";
   }
   return 0;
}
```

**Explanation:**

- All parts of the for loop (initialization, condition, update) are omitted.
- This creates an infinite loop that prints "GfG" endlessly.
- **Output:** "GfG" printed continuously until manually terminated.

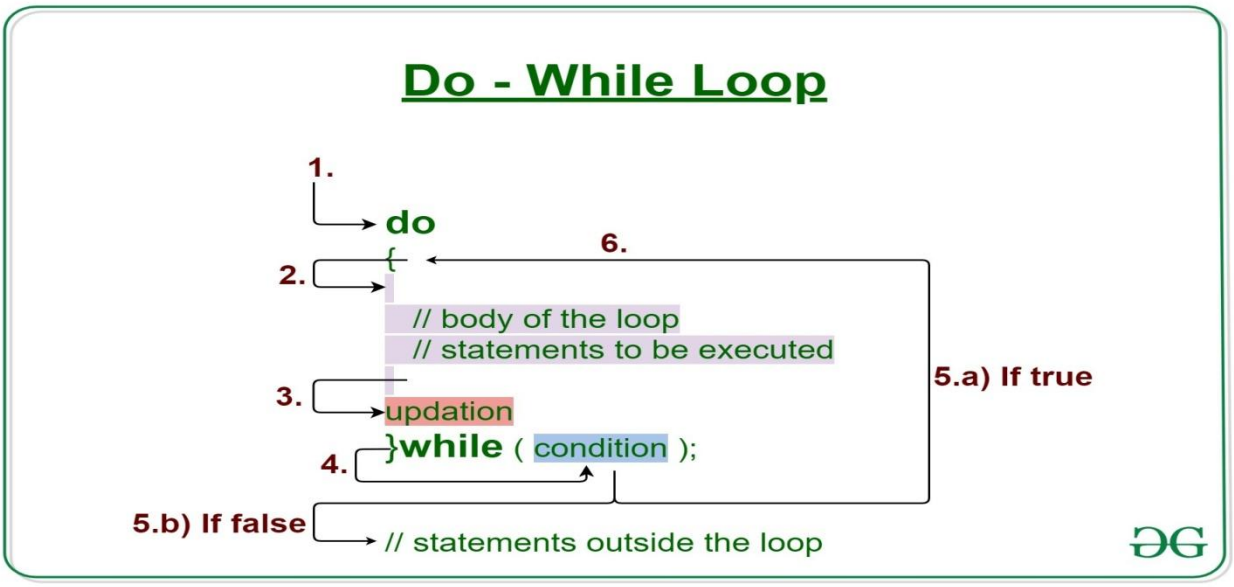
**When to Use a For Loop vs. While Loop**

1. **For Loop:**
   - Use when the number of iterations is known or can be calculated in advance.
   - Example: Iterating over an array or printing a sequence of numbers.
2. **While Loop:**
   - Use when the number of iterations depends on a condition that is evaluated during runtime.
   - Example: Waiting for user input or processing data until a certain condition is met.

**Key Difference:** The initialization, condition, and update are handled in one line in a for loop, making it concise and ideal for counters. A while loop offers more flexibility when the condition is dynamically determined.

**Do While Loop in C++**:- Loops come into use when we need to repeatedly execute a block of statements. Like **while** the **do-while loop** execution is also terminated on the basis of a test condition. The main difference between a do-while loop and a while loop is in the do-while loop the condition is tested at the end of the loop body, i.e do-while loop is exit controlled whereas the other two loops are entry-controlled loops.

***Note**: In the do-while loop, the loop body will execute at least once irrespective of the test condition.*




**Syntax:**

```
do
{
  /// Statements to be executed inside the loop

  update_expression;
}
while (test_expression);
```

***Note**: Notice the semi – colon(";") in the end of loop.*

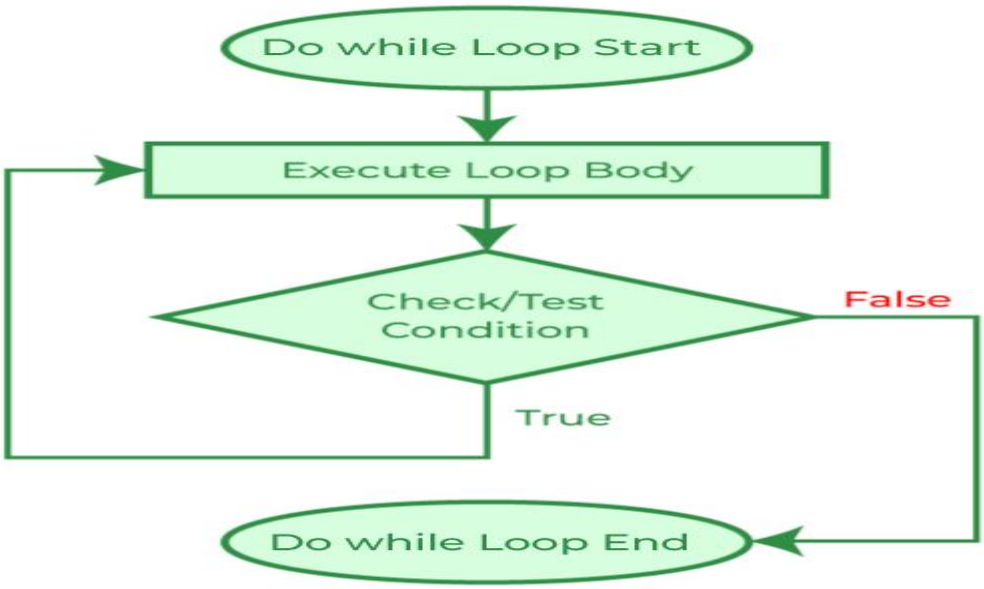
The various parts of the **do-while loop** are:

1. **Test Expression:** In this expression, we have to test the condition. If the condition evaluates to true then we will execute the body of the loop and go to the update expression. Otherwise, we will exit from the while loop.
2. **Update Expression:** After executing the loop body, this expression increments/decrements the loop variable by some value.
3. **Body:** It is the Collection of statements i.e, variables and functions, etc. The condition is not satisfied until the condition is executed automatically after a successful iteration. do-while loop, code can be used to print simple names, execute complex algorithms, or perform functional operations.

**How does a do-While loop execute?**

1. Control falls into the do-while loop.
2. The statements inside the body of the loop get executed.
3. Updation takes place.
4. The flow jumps to Condition
5. Condition is tested.
    - If the Condition yields true, go to Step 6.
    - If the Condition yields false, the flow goes outside the loop
6. The flow goes back to Step 2.
7. The do-while loop has been ended and flow has gone outside the loop.

**Flow Diagram of do-while loop**




**Example**

```
#include <iostream>
using namespace std;
   int main() {
   int i = 3;
   do {
      cout << "GfG\n";
      i++;
   } while (i < 3);

   return 0;
}
```

**Explanation:**

1. **Initialization:** The variable i is initialized to 3.
2. **Loop Body Execution:** The statement cout << "GfG\n"; is executed before the condition is checked.
3. **Condition:** The condition i < 3 is evaluated after the first iteration. Since i becomes 4 after the increment, the condition evaluates to false, and the loop terminates.

*Output:* ***GfG***

**Key Point:** The loop executes the body at least once, even if the condition is false initially.

**Characteristics of the Do-While Loop**

1. **Guaranteed Execution:** The loop body is executed at least once, regardless of whether the condition is true or false.
2. **Condition Checking:** The condition is checked after the loop body executes, making it suitable for scenarios where you want to ensure the code runs at least once.

**When to Use a Do-While Loop**

1. **Menu-Driven Programs:**

- When you need to display a menu and take user input at least once.
- Example:

```
char choice;
do {
   cout << "Enter your choice (y/n): ";
   cin >> choice;
} while (choice != 'n');
```

2. **Input Validation:**
   - When you need to validate user input and ensure the prompt is displayed at least once.
   - Example:

```
int num;
do {
   cout << "Enter a positive number: ";
   cin >> num;
} while (num <= 0);
```

3. **Running Code At Least Once:**
   - When the logic requires the code to run before evaluating a condition.

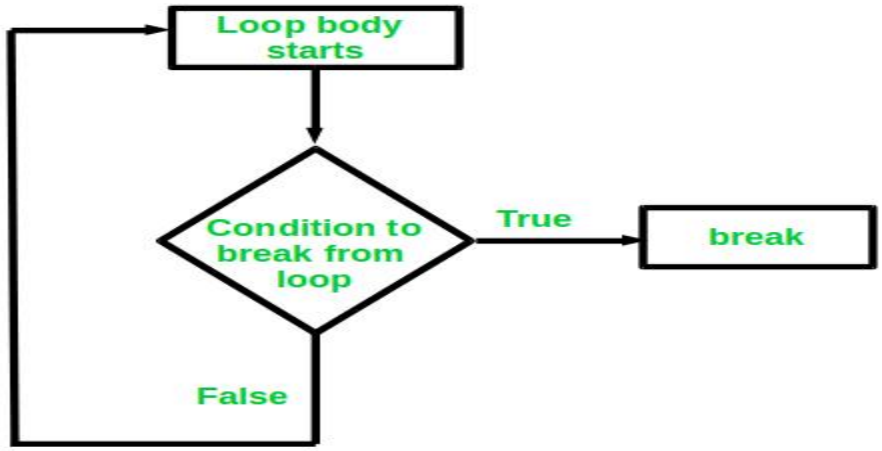
*Comparison with Other Loops*

| Feature | For Loop | While Loop | Do-While Loop |
|---|---|---|---|
| **Condition Check** | Before the loop body executes | Before the loop body executes | After the loop body executes |
| **Execution Guarantee** | Executes only if the condition is true | Executes only if the condition is true | Executes the loop body at least once |
| **Use Case** | Known number of iterations | Condition-based iterations | Run-at-least-once scenarios |

**Break in C++**:- The break in C++ is a loop control statement which is used to terminate the loop. As soon as the break statement is encountered from within a loop, the loop iterations stops there and control returns from the loop immediately to the first statement after the loop.

**Syntax:** break;

Basically break statements are used in the situations when we are not sure about the actual number of iterations for the loop or we want to terminate the loop based on some condition.



**We will see here the usage of break statement with three different types of loops:**

1. Simple loops
2. Nested loops
3. Infinite loops

*Example: Finding the Smallest Divisor*

The following program demonstrates how to use the break statement to find the smallest divisor of a number greater than 1:

```
#include <iostream>
using namespace std;

int main() {
   int n;
   cin >> n;
   for (int x = 2; x <= n; x++) {
      if (n % x == 0) {
         cout << "Smallest Divisor: " << x << endl;
         break;
      }
   }
   return 0;
}
```

**Explanation:**

1. The loop starts with x = 2 and increments x on each iteration.
2. The condition n % x == 0 checks if x is a divisor of n.
3. If the condition is true, the smallest divisor is printed, and the break statement terminates the loop.
4. The loop exits as soon as the first divisor is found, making the program efficient.

**Input and Output:**

- Input: n = 6
  Output

```
Smallest Divisor: 2
```

- Input: n = 49
  Output:

```
Smallest Divisor: 7
```

*Example: Using break in a While Loop*

Consider another example where break is used to terminate a while loop when a specific condition is met:

```
#include <iostream>
using namespace std;

int main() {
   int i = 1;
   while (i <= 5) {
      if (i == 3) {
         break;
      }
      cout << i << " ";
      i++;
   }
   cout << i << " ";
   return 0;
```

}

*Explanation:*

1. The loop starts with i = 1 and continues until i <= 5.
2. When i == 3, the break statement is executed, and the loop terminates.
3. The value of i is printed before the condition is checked.

**Output:** 1 2 3

**Applications of break**

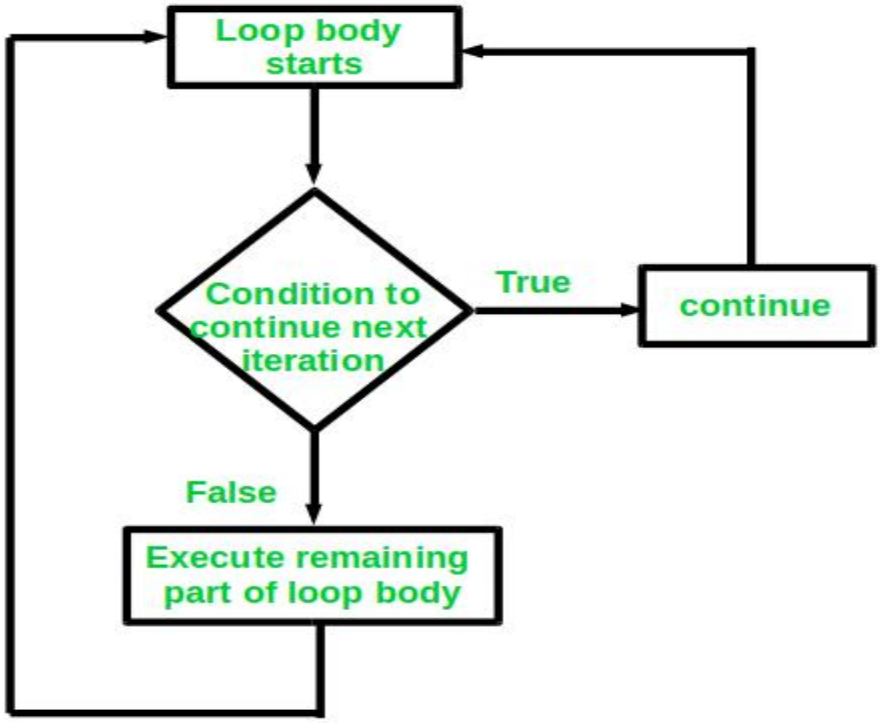
1. **Early Exit from Loops:**
   - Use break to stop a loop once the desired result is achieved, improving efficiency.
   - Example: Finding a specific element in an array.
2. **Menu-Driven Programs:**
   - Use break to exit from a loop when the user chooses to quit.
3. **Switch Statements:**
   - Use break to terminate a case block in a switch statement, ensuring the program doesn't execute subsequent cases unnecessarily.

**Continue in C++**:- Continue is also a loop control statement just like the break statement. *continue* statement is opposite to that of break *statement*, instead of terminating the loop, it forces to execute the next iteration of the loop.

As the name suggest the continue statement forces the loop to continue or execute the next iteration. When the continue statement is executed in the loop, the code inside the loop following the continue statement will be skipped and next iteration of the loop will begin.

**Syntax**: continue;

When the continue statement is encountered in a loop, the rest of the code inside the loop following the continue statement is skipped, and the next iteration begins.



**Example**:

Consider the situation when you need to write a program which prints number from 1 to n that are not multiples of x. It is specified that you have to do this using loop and only one loop is allowed to use.

Here comes the usage of continue statement. What we can do here is we can run a loop from 1 to n and every time we have to compare the value to the multiple of x. If it is multiple of x we will use the *continue* statement to continue to next iteration without printing anything otherwise we will print the value.

Below is the implementation of the above idea:

```
#include <iostream>
using namespace std;
   int main() {
   int n, x;
   cout << "Enter n \n";
   cin >> n;
   cout << "Enter x \n";
   cin >> x;
   for (int i = 1; i <= n; i++) {
      if (i % x == 0)
         continue;
      cout << i << " ";
   }
   return 0;
}
```

**Explanation:**

1. The user inputs two numbers n and x.
2. A for loop iterates from 1 to n.
3. The condition if (i % x == 0) checks if i is a multiple of x.
4. If the condition is true, the continue statement is executed, skipping the rest of the loop body and moving to the next iteration.
5. If the condition is false, the number i is printed.

**Input and Output:**

- Input: n = 10, x = 3
  Output: 1 2 4 5 7 8 10
- Input: n = 7, x = 5
  Output: 1 2 3 4 6 7

*Use Case: Skipping Specific Conditions in Loops*

Consider a situation where you need to print numbers from 1 to 10 but skip the number 6. This can be achieved using the continue statement as shown below:

```
#include <iostream>
using namespace std;
   int main() {
   for (int i = 1; i <= 10; i++) {
      if (i == 6)
         continue;
      cout << i << " ";
   }
   return 0;
}
```

Output: 1 2 3 4 5 7 8 9 10

**Explanation:**

- The loop skips the iteration where i == 6 and continues with the next iteration, thereby avoiding printing 6.

**Nested loop in C++** :- In C++, a **nested loop** is a loop inside another loop. Nested loops are useful for iterating over multi-dimensional data structures or performing repetitive tasks that involve multiple variables. This article explores the concept of nested loops, their syntax, and examples based on the provided lecture notes.

*Syntax of Nested Loops*

The general structure of nested loops is:

```
for ( initialization; condition; increment ) {

  for ( initialization; condition; increment ) {

    // statement of inside loop
  }

  // statement of outer loop
}
```

- **Outer Loop:** The outer loop controls the number of iterations for the inner loop.
- **Inner Loop:** The inner loop executes completely for every iteration of the outer loop.

**Syntax for Nested While loop:**

```
while(condition) {

  while(condition) {

    // statement of inside loop
  }

  // statement of outer loop
}
```

***Note:*** *There is no rule that a loop must be nested inside its own type. In fact, there can be any type of loop nested inside any type and to any level.*

**Syntax:**

```
do{

  while(condition) {

    for ( initialization; condition; increment ) {

      // statement of inside for loop
    }

    // statement of inside while loop
  }

  // statement of outer do-while loop
}while(condition);
```

Below are some examples to demonstrate the use of Nested Loops:

*Example 1: Multiplication Table*

The following program demonstrates the use of nested loops to print a multiplication table:

```
#include <iostream>
using namespace std;
   int main() {
   int n;
   cout << "Enter n:\n";
   cin >> n;
   for (int x = 1; x <= n; x++) {
      for (int i = 1; i <= 10; i++) {
         cout << (x * i) << " ";
      }
      cout << "\n";
   }
   return 0;
}
```

**Execution Flow:**

- For x = 1, the inner loop prints the multiples of 1: 1 2 3 4 5 6 7 8 9 10.
- For x = 2, the inner loop prints the multiples of 2: 2 4 6 8 10 12 14 16 18 20.
- For x = 3, the inner loop prints the multiples of 3: 3 6 9 12 15 18 21 24 27 30.

**Input and Output:**

- Input: n = 3
  Output: 1 2 3 4 5 6 7 8 9 10
  2 4 6 8 10 12 14 16 18 20
  3 6 9 12 15 18 21 24 27 30

**Example 2:** Below program uses a nested for loop to print all prime factors of a number.

```
// C++ Program to print all prime factors
// of a number using nested loop

#include <bits/stdc++.h>
using namespace std;

/* Driver program to test above function */
int main()
{
   int n = 315;
   // Print the number of 2s that divide n
   while (n % 2 == 0)
   {
      cout << 2;
      n = n / 2;
   }
    // n must be odd at this point. So we can skip
   // one element (Note i = i +2)
   for (int i = 3; i <= sqrt(n); i = i + 2)
   {
      // While i divides n, print i and divide n
      while (n % i == 0)
      {
         cout << i;
         n = n / i;
      }
   }
```

```
    // This condition is to handle the case when n
    // is a prime number greater than 2
    if (n > 2)
        cout << n;
    return 0;
}
```

**Output** 3357

**Patterns:-**Patterns are an essential concept in programming and a great way to practice and understand loops in C++. With proper use of **nested loops**, you can create various patterns such as stars, numbers, and other shapes. This article follows the lecture notes provided and focuses on the implementation of patterns using loops.

**Why Learn Patterns?** Patterns are a great way to:

- Understand how loops work.
- Enhance logical and analytical thinking.
- Gain confidence in handling nested loops. Let's explore a few patterns.

**Basic Star Pattern** This pattern prints a single column of stars based on user input:

```
#include <iostream>
using namespace std;
    int main() {
    int n;
    cin >> n;
    while (n--) {
        cout << "*" << endl;
    }
    return 0;
}
```

Input: 5

Output:*
*
*
*
*

**Row and Column Star Pattern** This pattern prints a grid of stars using nested loops:

```
#include <iostream>
using namespace std;
    int main() {
    int n;
    cin >> n;

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            cout << "* ";
        }
        cout << endl;
    }
    return 0;
}
```

Input: 3
Output: * * *
* * *
* * *

**Number Grid Pattern** Instead of *, print the column number (j):

```
#include <iostream>
using namespace std;
   int main() {
   int n;
   cin >> n;
     for (int i = 0; i < n; i++) {
      for (int j = 0; j < n; j++) {
         cout << j + 1 << " ";
      }
      cout << endl;
   }
   return 0;
}
```

Input: 3

Output:1 2 3

1 2 3

1 2 3

**Execution Flow of Nested Loops** Understanding the flow of nested loops is crucial. For every iteration of the outer loop, the inner loop runs completely. For example, in the number triangle pattern:

1. Outer loop (i): Controls the rows.
2. Inner loop (j): Controls the columns and runs i times for each i.
3. Total iterations = Sum of the first n natural numbers: n * (n + 1) / 2.

**Triangle pattern:-**Printing patterns is a fundamental programming exercise that helps in understanding nested loops and their behavior. In this article, we will explore how to create a triangle pattern in C++ using nested loops, accompanied by an analysis of the pattern.

**Triangle Pattern Description:-**A triangle pattern consists of rows and columns, where the number of rows corresponds to a given integer n. For instance:

- If n = 3:

```
*
* *
* * *
```

- If n = 4:

```
*
* *
* * *
* * * *
```

This exercise is perfect for understanding the relationship between indices in nested loops.

**Code to Print the** Triangle Pattern Here is a simple C++ program to print the triangle pattern:

```
#include <bits/stdc++.h>
using namespace std;
   int main() {
```

```
    int n;
    cin >> n;
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            if (j <= i)
                cout << "* ";
            else cout << "  ";
        }
        cout << endl;
    }
    return 0;
}
```

**Code Explanation**

1. **Input**: The program reads an integer n which determines the size of the triangle.
2. **Outer Loop**:
   - The loop variable i ranges from 1 to n, representing each row.
3. **Inner Loop**:
   - The loop variable j also ranges from 1 to n, representing each column.
   - If j <= i, an asterisk (*) is printed.
   - Otherwise, a blank space (" ") is printed to align the pattern correctly.
4. **Output**:
   - After each iteration of the inner loop, endl is used to move to the next row.

**Pattern Analysis** **The pattern builds row by row:**

- For i = 1: Only one star (*) is printed in the first column.
- For i = 2: Two stars are printed in the first two columns.
- For i = 3: Three stars are printed in the first three columns.
- This continues until all rows are filled with stars up to n.

**Relation Between Indices**

- The condition j <= i ensures that the number of stars in each row matches the current row number.
- This demonstrates the relationship between the outer (i) and inner (j) loop variables.

**Key Observations**

1. The stars align diagonally, forming a triangular shape.
2. Blank spaces (" ") are added for symmetry and clarity in the pattern layout.

To better understand the logic:

- Think of the pattern as a grid where:
  - Rows represent i (outer loop index).
  - Columns represent j (inner loop index).
- The stars are placed at positions where j <= i.

Here's an example of a grid for n = 4:

| i \ j | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| 1 | * | | | |
| 2 | * | * | | |

| i \ j | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| 3 | * | * | * | |
| 4 | * | * | * | * |

**Test the Program :-**Run the code with the following inputs to see the triangle patterns:

1. Input: 3
   Output:
   ```
   *
   * *
   * * *
   ```
2. Input: 5
   Output:
   ```
   *
   * *
   * * *
   * * * *
   * * * * *
   ```

**Inverted Triangle** Pattern printing exercises are crucial for mastering nested loops in programming. In this article, we will focus on creating an **inverted triangle pattern** in C++ using a while loop. Additionally, we will analyze the relationship between indices for rows and columns to understand how the pattern is formed.

### Inverted Triangle Pattern Description

An inverted triangle pattern is characterized by the stars (*) reducing row by row. For example:

- If n = 3:
  ```
  ***
   **
    *
  ```
- If n = 4:
  ```
  ****
   ***
    **
     *
  ```

Here, blank spaces (" ") are used to align the stars properly, creating the inverted triangle effect.

### Code to Print the Inverted Triangle Pattern

The following C++ code utilizes while loops to generate the inverted triangle pattern:

```
#include<iostream>
using namespace std;
   int main()
{
   int i = 1, n;
   cin >> n;
 while (i <= n)
   {
      int j = 1; // Print spaces
      while (j <= n - i)
      {
```

```
        cout << ' ';
        j++;
    } // Print stars
    while (j <= n)
    {
        cout << '*';
        j++;
    }
cout << '\n';
    i++;
  }
  return 0;
}
```

**Code Explanation**

1. **Input**:
   - The program reads an integer n which specifies the size of the triangle (number of rows).
2. **Outer Loop** (while (i <= n)):
   - This loop iterates through each row (i ranges from 1 to n).
3. **Inner Loop 1** (while (j <= n - i)):
   - This loop handles the printing of spaces (' ') before the stars on each row.
   - The number of spaces in each row is determined by n - i.
4. **Inner Loop 2** (while (j <= n)):
   - After printing the spaces, this loop prints the stars ('*') in the current row.
   - The number of stars decreases as the value of i increases.
5. **Output**:
   - After each row is printed, a newline (\n) is added to start the next row.

**Pattern Analysis :-Observations:**

- **Spaces**: The number of spaces in each row increases as i increases, starting from n - i for the first row.
- **Stars**: The number of stars in each row decreases as i increases.

**Relationship Between Indices: For any row i:**

1. Spaces are printed for indices j <= n - i.
2. Stars are printed for indices j > n - i.

**Run the Program** **Test the code with different inputs to observe the inverted triangle pattern:**

**Example 1** Input: **n = 3**
Output: ***
 **
  *

**Example 2** Input: **n = 5**
Output: *****
 ****
  ***
   **
    *

**Square pattern** :- Pattern printing is a common exercise for understanding loops in programming. This article explains how to create a **square pattern** in C++ using nested loops.

**Square Pattern Description:-**A square pattern is characterized by a grid of stars (*) where the number of rows and columns is equal. For example**:**

- If n = 3:
```
***
***
***
```
- If n = 4:
```
****
****
****
****
```

**Code to Print the Square Pattern :-** Here's the C++ code to generate the square pattern**:**

```
#include<iostream>
using namespace std;
  int main()
{
  int n;
  cin >> n;
  for (int i = 1; i <= n; i++)
  {
    for (int j = 1; j <= n; j++)
    {
      cout << '*';
    }
    cout << '\n';
  }
  return 0;
}
```

**Code Explanation**

1. **Input**:
   - The program reads an integer n representing the size of the square.
2. **Outer Loop** (for (int i = 1; i <= n; i++)):
   - The outer loop iterates over the rows. The loop variable i ranges from 1 to n.
3. **Inner Loop** (for (int j = 1; j <= n; j++)):
   - The inner loop iterates over the columns for each row. The loop variable j ranges from 1 to n.
   - A star (*) is printed in each column.
4. **Output**:
   - After the inner loop completes, a newline (\n) is printed to move to the next row.

**Pattern Analysis For n = 4, the pattern can be visualized as a grid of rows (i) and columns (j):**

| i \ j | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| 1 | * | * | * | * |
| 2 | * | * | * | * |

| i \ j | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| 3 | * | * | * | * |
| 4 | * | * | * | * |

**Observations:**

- Each row (i) has exactly n stars.
- The total number of stars is n × n.

**Test the Program Here are some sample inputs and outputs:**

**Example 1** Input: **n = 3**

Output:***
***
***

**Example 2** Input: **n = 5**

Output: *****
*****
*****
*****
*****

**Count Digits of a Number:-**Given a number N, the task is to return the count of digits in this number. Example:

**Simple Iterative Solution to count digits in an integer**

The integer entered by the user is stored in the variable n. Then the while loop is iterated until the test expression n != 0 is evaluated to 0 (false). We will consider 3456 as the input integer.

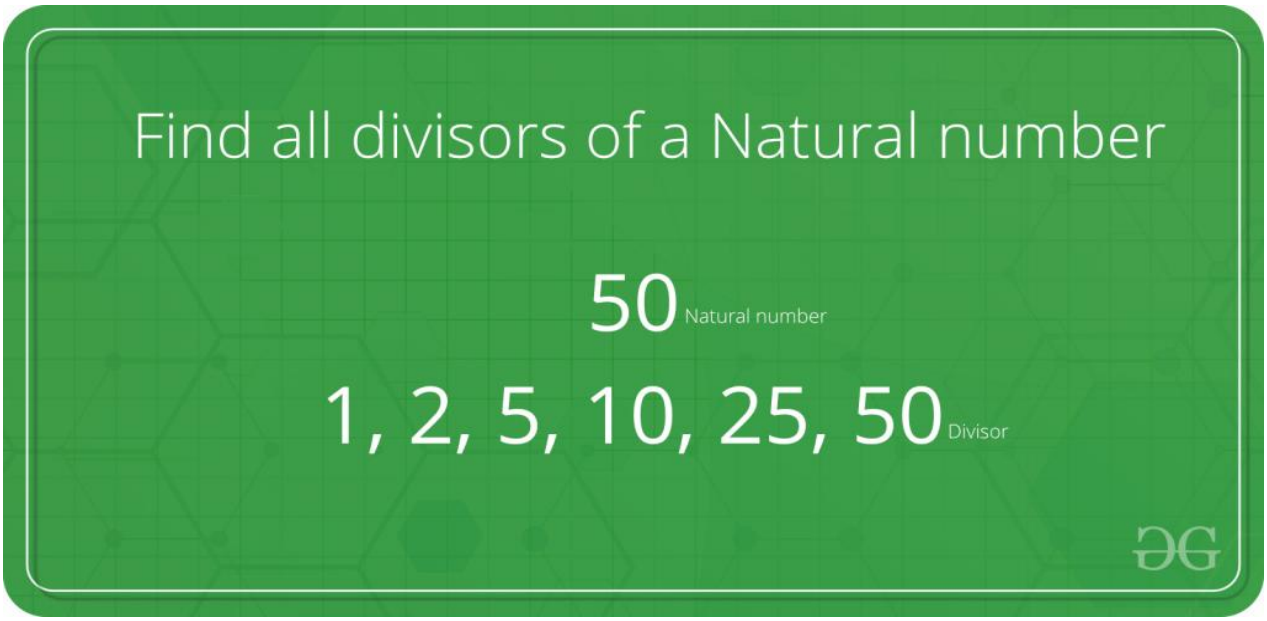
1. After the first iteration, the value of n will be updated to 345 and the count is incremented to 1.

2. After the second iteration, the value of n will be updated to 34 and the count is incremented to 2.
3. After the third iteration, the value of n will be updated to 3 and the count is incremented to 3.
4. In the fourth iteration, the value of n will be updated to zero and the count will be incremented to 4.
5. Then the test expression is evaluated ( n!=0 ) as false and the loop terminates with final count as 4.

```
#include<iostream>
using namespace std;
   int main()
{
   int count = 0 , n;
   cin >> n;
   while(n>0)
   {
      n /= 10;
      count ++;
   }
   cout << count;
   return 0;
}
```

**INPUT :** 231 **OUTPUT :**3

**All Divisor :-**All Divisor of a Number Given a natural number n, print all distinct divisors of it.




**What Are Divisors**? If an integer x divides another integer n completely (i.e., the remainder is 0), then x is called a divisor of n. For example:

- **Input**: 12
- **Output**: 1, 2, 3, 4, 6, 12

Here, the divisors of 12 are all numbers that divide 12 without leaving a remainder.

**Problem Statement:-**Given an integer n, find all its divisors. **Examples:**

**Input : n = 10 Output: 1 2 5 10**

**Input: n = 100 Output: 1 2 4 5 10 20 25 50 100**

**Input: n = 125 Output: 1 5 25 125**

**Approach to Solve the Problem :-**To find all divisors of a number n:

1. Iterate through all integers from 1 to n.
2. Check if the current number divides n without leaving a remainder.
3. If true, print the current number as a divisor.

**Code to Find Divisors**

Here's the C++ implementation of the above approach:

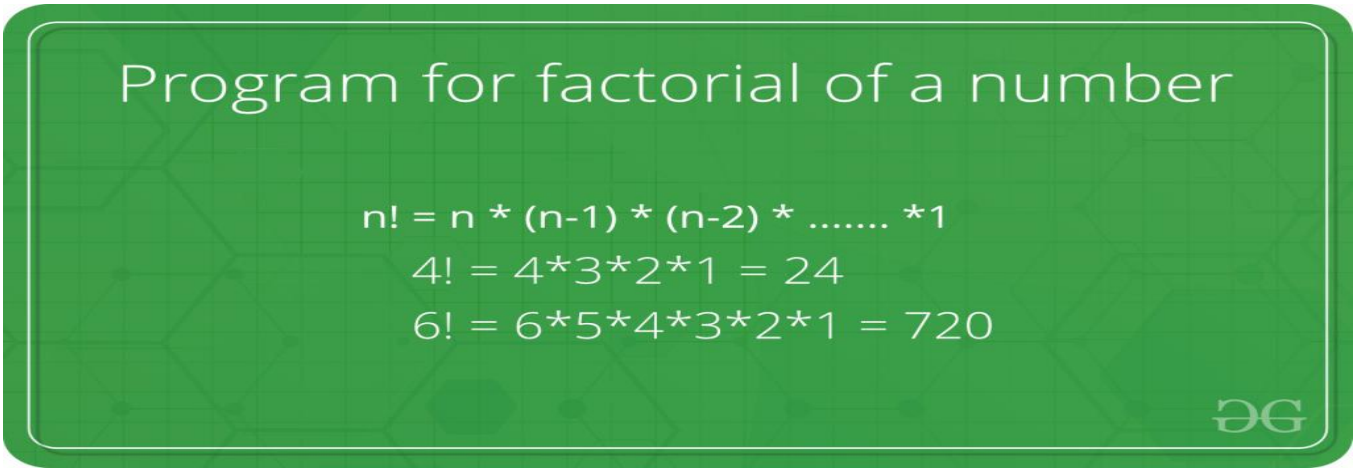
```
#include<iostream>
using namespace std;
  int main()
{
  int n;
  cin >> n;
  for(int i = 1; i <= n; i++)
  {
    if(n%i==0)
    {
      cout << i << ' ' ;
    }
  }
  return 0;
}
```

**Explanation of the Code**

1. **Input**:
   - The program reads an integer n from the user.
2. **Logic**:
   - A for loop iterates through all integers i from 1 to n.
   - The condition n % i == 0 checks if i divides n without leaving a remainder.
   - If true, i is printed as a divisor.
3. **Output**:
   - All divisors of n are printed, separated by spaces.

**INPUT : 6 OUTPUT :1 2 3 6**

**Factorial of A Number :-**Given an integer n, calculate its factorial. Ensure that n is non-negative as factorials are not defined for negative numbers.




**Code to Calculate Factorial** Below is the C++ implementation of calculating factorial using a while loop:

```
#include<iostream>
```

```
using namespace std;
int main()
{
   int n, ans = 1;
   cin >> n;
   while(n>0)
   {
      ans *= n;
      n--;
   }
   cout << ans;
   return 0;
}
```

**Explanation of the Code**

1. **Input**:
   - The program takes an integer n as input from the user.
2. **Initialization**:
   - The variable ans is initialized to 1 as it will store the final factorial value.
3. **Logic**:
   - The while loop iterates as long as n > 0.
   - On each iteration:
     - Multiply the current value of ans by n.
     - Decrease n by 1.
4. **Output**:
   - After the loop finishes, ans contains the factorial of the input number, which is printed to the console.

**INPUT :**5 **OUTPUT :**120

**Explanation**:

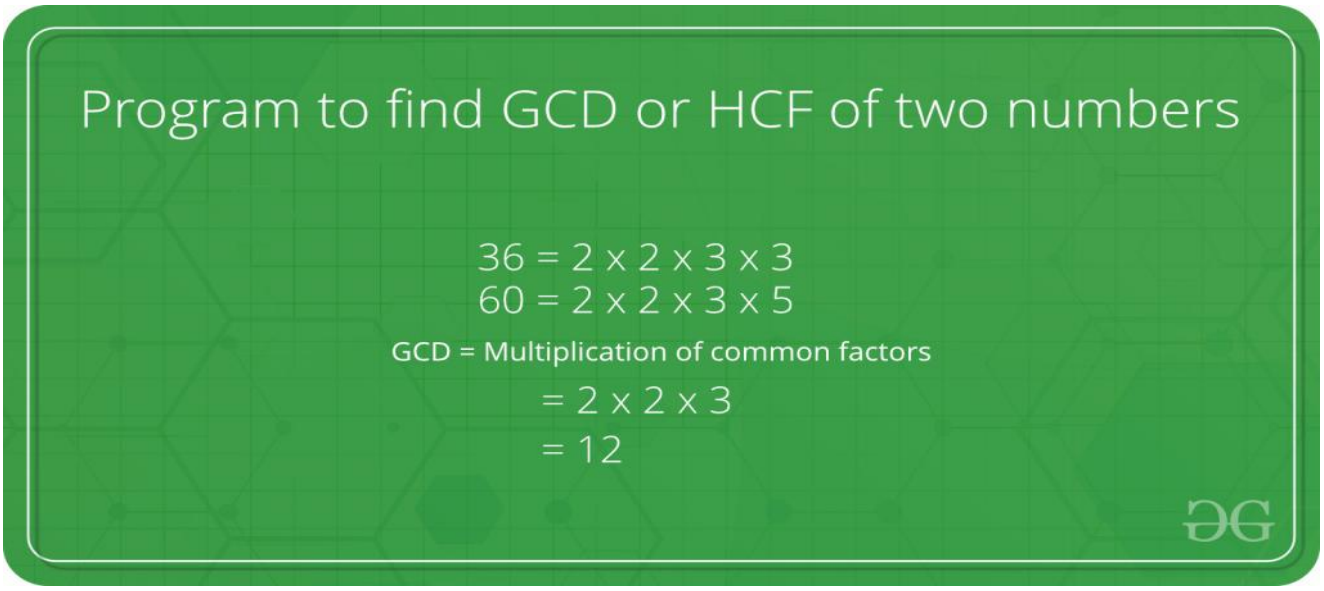
- Initially: ans = 1, n = 5
- First iteration: ans = 1 × 5 = 5, n = 4
- Second iteration: ans = 5 × 4 = 20, n = 3
- Third iteration: ans = 20 × 3 = 60, n = 2
- Fourth iteration: ans = 60 × 2 = 120, n = 1
- Fifth iteration: ans = 120 × 1 = 120, n = 0 (loop ends)
- Result: 120

**GCD of Two Numbers :-** The **GCD (Greatest Common Divisor)** of two numbers is the largest number that divides both numbers without leaving a remainder. It is also known as the **HCF (Highest Common Factor)**.

**What Is GCD? For two numbers a and b:**

- The GCD is the greatest integer x such that both a % x == 0 and b % x == 0. *Examples:*

1. **Input**: a = 10, b = 15
   **Output**: 5
   - Divisors of 10: 1, 2, 5, 10
   - Divisors of 15: 1, 3, 5, 15
   - Common divisors: 1, 5
   - GCD: 5
2. **Input**: a = 12, b = 18
   **Output**: 6
   - Divisors of 12: 1, 2, 3, 4, 6, 12
   - Divisors of 18: 1, 2, 3, 6, 9, 18
   - Common divisors: 1, 2, 3, 6
   - GCD: 6




**Steps to Solve the Problem**

1. Find the smaller of the two numbers, min(a, b), because the GCD cannot be larger than this value.
2. Iterate from 1 to min(a, b) and check if the current number divides both a and b.
3. Update the GCD (ans) whenever a common divisor is found.
4. At the end of the loop, ans will hold the greatest common divisor.

**C++ Code to Find GCD Here is the code implementation for finding the GCD of two numbers:**

```
#include<iostream>
using namespace std;
  int main()
{
  int a, b;
  cin >> a >> b;
  int x = min(a,b);
  int ans = 1;
  for(int i = 1; i<=x; i++){
    if(a%i==0 and b%i==0)
      ans = i;
  }
  cout << ans;
  return 0;
}
```

**INPUT :** 10 15 **OUTPUT :** 5

**Execution**:

x = min(10, 15) = 10 Iterations:

- i = 1: 10 % 1 == 0 and 15 % 1 == 0 → ans = 1
- i = 2: 10 % 2 == 0 and 15 % 2 != 0 → no update
- i = 3: 10 % 3 != 0 and 15 % 3 == 0 → no update
- i = 5: 10 % 5 == 0 and 15 % 5 == 0 → ans = 5

**Output**: 5

**LCM of Two Numbers** :- LCM (Least Common Multiple) of two numbers is the smallest number which can be divided by both numbers. For two numbers a and b, the LCM is the smallest number x such that:

- x % a == 0 (i.e., x is a multiple of a)
- x % b == 0 (i.e., x is a multiple of b)

**Input**: a = 6, b = 9 **Output**: 18

- Multiples of 6: 6, 12, 18, 24, 30, ...
- Multiples of 9: 9, 18, 27, 36, ...
- Common multiples: 18, 36, 54, ...
- Smallest common multiple: 18

**Steps:**

- Find the maximum of a and b since the LCM cannot be smaller than the larger number.
- Set the checking range from max(a, b) to a*b, where a*b is the largest possible LCM.
- Iterate through this range and check each number for divisibility by both a and b.
- The first number divisible by both is the LCM, and the loop is stopped.

```
#include<iostream>
using namespace std;
   int main(){
   int a = 6, b = 9;
   int x = a, y = b;
   while(y != 0){
      int temp = y;
      y = x % y;
      x = temp;
   }
   int gcd = x;
   int lcm = (a * b) / gcd;
   cout << lcm << endl;
   return 0;
}
```

**Output**18

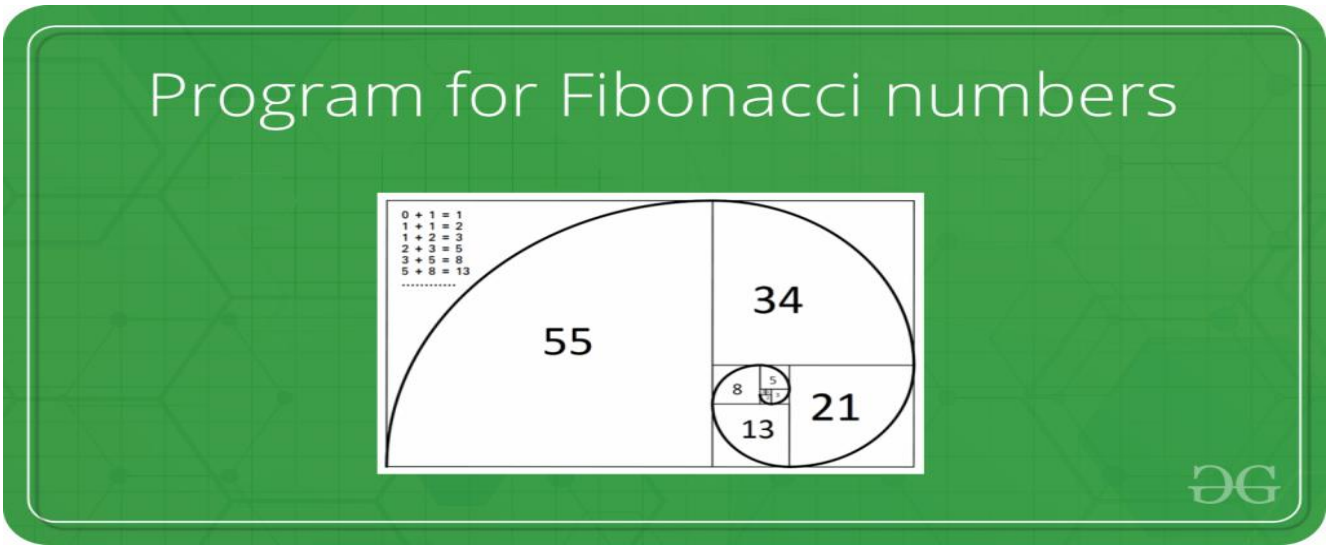
**Explanation:**

- "a" and "b" store the two numbers whose LCM is to be calculated, and "x" and "y" are used to find the GCD without changing the original values.
- while loop uses the Euclidean Algorithm to repeatedly divide and replace values until y becomes 0, leaving the GCD in x.
- value of x is stored in gcd, and the LCM is calculated using the formula LCM = (a × b) / GCD.

**Fibonacci Numbers** :- The Fibonacci sequence is a series of numbers where each number is the sum of the two preceding ones, starting from 0 and 1. 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, ……..

$F_n = F_{n-1} + F_{n-2}$ with seed values

$F_0 = 0$ *and* $F_1 = 1$.




**Problem Statement** Given a number n, print the first n numbers of the Fibonacci sequence.

Examples: *Input : n = 2*

*Output : 0, 1*

*Input : n = 9*

*Output : 0, 1, 1, 2, 3, 5, 8, 13, 21*

**Steps to Solve**

- Initialize the first two Fibonacci numbers (a = 0, b = 1).
- Print the first two numbers.
- Use a loop to calculate subsequent numbers up to the given value of n.
- Update the values of a and b in each iteration to compute the next Fibonacci number.

**C++ Code to Generate Fibonacci Sequence** **Here is the implementation of Fibonacci numbers using an iterative approach:**

```
#include<iostream>
using namespace std;
   int main()
{
   int n;
   cin >> n;
   int a = 0, b = 1, c = 0;
   cout << "0 1 ";
   for(int i = 2; i < n; i++)
   {
      c = a + b;
      cout << c << ' ';
      a = b;
      b = c;
   }
   return 0;
}
```

**INPUT :** 9 **OUTPUT :**0 1 1 2 3 5 8 13 21

***Check for Prime*** *:- A **prime number** is a natural number greater than **1**, which is only divisible by 1 and itself.*

- There are many prime numbers, such as 2, 3, 5, 7, 11, 13, etc.
- Keep in mind that 1 cannot be either prime or composite.
- The remaining numbers, except for 1, are classified as prime and composite numbers.

**Problem Statement**: Given an integer n, determine whether it is a prime number.

**Input:** An integer n. **Output:**

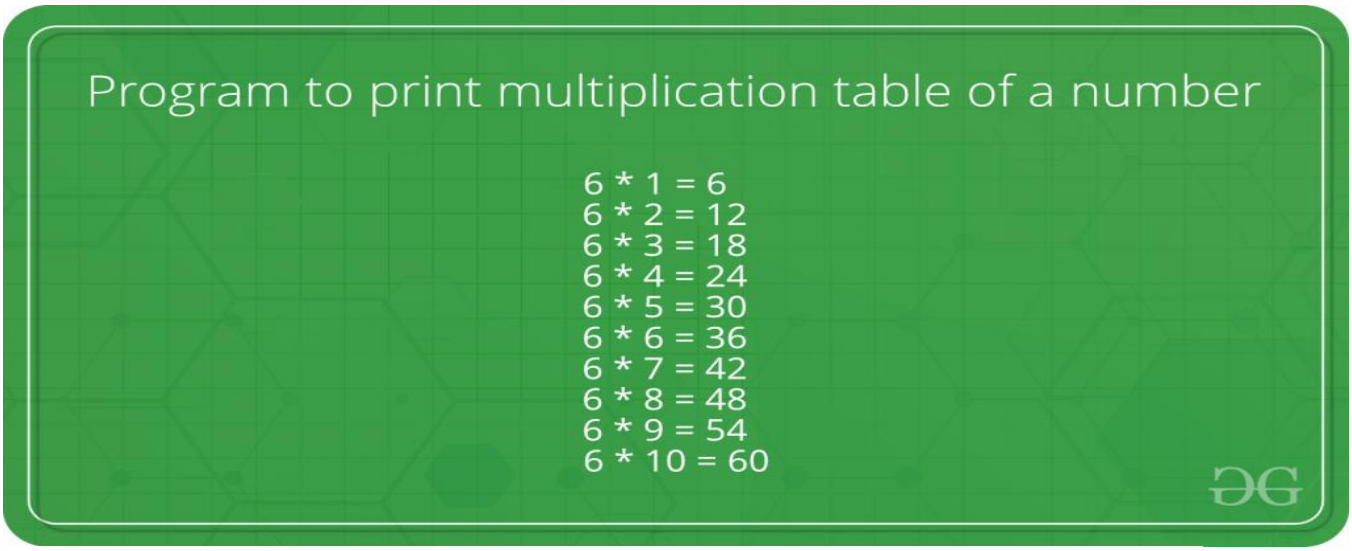
- Print "Yes" if n is prime.
- Print "No" if n is not prime.
- Print "Neither Prime nor Composite" if n is 0 or 1.

```
#include<bits/stdc++.h>
using namespace std;
   int main(){
   int n;
   cin >> n;

   if(n == 0 || n == 1){
      cout << "Neither Prime nor Composite";
      return 0;
   }int i = 2;
   while(i < n){
      if(n % i == 0){
         cout << "No";
         return 0;
      }  i++;
   }  cout << "Yes";
   return 0;
}
```

**INPUT :** 23 **OUTPUT :** Yes **Explanation:**

- The program accepts an integer n as input.
- It first checks if n is 0 or 1, which are neither prime nor composite.
- For other values, it tests divisibility from 2 to n−1.
- If any i satisfies n % i == 0, the program prints No (not prime) and stops.
- If no divisor is found, it prints Yes, indicating that n is a prime number.

**Table of a Number** :- Multiplication tables are a fundamental part of mathematics and programming. They help reinforce concepts of arithmetic and iteration. Given a number n as input, we need to print its table.




Input : 5

Output : 5 * 1 = 5

5 * 2 = 10

5 * 3 = 15

5 * 4 = 20

5 * 5 = 25

5 * 6 = 30

5 * 7 = 35

5 * 8 = 40

5 * 9 = 45

```
#include<iostream>
using namespace std;
   int main() {
   int n, m;
   cin >> n >> m;
   for(int i = 1; i <= m; i++)
   {
     cout << n*i << '\n';
   }
   return 0;
}
```

**INPUT :5 6**

**OUTPUT :**

```
5
10
15
20
25
30
```

**How It Works**

1. **Input Two Numbers**:

- The program first takes two integers as input:
  - n: the number whose multiplication table is to be generated.
  - m: the range up to which the table should be printed.

2. **Iteration with a for Loop**:
   - The for loop runs from 1 to m. Each iteration multiplies the number n by the current loop counter i and prints the result.
3. **Output**:
   - The multiplication result is printed line by line, creating the table.

## Binary to Decimal:- Understanding Binary to Decimal Conversion

The binary numeral system (base-2) represents numbers using two digits: 0 and 1. Each digit in a binary number is a power of 2, starting from 202^020 on the right. To convert a binary number to decimal, we calculate the sum of products of each binary digit and its corresponding power of 2. Example:

Binary: 101

Decimal: 1×2^2 + 0×2^1+1×2^0 = 4+0+1=5

Binary: 11

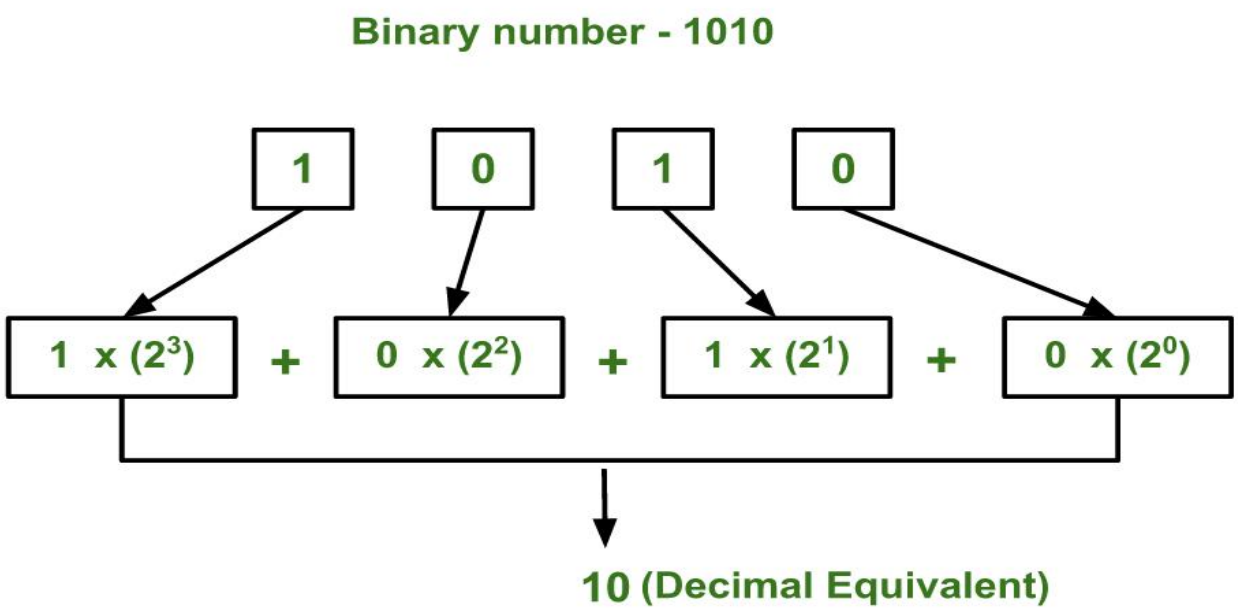
Decimal:1×2^1+1×2^0=2+1=3

**Problem Statement** :-Given a binary number as input, we need to write a program to convert the given binary number into an equivalent decimal number. Examples :

**Input :** 111 **Output :** 7

**Input :** 1010 **Output :** 10

**Input:** 100001
**Output:** 33




```
#include<iostream>

using namespace std;
```

```
int main()
{
  int n, val = 0, base = 1;
  cin >> n;
  while(n > 0)
  {
    int lastDigit = n % 10;  // Extract the last digit of the binary number
    val += (lastDigit * base); // Update the decimal value
    n /= 10;              // Remove the last digit from binary number
    base *= 2;            // Update the base for the next power of 2
  }
 cout << val;
  return 0;
}
```

**INPUT : 1010 OUTPUT :10**

**Next Prime** :-Number Given an integer N. The task is to find the next prime number i.e. the smallest prime number greater than N.

**Steps:**

- Start checking from n + 1 as the next possible prime number.
- Check whether the number is divisible by any value from 2 to n − 1.
- If it is divisible by any number, it is not a prime.
- Keep increasing the number until a prime number is found. **Examples:**

***Input:*** *N = 10* ***Output:*** *11*

*11 is the smallest prime number greater than 10.*

***Input:*** *N = 0* ***Output:*** *2*

```
#include <bits/stdc++.h>
using namespace std;
  int main(){
  int n;
  cin >> n;
  n++;
  while (true){
    int i;
    for (i = 2; i < n; i++){
      if (n % i == 0)
        break;
    }
    if (i == n){
      cout << n << endl;
      break;
    }
    n++;
  }
  return 0;
}
```

**INPUT :15 OUTPUT : 17**

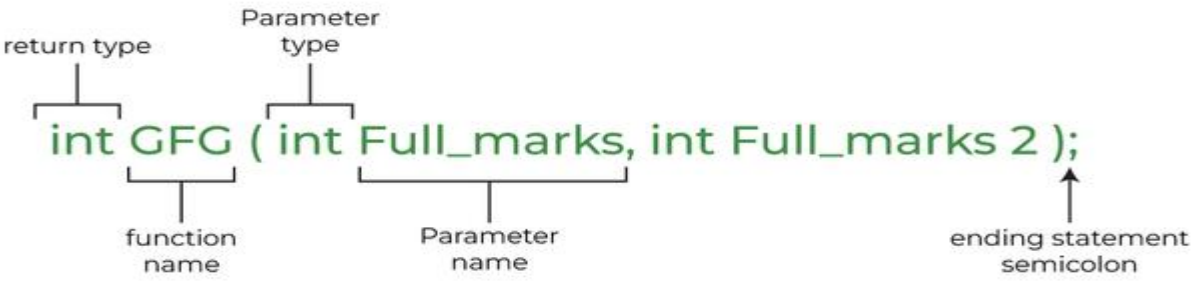
**Explanation:**

- The user enters 15. The program first increases it to 16 to start checking from the next number.
- The for loop checks whether 16 is divisible by any number from 2 to 15. Since 16 % 2 == 0, it is not a prime, so the program increases n to 17.

- Now 17 is checked for divisibility from 2 to 16. None of the divisions give remainder 0, so 17 is identified as a prime number.
- The program prints 17 and stops, which is the next prime number after 15.

**Functions in C++**:- Functions are one of the most essential features in C++ (and most programming languages), offering a modular way to write, organize, and maintain code. By encapsulating specific tasks within functions, developers can reduce redundancy, improve code readability, and simplify debugging. This article delves into the concept of functions in C++, exploring their syntax, use cases, and benefits.

**Syntax:**




**Key Benefits of Functions:**

1. **Code Reusability:** Write once, use multiple times.
2. **Reduced Redundancy:** Avoid repeating similar code in multiple places.
3. **Ease of Maintenance:** Changes in the function logic need to be done only once, and they automatically reflect wherever the function is called.
4. **Improved Readability:** Functions with meaningful names help make code self-explanatory.

**Example 1: A Simple Function**

```
#include <iostream>
using namespace std;
   void fun() {
   cout << "Fun() Called\n";
}
   int main() {
   cout << "Before calling fun()\n";
   fun();
   cout << "After calling fun()\n";
   return 0;
}
```

**Output:**

```
Before calling fun()
Fun() Called
After calling fun()
```

**Explanation:**

- The function fun() is defined to print a message.

- In the main() function, fun() is called, executing its logic.
- Once the function completes, control returns to the caller (main()), which continues execution.

**Example 2: Function with Multiple Calls**

```
#include <iostream>
using namespace std;
   void fun() {
   cout << "Fun() Called\n";
}
   int main() {
   cout << "Before calling fun()\n";
   fun();
   fun();
   cout << "After calling fun()\n";
   return 0;
}
```

**Output:**

```
Before calling fun()
Fun() Called
Fun() Called
After calling fun()
```

This demonstrates that a function can be called multiple times, each time executing its logic.

**Functions with Parameters and Return Values:-**A function can also take inputs (parameters) and return a result. This enhances flexibility and reusability.

Example 3: Finding the Maximum of Two Numbers

```
#include <iostream>
using namespace std;
   int getMax(int x, int y) {
 if (x > y)
       return x;
   else
       return y;
}
   int main() {
   int a = 10, b = 20;
   cout << getMax(a, b);
   return 0;
}
```

**Output:** 20

**Explanation:**

- The function getMax() takes two integers and returns the larger of the two.
- The main() function calls getMax() with a and b as arguments.

**Example: Modular Code with Multiple Functions**

Here's an example with two functions: greetMsg() for a greeting message and exitMsg() for a goodbye message.

```
#include <iostream>
```

```
using namespace std;
   void greetMsg() {
   cout << "Hi,\n";
   cout << "Welcome to GeeksforGeeks\n";
}
   void exitMsg() {
   cout << "Bye,\n";
   cout << "Visit again\n";
}
   int main() {
   greetMsg();
   cout << "Hope you are enjoying\n";
   exitMsg();
   return 0;
}
```

**Output:**

```
Hi,
Welcome to GeeksforGeeks
Hope you are enjoying
Bye,
Visit again
```

**Explanation:**

- greetMsg() is called at the start, printing a welcome message.
- After an intermediate message in the main() function, exitMsg() is called, printing a goodbye message.

**Why Do We Need Functions?**

- Functions help us in ***reducing code redundancy***. If functionality is performed at multiple places in software, then rather than writing the same code, again and again, we create a function and call it everywhere. This also helps in maintenance as we have to change at one place if we make future changes to the functionality.
- Functions make code ***modular***. Consider a big file having many lines of code. It becomes really simple to read and use the code if the code is divided into functions.
- Functions provide ***abstraction***. For example, we can use library functions without worrying about their internal work.

**How Functions Work in C**++?:-When working with C++ programs, understanding how functions are executed internally is critical. Let's explore this concept using an example program and the concept of the function call stack.

**The Code Example :-**Below is the program that demonstrates function calls in C++:

```
#include <iostream>
using namespace std;
   void fun2() {
   cout << "Inside fun2()\n";
}
   void fun1() {
   cout << "Before fun2()\n";
   fun2();
   cout << "After fun2()\n";
}
   int main() {
   cout << "Before fun1()\n";
```

```
    fun1();
    cout << "After fun1()\n";
    return 0;
}
```

**Output of the Program**

The output of this program will be:

```
Before fun1()
Before fun2()
Inside fun2()
After fun2()
After fun1()
```

**Program Execution Flow:-**The program consists of three functions: main, fun1, and fun2. Here's the step-by-step flow of execution:

1. The main function starts execution.
2. main calls fun1 and pauses its execution.
3. fun1 begins execution and prints "Before fun2()".
4. fun1 calls fun2 and pauses its execution.
5. fun2 begins execution, prints "Inside fun2()", and completes.
6. Control returns to fun1, which resumes execution and prints "After fun2()". Then, fun1 finishes.
7. Control returns to main, which resumes execution and prints "After fun1()". Finally, main finishes.

**Function Call Stack**:-The **function call stack** is a key concept to understand how function calls work in C++:

1. **What is a Stack?**
   - A stack is a data structure with a Last In, First Out (LIFO) property. Elements are added and removed from the top of the stack.
2. **How Function Calls Use a Stack**
   - When a function is called, an entry is pushed onto the function call stack.
   - This entry contains:
     - The function's local variables.
     - The current execution state of the function (e.g., the next line to execute).
   - When the function completes, its entry is removed (popped) from the stack, and control returns to the caller function.

**Step-by-Step Function Call Stack Execution:-**Let's see how the stack evolves for the provided program:

1. **Initial State**
   - main starts, and its entry is pushed onto the stack.
2. **Calling fun1**
   - main calls fun1.
   - The state of main is saved (paused), and an entry for fun1 is pushed onto the stack.
3. **Calling fun2**
   - fun1 calls fun2.
   - The state of fun1 is saved, and an entry for fun2 is pushed onto the stack.
4. **Completion of fun2**
   - fun2 completes execution and is popped from the stack.
   - Control returns to fun1.
5. **Completion of fun1**
   - fun1 completes execution and is popped from the stack.
   - Control returns to main.
6. **Completion of main**
   - main completes execution and is popped from the stack.

**Visualizing the Stack** Here's how the stack evolves during execution:

| Step | Stack (Top to bottom) |
|---|---|
| Start main | main |
| main calls fun1 | fun1,main |
| fun1 calls fun2 | fun2,fun1,main |
| fun2 completes | fun1,main |
| fun1 completes | main |
| main completes | (Empty) |

**Key Points to Remember**

1. The function call stack ensures proper sequencing of function calls and returns.
2. When a function is called, its execution is suspended until the called function completes.
3. The LIFO nature of the stack allows the most recently called function to complete first.
4. Local variables and execution state are stored in the stack for each function call.

**Applications of Functions:-**Functions play a crucial role in programming by enhancing code efficiency, readability, and maintainability. Here's an overview of functions' various applications and why they are essential in software development.

**Avoiding Code Redundancy:-**There are certain functionalities are needed repeatedly. For example, printing an array of data, sorting the array, and searching for an item in the array.
Instead of rewriting the same code every time, you can create dedicated functions. For instance:

- printArray() function handles printing the array.
- sortArray() function handles sorting.
- searchArray() function handles searching.

**By using functions, you:**

- **Avoid Redundancy:** Write the logic once and reuse it as needed.
- **Simplify Maintenance:** If a functionality changes, you only update the function definition, and all calls automatically reflect the update.

**Example**: If a company prints its tagline on every form:

- **Without functions**: You'd need to update every occurrence if the tagline changes.
- With a printTagline() function, a single update applies universally, reducing errors and ensuring consistency.

**Make Code Modular** :-Functions make code modular, readable, and maintainable.

**Example**: You write a program, that takes a large amount of data, then process the data and produces some useful data. To handle this thing you divide the whole process into functions. Like:

- **takeInputu():** To take input from users or somewhere.
- **processData():** This function performs some operation of input data.
- **produceOutput():** In this function, we simply return the data or result after processing the data.

By creating separate functions for these tasks, you make the program more understandable. Anyone reading the code can quickly grasp its structure and functionality.

**Abstraction**:-Functions provide abstraction by hiding implementation details. Library functions are a prime example. For example:

- When using printf() to display output, you don't need to understand its internal workings.
- Sorting functions in libraries (like qsort in C) can be updated or optimized without affecting your code.

**Avoiding Variable Name Collisions** :-When everything is written in a single block of code, you may run out of unique variable names. Functions help avoid this problem by introducing scope. Each function has its local variables, meaning two functions can use the same variable name without conflict.

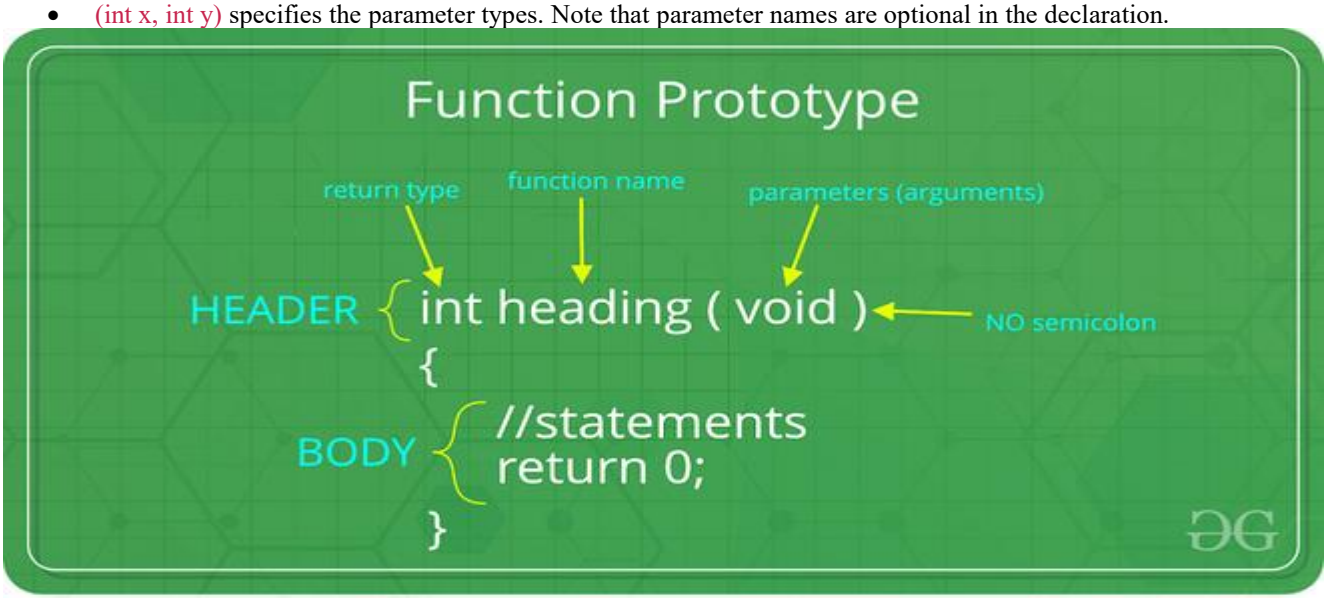
**Function declaration & definition**:- C++, functions are essential tools that allow programmers to write reusable, efficient, and organized code. To use functions effectively, it is important to understand their declaration, definition, and usage. Let's break this down step by step.

**Function Declaration** **Just like variables need to be declared before they are used, functions in C++ must also be declared to inform the compiler about their existence, the number and types of parameters, and their return type.**

A function declaration (also called a prototype) tells the compiler about the function's return type, name, and parameters without providing the implementation. It is typically written before the main function or in a header file.**Example:**

```
int getMax(int x, int y); // Declaration or Prototype   Here:
```

- int specifies the return type of the function.
- getMax is the name of the function.
- (int x, int y) specifies the parameter types. Note that parameter names are optional in the declaration.




**Example Program**

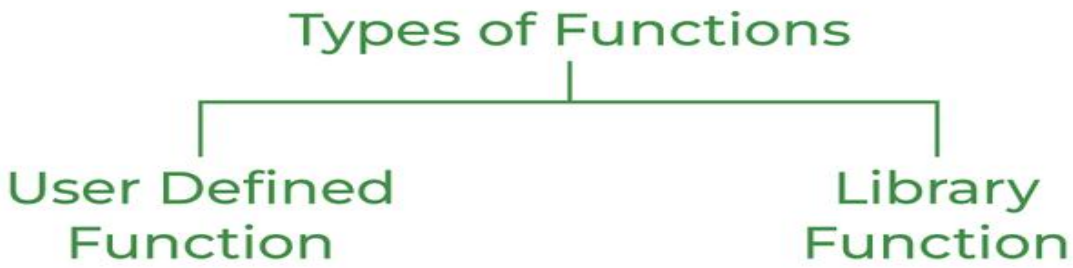
```
#include <iostream>
using namespace std;
```

```
   int getMax(int x, int y); // Declaration
   int main() {
   int a = 10, b = 20;
   cout << getMax(a, b); // Using the function
   return 0;
}
   int getMax(int x, int y) { // Definition
   if (x > y)
      return x;
   else
      return y;
} Output: 20
```

**Importance of Declaration in C++ :-**In C++, if you try to use a function before declaring or defining it, the compiler will throw an error because it does not know the function's signature.

- If a function is **declared** before its usage but not defined, a **linker error** will occur during compilation.
- If a function is **used** without being declared or defined, a **compiler error** will occur.

**Types of Functions**




**User Defined Function** :-User Defined functions are user/customer-defined blocks of code specially customized to reduce the complexity of big programs. They are also commonly known as "***tailor-made functions***" which are built only to satisfy the condition in which the user is facing issues meanwhile reducing the complexity of the whole program.

**Library Function :-**In addition to creating your own functions, C++ provides numerous built-in functions in libraries, such as those in the <cmath> header file. These functions are pre-declared in the respective libraries and can be used after including the relevant header. Example:

```
#include <iostream>
#include <cmath> // Header file for math functions
using namespace std;
   int main() {
   double a = 2, b = 4;
   cout << pow(a, b) << endl; // 2^4 = 16
double x = 100;
   cout << log10(x) << endl; // log10(100) = 2
     return 0;
}
```

**Output:**

```
16
2
```

Explanation:

- pow(a, b): Raises a to the power b.
- log10(x): Returns the base-10 logarithm of x.

**How Library Functions Work** :-Library functions are efficient and tested. They are declared in header files (e.g., <cmath> for mathematical functions). When you include a header file, its function declarations are made available to the program. During linking, the compiler matches the function calls with their definitions provided by the library.

**Key Points to Remember**

1. **Function Declaration:** Specifies the function's name, parameters, and return type. Optional in some cases if the function is defined before usage.
2. **Function Definition:** Provides the actual logic of the function.
3. **Compiler and Linker Errors:**
   - Missing declaration: Compiler error.
   - Missing definition: Linker error.
4. **Library Functions:** Use pre-declared and pre-defined functions to save time and ensure reliability.

Default Arguments in C++Default arguments are a feature in C++ that allows a function to be called without explicitly providing values for some of its parameters. If no value is provided for a parameter, the function automatically assigns it a pre-defined default value. Let's explore this concept in detail.

**What Are Default Arguments**? Default arguments allow flexibility in function calls by assigning a default value to one or more parameters during the function declaration or definition. If the caller does not provide a value for these parameters, the default value is used.

**Example:**

```
#include <iostream>
using namespace std;
   void printDetails(int id, string name = "NA", string address = "NA") {
   cout << "Id is " << id << "\n";
   cout << "Name is " << name << "\n";
   cout << "Address is " << address << "\n";
}
   int main() {
   printDetails(101, "Sandeep", "Noida");
   cout << "\n";
   printDetails(201, "Shivam");
   cout << "\n";
   printDetails(301);
   return 0;
}
```

**Output:**

```
Id is 101
Name is Sandeep
Address is Noida

Id is 201
Name is Shivam
Address is NA
```

```
Id is 301
Name is NA
Address is NA
```

In this example:

- The printDetails function has default values "NA" for the name and address parameters.
- When all arguments are provided, the default values are ignored.
- If fewer arguments are provided, the remaining parameters take their default values.

**Rules for Using Default Arguments**

*1. **Default Arguments Must Appear at the End***

All default arguments must be declared after non-default arguments. This ensures that the compiler can correctly assign values to parameters.

**Incorrecct Code:**

```
void printDetails(string name = "NA", int id) { // Compiler error
   cout << "Id is " << id << "\n";
   cout << "Name is " << name << "\n";
}
```

**Output:** Compiler error.**Correct Code:**

```
void printDetails(int id, string name = "NA") {
   cout << "Id is " << id << "\n";
   cout << "Name is " << name << "\n";
}
```

*2. **Default Values Can Be Provided in Declaration or Definition***

You can specify default values in either the declaration or the definition of the function, but not both.

**Example:**

```
#include <iostream>
using namespace std;
   int sum(int a, int b, int c = 0, int d = 0); // Declaration with defaults
   int main() {
   cout << sum(10, 20, 30) << "\n"; // Output: 60
   cout << sum(10, 5) << "\n";      // Output: 15
   return 0;
}
   int sum(int a, int b, int c, int d) { // Definition without defaults
   return (a + b + c + d);
} Output:
60
15
```

*Key Point:*

- If default arguments are provided in the declaration, they should not be repeated in the definition.

- The preferred approach is to provide defaults in the declaration, as it helps with documentation and readability.

**Inline Functions in C++**:- Inline functions in C++ are a feature designed to optimize program execution by reducing the overhead associated with function calls. By suggesting the compiler to replace a function call with the actual code of the function, inline functions can help improve performance, especially for small, frequently used functions.

**What Are Inline Functions**? An inline function is a special type of function where the compiler attempts to replace the function call with the function's body during compilation. This avoids the overhead of function calls, such as parameter passing, stack operations, and returning values.

**Syntax:** To make a function inline, use the inline keyword before the function definition.

**Example:**

```
#include <iostream>
using namespace std;
   inline int getMax(int x, int y) {
   return (x > y) ? x : y;
}
   int main() {
   cout << getMax(10, 20); // Output: 20
   return 0;
}
```

In this example:

- The getMax function is defined as inline.
- During compilation, the compiler replaces getMax(10, 20) with (10 > 20) ? 10 : 20 to avoid the function call overhead.

**Why Use Inline Functions?**

1. **Optimization for Small Functions** For small functions, the overhead of function calls can be significant compared to the actual execution of the function body. Inline functions eliminate this overhead by directly placing the function code at the call site.
2. **Improves Execution Speed** By avoiding function calls, inline functions can improve the execution speed of the program, especially when the function is called multiple times in performance-critical sections.

**Inline Functions vs. Macros Inline** functions are often compared to macros because both can replace code at the call site. However, inline functions have significant advantages over macros:

1. **Type Checking** Inline functions perform type checking, while macros do not. This makes inline functions safer and less error-prone.
2. **Scope and Syntax** Inline functions follow the same scoping rules as regular functions, whereas macros are simple text replacements that can lead to unexpected behavior if not written carefully.
3. **Example Comparison:**

```
#define add(x, y) x + y
int main() {
cout << 4 * add(10, 20); // Output: 60 (Unexpected result due to lack of parentheses)
 return 0;
```

```
}
```

5. Here, the macro expands to 4 * 10 + 20, which evaluates as 40 + 20 = 60 instead of the expected 4 * (10 + 20) = 120. This issue does not occur with inline functions.

**Important Facts About Inline Functions**

1. **Compiler's Choice** Declaring a function as inline is merely a suggestion to the compiler. The compiler may decide not to inline the function if the function body is too large or if inlining would increase the program size significantly.
2. **Binary File Size** Overusing inline functions in large programs can increase the size of the binary executable because the function code is copied at every call site.
3. **Inline Functions in Classes** When you define a function inside a class, it is automatically considered an inline function.
4. **Modern Compiler Behavior** Modern compilers may inline some functions even if they are not explicitly marked as inline. Conversely, they may ignore the inline keyword for functions that are unsuitable for inlining.

**Function Overloading** :-Function overloading is a concept in C++ that allows multiple functions with the same name but different parameter lists. It enhances code readability and reusability, enabling developers to use the same name for functions performing similar tasks with different types or numbers of parameters.

**What is Function Overloading?:-**Function overloading allows defining multiple versions of a function where the **number** or **type** of parameters differs. When you invoke a function, the compiler determines which version to execute based on the arguments passed. This feature eliminates the need for unique function names, making the code more intuitive and manageable.Example 1

```
#include <iostream>
using namespace std;  // Function to find the maximum of two integers
int myMax(int x, int y) {
   return (x > y) ? x : y;  // Ternary operator to find the maximum
}          // Function to find the maximum of three integers
int myMax(int x, int y, int z) {
   return myMax(myMax(x, y), z);  // Compare maximum of x and y with z
}
   int main() {
   int a = 10, b = 30, c = 5;     // Call to the two-parameter version of myMax
   cout << myMax(a, b) << "\n";   // Call to the three-parameter version of myMax
   cout << myMax(a, b, c) << "\n";
    return 0;
}
```

**Output:**30
30

***Explanation***

1. **myMax(int x, int y)**: This function takes two integers and returns the larger value using the ternary operator.
2. **myMax(int x, int y, int z)**: This function calculates the maximum of three integers by first finding the maximum of x and y using myMax(int x, int y) and then comparing the result with z.
3. **In main()**: The appropriate function is invoked based on the number of arguments passed:
   - Passing two integers calls the two-parameter version.
   - Passing three integers calls the three-parameter version.

**Example 2**

```
#include <iostream>
#include <string>
using namespace std;    // Function to print a string
```

```
void print(string s) {
 cout << s << "\n";
}     // Function to print an integer
void print(int x) {
cout << x << "\n";
}    // Function to print a character
void print(char c) {
 cout << c << "\n";
}
int main() {
 print('a');  // Calls the print(char) function
 print(10);   // Calls the print(int) function
 print("GfG");  // Calls the print(string) function
 return 0;
}
```

**Output:**

```
a
10
GfG
```

***Explanation***

1. **print(string s)**: Prints the input string.
2. **print(int x)**: Prints the input integer.
3. **print(char c)**: Prints the input character.
4. **In main()**:
    - The compiler determines the function to call based on the type of the argument:
        - 'a' calls print(char c).
        - 10 calls print(int x).
        - "Hello, World!" (C-style string) automatically converts to a std::string and calls print(string s).

**Handling Implicit Type Conversion:-**If the print(char c) function is removed, calling print('a') will invoke print(int x) because a char can be implicitly converted to an int. The ASCII value of 'a' (97) will be printed instead of the character.

```
#include <iostream>
#include <string>
using namespace std;
   void print(string s) {
 cout <<s << "\n";
}
   void print(int x) {
   cout << x << "\n";
}
   int main() {
   print('a');  // 'a' is implicitly converted to int, printing ASCII value
   print(10);
   print("GfG");
 return 0;
}
```

**Output:**97

10

GfG

**Important Notes About Function Overloading**

1. **Parameters Must Differ**: Overloading relies on differences in parameter types or counts. Functions cannot be overloaded based solely on the return type.
```
void print(int x);

int print(int x); // Error: Functions cannot differ only in return type.
```
3. **Automatic Type Conversion**: If no exact match is found, the compiler performs implicit type conversion to find the best match.
4. **Templates vs. Overloading**:
   - Use **function overloading** when logic differs for each implementation.
   - Use **templates** when logic remains the same but supports different data types. For instance, sorting integers, floats, or strings using the same algorithm.

**Default Arguments in Function Overloading**

Default arguments are parameters with predefined values. If a value for the parameter is not provided during the function call, the default value is used. While default arguments enhance function flexibility, they may introduce ambiguity when multiple overloaded functions exist.

**Example of Ambiguity in Function Overloading**

```
#include <iostream>
using namespace std;    // Function with a default argument
void print(int x = 0) {
cout << x << "\n";
}  // Overloaded function with no arguments
void print() {
 cout << "No arguments" << "\n";
}
   int main() {
   print();  // Ambiguous call
   return 0;
}
```

***Explanation***

1. **print(int x = 0)**: This function accepts an integer parameter with a default value of 0. If no argument is provided during the function call, the value 0 is used.
2. **print()**: This overloaded function takes no arguments and prints a fixed message.
3. **Function Call in main()**:
   - The call to print() is ambiguous because both functions are potential matches:
     - The first function can be called with the default value 0.
     - The second function can also be called, as it explicitly takes no arguments.
   - The compiler cannot decide which function to invoke, resulting in an error.

**Practice Problems on C++ FunctionsProblem 1: Undefined Function Call**

```
#include <iostream>
using namespace std;
   void fun(int, string="NA");
   int main() {
   fun(101);
   return 0;
}
   void fun(int id, string name ) {
   cout << id << " " << name;
}
```

**Output**

```
101 NA
```

**Key Points:**

- Initially, this program caused a linker error because the function fun was called before being declared.
- The issue was resolved by declaring the function prototype before main.
- The output of this program is 101 NA because the name parameter uses the default value "NA."
- **Best Practice:** Default arguments should be specified in the function declaration, not in the definition, although the compiler allows it.

**Problem 2: Static vs. Local Variables**

```
#include <iostream>
using namespace std;
   void fun() {
   static int x = 1;
   int y = 1;
   x++;
   y++;
   cout << x << " " << y << endl;
}
   int main() {
   fun();
   fun();
   fun();
   return 0;
}
```

**Output**

```
2 2
3 2
4 2
```

**Key Points:**

- x is a static variable, so its value persists across function calls.
- y is a local variable, so it is reinitialized each time the function is called.
- **Execution:**
  - First call: x = 2, y = 2 (incremented from their initial values).
  - Second call: x = 3, y = 2 (static x retains its value, but y is reinitialized).
  - Third call: x = 4, y = 2.

**Problem 3: Compiler-Dependent Evaluation Order**

```
#include <iostream>
using namespace std;
   void fun(int x, int y, int z) {
   cout << x << " " << y << " " << z;
}
   int main() {
   int i = 2;
   fun(++i, ++i, ++i);
   return 0;
}
```

**Output**

```
5 5 5
```

**Key Points:**

- The output of this program is compiler-dependent due to the order of evaluation of function arguments.

- The standard does not define whether arguments are evaluated from left to right or right to left.
- Example outcomes:
  - If evaluated left to right: x = 3, y = 4, z = 5.
  - If evaluated right to left: x = 5, y = 4, z = 3.
- **Best Practice:** Avoid writing such code to ensure predictable behavior.

**Problem 4: Recursive Function Example**

```
#include <iostream>
using namespace std;
   void fun(int x) {
   if (x == 0)
      return;
   else {
      cout << "gfg\n";
      fun(x - 1);
   }
}
   int main() {
   fun(3);
   return 0;
}
```

**Output**

```
gfg
gfg
gfg
```

**Key Points:**

- The function fun is recursive and calls itself until the base case (x == 0) is met.
- Execution flow:
  1. fun(3) prints "gfg" and calls fun(2).
  2. fun(2) prints "gfg" and calls fun(1).
  3. fun(1) prints "gfg" and calls fun(0).
  4. fun(0) meets the base case and returns.

**Problem 5: Infinite Recursion**

```
#include <iostream>
using namespace std;
   int main() {
   cout << "gfg";
   main();
   return 0;
}
```

**Key Points:**

- This program makes the main function call itself recursively, leading to infinite recursion.
- There is no base case to terminate the recursion.
- **Outcome:**
  - It keeps printing "gfg" until the stack overflows, causing a runtime error (segmentation fault).

**First Digit of a Number**:- The first digit of a number is simply the leftmost digit in the number. For example:

- The first digit of 9768 is 9.
- The first digit of 763 is 7.

The key idea is to isolate the leftmost digit by dividing the number repeatedly by 10 until it becomes a single-digit number. **Steps to find first digit**

- Start with the given number.
- Divide the number by 10 repeatedly.
- Continue dividing until the number becomes less than 10.
- The remaining single digit is the first digit of the number.
- Example: 9768 → 976 → 97 → 9 (first digit is 9).

**Code Example**

```
#include <bits/stdc++.h>
using namespace std;
   int find_first_digit(int n) {
   while (n >= 10) { // Loop until the number becomes a single digit
      n = n / 10;
   }
   return n; // Return the single-digit number
}
   int main() {
   int n;
   cin >> n;
   cout << find_first_digit(n);
   return 0;
}
```

**Input:***9768* **Output:***9*

**Explanation:**

- The user enters 9768, which is stored in "n".
- The function "find_first_digit(9768)" is called.
- Inside the while loop, 9768 is divided by 10 to become 976, then 97, and finally 9.
- When the value becomes 9 (less than 10), the loop stops.

The first digit of a number is simply the leftmost digit in the number. For example:

- The first digit of 9768 is 9.
- The first digit of 763 is 7.

The key idea is to isolate the leftmost digit by dividing the number repeatedly by 10 until it becomes a single-digit number.

**Steps to find first digit**

- Start with the given number.
- Divide the number by 10 repeatedly.
- Continue dividing until the number becomes less than 10.
- The remaining single digit is the first digit of the number.
- Example: 9768 → 976 → 97 → 9 (first digit is 9).

**Code Example**

```
#include <bits/stdc++.h>
using namespace std;
   int find_first_digit(int n) {
   while (n >= 10) { // Loop until the number becomes a single digit
      n = n / 10;
```

```
    }
    return n; // Return the single-digit number
}
    int main() {
    int n;
    cin >> n;
    cout << find_first_digit(n);
    return 0;
}
```

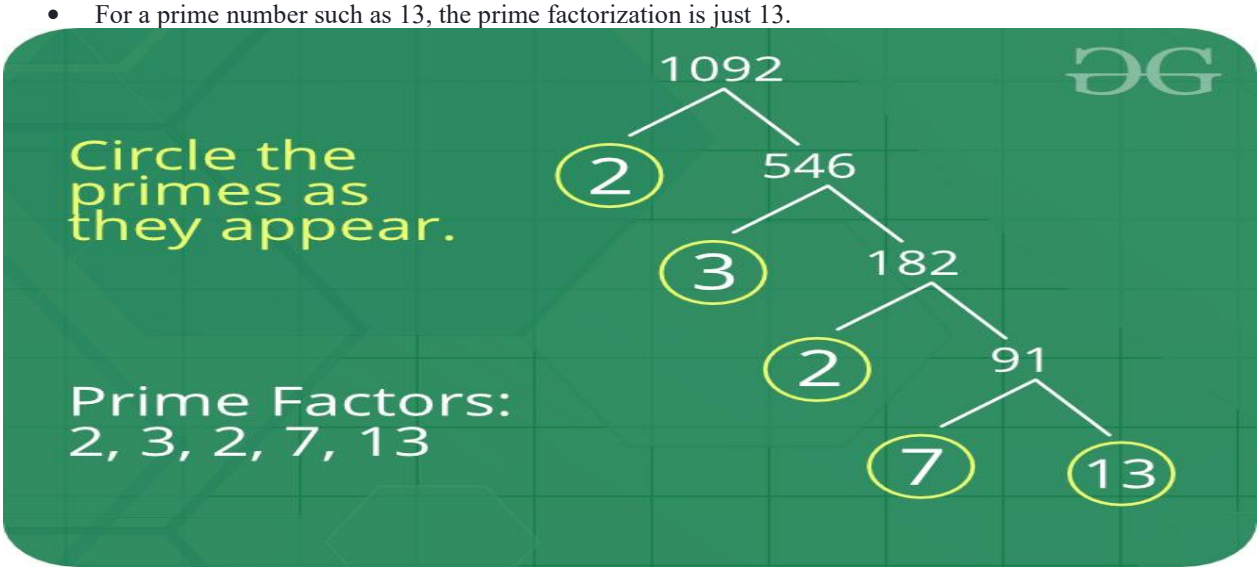
**Input:** *9768* **Output:** *9*

**Explanation:**

- The user enters 9768, which is stored in "n".
- The function "find_first_digit(9768)" is called.
- Inside the while loop, 9768 is divided by 10 to become 976, then 97, and finally 9.
- When the value becomes 9 (less than 10), the loop stops.

**Prime Factorization**: - Prime factorization is a mathematical process used to express a given number as a product of its prime factors. For example:

- The prime factorization of 50 is 2×5×5.
- The prime factorization of 20 is 2×2×5.
- For a prime number such as 13, the prime factorization is just 13.




**Steps:**

- Start from number 2 (smallest prime).
- Check each number i up to n.
- If i is prime and n is divisible by i, then i is a prime factor.
- Print i and divide n by i until it is no longer divisible.
- Move to the next number and repeat until n becomes 1.

```
#include <iostream>
using namespace std;
    bool isPrime(int n) {
    for (int i = 2; i < n; i++) {
        if (n % i == 0) {
            return false;
        }
    }
```

```
    return true;
}
    void primeFactors(int n) {
    for (int i = 2; i <= n; i++) {
        if (isPrime(i)) {
            int x = i;
            while (n % x == 0) {
                cout << i << " ";
                x = x * i;
            }
        }
    }
}

int main() {
    int n;
    cout << "Enter a number: ";
    cin >> n;
    primeFactors(n);
    return 0;
}
```

**Input:** *Enter a number: 20*

**Output:***2 2 5*

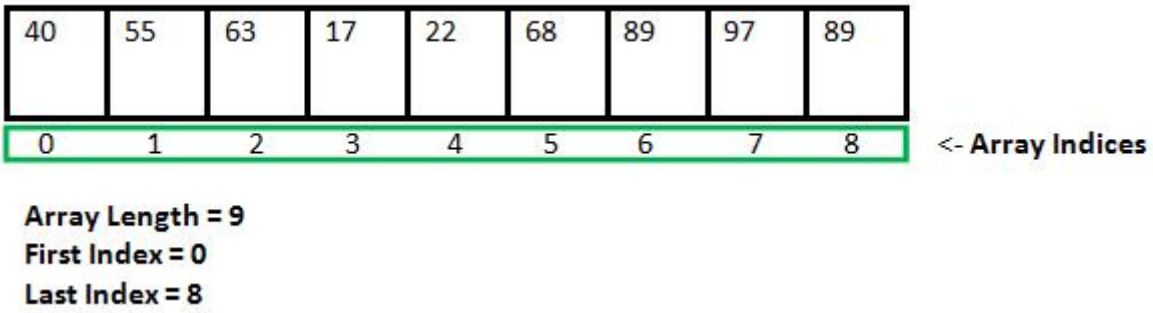
**Explanation:**

- The user enters 20, and primeFactors(20) is called.
- The loop starts checking numbers from 2 to 20.
- For i = 2, since it is prime and divides 20, the program prints 2 twice.
- For i = 5, since it is prime and divides 20, the program prints 5.

## Introduction to Arrays in C++

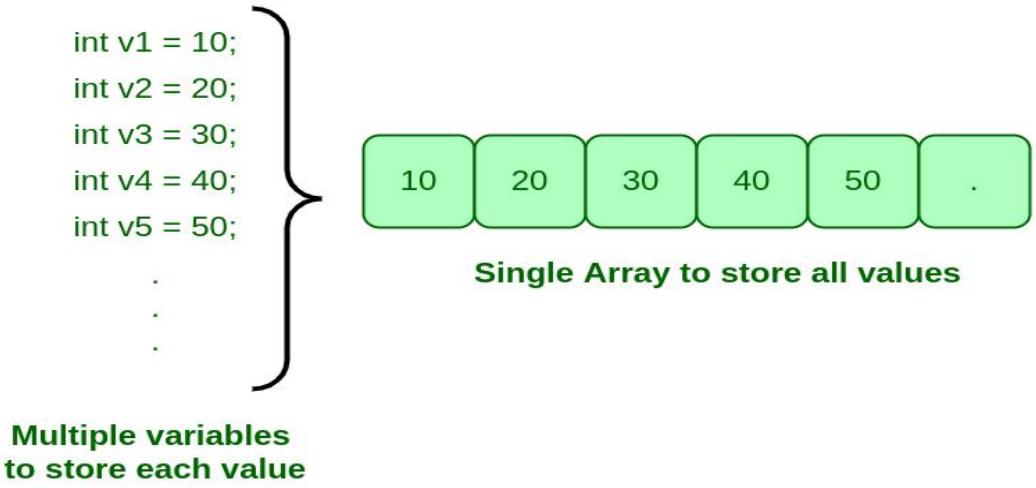
1. It is a group of variables of similar data types referred to by a single element.
2. Its elements are stored in a contiguous memory location.
3. The size of the array should be mentioned while declaring it.
4. Array elements are always counted from zero (0) onward.
5. Array elements can be accessed using the position of the element in the array.
6. The array can have one or more dimensions.

An array in C++ a collection of similar data items stored at contiguous memory locations and elements can be accessed randomly using indices of an array. They can be used to store the collection of primitive data types such as int, float, double, char, etc of any particular type. To add to it, an array in C++ can store derived data types such as structures, pointers etc. Given below is the picture representation of an array.

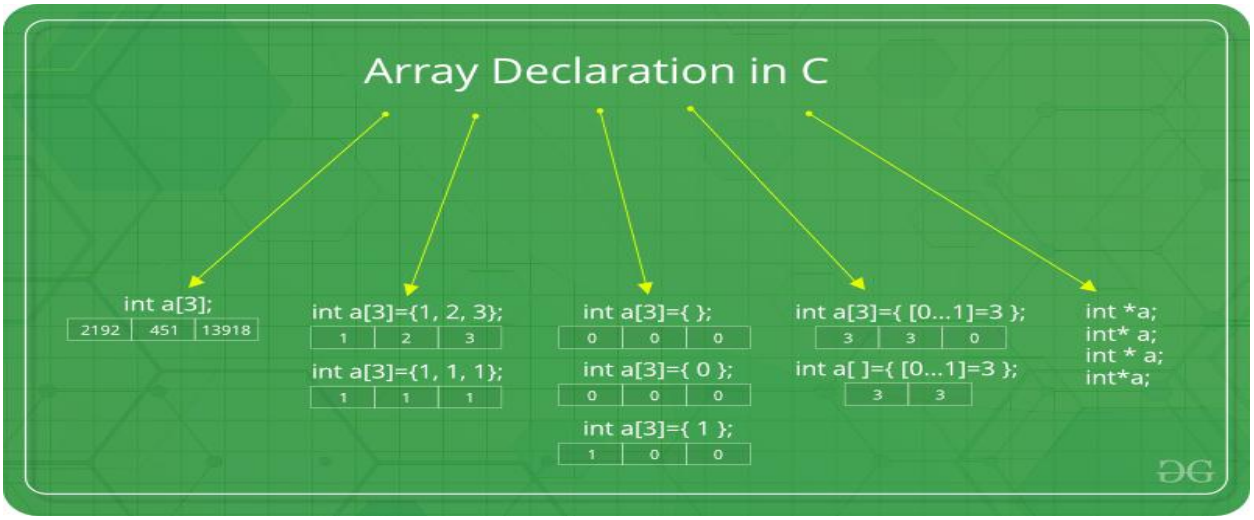



**Why do we need arrays?**

We can use normal variables (v1, v2, v3, ..) when we have a small number of objects, but if we want to store a large number of instances, it becomes difficult to manage them with normal variables. The idea of an array is to represent many instances in one variable.




Array declaration in C++Array declaration in C++ is same as C.




**Note:** In the above image int a[3]={[0…1]=3}; this kind of declaration has been obsolete since GCC 2.5

There are various ways in which we can declare an array. It can be done by specifying its type and size, initializing it or both.

**Array declaration by specifying the size**

```
#include <iostream>
using namespace std;
   int main()
{     // array declaration by specifying size
   int arr1[10];    // With recent C/C++ versions, we can also     // declare an array of user specified size
   int n = 10;
   int arr2[n];
    return 0;
}
```

**Array declaration by initializing elements** // Array declaration by initializing elements

```
#include <iostream>
using namespace std;
int main()
{
 int arr[] = { 10, 20, 30, 40};
   return 0;     // Compiler creates an array of size 4.   // above is same as "int arr[4] = {10, 20, 30, 40}"
}
```

**Array declaration by specifying the size and initializing elements**

```
#include <iostream>
using namespace std;
   int main()
{      // Array declaration by specifying size and initializing   // elements
   int arr[6] = { 10, 20, 30, 40 }       // Compiler creates an array of size 6, initializes first
  ] // 4 elements as specified by user and rest two elements as
   // 0. above is same as "int arr[] = {10, 20, 30, 40, 0, 0}"

   return 0;
}
```

**Advantages of an Array in C/C++:**

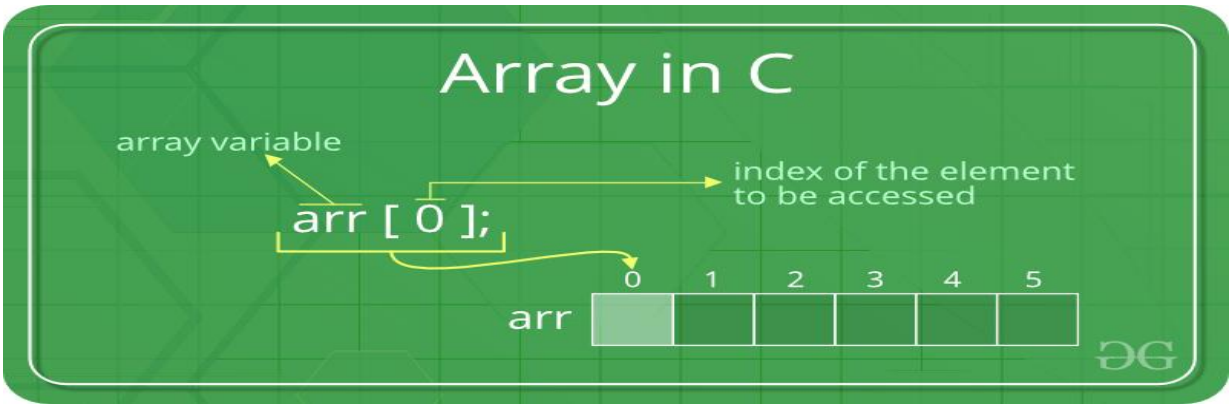
1. Random access of elements using the array index.
2. Use of fewer lines of code as it creates a single array of multiple elements.
3. Easy access to all the elements.
4. Traversal through the array becomes easy using a single loop.
5. Sorting becomes easy as it can be accomplished by writing fewer lines of code.

**Disadvantages of an Array in C/C++:**

1. Allows a fixed number of elements to be entered which is decided at the time of declaration. Unlike a linked list, an array in C is not dynamic.
2. Insertion and deletion of elements can be costly since the elements are needed to be managed in accordance with the new memory allocation.

**Accessing Array Elements in C++Accessing Array Elements:**

- Array elements are accessed by using an integer index. Array index starts with 0 and goes till the size of the array minus 1.Its same as C language.
- The name of the array is also a pointer to the first element of the array.




**Basics of Accessing Array Elements :-**An array's elements can be accessed using their indices. In C++, array indices start from 0 and go up to size - 1.

*Example 1: Accessing and Modifying Elements*

```
#include <iostream>
using namespace std;
   int main() {
  int arr[] = {10, 20, 30, 40};  // Accessing elements
   cout << arr[2] << " " << arr[0] << "\n"; // Output: 30 10    // Modifying elements
   arr[2] = 50;
   cout << arr[2] << "\n"; // Output: 50
     return 0;
}
```

**Output:**30  10

50

**Explanation:**

- The array arr has four elements.
- Accessing arr[2] fetches the third element, which is 30.
- Modifying arr[2] to 50 updates the array, and subsequent accesses to arr[2] reflect this change.

***How Does Array Indexing Work?***

Array indexing works by using the address of the first element and adding an offset based on the index. The size of the data type determines the offset.

**Example 2: Understanding Indexing Internals**

```
#include <iostream>
using namespace std;
   int main() {
   int arr[] = {10, 20, 30, 40};  // Prints the address of the first element
   cout << arr << "\n";    // Accessing the third element
   cout << arr[2] << "\n"; // Output: 30
    return 0;
}
```

**Output:** 0x7fff89aa1610

30

**Explanation:**

- arr points to the address of the first element in memory.
- To access arr[2], the compiler calculates the address as:
  Address of arr[2] = Address of arr[0] + (2 * sizeof(int))
- For a 4-byte integer, this would result in an address offset of 8 bytes from arr[0].

***Out-of-Bounds Access in C++***

In C++, there is no automatic out-of-bounds checking for arrays. Accessing an index beyond the array size leads to undefined behavior.

*Example 3: Out-of-Bounds Access*

```
#include <iostream>
using namespace std;
   int main() {
   int arr[] = {10, 20, 30, 40};   // Accessing an out-of-bounds index
   cout << arr[5] << "\n"; // Undefined behavior
   return 0;
}
```

**Explanation:**

- Accessing arr[5] is invalid as the array has only four elements.
- The compiler does not prevent this, but the output could be random or lead to a segmentation fault.

**Key Takeaway:** Always ensure that the indices used are within valid bounds.

**Uninitialized Elements in Arrays:-**If an array is partially initialized, the uninitialized elements are set to 0 for statically allocated arrays.

Example 4: Partially Initialized Array

```
#include <iostream>
using namespace std;
   int main() {
   int arr[4] = {10, 20};   // Accessing an uninitialized element
   cout << arr[2] << "\n"; // Output: 0
     return 0;
}
```

**Output:** 0

**Explanation:**

- Only the first two elements are explicitly initialized.
- The remaining elements are automatically set to 0.

**Summary**

1. **Indexing:** Array elements are accessed using indices starting from 0 to size - 1.
2. **Memory Calculation:** Indexing internally calculates the memory address based on the element size.
3. **No Bounds Checking:** C++ does not provide automatic checks for out-of-bounds access, leading to potential undefined behavior.
4. **Uninitialized Elements:** Statically declared arrays have uninitialized elements set to 0 if partially initialized.

**Array Traversal in C++**:- Array traversal is a fundamental operation in programming, allowing you to access, process, or modify each element of an array. This article discusses different methods of array traversal in C++, how to modify array elements during traversal, and the nuances of both methods.

**1. Using Normal Loops** A common way to traverse arrays in C++ is by using traditional for loops. This method requires knowing the size of the array, which can be calculated using the sizeof operator.

```
#include <iostream>
using namespace std;
   int main() {
 int arr[] = {10, 40, 30, 45};
 int n = sizeof(arr) / sizeof(arr[0]); // Calculate number of elements
    for (int i = 0; i < n; i++) {
  cout << arr[i] << " ";
   }
   return 0;
}
```

**Output:** 10 40 30 45

**Explanation:**

- The sizeof(arr) gives the total size of the array in bytes.
- sizeof(arr[0]) gives the size of a single element.
- Dividing the two calculates the number of elements in the array.
- A for loop iterates through indices from 0 to n-1, accessing elements using arr[i].

**2. Using Range-Based for Loop** The range-based for loop simplifies array traversal by eliminating the need to calculate the size of the array. It directly iterates over elements, making the code cleaner and easier to write.

```
#include <iostream>
using namespace std;
   int main() {
int arr[] = {10, 40, 30, 45};
for (int x : arr) {
      cout << x << " ";
   }
    return 0;
}
```

**Output:** 10 40 30 45

**Explanation:**

- The loop variable x takes the value of each element in the array, one by one.
- The syntax for (int x : arr) automatically traverses all elements without needing explicit size calculation.

**3. Modifying Array Elements During Traversal** You can modify array elements during traversal. This can be done using either a normal loop or a range-based for loop with a reference variable.

**Using Normal Loop:**

```
#include <iostream>
using namespace std;
   int main() {
   int arr[] = {10, 40, 30};
   int n = sizeof(arr) / sizeof(arr[0]);
     for (int i = 0; i < n; i++) {
       arr[i] *= 2; // Modify each element to its double
   }
    for (int i = 0; i < n; i++) {
```

```
        cout << arr[i] << " ";
    }
 return 0;
}
```

**Using Range-Based Loop:**

```
#include <iostream>
using namespace std;
    int main() {
 int arr[] = {10, 40, 30};
     for (int &x : arr) { // Use reference (&) to avoid copying
        x *= 2; // Modify each element
    }
     for (int x : arr) {
 cout << x << " "; }
  return 0;
}
```

**Key Points:**

- In the normal loop, elements are accessed using their index.
- In the range-based loop, the reference (&) ensures modifications are applied directly to the original array elements rather than a copy.

**Different Types of Arrays in C++** Arrays in C++ can be classified based on their size management: **fixed-size arrays** and **dynamic-size arrays**. Understanding the distinctions between these types helps programmers choose the right approach depending on the use case. Below, we'll explore these arrays in detail.

**1. Fixed-Size Arrays :-** Fixed-size arrays have a pre-defined size that remains constant throughout the program. Once the size is decided, it cannot be altered. These arrays are memory-efficient in specific scenarios but can lead to wastage if the allocated size exceeds actual requirements.**Examples of Fixed-Size Arrays:**

**1. Local Arrays (Function Call Stack):Syntax Example:**

- Created within a function and stored in the function call stack.
- Temporary in nature; their memory is automatically released once the function execution completes.
- Commonly initialized using constants, user inputs, or variable values.

**Syntax Example:**

```
int arr[10]; // Fixed-size array of 10 elements
int n = 5;
int arr2[n]; // Variable size determined at runtime
```

**2. Heap-Allocated Fixed-Size Arrays:**

- Allocated in the heap memory using pointers.
- Memory persists even after the function execution, as long as the pointer is valid.
- Provides flexibility to manage memory beyond the scope of a single function.

**Syntax Example:**

```
int* arr = new int[10]; // Dynamically allocated fixed-size array
delete[] arr;           // Manually deallocate memory
```

**2. Dynamic-Size Arrays** Dynamic-size arrays can grow or shrink during runtime. This makes them highly flexible and efficient, especially in scenarios where the exact size of data is unknown beforehand.

**Advantages:**

- **Flexible Memory Usage:** Adapts to the required size without wasting memory.
- **Space Efficiency:** Avoids over-allocation compared to fixed-size arrays.
- **Ease of Use:** Automatic resizing saves time and reduces complexity. **Examples of Dynamic Arrays:**

**1. Vectors in C++ (STL):Syntax Example:**

- A part of the Standard Template Library (STL).
- Automatically manages memory by resizing itself as elements are added or removed. **Syntax Example:**

```
#include <vector>
std::vector<int> vec; // Declare an empty dynamic array
vec.push_back(10);    // Add an element
vec.pop_back();       // Remove an element
```

**2. Dynamic Arrays Using Pointers:**

- Offers manual control over memory allocation and deallocation.
- Useful when precise memory management is required. **Syntax Example:**

```
int* arr;
int size = 10;
arr = new int[size]; // Allocate memory
// Modify elements
delete[] arr;       // Deallocate memory
```

**Maximum in an Array Problem Statement** Given an array of integers, determine the maximum element in the array. **Examples**

1. **Input:**
   - Array: [1, 2, 3, 4, 5]
   - **Output:** 5
   - Explanation: Among all elements, 5 is the largest.
2. **Input:**
   - Array: [1, 2, 3]
   - **Output:** 3
   - Explanation: The largest value is 3.

**Approach** The task is solved by iterating through the array, comparing each element to a variable that stores the current maximum value. If a larger element is found during iteration, the maximum value is updated.

***Steps:***

1. Initialize a variable max to hold the maximum value. Set it to the first element of the array initially.
2. Iterate through the array starting from the second element.
3. Compare each element with the current max value:
   - If the current element is greater, update max with the current element.
4. After completing the iteration, max will hold the maximum value.

**Code Implementation:**

```
#include <bits/stdc++.h>
using namespace std;
// Function to find the maximum element in the array
int find_max(int arr[], int n) {
    int ans = arr[0]; // Initialize the maximum with the first element
    for (int i = 1; i < n; i++) {
```

```
    if (arr[i] > ans) { // If the current element is greater, update max
        ans = arr[i];
    }  }
  return ans; // Return the maximum element
}
int main() {
  int n;
  cin >> n; // Input the size of the array
  int arr[n]; // Declare an array of size n   // Input array elements
  for (int i = 0; i < n; i++) {
     cin >> arr[i];
  }  // Find and print the maximum element
  cout << find_max(arr, n);
return 0;
}
```

**Code Explanation**

1. **Function find_max:**
   - The function takes two parameters: the array arr and its size n.
   - Initializes ans to the first element of the array (arr[0]).
   - Iterates through the array starting from the second element (i = 1).
   - Updates ans whenever a larger element is found.
   - Returns the final value of ans as the maximum element.
2. **Main Function:**
   - Inputs the size of the array and its elements from the user.
   - Calls the find_max function, passing the array and its size.
   - Outputs the result.

**Check if Array is SortedProblem Statement:_** We are tasked with determining whether a given array is sorted in ascending order. Here are the requirements**:**

- The array is sorted if each element is greater than or equal to the previous element.
- Arrays with a single element or no elements are considered sorted.
- If the array is not sorted, we return false. Otherwise, we return true.

**Input and Output Examples**

1. **Input:** [10, 20, 30]
   - **Output:** Sorted
   - Explanation: All elements are in ascending order.
2. **Input:** [10, 20, 20, 30]
   - **Output:** Sorted
   - Explanation: Equal elements are allowed.
3. **Input:** [10, 5, 2]
   - **Output:** Not sorted
   - Explanation: The sequence decreases, so it's not sorted.
4. **Input:** [] (empty array)
   - **Output:** Sorted
   - Explanation: An empty array is considered sorted.
5. **Input:** [42] (single element)

- **Output:** Sorted
- Explanation: A single element array has no comparisons and is inherently sorted.

**Solution Approach** To solve the problem, we use the following logic:

1. Traverse the array from the second element to the last.
2. For each element, compare it to the previous element:
   - If the current element is less than the previous element, return false.
3. If all comparisons are valid, return true

```
#include <bits/stdc++.h>
using namespace std;    // Function to check if the array is sorted
bool check_if_sorted(int arr[], int n) {
  for (int i = 1; i < n; i++) {
     if (arr[i] < arr[i - 1]) { // If current element is less than the previous one
         return false; // Not sorted
      }}
   return true; // Sorted
}
int main() {
   int n;
   cin >> n; // Input the size of the array
   int arr[n]; // Declare an array of size n
   for (int i = 0; i < n; i++) {
      cin >> arr[i]; // Input array elements
   } // Check if the array is sorted
   if (check_if_sorted(arr, n) == false)
      cout << "Not sorted";
   else cout << "Sorted";
 return 0;
}
```

**Code Explanation**

1. **check_if_sorted Function:**
   - Takes two parameters: the array arr and its size n.
   - Iterates through the array from index 1 to n-1.
   - Compares each element arr[i] with the previous element arr[i-1].
   - If arr[i] is smaller, it returns false. Otherwise, it completes the loop and returns true.
2. **Main Function:**
   - Reads the size of the array and its elements from the user.
   - Calls the check_if_sorted function and prints "Sorted" or "Not sorted" based on the return value.

**Count Distinct in an ArrayProblem Statement** Given an array of integers, the goal is to determine the total number of distinct elements in the array. For example:

1. **Input:** [10, 20, 10, 20, 30]
   - **Output:** 3
   - Explanation: The distinct elements are 10, 20, and 30.

2. **Input:** [1, 2, 3, 3]
    - **Output:** 3
    - Explanation: The distinct elements are 1, 2, and 3.
3. **Input:** [1, 1, 1, 1]
    - **Output:** 1
    - Explanation: The distinct element is 1.

**Approach** The idea is to iterate through the array and check for duplicate elements to determine distinctness. Here's how it can be achieved:

1. Start with a counter initialized to 0 to keep track of the distinct elements.
2. For each element in the array:
    - Check whether the element exists in the part of the array to its left.
    - If the element is not found in the left part, increment the counter.
3. Repeat this process for all elements in the array.
4. Return the value of the counter as the result.

**Code Implementation:**

```
#include <iostream>
using namespace std; // Function to count distinct elements in an array
int count_distinct(int arr[], int n) {
   int count = 0; // Counter for distinct elements
 for (int i = 0; i < n; i++) {
      bool isDistinct = true; // Assume the current element is distinct
 // Check the elements to the left of the current element
      for (int j = i - 1; j >= 0; j--) {
         if (arr[i] == arr[j]) { // If a duplicate is found
            isDistinct = false;
            break;
         } }
 if (isDistinct) {
         count++; // Increment the count for distinct elements
      }
   }
return count; // Return the total count of distinct elements
}
int main() {
   int n;
   cin >> n; // Input the size of the array
 int arr[n];
   for (int i = 0; i < n; i++) {
   cin >> arr[i]; // Input the elements of the array
   } // Call the function and output the result
   cout << count_distinct(arr, n);
return 0;
}
```

**Code Explanation**

1. **Function count_distinct:**
   - The function takes the array arr and its size n as parameters.
   - It initializes a counter count to 0.
   - For each element, it assumes the element is distinct (isDistinct = true).
   - It then checks the elements to the left of the current element:
     - If a duplicate is found, isDistinct is set to false, and the loop is terminated.
   - If no duplicate is found, the counter is incremented.
   - Finally, the function returns the value of count.
2. **Main Function:**
   - Inputs the size of the array and the array elements from the user.
   - Calls the count_distinct function with the array and its size.
   - Outputs the result.

**Sum of an ArrayProblem Statement** Given an array of integers, calculate the sum of all its elements. For example:

1. **Input:**
   - Array size: 5
   - Elements: [1, 2, 3, 4, 5]
   - **Output:** 15
   - Explanation: The sum is calculated as 1 + 2 + 3 + 4 + 5 = 15.
2. **Input:**
   - Array size: 3
   - Elements: [1, 2, 3]
   - **Output:** 6
   - Explanation: The sum is 1 + 2 + 3 = 6.

**Approach** To solve this problem, we can take the following approach:

1. Use a loop to iterate over all elements of the array.
2. Maintain a variable (sum) to store the running total.
3. Add each array element to the sum variable during the iteration.
4. Return the final value of sum.

**Code Implementation:**

```
#include <iostream>
using namespace std; // Function to calculate the sum of array elements
int add(int arr[], int n) {
   int answer = 0; // Variable to store the sum of elements
   for (int i = 0; i < n; i++) {
      answer += arr[i]; // Add each element to the sum
   }
  return answer; // Return the final sum
}
int main() {
   int n;
   cin >> n; // Input the size of the array
 int arr[n]; // Declare an array of size n
 // Input array elements
```

```
  for (int i = 0; i < n; i++) {
     cin >> arr[i];
  } // Call the add function and store the result in sum
  int sum = add(arr, n);// Output the sum
  cout << sum;
 return 0;
}
```

**Code Explanation**

1. **add Function:**
   - Takes two parameters: the array arr and its size n.
   - Initializes an answer variable to 0.
   - Iterates through the array using a for loop and adds each element to the answer.
   - Returns the value of answer after the loop ends.
2. **Main Function:**
   - Inputs the size of the array (n) and its elements.
   - Calls the add function, passing the array and its size as arguments.
   - Prints the result (sum of elements).

**Average of an Array** :- Given an array, the task is to find average of that array. Average is the sum of array elements divided by the number of elements. **Examples :**

```
Input : arr[] = {1, 2, 3, 4, 5}
Output : 3
Sum of the elements is 1+2+3+4+5 = 15
and total number of elements is 5.
So average is 15/5 = 3
Input : arr[] = {5, 3, 6, 7, 5, 3}
Output : 4.83333
Sum of the elements is 5+3+6+7+5+3 = 29
and total number of elements is 6.
So average is 29/6 = 4.83333.
```

```
#include<iostream>
using namespace std;
   double average(int a[], int n)
{
   int sum = 0 ;
   for(int i = 0; i < n; i++)
   {
     sum += a[i];
   }
   return sum/double(n);
}
int main()
{
   int n;
   cin >> n;
   int a[n];
   for(int i = 0; i < n; i ++)
   {
```

```
    cin >> a[i] ;
  }
  cout << average(a,n);
  return 0;  }
```

**INPUT :3  1 2 4  OUTPUT : 2.33333**

**References in C++**:- A **reference** in C++ is a variable that acts as an alias for another variable. It is created by using the **ampersand (&)** symbol during declaration. Once a reference is initialized, it cannot be changed to refer to another variable.

**Key Properties of References:**

1. **Must Be Initialized**: A reference must be assigned a valid variable at the time of declaration.
2. **Cannot Be NULL**: References cannot hold NULL values; they must always refer to a valid variable.
3. **Cannot Be Changed**: After initialization, a reference cannot be made to refer to a different variable.

**Syntax of References**

```
int x = 10;      // Original variable
int &ref = x;    // Reference variable pointing to 'x'
```

- ref is a reference to x. Any changes made to ref will directly affect the value of x and vice versa.

Below is the code provided for demonstrating references:

```
#include<iostream>
using namespace std;
  int main()
{
  int x = 10;
  int &y = x;
  cout << y << ' '; // Print value of 'y' (10, because it refers to 'x')
   x += 5;          // Modify 'x'
  cout << y << ' '; // Print value of 'y' (15, because it still refers to 'x')
 y += 5;          // Modify 'y'
  cout << x << ' '; // Print value of 'x' (20, because 'y' modifies 'x')
  return 0;
}
```

**Output:**10 15 20

- x and y refer to the same memory location.
- Any changes made to either x or y will affect the other.

Memory layout:

| Variable | Memory Address | Value |
|---|---|---|
| x | 0x123 | 20 |
| y | 0x123 | 20 |

**Function Parameters & References** :- In C++, function parameters are variables that are used to pass data into a function when it is called. Function parameters are defined in the function's declaration and are used to receive the data that is passed to the function when it is called.

Function parameters can be either pass-by-value or pass-by-reference. Pass-by-value means that the function receives a copy of the data that is passed to it, while pass-by-reference means that the function receives a reference to the original data.

**Issues with Passing by Value** When variables are passed to functions by value, the function works on a **copy** of the variable, not the original. This approach has two major limitations**:**

**1. Changes Are Not Reflected Outside the Function**

- Any modifications made to the variable inside the function are not visible outside it. This is because the function operates on a copy of the variable.
- Example:

```
void fun1(int x) {
   x += 2;
}
int main() {
   int x = 2;
   fun1(x); // 'x' remains unchanged
   cout << x; // Output: 2
}
```

**2. Performance Issues with Large Data Structures**

- When large objects (e.g., strings, arrays, or custom data structures) are passed by value, the entire object is copied. This can lead to significant memory and time overhead.
- Example:

```
void fun(string s) {
   // Operates on a copy of the string 's'
   cout << s;
}
```

## Solving Issues with References **Passing by** reference **allows a function to operate directly on the original variable. This is done by declaring the parameter with an** ampersand (&)**.**

**Changes Are Reflected Outside the Function**

- The function works directly with the original variable, so any modifications are visible outside.

```
#include<iostream>
using namespace std;

void fun1 (int x)
{
   x += 2;
}
void fun2 (int &x)
{
   x += 2;
}
int main()
{
   int x = 2;
   fun1(x);
   cout << x << ' ';
   fun2(x);
   cout << x;
   return 0;
```

}
**Output:**2 4

Passing parameters by reference can be more efficient than pass-by-value because it avoids the overhead of creating and copying the data. However, it is important to be careful when using pass-by-reference, as it can lead to unintended side effects if the function modifies the original data.

*Performance issue:*

The const keyword is often used with references to indicate that the function should not modify the variable being referred to. This is especially useful when working with large objects (like strings or data structures) that are passed by reference for performance reasons, but the function should only read the data, not change it.

*Code Example: Using const Reference*

```
#include<iostream>
using namespace std;
   void fun(const string &s) { // 's' is a constant reference
   cout << s; // Only allowed to read 's', not modify it
}
   int main() {
   string s = "geeksforgeeks course";
   fun(s);
   return 0;
}
```

*Output:* ***geeksforgeeks course***

***Range Based for Loop*** :- The range-based for loop in C++ provides a simpler and more elegant way to iterate over collections like arrays, vectors, or other containers. When combined with references, it becomes even more powerful, addressing issues like element modification and performance.

**Syntax : for (data_type variable : collection) {**

```
   // Code to execute for each element
}
```

- **data_type**: The type of the elements in the collection.
- **variable**: A variable to hold each element during iteration.
- **collection**: The array or container to iterate over.

**Example Without References**

```
#include<iostream>
using namespace std;
   int main() {
   int arr[] = {10, 20, 30};
   for (int x : arr) { // 'x' is a copy of each element
      x += 2; // Modifies the copy, not the original array
   }
   for (int x : arr) {
      cout << x << " "; // Original array remains unchanged
   }
   return 0;
}
```

**Output:** 10 20 30

**Problems with Normal Loop Variables**

1. **Cannot Change Elements**:
   - The loop variable (x) is a **copy** of each element.
   - Any modifications to x do not affect the original array.
2. **Performance Issues**:
   - For large objects (e.g., strings or vectors), copying each element can be computationally expensive.

**Solving Problems with References :-**By using references (&) in the loop, we can iterate directly over the elements of the collection, avoiding unnecessary copying and allowing modifications to the original elements.

```
#include<iostream> using namespace std; int main() {

int arr[] = {10, 20, 30};
for (int &x : arr) {
x += 2; // Modifies the original array
}
for (int x : arr) {
cout << x << " ";
}
return 0;
}
```

**Output:**

```
12 22 32
```

**Combining const with References** If the goal is only to read the elements without modifying them, you can use const references. This ensures the safety of the data while avoiding the cost of copying.Example:

```
#include<iostream>
using namespace std;
   int main() {
   string arr[] = {"GFG", "C++ Course", "learning"};
   for (const string &s : arr) { // 's' is a const reference
      cout << s << "\n";
   }  return 0;
}
```

**Output:**

```
GFG
C++ Course
learning
```

**Const & R Value References** C++ references provide an efficient way to manipulate data. However, the distinction between **L-value references**, **R-value references**, and the use of the const keyword introduces more flexibility, especially for temporary objects or immutable data.

In C++, you can use the **const** keyword to declare a reference that cannot be modified, and you can use the **&&** operator to declare an rvalue reference, which is a reference to a temporary object that will be destroyed after the reference goes out of scope.

**What Are L-Values and R-Values?**

- **L-Values**: Objects that persist beyond a single expression, such as variables or references. You can take their address using the & operator.

  ```
  int x = 10; // 'x' is an L-value
  ```

- **R-Values**: Temporary objects or literals that exist only during the evaluation of an expression and cannot be referenced directly. They cannot have their address taken.

```
int x = 10 + 20; // '10 + 20' is an R-value
```

**const References Problem with Regular References:**

Regular references cannot bind to R-values, meaning you cannot reference temporary objects directly. (e.g., literals or temporary objects):

```
int &x = 3; // Invalid: '3' is an R-value
```

**Solution: Use const References** A const reference allows binding to both L-values and R-values. It ensures the referenced object is immutable within the scope of the reference. Example:

```
#include<iostream>
using namespace std;
void print(const string &s) { // 's' can bind to temporary strings
   cout << s << '\n';
}
   int main() {
   print("Welcome to GeeksforGeeks"); // Valid: temporary string bound to 's'
   return 0;
}
```

Output: Welcome to GeeksforGeeks

**Key Points:**

- const string &s prevents modifications to the string s inside the print function.
- const references are useful for read-only access to large objects or temporary R-values.

**R-Value References (&&)**

R-value references, denoted by &&, allow functions to modify temporary objects (R-values). They provide a way to work with temporary data without unnecessary copies.

```
#include<iostream>
using namespace std;
   void editAndPrint(string &&s) { // 's' is an R-value reference
   s = "Hi, " + s; // Modifies the temporary string
   cout << s << "\n";
}
   int main() {
   editAndPrint("Welcome to GeeksforGeeks");
   return 0;
}
```

**Output:** Hi, Welcome to GeeksforGeeks

**Explanation:**

- "Welcome to GeeksforGeeks" is an R-value.
- editAndPrint modifies the temporary string and avoids copying the data.

**References Practice Questions in C++Approach to solving questions involving References in C++**

When solving coding questions related to references in C++, there are a few key things to keep in mind:

- Understand the difference between references and pointers. A reference is an alias for an existing variable, whereas a pointer holds the memory address of a variable.
- Remember that once a reference is initialized, it cannot be made to refer to a different object.
- Be aware of the advantages and disadvantages of using references. For example, references can improve code readability and make it easier to work with large or complex data structures. However, they also can also make your code more difficult to understand if used improperly.
- Pay attention to the problem statement and constraints. Make sure you understand the requirements of the problem and how references can be used to solve it.
- Write clear, readable, and well-commented code. This will help you and others understand the logic behind your solution, and make it easier to debug if necessary.
- Test your code thoroughly before submitting, using sample inputs as well as edge cases. This will help you catch any bugs or errors that you might have missed during the development process.

**Question 1: Reference Assignment**

```
#include<iostream>
using namespace std;
  int main()
{
  int x = 10, y = 20;
  int &z = x;
  z = y;
  z =z + 5;
  cout << x << ' ' << y << ' ' << z;
  return 0;
}
```

Explanation:

1. **Reference Declaration**:
   - int &z = x; declares z as a reference to x. This means that z and x refer to the same memory location.
2. **Assignment**:
   - z = y; assigns the value of y (which is 20) to z, and since z is a reference to x, x now also becomes 20.
3. **Modification**:
   - z += 5; modifies z, which is a reference to x, so x becomes 25 (20 + 5).

*Output:* ***25 20 25***

**Question 2: Returning a Reference from a Function**

```
#include<iostream>
using namespace std;
  int &fun()
{
  static int x = 10;
  return x;
}
  int main()
{
  int &y = fun();
  y = 20;
  cout << fun();
  return 0;
}
```

**Explanation:**

1. **Static Variable**:
   - static int x = 10; ensures that x retains its value even after the fun() function exits. If x were not static, returning a reference to a local variable would result in undefined behavior, as the variable would go out of scope after the function returns.
2. **Returning a Reference**:

- return x; returns a reference to x. Since x is static, the reference is valid even after the function call completes.

3. **Main Function**:
   - int &y = fun(); assigns a reference y to the value returned by fun(). Modifying y modifies x directly.
   - y = 20; assigns the value 20 to x through y.
   - cout << fun(); prints the modified value of x, which is now 20.

*Output:* ***20***

**Question 3: R-Value References**

```
#include<iostream>
using namespace std;
  int main()
{
  string s1 = "GFG ", s2 = "Courses";
  string &&s3 = s1 + s2;
  s3 = "Welcome to " + s3;
  cout << s3 << endl;
  return 0;
}
```

**Explanation:**

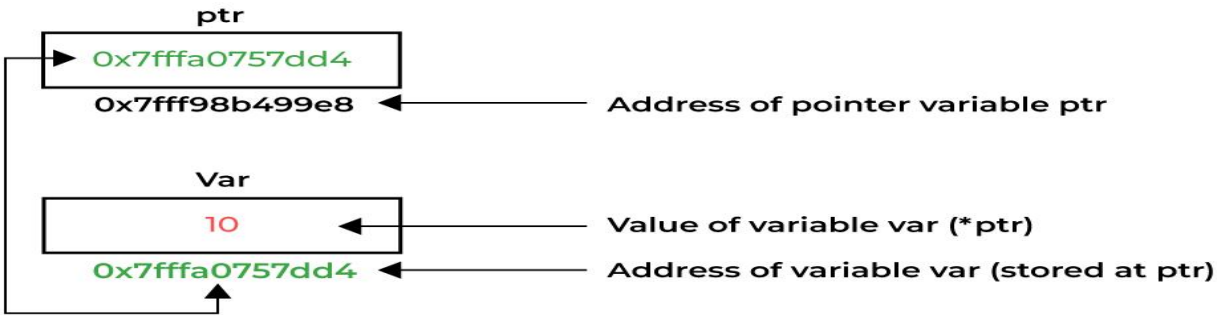
1. **R-Value Reference**:
   - string &&s3 = s1 + s2; declares s3 as an r-value reference. s1 + s2 is a temporary string, and s3 is bound to this temporary object. R-value references allow us to modify temporary objects directly.
2. **Modifying R-Value**:
   - s3 = "Welcome to " + s3; modifies the temporary string s3. Since s3 is an r-value reference, this is allowed.
   - The concatenation results in the string "Welcome to GFG Courses".
3. **Output**:
   - cout << s3 << endl; prints the modified string s3.

*Output:* ***Welcome to GFG Courses***

## Address and Dereference Operators in C++

**1. Address Operator (&) :-**The address operator (&) is used to get the **memory address** of a variable. When you use the address operator in front of a variable, it returns the location in memory where the variable is stored.

**2. Dereference Operator (*):-**The dereference operator (*) is used to access the **value** stored at a specific memory address. It is used when we have a pointer (which stores the address of a variable) and we want to retrieve the value stored at that address.




```
#include<iostream>
using namespace std;
   int main() {
   int x = 10;
   cout << &x << '\n';    // Print the address of x
   cout << *(&x) << '\n';  // Print the value at the address of x
   return 0;
}
```

*Output:*

```
0x7fff5354ae8c
10
```

*1. **&x**:*

- The expression &x returns the **memory address** of the variable x. The output will look like an address in hexadecimal format (e.g., 0x7ff5e20af2284).

*2. ***(&x)**:*

- The expression *(&x) first uses &x to get the address of x. Then, the * operator dereferences the address to access the **value** stored at that address. Since x is 10, the output will be 10.

### Three Things Associated with a Variable

In C++, every variable is associated with three key elements:

1. **Name**: This is the label or identifier used to refer to the variable. For example, in int x = 10;, x is the name.
2. **Address**: This is the location in memory where the variable's value is stored. You can retrieve the address using the & operator. For example, &x gives the memory address of x.
3. **Value**: This is the actual data stored at the memory address. You can access the value using the variable name itself or by dereferencing a pointer to the variable's address using the * operator. For example, x gives you the value stored in x, and *(&x) gives you the value at the address of x.

Using & and * operators, you can work with the address and value of variables, which is essential for efficient memory management and pointer-based operations in C++.

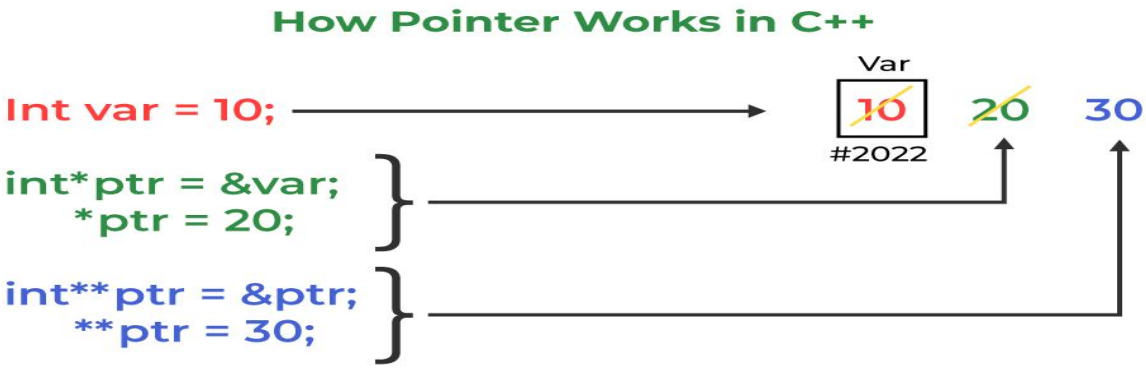
# Introduction to Pointers in C++:-

Pointers are symbolic representations of addresses. They enable programs to simulate call-by-reference as well as to create and manipulate dynamic data structures. Iterating over elements in arrays or other data structures is one of the main use of pointers.

**What is a Pointer?**

A pointer is a variable that stores the memory address of another variable. In C++, pointers are declared using the * symbol. Here's the syntax:

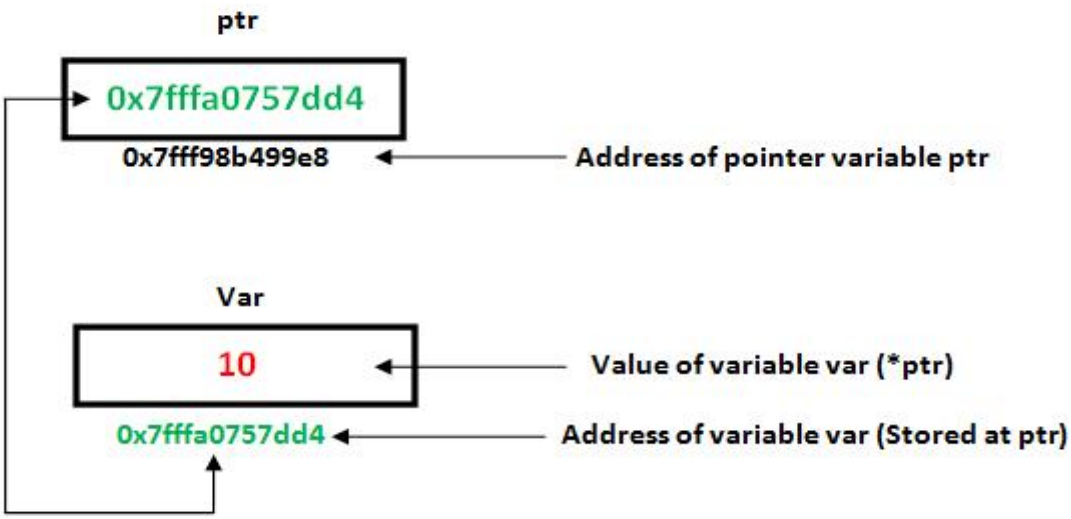
```
datatype *var_name;
int *ptr;   // ptr can point to an address which holds int data
```




How to use a pointer?

- Define a pointer variable
- Assigning the address of a variable to a pointer using the unary operator (&) which returns the address of that variable.
- Accessing the value stored in the address using unary operator (*) which returns the value of the variable located at the address specified by its operand.

The reason we associate data type with a pointer is **that it knows how many bytes the data is stored in**. When we increment a pointer, we increase the pointer by the size of the data type to which it points.



## Example 1: Basics of Pointers

```
#include <iostream>
using namespace std;
   int main() {
   int x = 10;
   int *p;
   p = &x;
      cout << x << '\n';
   cout << *p << '\n'; // Dereferences p to get the value at the address (10)
   cout << p << '\n';  // Prints the address stored in p (address of x)
    return 0;
}
```

**Explanation:**

1. **Variable and Pointer Declaration**:
   int x = 10; declares an integer variable x.
   int *p; declares a pointer p that can hold the address of an integer.
2. **Address Assignment**:
   p = &x; assigns the address of x to the pointer p.
3. **Dereferencing**:
   *p retrieves the value stored at the address p points to.

**Memory Diagram:**

- **x = 10** is stored at address 0x1234.
- **p** holds the value 0x1234, pointing to x.

*Output:* ***10***
***10***
***0x1234***

**Example 2: Modifying Values Using Pointers** Pointers allow direct manipulation of the variable they point to. Here's an example:

```
#include<iostream>
using namespace std;
   int main()
{
   int x = 10;
   int *ptr = &x;
   x += 30;
   cout << *ptr << '\n';
   *ptr += 40;
   cout << x << '\n';
   return 0;
}
```

**Explanation:**

1. The pointer ptr is initialized with the address of x.
2. Modifying x directly reflects when accessing it through *ptr.
3. Similarly, modifying *ptr changes the value of x because they both refer to the same memory location.

*Output:**40***
***80***

**Example 3: Size of Pointers** The size of a pointer in memory depends on the system's architecture. For instance, on a 64-bit system, all pointers (regardless of their type) typically occupy 8 bytes**.**

```
#include <iostream>
using namespace std;
   int main() {
   int *p1;
   double *p2;
   string *p3;
    cout << sizeof(p1) << ' '
       << sizeof(p2) << ' '
       << sizeof(p3);
     return 0;
}
```

**Explanation:**

1. sizeof gives the size in bytes of a data type or variable.
2. Although p1, p2, and p3 point to different types (int, double, and string), their sizes are the same because all pointers store memory addresses.

**Output** (on a 64-bit system): 8 8 8

**Key Points to Remember:**

1. A pointer stores the address of a variable.
2. The & operator is used to get the address of a variable.
3. The * operator is used to access the value at the address (dereferencing).
4. The size of a pointer depends on the system architecture and is independent of the data type it points to.

## Applications of Pointers in C++

Pointers in C++ are an essential concept for efficient memory management and dynamic programming. Their versatile nature makes them incredibly useful in a variety of scenarios. Below are some common applications of pointers in C++

**1. Changing Passed Parameters** When a function needs to modify the original variable passed to it, pointers are used. Instead of creating a copy of the variable, pointers allow direct access to the variable's memory address.

```
void updateValue(int *p) {
   *p = 20; // Modify the value at the pointer's address
}
   int main() {
   int x = 10;
   updateValue(&x); // Pass address of x
   cout << x;       // Output: 20
   return 0;
}
```

Pointers enable functions to change the original value, making them ideal for reference-based modifications.

**2. Passing Large Objects** Instead of passing large objects (e.g., arrays or structures) by value, which consumes more memory and processing time, pointers allow these objects to be passed by reference efficiently.

**Advantage:** Only the memory address is passed, not the entire object, reducing overhead.

**3. Dynamic Memory Allocation** Pointers are crucial for managing memory dynamically during runtime. Using functions like new and delete, you can allocate and deallocate memory as needed.

**4. Implementing Data Structures** Pointers form the backbone of many data structures such as:

- **Linked Lists**
- **Trees**
- **Binary Search Trees (BSTs)**

These structures rely on pointers to link nodes and create dynamic, non-contiguous storage.

**Example:** A linked list uses pointers to connect each node to the next.

**5. System-Level Programming** Pointers allow low-level programming, such as:

- Direct memory access
- Manipulating hardware resources

System-level tasks like implementing operating systems or drivers often require pointer-based operations.

**6. Returning Multiple Values** By using pointers, a function can return multiple values by modifying variables directly at their memory locations.

**7. Accessing Array Elements** Pointers can be used to traverse arrays. Instead of using array indices, pointer arithmetic simplifies array navigation.

**8. Passing Array Arguments** Pointers are used to pass arrays to functions. Instead of passing the entire array, passing a pointer to its first element is more efficient.

**Why Are Pointers Important?** Pointers are indispensable in C++ programming. They provide flexibility and efficiency in handling memory, especially in scenarios where direct control over memory is crucial. However, working with pointers requires careful handling to avoid issues like memory leaks or invalid memory access.

Function Parameter and PointersWhen working with functions in C++, one common challenge is managing how data is passed and manipulated between functions. By default, C++ supports different parameter-passing techniques, but sometimes, these default methods may lead to inefficiencies or undesired outcomes. This is where **pointers** come into play.

**Problems with Normal Parameter Passing**

When you pass parameters by **value** (normal parameter passing), the following issues arise:

1. **Changes Are Not Reflected**: The function works with a copy of the original variable, so any changes made inside the function are not reflected outside.
2. **Whole Object Is Copied**: For large objects (like strings or arrays), passing by value creates a copy of the object, which consumes more memory and processing time.

Example 1: Passing an Integer by Value

```
#include <iostream>
```

```
using namespace std;
   void fun(int x) {
   x = x + 5;
}
   int main() {
   int x = 10;
   fun(x);
   cout << x; // Output: 10
   return 0;
}
```

**Explanation**:

- The function fun takes x as an argument and modifies its value. However, the modification is done on a **local copy** of x, so the original variable remains unchanged.

Example 2: Passing a String by Value

```
#include <iostream>
using namespace std;
   void fun(string s) {
   cout << s;
}
   int main() {
   string s = "geeksforgeeks course";
   fun(s);
   return 0;
}
```

**Explanation**:

- Although the string s is printed in the function, a **copy** of the string is created when the function is called. This can be inefficient for large strings or objects.

## Using Pointers in Function Parameters

To address the limitations of normal parameter passing, pointers can be used. Pointers enable:

1. **Changing Values**: By passing a pointer, the function can directly modify the original variable.
2. **Avoiding Copying of Large Objects**: Instead of copying the entire object, only its address is passed, improving efficiency.

Example 1: Passing an Integer by Pointer

```
#include <iostream>
using namespace std;
   void fun(int *p) {
   *p = *p + 5;
}
   int main() {
   int x = 10;
   fun(&x);
   cout << x; // Output: 15
   return 0;
}
```

**Explanation**:

- The function fun takes a pointer to x as an argument. By dereferencing the pointer (*p), it directly modifies the value of x. This ensures that the change is reflected outside the function.

Example 2: Passing a String by Pointer

```
#include <iostream>
```

```
using namespace std;
   void fun(string *s) {
   cout << *s; // Dereference the pointer to access the string
}
   int main() {
   string s = "geeksforgeeks course";
   fun(&s); /
   return 0;
}
```

**Explanation**:

- Instead of copying the string, the function receives a pointer to the original string. Dereferencing the pointer (*s) allows the function to access and use the string efficiently.

**Key Advantages of Using Pointers**

1. **Efficiency**: Passing pointers avoids the overhead of copying large objects, improving performance.
2. **Direct Modification**: Functions can directly modify the original variables or objects, which is essential in many programming scenarios.

**Array Parameters and Pointers** :-Arrays are often passed to functions as parameters in C++. However, since arrays decay into pointers when passed to a function, there are some subtle issues and limitations to keep in mind.

**Understanding Array Parameters in Functions** When you pass an array to a function, only the **pointer to the first element** of the array is passed, not the entire array. Therefore, any attempt to calculate the size of the array inside the function using sizeof will yield incorrect results.

```
#include <iostream>
using namespace std;
   void f(int a[]) {
   int n = sizeof(a) / sizeof(int); // Incorrect size calculation
   for (int i = 0; i < n; i++) {
      cout << a[i] << ' ';
   }
   cout << '\n';
}
   void f(int a[], int N) {
   int n = N; // Correct size passed explicitly
   for (int i = 0; i < n; i++) {
      cout << a[i] << ' ';
   }
   cout << '\n';
}
   int main() {
   int a[] = {1, 2, 3, 4, 5};
   int n = sizeof(a) / sizeof(int); // Correct size calculation in main
   for (int i = 0; i < n; i++) {
      cout << a[i] << ' ';
   }
   cout << '\n';

   f(a);
   f(a, n);
   return 0;
}
```

**Explanation**

1. **Inside the main Function**:
   - The array a is declared with 5 elements.
   - The size of the array is correctly calculated using sizeof(a) / sizeof(int) because a is treated as an array.
2. **Inside the Function f(int a[])**:
   - The array decays into a pointer when passed as a parameter.
   - The sizeof(a) calculates the size of the pointer, **not the size of the array**. This results in an incorrect size calculation.
3. **Inside the Function f(int a[], int N)**:
   - The size of the array is passed explicitly as N. This is the correct way to pass an array when its size is required.

**Output** The output of the program is as follows:

```
1 2 3 4 5
0 0 0 0 0
1 2 3 4 5
```

- The first line corresponds to the main function printing the array elements.
- The second line corresponds to the incorrect size calculation in f(int a[]).
- The third line corresponds to the correct size calculation in f(int a[], int N).

**Common Pitfalls and Recommendations**

1. **Incorrect Size Calculation**:
   - Inside a function, calculating the size of the array using sizeof is incorrect because the array is treated as a pointer. This leads to incorrect results.
   - **Solution**: Always pass the size of the array as an additional parameter.
2. **Array Decay**:
   - Arrays decay into pointers when passed to functions, which means the size information is lost.
   - **Tip**: Be cautious about this behavior when designing functions.
3. **Avoid Using sizeof on Array Parameters**:
   - Since sizeof on a parameter array returns the size of the pointer, not the actual array, avoid using it to determine the size.

**Array Parameters and Pointers** Arrays are often passed to functions as parameters in C++. However, since arrays decay into pointers when passed to a function, there are some subtle issues and limitations to keep in mind.

**Understanding Array Parameters in Functions** When you pass an array to a function, only the **pointer to the first element** of the array is passed, not the entire array. Therefore, any attempt to calculate the size of the array inside the function using sizeof will yield incorrect results.

```
#include <iostream>
using namespace std;
   void f(int a[]) {
   int n = sizeof(a) / sizeof(int); // Incorrect size calculation
   for (int i = 0; i < n; i++) {
      cout << a[i] << ' ';
   }
   cout << '\n';
}
```

```
void f(int a[], int N) {
   int n = N; // Correct size passed explicitly
   for (int i = 0; i < n; i++) {
      cout << a[i] << ' ';
   }
   cout << '\n';
}
   int main() {
   int a[] = {1, 2, 3, 4, 5};
   int n = sizeof(a) / sizeof(int); // Correct size calculation in main
   for (int i = 0; i < n; i++) {
      cout << a[i] << ' ';
   }
   cout << '\n';
     f(a);
   f(a, n);
   return 0;
}
```

**Explanation**

1. **Inside the main Function**:
   - The array a is declared with 5 elements.
   - The size of the array is correctly calculated using sizeof(a) / sizeof(int) because a is treated as an array.
2. **Inside the Function f(int a[])**:
   - The array decays into a pointer when passed as a parameter.
   - The sizeof(a) calculates the size of the pointer, **not the size of the array**. This results in an incorrect size calculation.
3. **Inside the Function f(int a[], int N)**:
   - The size of the array is passed explicitly as N. This is the correct way to pass an array when its size is required.

**Output**

The output of the program is as follows:

```
1 2 3 4 5
0 0 0 0 0
1 2 3 4 5
```

- The first line corresponds to the main function printing the array elements.
- The second line corresponds to the incorrect size calculation in f(int a[]).
- The third line corresponds to the correct size calculation in f(int a[], int N).

**Common Pitfalls and Recommendations**

1. **Incorrect Size Calculation**:
   - Inside a function, calculating the size of the array using sizeof is incorrect because the array is treated as a pointer. This leads to incorrect results.
   - **Solution**: Always pass the size of the array as an additional parameter.
2. **Array Decay**:
   - Arrays decay into pointers when passed to functions, which means the size information is lost.
   - **Tip**: Be cautious about this behavior when designing functions.
3. **Avoid Using sizeof on Array Parameters**:
   - Since sizeof on a parameter array returns the size of the pointer, not the actual array, avoid using it to determine the size.

**Pointers versus Arrays** At first glance, pointers and arrays in C++ may look similar, and they indeed share some common behaviors. However, they are fundamentally different in how they are implemented, used, and accessed. **Syntax Similarities and Key Differences**

**1. Assigning Arrays to** Pointers You can assign the base address of an array to a pointer, but you cannot assign a pointer back to an array.

```
int arr[] = {10, 20, 30};
int *ptr = arr; // Valid: Pointer gets the base address of the array
arr = ptr;      // Invalid: Arrays are not modifiable lvalues
```

**Why?**: Arrays are not pointers; they represent a block of memory with a fixed size, and their base address cannot be changed. A pointer, on the other hand, is a variable that can point to different addresses.

**Incrementing Pointers vs Arrays** You can increment or decrement a pointer to navigate through memory, but you cannot increment or decrement an array.

```
int arr[] = {10, 20, 30};
int *ptr = arr;
ptr++;      // Valid: Pointer now points to the second element
arr++;      // Invalid: Arrays cannot be incremented
```

**Why?**: A pointer is a variable holding an address, so you can manipulate it. An array name, however, is a constant that represents the starting address of the array and cannot be changed.

**Accessing Elements: *(arr + 2) vs ptr[2] Both *(arr + 2) and ptr[2] can be used to access the third element of an array. However, they have distinct meanings:**

1. ***(arr + 2)**:
   - This uses **pointer arithmetic** to calculate the memory address of the third element and dereferences it to get the value.
   - arr + 2 calculates the address of the element at index 2 (base address + 2 * size of element).
2. **ptr[2]**:
   - This uses the **array indexing** syntax but operates on a pointer.
   - Internally, ptr[2] is equivalent to *(ptr + 2).

```
int arr[] = {10, 20, 30};
int *ptr = arr;

cout << *(arr + 2) << endl; // Output: 30
cout << ptr[2] << endl;     // Output: 30
```

**Best Practices for Readability**

1. Use **square brackets ([])** for array variables:
   - **Example**: arr[2]
   - This makes it clear that you are accessing an element in an array.
2. Use **pointer arithmetic (*(ptr + 2))** for pointers:
   - **Example**: *(ptr + 2)
   - This ensures the code is easy to understand, indicating you are accessing elements via a pointer.

By following these conventions, your code will be more readable and easier to maintain.

**Differences Between Array and Pointer in Detail**

Here's an example to illustrate key differences:

```
#include<iostream>
using namespace std;
   int main()
{
   int a[] = {1, 2, 3};
   int *p1 = a;
   int (*p2)[3] = &a;
   cout << *p1 << ' ' << **p2;
   return 0;
}
```

**Explanation**:

1. p1 points to the first element of the array, so *p1 accesses the value 1.
2. p2 points to the entire array, so **p2 accesses the value of the first element.

| **Array** | **Pointer** |
|---|---|
| Arrays are declared as type var_name[size]; | Pointers are declared as type * var_name; |
| Collection of elements of similar data type. | Store the address of another variable. |
| An array can decide the number of elements it can store. | The pointer can store the address of only one variable. |
| Memory allocation is in sequence. | Memory allocation is random. |
| Arrays are static in nature i.e. they cannot be resized according to the user requirements. | Pointers are dynamic in nature i.e. memory allocated can be resized later. |
| An array of pointers can be generated. | A pointer to an array can be generated. |

**NULL in C++** :- In C++, NULL is a symbolic constant used to represent a **null pointer**, i.e., a pointer that doesn't point to any valid memory address. Understanding NULL and its use cases is crucial for safe pointer management in C++ programs.

**What is NULL in C++? Definition**

NULL is typically defined as 0 in C++ and represents a **null pointer constant**. It is used to indicate that a pointer is not currently pointing to a valid memory location.

**How is NULL Defined?**

- **Correct Usage**:
  int *ptr = 0; // Compiles fine; NULL is equivalent to 0
- **Incorrect Usage**:
  int *ptr = 10; // Compilation Error; pointer must store a valid memory address **Example:**

```
int *ptr = NULL;
if (ptr == NULL) {
   cout << "Pointer is NULL";
}
```

**Code Without NULL vs. Code With NULL** **Without NULL:**

```
int main() {
   int *ptr;
   cout << *ptr; // Dangerous: Dereferencing an uninitialized pointer
   return 0;
}
```

- This code is **unsafe** because ptr is uninitialized and may point to a random memory location, leading to undefined behavior.

**With NULL:**

```
int main() {
   int *ptr = NULL;
   if (ptr != NULL) {
      cout << *ptr;
   }
   return 0;
}
```

- This code is **safe** because the pointer is initialized to NULL, ensuring no accidental dereferencing.

**Applications of NULL in C++**

1. **For Pointers with No Valid Memory Address**:
   - NULL is used to indicate that a pointer is not currently pointing to any valid memory address.
2. **In Functions Returning Invalid Outputs**:
   - Functions often return NULL to indicate an invalid or failure state.
3. **In Data Structures**:
   - NULL is widely used in data structures like **linked lists** and **trees** to represent the end of a structure or an empty node.

**Some Important Points About NULL in C++**

1. **NULL Converts to false in Boolean Contexts**:
   - A NULL pointer is treated as false in conditional statements.
   - Any non-NULL pointer is treated as true.

   **Example**:

   ```
   int *ptr = NULL;
   if (ptr) {
      cout << "Not NULL";
   } else {
      cout << "NULL";
   }
   // Output: NULL
   ```

2. **NULL Can Be Used with Any Pointer Type**:
   - NULL is a generic constant and can be assigned to any pointer type.

   **Example**:

   ```
   double *dptr = NULL;
   char *cptr = NULL;
   ```

   **Best Practices When Using NULL**
   - Always initialize pointers with NULL when declaring them, especially if they won't immediately point to valid memory.

- Always check if a pointer is NULL before dereferencing it to avoid segmentation faults or undefined behavior.
-

**nullptr in C++**:- nullptr is a keyword introduced in C++11 as a better alternative to NULL. While NULL has served its purpose for decades, it has limitations that can lead to bugs or ambiguous code.

**The Problem with NULL How is NULL Defined?**

In C++, NULL is typically defined as a macro with the integer value 0.

This means that NULL is essentially an integer constant and not a distinct type. While this works fine in most cases, issues can arise when functions are overloaded, or when automatic integer conversions occur.

**The Issue with NULL Consider the following example:**

```
#include <bits/stdc++.h>
using namespace std;
   void fun(int x) {
   cout << "Called fun(int)" << endl;
}
   void fun(int* ptr) {
   cout << "Called fun(int*)" << endl;
}
   int main() {
   fun(NULL); // Ambiguous
   return 0;
}
```

In this case:

1. NULL is treated as 0 (an integer constant).
2. The compiler cannot decide whether to call fun(int) or fun(int*) because NULL could match both signatures:
   - fun(int) (as NULL is 0)
   - fun(int*) (as NULL represents a null pointer).

Output: 16:13: error: call of overloaded 'fun(NULL)' is ambiguous
fun(NULL);

**nullptr: A Safer Alternative** To solve the issues associated with NULL, C++11 introduced nullptr. It is a **keyword** representing a null pointer literal and is of type nullptr_t.

**Key Features of nullptr**

1. **Strong Typing**:
   - Unlike NULL, nullptr has its own distinct type: nullptr_t.
   - This prevents accidental or ambiguous conversions to other types, such as integers.
2. **Unambiguous Overloading**:
   - nullptr is explicitly treated as a null pointer, so it only matches pointer-type function overloads.
3. **Compatibility**:
   - nullptr can be used with any pointer type.

Example: Fixing Ambiguity with nullptrThe previous example can be fixed using nullptr:

```
#include<iostream>
using namespace std;
```

```
void f (int x)
{
   cout << "Integer" << '\n';
}
void f (int *ptr)
{
   cout << "Pointer" << '\n';
}
int main()
{
   // f(NULL);
   f(0);
   f(nullptr);
   return 0;
}
```

**Explanation:**

- nullptr is of type nullptr_t and matches only the fun(int*) overload.
- The function call is no longer ambiguous.

*Output:* ***Integer***
***Pointer***

**Pointer Arithmetic** :- Pointer arithmetic refers to operations performed on pointers to navigate through memory. The key operations include:

1. **Incrementing a pointer (ptr++).**
2. **Decrementing a pointer (ptr--).**
3. **Adding or subtracting an integer to/from a pointer (ptr + n, ptr - n).**
4. **Subtracting two pointers (ptr2 - ptr1).**

These operations are not performed in terms of bytes, but in terms of the size of the data type the pointer is pointing to**. (**Note: **Pointer arithmetic is meaningless unless performed on an array**.)

**Key Concept: Pointer Incrementation** When a pointer is incremented or decremented, it moves by the size of the data type it points to. For example:

- For an int pointer, it moves **4 bytes** (assuming sizeof(int) = 4).
- For a char pointer, it moves **1 byte** (assuming sizeof(char) = 1).
- For a double pointer, it moves **8 bytes** (assuming sizeof(double) = 8).

**Example 1: Pointer Increment and Decrement**

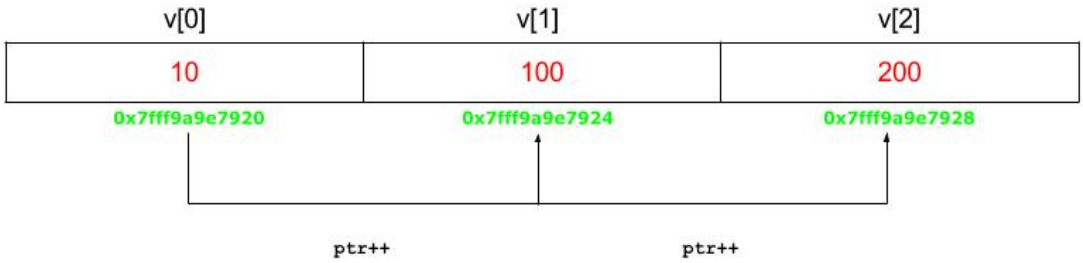
```
#include <iostream>
using namespace std;
   int main() {
   int arr[] = {10, 20, 30, 40};
   int *ptr = arr;
      cout << *ptr << " " << ptr << "\n";  // Output: 10 [address of arr[0]]
  ptr++;
   cout << *ptr << " " << ptr << "\n";  // Output: 20 [address of arr[1]]
   ptr--;
   cout << *ptr << " " << ptr << "\n";  // Output: 10 [address of arr[0]]
    return 0;
}
```

Output:

```
10 0x7ffe76c5b940
20 0x7ffe76c5b944
10 0x7ffe76c5b940
```

*Explanation:*

- Pointer incrementation (ptr++) moves the pointer from arr[0] to arr[1], increasing the address by **4 bytes** (size of an int).
- Pointer decrementation (ptr--) moves it back to arr[0].




**Example 2: Adding/Subtracting an Integer to/from a Pointer**

```
#include <iostream>
using namespace std;
    int main() {
   int arr[] = {10, 20, 30, 40};
   int *ptr = arr;
       cout << *ptr << " " << ptr << "\n";
    ptr = ptr + 3;
   cout << *ptr << " " << ptr << "\n";
   ptr = ptr - 2;
   cout << *ptr << " " << ptr << "\n";
      return 0;
}
```

**Output:**

```
10 0x7ffe76c5b940
40 0x7ffe76c5b94c
20 0x7ffe76c5b944
```

*Explanation:*

- Adding an integer (ptr + 3) moves the pointer forward by 3 elements, corresponding to **12 bytes** (3 × 4 bytes for int).
- Subtracting an integer (ptr - 2) moves it backward by 2 elements, corresponding to **8 bytes** (2 × 4 bytes for int).

**Example 3: Subtracting Two Pointers**

```
#include <iostream>
using namespace std;
   int main() {
   int arr[] = {10, 20, 30, 40};
   int *ptr1 = arr;
   int *ptr2 = arr + 3;
       cout << *ptr1 << " " << *ptr2 << "\n";
   cout << (ptr2 - ptr1) << "\n";
      return 0;
}
```

**Output:**

```
10 40
3
```

*Explanation:*

- Subtracting ptr1 from ptr2 gives the number of elements between the two pointers, not the difference in bytes.

**Pointer Practice QuestionsKey Tips for Solving Pointer Questions in C++:**

1. **Master Pointer Basics**: Understand how to declare, initialize, dereference pointers, and use the address-of operator (&) to access memory addresses.
2. **Pointers vs References**: Pointers can change the memory they point to, while references are fixed to one memory location.
3. **Pointer Arithmetic**: Learn how to increment, decrement, and navigate arrays or data structures using pointers.

**Question 1**

```
#include <iostream>
using namespace std;
   int main() {
   int arr[] = {10, 20, 30};
   int *ptr = arr;
   cout << sizeof(arr) << "\n";
   cout << sizeof(ptr);
   return 0;
}
```

Explanation:

- sizeof(arr) gives the size of the entire array. Since arr contains 3 integers and each int is 4 bytes (assumption), the result is **12 bytes**.
- sizeof(ptr) gives the size of a pointer, which is **8 bytes** on a 64-bit system (fixed size for pointers regardless of data type).

*Output:**12***
***8***

***Question 2***

```
#include <iostream>
using namespace std;
   int main() {
   char arr[] = {'g', 'f', 'g'};
   char *ptr = arr;
   cout << sizeof(arr) << "\n";
   cout << sizeof(ptr);
   return 0;
}
```

Explanation:

- sizeof(arr) gives the size of the entire char array. Since the array contains 3 characters and each char is 1 byte, the result is **3 bytes**.
- sizeof(ptr) gives the size of the pointer, which is **8 bytes** on a 64-bit system.

*Output:**3***
***8***

***Question 3***

```
#include <iostream>
using namespace std;

int main() {
   int arr[] = {10, 20};
```

```
    int *ptr = arr;
    ++*ptr;
    cout << arr[0] << " " << arr[1] << " " << *ptr;
    return 0;
}
```

Explanation:

- The expression ++*ptr increments the value pointed to by ptr. Since ptr initially points to arr[0], this operation increments arr[0] by 1.
- The final values are:
  - arr[0] = 11
  - arr[1] = 20
  - *ptr = 11 (as ptr still points to arr[0]).

*Output:* ***11 20 11***

### *Question 4*

```
#include <iostream>
using namespace std;
    int main() {
    int arr[] = {10, 20};
    int *ptr = arr;
    *ptr++;
    cout << arr[0] << " " << arr[1] << " " << *ptr;
    return 0;
}
```

Explanation:

- *ptr++ increments the pointer ptr, moving it to the next element (arr[1]).
- Since *ptr++ increments the pointer after dereferencing it, the value at arr[0] is not modified.
- The final values are:
  - arr[0] = 10
  - arr[1] = 20
  - *ptr = 20 (as ptr now points to arr[1]).

*Output:* ***10 20 20***

### *Question 5*

```
#include <iostream>
using namespace std;
    int main() {
    int arr[] = {10, 20};
    int *ptr = arr;
    ++*++ptr;
    cout << arr[0] << " " << arr[1] << " " << *ptr;
    return 0;
}
```

Explanation:

- ++ptr increments the pointer, making it point to arr[1].
- *++ptr dereferences the new pointer value, accessing arr[1].
- ++*++ptr increments the value at arr[1] by 1.
- The final values are:
  - arr[0] = 10
  - arr[1] = 21
  - *ptr = 21 (as ptr points to arr[1]).

***Output: 10 21 21***

**Dynamic Memory Allocation in C++**:- Dynamic memory allocation in C++ refers to performing memory allocation manually by a programmer. Dynamically allocated memory is allocated on Heap, and non-static and local variables get memory allocated on Stack.

**What is Dynamic Memory Allocation?**

Dynamic memory allocation gives programmers the ability to:

- Allocate memory at runtime on the Heap.
- Deallocate memory when it's no longer needed.

Dynamic memory allocation is useful when the required memory size cannot be determined during compile time.

**Key Functions for Dynamic Memory Allocation:**

- new: Allocates memory in C++.
- delete: Deallocates memory in C++.
- malloc and calloc: Legacy C-style functions used for memory allocation.

**new operator**:-The new operator denotes a request for memory allocation on the Free Store. If sufficient memory is available, a new operator initializes the memory and returns the address of the newly allocated and initialized memory to the pointer variable.

**Syntax to use new operator** *pointer-variable = **new** data-type;*

**Initialize memory:** We can also initialize the memory for built-in data types using a new operator. For custom data types, a constructor is required (with the data type as input) for initializing the value.

Example: *pointer-variable = **new** data-type(value);*

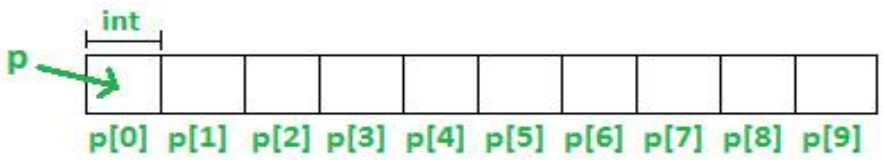
**Allocate a block of memory:** a new operator is also used to allocate a block(an array) of memory of type *data type*. *pointer-variable = **new** data-type[size];*

. Example1: Allocating and Accessing Memory

```
#include <iostream>
using namespace std;
   int main() {
   int *ptr = new int[10];
   return 0;
}
```

### *Memory Visualization:*

1. **Heap Allocation**: Memory is allocated on the heap for 10 integers. The pointer ptr stores the address of the first element.




**Example 2: Accessing Uninitialized Memory**

```
#include <iostream>
using namespace std;

int main() {
```

```
    int *ptr = new int[5];
    cout << *(ptr + 2);  // Access the third element in the array
    return 0;
}
```

**Output** 0 ***Explanation:*** *The value at *(ptr + 2) is uninitialized and may contain garbage data.*

**Example 3: Initializing and Accessing Dynamic Memory**

```
#include <iostream>
using namespace std;
    int main() {
    int *ptr = new int[5];
    *(ptr + 2) = 10;
    cout << *(ptr + 2);
    return 0;
}
```

**Output** 10 ***Explanation:*** *The third element of the allocated array (*(ptr + 2)) is initialized to 10.*

**Example 4: Deallocating Memory**

```
#include <iostream>
using namespace std;
    int main() {
    int *ptr = new int[5];
    *(ptr + 2) = 10;
    delete[] ptr;
    ptr = NULL;
}
```

**Explanation:**

- After delete[] ptr, the allocated memory is returned to the system.
- Setting ptr = NULL ensures it does not point to invalid memory.

**Example 5: Returning Address of Local Variable**

```
#include <iostream>
using namespace std;
    int *fun() {
    int a = 10;
    int *ptr = &a;
    return ptr;
}
    int main() {
    cout << *fun();        // Undefined behavior
    return 0;
}
```

**`Explanation:`** The function returns the address of a local variable that is destroyed after the function ends, so the output is an undefined or garbage value.

**Example 6: Returning Dynamically Allocated Memory**

```
#include <iostream>
using namespace std;
    int *fun() {
    int *ptr = new int;
```

```
    *ptr = 10;
    return ptr;
}

int main() {
    cout << *fun();        // Access dynamically allocated value
    return 0;
}
```

**Explanation:** Memory allocated inside fun persists even after the function ends since it is allocated on the Heap.

**Example 7: Proper Deallocation After Function Call**

```
#include <iostream>
using namespace std;
    int *fun() {
    int *ptr = new int;
    *ptr = 10;
    return ptr;
}
    int main() {
    int *ptr = fun();
    cout << *ptr;
    delete ptr;          // Deallocate memory
    ptr = NULL;          // Set pointer to NULL
    return 0;
}
```

**Output** 10

***Note:*** *Always deallocate memory using delete to avoid memory leaks.*

**Common Mistakes with Dynamic Memory Allocation**

1. **Memory Leaks**: Forgetting to deallocate memory results in unused memory occupying the heap.
2. **Dangling Pointers**: Accessing memory after it has been deallocated leads to undefined behavior.

   ```
   delete ptr;
   cout << *ptr; // Undefined behavior
   ```
3. **Accessing Invalid Memory**: Returning addresses of local variables (stack memory) is invalid.

**Strings in C++** :- A **string** is essentially a sequence of characters. For instance, "gfg" is a string made up of three characters, and "courses" is another example of a string. Any sequence of characters that we write can be referred to as a string. But why do we specifically use the term "string" instead of calling it just an array of characters? Let's explore what makes strings unique in C++.

**Why Strings Are Special** Strings differ from standard arrays (like integer or floating-point arrays) in a few ways:

1. **Small Character Set:**
   - Strings are composed of characters, which usually belong to a small set. For example:
     - Capital letters (A-Z): 26 characters.
     - Lowercase letters (a-z): Another 26 characters.
     - This totals to 52 characters, with additional special characters and control characters.
   - Originally, strings in C++ used the **ASCII character set**, which consists of 128 characters, later extended to 256 characters. This small, predefined set makes it easier to work with strings in many scenarios.

2. **Separator-Free Representation:**
   - Unlike arrays where elements are separated by commas or spaces (e.g., 1, 2, 3), strings do not need separators. For example, the string "GFG" can be directly interpreted as having characters 'G', 'F', and 'G' without any additional delimiters.
3. **Internal Representation:**
   - String characters are internally stored as **integers**. This means:
     - Characters like 'A' to 'Z' are mapped to integer values 65 to 90.
     - Characters like 'a' to 'z' are mapped to integer values 97 to 122.
   - This contiguous mapping helps in programming tasks, such as comparing characters alphabetically. For example, 'A' (65) is less than 'B' (66).

**Unicode and Extended Support** While ASCII serves the purpose in many cases, modern C++ versions also support **Unicode** for a broader range of characters:

- Formats like **UTF-8**, **UTF-16**, and **UTF-32** are available.
- Additionally, wide character types (wchar_t) allow handling of extended character sets, enabling global and multilingual support.

**Types of Strings in C++**

1. **C-Style Strings:**
   - These are essentially arrays of characters terminated with a special character \0 (null character).
   - Example: char str[] = "hello";
2. **C++ String Class:**
   - Introduced as part of the Standard Template Library (STL), the std::string class provides a more intuitive and feature-rich way to handle strings.

**C Style String in C++**:- In C, strings are defined as an array of characters. The difference between a character array and a string is the string is terminated with a special character '\0'. In C, the string is actually represented as an array of characters terminated by a null string. Therefore the size of the character array is always one more than that of the number of characters in the actual string. This thing continues to be supported in C++ too. The C++ compiler automatically sets "\0" at the end of the string, during initialization of the array.**Initializing a String in C++:**

```
1. char str[] = "Geeks";
2. char str[6] = "Geeks";
3. char str[] = {'G', 'e', 'e', 'k', 's', '\0'};
4. char str[6] = {'G', 'e', 'e', 'k', 's', '\0'};
```

Below is the memory representation of a string "Geeks" in C++.

**Commonly Used Functions for C-Style Strings** C-style strings rely on a number of functions defined in the <cstring> header. Let's explore three common functions:

***1. strlen (String Length) :-***The strlen function calculates the length of a string by counting characters until it encounters \0. *Code Example:*

```
#include <iostream>
#include <cstring>
using namespace std;
   int main() {
char s1[] = "gfg";
 char s2[] = {'C', 'P', 'P', '\0'};
   cout << strlen(s1) << endl; // Output: 3
 cout << strlen(s2) << endl; // Output: 3
    return 0;
}
```

**Key Points:**

- strlen ignores the null character while calculating the length.
- If the null terminator is missing, strlen may traverse unintended memory, causing incorrect results or runtime errors.

***2. strcmp (String Comparison) :-*** The strcmp function compares two strings lexicographically:

- Returns 0 if both strings are equal.
- Returns a negative value if the first string is smaller.
- Returns a positive value if the first string is greater. **Code Example:**

```
#include <iostream>
#include <cstring>
using namespace std;
   int main() {
   char s1[] = "gfg";
   char s2[] = "abcd";
      int result = strcmp(s1, s2);
      if (result == 0) {
      cout << "Same" << endl;
   } else if (result < 0) {
      cout << "Smaller" << endl;
   } else {
      cout << "Greater" << endl; // Output: Greater
   }
   return 0;
}
```

**How It Works:**

- Characters are compared one by one based on their ASCII values.
- Comparison stops as soon as a mismatch is found, or one string ends.

***3. strcpy (String Copy) :-****The strcpy function copies the content of one string into another.* ***Code Example:***

```
#include <iostream>
#include <cstring>
using namespace std;
   int main() {
   char str[5];
   strcpy(str, "gfg");
       cout << str << endl; // Output: gfg
   return 0;
}
```

**Key Points:**

- strcpy copies characters, including the null terminator, from the source to the destination.
- Ensure the destination array has enough space to hold the source string and the null terminator.

**String Class in C++**:- The **C++ string class (std::string)** is part of the Standard Template Library (STL) and offers significant advantages over C-style strings. It eliminates many of the limitations and challenges associated with character arrays. **Creating a C++ String**

```
#include <iostream>
using namespace std;
int main() {
   string str = "geeksforgeeks";
   cout << str; // Output: geeksforgeeks
   return 0;
}
```

**Output:**geeksforgeeks

**Advantages of C++ Strings 1. Dynamic Size Management:**

- You do not need to worry about predefining the size of the string. The class handles dynamic memory allocation internally.

**2. Rich Operator Support:**

- Operators like +, <, >, <=, and >= are supported for operations like concatenation and comparison.

**3. Iterator Support:**

- Provides random-access iterators (begin() and end()) for compatibility with STL algorithms.

**4. Easier Assignment:**

- Strings can be initialized as empty and assigned later.

**5. Conversion to C-Style Strings:**

- The c_str() function allows conversion from a C++ string to a C-style string.

**String Operations (Length, Substring and Find)** :-The std::string class in C++ offers a rich set of functionalities that make string manipulation efficient and straightforward. This article delves into some key functions, such as length(), substr(), and find(), along with practical examples to demonstrate their usage.

```
#include <iostream>
using namespace std;
   int main() {
   string str = "gfg";  // Get the length of the string
   cout << str.length() << endl; // Concatenate another string
   str = str + "xyz";
   cout << str << endl;  // Get a substring
   cout << str.substr(1, 3) << endl;    // Find a substring
   cout << str.find("fg") << endl;
     return 0;
}
```

**Explanation of Key Functions**

***1. length() Function***:-*The length() function returns the number of characters in the string, excluding the null terminator (\0). Example:*

```
string str = "gfg";
cout << str.length(); // Output: 3
```

**Details:**

- The function counts the characters in the string.
- For the string "gfg", the length is 3.

**2. String Concatenation** In C++, you can use the + operator to concatenate two strings. This operation appends one string to the end of another. **Example:**

```
string str = "gfg";
str = str + "xyz";
cout << str; // Output: gfgxyz
```

**Details:**

- After concatenation, the string "gfg" becomes "gfgxyz".
- Concatenation does not require manual memory allocation or resizing, as the std::string class manages it automatically.

**3. substr() Function** The substr() function extracts a substring from a string. It takes two arguments:

1. **Start Index:** The position where the substring begins.
2. **Length:** The number of characters to include in the substring (optional). **Example:**

```
string str = "gfgxyz";
cout << str.substr(1, 3); // Output: fgx
```

**Details:**

- The substring starts at index 1 and includes 3 characters (f, g, and x).
- If the length parameter is omitted, the substring includes all characters from the start index to the end of the string.

***4. find() Function :-****The find() function searches for a substring within a string and returns the index of its first occurrence. If the substring is not found, it returns std::string::npos.* Example:

```
string str = "gfgxyz";
cout << str.find("fg"); // Output: 1
```

**Details:**

- The substring "fg" is found starting at index 1.
- If the substring occurs multiple times, find() returns the index of its first occurrence.

**String Comparison in C++** :- The std::string class in C++ provides robust functionalities to search for substrings and compare strings lexicographically. This article delves into the usage of the find() function and comparison operators with practical examples.

**1. Using the find() Function** The find() function in the std::string class searches for the first occurrence of a substring within a string. It returns the index of the first occurrence if found, or std::string::npos if the substring is not present.

**Code Example: Searching for a Substring**

```
#include <iostream>
using namespace std;
```

```
   int main() {
   string str = "geeksforgeeks";
     int res = str.find("eek");
   if (res == string::npos)
      cout << "Not Present\n";
else
      cout << "First occurrence at index " << res << "\n";
   return 0;
}
```

**Output:** First occurrence at index 1

**Explanation:**

- The substring "eek" appears twice in "geeksforgeeks", but the find() function only returns the index of the **first occurrence** (1).

**Code Example: Substring Not Found**

```
#include <iostream>
using namespace std;
   int main() {
   string str = "geeksforgeeks";
     int res = str.find("xyz");
   if (res == string::npos)
      cout << "Not Present\n";
   else
      cout << "First occurrence at index " << res << "\n";
 return 0;
}
```

**Output:** Not Present

**Explanation:**

- When the substring "xyz" is not found in the string, the function returns std::string::npos, a constant representing "not found."

**2. Comparing Strings** C++ provides built-in operators to compare strings lexicographically. These operators include:

- ==: Checks if two strings are equal.
- <: Checks if one string is lexicographically smaller.
- >: Checks if one string is lexicographically greater.
- <=, >=: Additional comparison options.

**Code Example: Lexicographical Comparison**

```
#include <iostream>
using namespace std;
   int main() {
   string s1 = "abc";
   string s2 = "bcd";
       if (s1 == s2)
      cout << "Same\n";
   else if (s1 < s2)
      cout << "Smaller\n";
   else
      cout << "Greater\n";
    return 0;
}
```

**Output:** Smaller

**Explanation:**

- The strings "abc" and "bcd" are compared character by character. Since 'a' is lexicographically smaller than 'b', "abc" is considered smaller than "bcd".

**Code Example: Equal Strings**

```
#include <iostream>
using namespace std;
   int main() {
   string s1 = "abc";
   string s2 = "abc";
   if (s1 == s2)
      cout << "Same\n";
   else if (s1 < s2)
      cout << "Smaller\n";
   else
      cout << "Greater\n";
    return 0;
}
```

**Output:** Same **Explanation:**

- The strings "abc" and "abc" are identical, so the == operator returns true.

**3. How Comparison Operators Work** When comparing two strings:

1. Characters are compared **one by one** starting from the first character.
2. If characters differ at any position, the comparison result is based on the ASCII values of those characters.
3. If one string ends before the other, it is considered smaller.

**Reading a string with spaces** :- When working with strings in C++, reading user input is a common task. The cin and getline() functions are the two primary methods for reading strings. This article explores their functionality, highlights their differences, and demonstrates how to use them effectively, especially when handling strings with spaces.

**1. Using cin to Read Strings:-** The cin function is the simplest way to read a string from the console. It stops reading input at the first whitespace character (space, tab, or newline), which can be a limitation when dealing with multi-word strings. **Code Example: Using cin**

```
#include <iostream>
using namespace std;
int main() {
 string name;
cout << "Enter your name: ";
  cin >> name;
  cout << "Your name is " << name;
    return 0;
}
```

**Output:**

```
Enter your name: Sandeep Jain
Your name is Sandeep
```

**Explanation:**

- When the user enters "Sandeep Jain," cin only captures "Sandeep" because it stops reading at the first space.
- This behavior makes cin unsuitable for reading full names or sentences.

**2. Using getline() to Read Strings:**The getline() function provides a more robust way to read strings. It reads input until it encounters a newline character (\n) or a specified delimiter.

***Code Example: Using getline()***

```
#include <iostream>
using namespace std;
int main() {
 string name;
 cout << "Enter your name: ";
 getline(cin, name);
cout << "Your name is " << name;
 return 0;
}   Output:
Enter your name: Sandeep Jain
Your name is Sandeep Jain
```

**Explanation:**

- Unlike cin, getline() continues reading input, including spaces, until the user presses Enter.
- This makes it ideal for capturing multi-word strings, such as full names or sentences.

**3. Using getline() with a Custom Delimiter:-**The getline() function also allows you to specify a custom delimiter instead of the default newline character. This can be useful when reading data formatted with specific delimiters, such as commas or dollar signs.

***Code Example: Using getline() with a Custom Delimiter***

```
#include <iostream>
using namespace std;
int main() {
 string name;
 cout << "Enter your name: ";
  getline(cin, name, '\$');
 cout << "Your name is " << name;
   return 0;
}   Input:   Sandeep Jain$
```

Output: Your name is Sandeep Jain

**Explanation:**

- In this example, the input stops being read when the \$ character is encountered.
- This feature is particularly useful when processing input data with specific delimiters.

**String Traversal**:- Strings in C++ can be traversed using different techniques. This article demonstrates how to traverse a string using indices, range-based for loops, and iterators, with a focus on a concise explanation of each approach.

```
#include <iostream>
using namespace std;
   int main() {
```

```
 string str = "gfg";  // Traversing using indices
   for (int i = 0; i < str.length(); i++) {
    cout << str[i];
   }
   cout << endl;  // Traversing using range-based for loop
   for (char x : str) {
  cout << x;
   }
   cout << endl;  // Traversing using iterators
   for (auto it = str.begin(); it != str.end(); ++it) {
       cout << *it;
   }
   cout << endl;
 return 0;
}
```

**Explanation**

***1. Traversal Using Indices :-****This approach treats the string as an array and accesses its characters using their indices.*

- str.length() provides the number of characters in the string.
- Each character is accessed using str[i].

**Output for this section:** gfg

**2. Traversal Using a Range-Based for Loop** :- This modern approach simplifies traversal by iterating directly over the characters of the string.

- char x captures each character of the string during iteration.
- The loop eliminates the need for explicit indexing.

**Output for this section:** gfg

***3. Traversal Using Iterators :-****The std::string class supports STL iterators, which allow for efficient traversal.*

- str.begin() returns an iterator pointing to the first character.
- str.end() returns an iterator pointing past the last character.
- The loop iterates through the string using *it to access each character.

**Output for this section:** gfg

**Reverse a StringProblem Statement** Given a string, reverse its characters and output the reversed string.

***Example 1: Input: "geeks" Output: "skeeg"***

***Example 2: Input: "abba" Output: "abba"***

**Approach :-** The goal is to iterate through the string from the last character to the first and construct a new string by appending characters in reverse order**.**

*Steps:*

1. Initialize an **empty string** to hold the reversed string.
2. Find the **last index** of the input string using s.length() - 1.
3. Traverse the string **backward** using a loop:
    - Append the character at the current index to the new string.
    - Decrement the index by 1 after each iteration.
4. Output the reversed string.

**Code Implementation:**

```
#include <bits/stdc++.h>
```

```
using namespace std;
int main() {
   string s;
   cin >> s; // Input the string

 string newstring = ""; // Initialize an empty string for the reversed string
 int l = s.length() - 1; // Calculate the last index of the string
 // Traverse the string in reverse order
 while (l >= 0) {
  newstring = newstring + s[l]; // Append the current character to the new string
   l--; // Decrement the index
   }
 cout << newstring; // Output the reversed string
 return 0;
}
```

**Explanation of the Code**

1. **Input the String:**
   - The input string is read using cin.
2. **Initialization:**
   - newstring is initialized as an empty string to store the reversed string.
   - The variable l is set to s.length() - 1 to point to the last character of the string.
3. **Reversing the String:**
   - A while loop is used to traverse the string from the end (l) to the beginning (l = 0).
   - In each iteration:
     - The character at index l is appended to newstring.
     - The index l is decremented by 1 to move to the previous character.
4. **Output the Result:**
   - After the loop, the reversed string stored in newstring is printed.

**Check for Palindrome in Cpp**:- A **palindrome** is a string that reads the same backward as forward. In this article, we'll walk through the implementation of a program in C++ to check if a given string is a palindrome.

**What is a Palindrome?** A string is considered a palindrome if:

1. Reading the string from the front gives the same result as reading it from the back.
2. Examples:
   - "abba" → Palindrome
   - "abcba" → Palindrome
   - "abc" → Not a palindrome

**Approach** To determine if a string is a palindrome, we compare characters from the beginning and end of the string:

1. Start with two pointers:
   - One at the beginning (start).
   - One at the end (end).
2. Compare characters at start and end.

3. If they differ, the string is not a palindrome.
4. If they are the same, move the start pointer forward and the end pointer backward.
5. Continue until the pointers cross each other.
6. If all characters match, the string is a palindrome.

**Code Implementation**

```
#include <bits/stdc++.h>
using namespace std;
int main() {
   string s;
   cin >> s; // Input the string
  int start = 0;            // Pointer to the beginning of the string
   int end = s.length() - 1;   // Pointer to the end of the string
// Check for palindrome
   while (start < end) {
      if (s[start] != s[end]) {
         cout << "No";      // Not a palindrome
         return 0;         // Exit the program
 }
      start++;              // Move start pointer forward
      end--;               // Move end pointer backward
   }
 cout << "Yes";            // The string is a palindrome
   return 0;
}
```

**Explanation of the Code**

1. **Input the String:**
   - The string s is taken as input from the user.
2. **Initialize Pointers:**
   - start is initialized to 0 (the first character).
   - end is initialized to s.length() - 1 (the last character).
3. **Check Characters:**
   - A while loop runs as long as start < end.
   - If s[start] is not equal to s[end], the string is not a palindrome, and the program prints "No" and exits.
   - If the characters match, increment start and decrement end.
4. **Palindrome Confirmation:**
   - If the loop completes without finding any mismatched characters, the program prints "Yes".

**Binary ProblemsProblem Statement** Convert a given binary string into its decimal representation. **Examples**

1. Binary: "100"
   Decimal: 4
2. Binary: "1010"
   Decimal: 10

**Approach to Solve the Problem :-**The binary-to-decimal conversion can be achieved by leveraging the powers of 2. Each binary digit represents a power of 2, starting from 202^020 at the least significant bit (rightmost) to 2n−12^{n-1}2n−1 at the most significant bit (leftmost).

***Steps:***

1. **Traverse the Binary String:**
   - Start from the last character (least significant bit) and move to the first character (most significant bit).
2. **Calculate Decimal Value:**
   - For each character in the binary string:
     - Convert the character to its integer value.
     - Multiply it by the current base (a power of 2).
     - Add the result to the running total (decimal value).
3. **Update Base:**
   - Multiply the base by 2 after processing each character.
4. **Output the Result:**
   - Print the final decimal value after processing all characters.

**Code Implementation:**

```
#include <bits/stdc++.h>
using namespace std;
void binary_to_decimal(string s) {
   int base = 1; // Represents the power of 2
   int ans = 0;  // Stores the final decimal value
  int size = s.length(); // Traverse the binary string from the end
   for (int i = size - 1; i >= 0; i--) {
       ans += ((int)(s[i] - '0') * base); // Convert character to integer and multiply by base
       base = base * 2; // Update base (power of 2)
   }
 cout << ans; // Output the decimal value
}
int main() {
   string s;
   cin >> s; // Input the binary string
   binary_to_decimal(s); // Call the conversion function
   return 0;
}
```

**Explanation of the Code**

1. **Input:**
   - A binary string s is taken as input.
2. **Initialization:**
   - base is initialized to 1 (corresponding to 202^020).
   - ans is initialized to 0 to store the resulting decimal value.
3. **Iterating Through the String:**
   - The loop starts from the last character (index size - 1) and moves to the first character (index 0).
   - For each character:
     - Convert it to an integer using s[i] - '0'.
     - Multiply it by the current base (power of 2).

        - Add the result to ans.
4. **Updating the Base:**
    - After each iteration, multiply base by 2 to move to the next power of 2.
5. **Output:**
    - Print the final value of ans, which is the decimal equivalent of the binary string.

**Dry Run**

Input: "1010"

| Step | i | s[i] | base | calculation | ans |
|---|---|---|---|---|---|
| 1 | 3 | '0' | 1 | 0 x 1 =0 | 0 |
| 2 | 2 | '1' | 2 | 1 x 2 =2 | 2 |
| 3 | 1 | '0' | 4 | 0 x 4 =0 | 0 |
| 4 | 0 | '1' | 8 | 1 x 8 =8 | 8 |

**Output: 10**

## Pattern Searching | C++ FoundationProblem Statement

Given a text and a pattern:

- Find all occurrences of the pattern in the text.
- Print the starting index of each occurrence.

*Example 1:*

- **Text:** "GeeksforGeeks"
- **Pattern:** "Geek"
- **Output:**

```
Pattern found at 0
Pattern found at 8
```

*Example 2:*

- **Text:** "abababab"
- **Pattern:** "ab"
- **Output:**

```
Pattern found at 0
Pattern found at 2
Pattern found at 4
Pattern found at 6
```

**Approach :-**The C++ std::string class provides a built-in function find() to locate substrings within a string. This function can be used to solve the pattern searching problem efficiently.

*Steps:*

1. Use the find() function to locate the first occurrence of the pattern in the text.
2. If the pattern is found, store the starting index and print it.
3. Use the second parameter of the find() function to specify the position to start the next search (found + 1).
4. Repeat the process until the find() function returns string::npos, indicating no more occurrences.

**Code Implementation**

```
#include <bits/stdc++.h>
using namespace std;// Function to find and print all occurrences of the pattern
void printIndex(string text, string pat) {
   int found = text.find(pat); // Find the first occurrence of the pattern
 // Loop until no more occurrences are found
   while (found != string::npos) {
      cout << "Pattern found at " << found << endl;
      found = text.find(pat, found + 1); // Search for the next occurrence
   }
}
int main() {
   string text, pat;
   cin >> text >> pat; // Input text and pattern
 printIndex(text, pat); // Call the function to search for the pattern
return 0;
}
```

**Explanation of the Code**

1. **Input:**
   - text and pat are taken as input strings, representing the text and the pattern to search for, respectively.
2. **Finding the Pattern:**
   - text.find(pat) locates the first occurrence of the pattern in the text and returns its starting index.
   - If the pattern is not found, text.find(pat) returns string::npos.
3. **Iterating Through the Text:**
   - A while loop runs as long as found != string::npos.
   - Inside the loop:
     - Print the current starting index of the pattern.
     - Update found to search for the next occurrence, starting from found + 1.
4. **Output:**
   - All occurrences of the pattern in the text are printed with their respective starting indices.

## String: Decimal To Binary **Problem Statement**

Given a decimal integer n, convert it into its binary representation and return the result as a string.

*Example 1:*

- **Input:** n = 5
- **Output:** "101"

*Example 2:*

- **Input:** n = 3
- **Output:** "11"

**Understanding the Conversion :-**To convert a decimal number to binary:

1. Continuously divide the number by 2.
2. Store the remainder of each division.
3. The binary representation is the sequence of remainders read in reverse order.

*Example for n = 5:*

1. 5÷2=2, remainder = 1
2. 2÷2=1, remainder = 0
3. 1÷2=0, remainder = 1
4. Binary representation = "101"

*Example for n = 3:*

1. 3÷2=1, remainder = 1
2. 1÷2=0, remainder = 1
3. Binary representation = "11"

**Approach**

1. Initialize an empty string to store the binary representation.
2. Use a loop to:
   - Divide the number by 2 and store the remainder.
   - Append the remainder to the string.
   - Update the number by dividing it by 2.
3. Reverse the string since the binary digits are calculated from least significant to most significant.
4. Return the reversed string.

**Code Implementation:**

```
#include <bits/stdc++.h>
using namespace std; // Function to convert decimal to binary
string decimal_to_binary(int n) {
   string s = ""; // Initialize an empty string to store binary representation
// Loop until the number becomes 0
   while (n > 0) {
      int t = n % 2;          // Calculate the remainder (0 or 1)
      s = s + to_string(t);    // Convert the remainder to string and append it
      n = n / 2;              // Update the number by dividing it by 2
   } // Reverse the string to get the correct binary representation
   reverse(s.begin(), s.end());
 return s; // Return the binary string
}
int main() {
   int n;
   cin >> n; // Take input for the decimal number
// Convert the decimal number to binary and store the result
   string ans = decimal_to_binary(n);
// Print the binary representation
   cout << ans;
return 0;
}
```

**Explanation of the Code**

1. **Input Handling:**
   - The decimal number n is taken as input.
2. **Binary Conversion:**

- Remainders of n%2n \% 2n%2 are appended to the string.
- nnn is updated by dividing it by 2 at each step.
- This process continues until n>0n > 0n>0.

3. **Reversing the String:**
   - Since the remainders are calculated from the least significant bit to the most significant bit, the string is reversed to form the correct binary representation.
4. **Output:**
   - The final binary string is printed.

**Find one extra character in stringProblem Statement Given:**

1. String s1 of length n.
2. String s2 of length n+1.

Both strings consist of the same characters except for one additional character in s2. The task is to find the extra character.**Example**

- **Input:**
  - s1 = "abcd"
  - s2 = "abecd"
- **Output:**
  - Extra Character: e

**Approach 1: Sorting and Comparison Steps:**

1. Sort both strings s1 and s2.
2. Compare characters of s1 and s2 sequentially.
3. If characters differ at any index i, return the character from s2.
4. If all characters match, the last character of s2 is the extra character. **Code:**

```
#include <iostream>
#include <algorithm>
using namespace std;
char findExtra(string s1, string s2) {
    sort(s1.begin(), s1.end());
    sort(s2.begin(), s2.end());
    int n = s1.length();
    for (int i = 0; i < n; i++) {
        if (s1[i] != s2[i]) {
            return s2[i];
        }
    }
    return s2[n];
}
int main() {
    string s1 = "abcd";
    string s2 = "abecd";
    cout << "Extra character is: " << findExtra(s1, s2) << endl;
    return 0;
}
```

**Time Complexity:**

- Sorting both strings: O(nlogn)
- Comparison: O(n)
- **Overall:** O(nlogn)

**Approach 2: Count Array (Frequency Array) Steps:**

1. Initialize a count array of size 256 (for all ASCII characters) with zeros.
2. Traverse s2 and increment the frequency of each character in count.
3. Traverse s1 and decrement the frequency of each character in count.
4. The character with a count of 1 in the count array is the extra character. **Code:**

```
#include <iostream>
#include <string>
using namespace std;
char findExtra(string s1, string s2) {
 int count[256] = {0}; // Increment counts for all characters in s2
  for (char c : s2) {
     count[c]++;
  }  // Decrement counts for all characters in s1
  for (char c : s1) {
     count[c]--;
  } // Find the character with count 1
  for (int i = 0; i < 256; i++) {
     if (count[i] == 1) {
        return (char)i;
     }
  }
 return 0; // Should never reach here
}
int main() {
  string s1 = "abcd";
  string s2 = "abecd";
  cout << "Extra character is: " << findExtra(s1, s2) << endl;
  return 0;
}
```

**Time Complexity:**

- Traversal of s1 and s2: O(n)
- Traversing the count array: O(256)≈O(1)
- **Overall:** O(n)

**Approach 3: Bitwise XOR Steps:**

1. Use the XOR operation on all characters in s1 and s2.
2. Characters common in both strings will cancel out due to the property of XOR.

3. The remaining result will be the ASCII value of the extra character. **Code:**

```
#include <iostream>
using namespace std;
   char findExtra(string s1, string s2) {
   char res = 0;   // XOR all characters in s1
   for (char c : s1) {
      res ^= c;
   }  // XOR all characters in s2
   for (char c : s2) {
      res ^= c;
   }
    return res;
}
int main() {
   string s1 = "abcd";
   string s2 = "abecd";
   cout << "Extra character is: " << findExtra(s1, s2) << endl;
   return 0;
}
```

**Time Complexity:**

- Traversal of s1 and s2: O(n)
- **Overall:** O(n)

**Properties of XOR Used:**

- $x \oplus x = 0$
- $x \oplus 0 = x$
- XOR operation is commutative and associative.

## Check for Pangram

A **pangram** is a string that contains every letter of the English alphabet at least once. In this article, we'll discuss how to determine if a string is a pangram using an efficient algorithm.

A string is considered a pangram if it contains all 26 letters of the English alphabet (case insensitive). The letters can appear in any order and can be either uppercase or lowercase. **Examples:**

1. "The quick brown fox jumps over the lazy dog"
   - **Pangram:** It contains all letters from 'a' to 'z'.
2. "Hello World"
   - **Not a Pangram:** It is missing several letters such as 'a', 'b', 'c', etc.

**Steps in the Algorithm**

1. **Initialize a Visited Array:**
   Create a boolean array visited[26] initialized to false. This array will represent whether each letter ('a' to 'z') has been found in the string.
2. **Traverse the String:**
   Iterate through the string character by character. For each character:
   - If it is a lowercase letter ('a' <= x <= 'z'), mark visited[x - 'a'] as true.
   - If it is an uppercase letter ('A' <= x <= 'Z'), mark visited[x - 'A'] as true.
3. **Check the Visited Array:**
   After traversing the string, if any value in the visited array is still false, it means the corresponding letter is missing, and the string is not a pangram. Otherwise, it is a pangram.
4. **Return the Result:**
   If all entries in visited are true, return true. Otherwise, return false.

**Code Implementation:**

```
#include <iostream>
#include <string>
using namespace std;
   bool isPangram(string s) {
   int n = s.length();
 if (n < 26) {
       return false; // A pangram must have at least 26 characters
   }
bool visited[26] = {false}; // Track presence of letters  // Traverse the string
   for (int i = 0; i < n; i++) {
       char x = s[i];   // For lowercase letters
       if (x >= 'a' && x <= 'z') {
           visited[x - 'a'] = true;
       }  // For uppercase letters
       else if (x >= 'A' && x <= 'Z') {
           visited[x - 'A'] = true;
       }
   }   // Check if all letters are visited
   for (int i = 0; i < 26; i++) {
       if (!visited[i]) {
           return false; // If any letter is missing
       }
   }
   return true;
}
int main() {
   string test1 = "The quick brown fox jumps over the lazy dog";
   string test2 = "Hello World";
   cout << (isPangram(test1) ? "Pangram" : "Not a Pangram") << endl;
   cout << (isPangram(test2) ? "Pangram" : "Not a Pangram") << endl;
    return 0;
}
```

**Explanation of the Code**

1. **Input and Initialization:**
   - The string is passed to the isPangram function.
   - If the length of the string is less than 26, it is immediately ruled out as a non-pangram.
2. **Marking Visited Letters:**
   - For each character, its corresponding position in the visited array is marked true.
3. **Final Check:**
   - After processing the string, if all entries in visited are true, the string is a pangram.

**Time Complexity**

- **Traversing the string:** O(n), where n is the length of the string.
- **Checking the visited array:** O(26) = O(1) (constant time).
- **Total Complexity:** O(n).

**Space Complexity**

- **Visited Array:** O(26) = O(1) (constant space).

**Check whether two strings are anagram of each other** | :- C++ foundationAn anagram is a word or phrase formed by rearranging the letters of a different word or phrase, typically using all the original letters exactly once. In this article, we explore how to check whether two given strings are anagrams of each other using different methods.

**Definition of Anagram :- Two strings are considered anagrams if:**

- Both strings have the same characters.
- The characters in both strings appear in the same frequency.

For example:

- **"listen"** and **"silent"** are anagrams.
- **"triangle"** and **"integral"** are anagrams.
- **"abc"** and **"abd"** are not anagrams.

**Approaches to Solve the Problem** We discuss two approaches to check whether two strings are anagrams:

***1. Method 1: Sorting-*** *This method involves sorting both strings and comparing them character by character. Steps:*

1. Check if the lengths of the two strings are equal. If not, return false.
2. Sort both strings.
3. Compare the sorted strings character by character.
4. If all characters match, the strings are anagrams; otherwise, they are not.

**Code Implementation:**

```
#include <bits/stdc++.h>
using namespace std; // Function to check if two strings are anagrams using the sorting method
bool areAnagramsSorting(string s1, string s2) {  // If lengths are not equal, they cannot be anagrams
   if (s1.length() != s2.length()) {
      return false;
   }  // Sort both strings
   sort(s1.begin(), s1.end());
   sort(s2.begin(), s2.end());// Compare sorted strings
   for (int i = 0; i < s1.length(); i++) {
      if (s1[i] != s2[i]) {
         return false;
      }
   }
 return true;
}
int main() {
   string s1, s2;
   cout << "Enter the first string: ";
   cin >> s1;
   cout << "Enter the second string: ";
   cin >> s2;
if (areAnagramsSorting(s1, s2)) {
      cout << "Yes, the strings are anagrams." << endl;
   } else {
      cout << "No, the strings are not anagrams." << endl;
   }
```

```
return 0;
}
```

**Example:**

- Input: s1 = "listen", s2 = "silent"
  - After sorting: s1 = "eilnst", s2 = "eilnst"
  - Output: Yes
- Input: s1 = "abc", s2 = "abd"
  - After sorting: s1 = "abc", s2 = "abd"
  - Output: No

**Time Complexity:**

- Sorting: O(nlogn)
- Comparison: O(n)
- Total: O(nlogn)

### *2. Method 2: Count Array*

Instead of sorting, we can count the frequency of each character in both strings and then compare these frequencies. **Steps:**

1. Check if the lengths of the two strings are equal. If not, return false.
2. Create a count array of size 256 (to handle all ASCII characters) and initialize it to 0.
3. Traverse the first string and increment the count for each character.
4. Traverse the second string and decrement the count for each character.
5. If the count array has any non-zero value, the strings are not anagrams.

**Code Implementation:**

```
#include <bits/stdc++.h>
using namespace std;
// Function to check if two strings are anagrams using the count array method
bool areAnagramsCountArray(string s1, string s2) {
    // If lengths are not equal, they cannot be anagrams
    if (s1.length() != s2.length()) {
        return false;
    }
    // Initialize a count array of size 256 (for all ASCII characters)
    int count[256] = {0};
    // Increment counts for characters in s1
    for (int i = 0; i < s1.length(); i++) {
        count[s1[i]]++;
    }
    // Decrement counts for characters in s2
    for (int i = 0; i < s2.length(); i++) {
        count[s2[i]]--;
    }
    // If any count is non-zero, the strings are not anagrams
    for (int i = 0; i < 256; i++) {
```

```
        if (count[i] != 0) {
            return false;
        }
    }
 return true;
}
int main() {
    string s1, s2;
    cout << "Enter the first string: ";
    cin >> s1;
    cout << "Enter the second string: ";
    cin >> s2;
 if (areAnagramsCountArray(s1, s2)) {
        cout << "Yes, the strings are anagrams." << endl;
    } else {
        cout << "No, the strings are not anagrams." << endl;
    }
 return 0;
}
```

**Example:**

- Input: s1 = "listen", s2 = "silent"
  - Frequency count:
    - For s1: {l:1, i:1, s:1, t:1, e:1, n:1}
    - For s2: {l:1, i:1, s:1, t:1, e:1, n:1}
  - Output: Yes
- Input: s1 = "abc", s2 = "abd"
  - Frequency count:
    - For s1: {a:1, b:1, c:1}
    - For s2: {a:1, b:1, d:1}
  - Output: No

**Time Complexity:**

- Traversal: O(n)
- Total: O(n)

**Space Complexity:**

- Auxiliary Space: O(1) (constant size array)

**Struct in C++**:- A **struct** in C++ (short for structure) is a user-defined data type that allows grouping of different data types into a single composite object. Structs are particularly useful for managing composite data such as points, student information, or web page attributes.

**Why Use Structs?** Structs enable us to define composite data types to handle situations like:

1. Storing 2D coordinates of a point with x and y values.
2. Managing student details, such as roll number, name, and address, in one structure.
3. Grouping related attributes of a web page (e.g., URL, views, content).

**Defining and Using Structs** A struct is a **user-defined data type** that contains multiple members, which can be of different data types.

*Example 1: Defining a Simple Struct*

```
#include <iostream>
using namespace std;
   typedef struct Point {
   int x;
   int y;
} P;
   int main() {
   P p;
   p.x = 10;
   p.y = 20;
   cout << p.x << " " << p.y;
   return 0;
}
```

***Explanation:***

- typedef creates an alias (P) for the struct type to simplify usage.
- Members of the struct (x and y) are accessed using the **dot operator** (p.x, p.y).

***Output: 10 20***

**Struct Syntax in C++ vs. C**

In **C**, the keyword struct is mandatory when declaring a variable: struct Point p;

In **C++**, the struct keyword is optional: Point p;

**Struct with Multiple Data Types** Structs can group members of different data types.

**Example 2: Student Information**

```
#include <iostream>
using namespace std;
   struct Student {
   int rollNo;
   string name;
   string address;
};
   int main() {
   Student s = {101, "ABC", "xyz"};
   cout << s.rollNo << " "
       << s.name   << " "
       << s.address<< "\n";
   return 0;
}
```

Explanation:

- The struct Student contains an integer and two strings.
- Members are initialized using **curly braces**, maintaining the declared order.

***Output: 101 ABC xyz***

**Designated Initialization** Designated initialization allows initializing specific members of a struct by name, enabling flexibility and clarity.

***Example 3: Designated Initialization***

```
#include <stdio.h>
```

```
   struct Point {
   int x;
   int y;
};
   int main() {
   struct Point p = {.y = 10, .x = 20};
   printf("%d %d", p.x, p.y);
   return 0;
}
```

Explanation:

- Members can be initialized in **any order** by specifying their names (.x, .y).
- Designated initialization is primarily a C-style construct.

***Output: 20 10***

**Default Values in Structs**

If a member of a struct is not explicitly initialized, it may take a **default value** (e.g., 0 for integers). However, uninitialized members can also contain **garbage values**.

***Example 4: Default Values***

```
#include <stdio.h>
   struct Point {
   int x;
   int y;
};
   int main() {
   struct Point p = {.y = 10};
   struct Point t;
   printf("%d %d", t.x, p.x);
   return 0;
}
```

Explanation:

- p.x is not initialized but takes the default value 0.
- t.x is not initialized and contains **garbage data**.

Output:<garbage value> 0

**Struct vs Class in C++** C++ provides two constructs for grouping related variables and functions: **struct** and **class**. While they may seem similar, there are subtle differences that influence how and when to use each.

**Key Differences Between Struct and Class**

**a. Default Access Specifier :-**The most significant difference between structs and classes lies in the **default access specifier**:

- In a **struct**, members are **public** by default.
- In a **class**, members are **private** by default.

**Example 1: Default Access Specifier** Struct Example:

```
#include<iostream>
using namespace std;
   struct Point {
   int x;
   int y;
};
```

```
   int main() {
   Point p;
   p.x = 10;  // Accessible since default is public
   cout << p.x;
   return 0;
} Output: 10
```

*Class Example:*

```
#include<iostream>
using namespace std;

class Point {
   int x;  // Default is private
   int y;
};
   int main() {
   Point p;
   p.x = 10;  // Compiler Error: x is private
   cout << p.x;
   return 0;
}Output:
Compiler Error
```

**Structs Can Act Like Classes Structs in C++ support all features of classes, including:**

- Constructors
- Destructors
- Member functions
- Inheritance

**Example 2: Struct with Constructor and Member Function**

```
#include<iostream>
using namespace std;
   struct Point {
   int x, y;
    Point(int a, int b) {  // Constructor
      x = a;
      y = b;
   }
   void print() {  // Member function
      cout << x << " " << y << endl;
   }
};

int main() {
   Point p(10, 20);
   p.print();  // Output: 10 20
   return 0;
}
```

**Inheritance Default Behavior The default type of inheritance differs for structs and classes:**

- In a **class**, inheritance is **private** by default.
- In a **struct**, inheritance is **public** by default.

**Example 3: Default Inheritance**Class Inheritance:

```
#include<iostream>
using namespace std;
```

```
   class Base {
public:
   int x;
};
   class Derived : Base {};  // Private inheritance by default
   int main() {
   Derived d;
   d.x = 10;  // Compiler Error: x is private in Derived
   return 0;
}
```

*Struct Inheritance:*

```
#include<iostream>
using namespace std;
   struct Base {
   int x;
};
   struct Derived : Base {};  // Public inheritance by default
   int main() {
   Derived d;
   d.x = 10;  // Accessible since default is public
   cout << d.x;
   return 0;
}
```

**Choosing Between Struct and Class** **General Guidelines:**

- Use a **struct**:
  - When you want to **bundle multiple related data items** together.
  - When your object does not require complex behavior (e.g., a mathematical point or a student record).
- Use a **class**:
  - When you care about **data hiding** and **object-oriented design principles** like encapsulation.
  - When your object requires **complex behavior** or interaction with other objects.

| **Class** | **Structure** |
|---|---|
| Members of a class are private by default. | Members of a structure are public by default. |
| It is declared using the **class** keyword. | It is declared using the **struct** keyword. |
| It is normally used for data abstraction and further inheritance. | It is normally used for the grouping of data |

Structure(Pointer Array and Argument) Structs are versatile in C++ programming and can be combined with pointers, arrays, and functions to create flexible and efficient programs.

## Structure Pointers

A **pointer to a structure** allows access to the struct's members indirectly. When accessing struct members through a pointer, the **arrow operator (->)** is used instead of the dot operator.

***Example: Pointer to a Structure***

```
#include <iostream>
using namespace std;
   struct Point {
   int x;
   int y;
};
```

```
int main() {
Point p = {10, 20};   /
Point *ptr = &p;      // Pointer to the struct
   cout << ptr->x << " ";  // Access members using the arrow operator
ptr->x = 30;
cout << p.x;
return 0;
}
```

Output: 10 30

***Explanation***:

- ptr->x accesses x of the struct pointed to by ptr.
- Changing x through the pointer also updates p.x because both refer to the same memory location.

**Array of Structures**

An **array of structures** allows creating and managing multiple instances of a struct simultaneously.

***Example: Array of Structures***

```
#include <iostream>
using namespace std;
   struct Point {
   int x;
   int y;
};
   int main() {
   Point arr[5];   // Initialize members of each struct in the array
   for (int i = 0; i < 5; i++) {
      arr[i].x = i;
      arr[i].y = i * 10;
   }// Print the members
   for (int i = 0; i < 5; i++) {
      cout << arr[i].x << " " << arr[i].y << endl;
   }
    return 0;
}
```

Output: 0 0
1 10
2 20
3 30
4 40

***Explanation***:

- Each element in the array is a separate struct object.
- Use arr[i].x and arr[i].y to access members of the i-th struct in the array.

**Passing Structures as Arguments to Functions**

Structs can be passed to functions either **by value**, **by reference**, or **by pointer**.

**Passing by Value:-**When a struct is passed by value, a copy is created. This is inefficient for large structs and does not allow modification of the original struct.

**Passing by Reference** Passing by reference avoids creating a copy and allows the function to modify the original struct.  Example:

```
#include <iostream>
using namespace std;
   struct Student {
   int roll;
   string name;
   string address;
};
   void print(const Student &s) {
   cout << s.roll << " "
       << s.name << " "
<< s.address << endl;
}
   int main() {
   Student s = {101, "Alice", "Wonderland"};
   print(s); // Pass by reference
   return 0;
}
```

**Output**: 101 Alice Wonderland

***Advantages***:

- Avoids unnecessary copying of large structs.
- Using const ensures the function cannot modify the struct.

**Passing by Pointer** Passing by pointer is another efficient method. Members of the struct are accessed using the arrow operator (->).

```
#include <iostream>
using namespace std;
   struct Student {
   int roll;
   string name;
   string address;
};
   void print(const Student *s) {
   cout << s->roll << " "
       << s->name << " "
       << s->address << endl;
}
   int main() {
   Student s = {101, "Bob", "Narnia"};
   print(&s); // Pass by pointer
   return 0;
}
```

Output**:**

```
101 Bob Narnia
```

***Advantages***:

- Similar to passing by reference but explicitly uses pointers.
- Useful when working with dynamic memory.

**Avoiding Common Mistakes**

1. **Copy Overhead**: Passing structs by value is inefficient for large structs. Always prefer passing by reference or pointer for larger objects.
2. **Unnecessary Modifications**: Use const with references or pointers to prevent accidental modifications of struct members.

**Structure Alignment and Padding in C++ What is Structure Alignment?**

Alignment refers to arranging data in memory based on certain rules defined by the processor and compiler. Each data type (e.g., int, float, double) has specific alignment requirements, which determine how data is stored in memory.

**Key Points:**

1. **Alignment Requirements**:
   - Each data type requires memory to be aligned at specific byte boundaries.
   - For example:
     - int might require alignment to 4-byte boundaries.
     - double might require alignment to 8-byte boundaries.
2. **Largest Member Alignment**:
   - A structure aligns itself based on its largest member's alignment requirement.

**Padding in Structures** Padding is the extra memory added between structure members to ensure alignment requirements are met. This ensures efficient memory access by the processor.

**Example 1: Structure Alignment and Padding**

```
#include <iostream>
using namespace std;
   struct S1 {
   char c1;    // 1 byte
   double d1;  // 8 bytes
   char c2;    // 1 byte
};
   int main() {
   cout << sizeof(S1);  // Output depends on alignment rules
   return 0;
}
```

*Memory Layout of S1:*

```
c1 [1 byte] + padding [7 bytes]

d1 [8 bytes]

c2 [1 byte] + padding [7 bytes]

Total = 24 bytes (on a 64-bit system)
```

**Why Padding is Necessary** **Reasons for Alignment:**

1. **Memory Access Efficiency**:
   - Modern CPUs access memory in chunks (e.g., 4 or 8 bytes). Misaligned data may require multiple memory accesses, reducing efficiency.
2. **Processor Architecture**:
   - 32-bit and 64-bit processors impose alignment rules to improve performance. Example:

Consider the following structure:

```
struct S2 {
   char c1;    // 1 byte
   char c2;    // 1 byte
   double d1;  // 8 bytes
```

```
};
```

- Without alignment:
  - d1 would span across multiple memory chunks.
- With alignment:
  - d1 starts at a memory address divisible by 8, ensuring efficient access.

**Reducing Padding** In some cases, padding can be minimized or eliminated to save memory. This is achieved by:

1. **Reordering Members**:
   - Place larger data types first, followed by smaller ones. Example:

```
struct S3 {
   double d1; // 8 bytes
   char c1;   // 1 byte
   char c2;   // 1 byte
};
```

**Memory Layout**:

*d1 [8 bytes]*

*c1 [1 byte]*

*c2 [1 byte] + padding [6 bytes]*

*Total = 16 bytes*

By placing the largest member (double d1) first, we minimize padding compared to placing char members before it.

**Using Compiler Directives for Packing :-** Many compilers provide directives to enforce compact representation of structures by eliminating or reducing padding.

*Example 2: Compact Representation Using attribute ((packed)) (GCC/Clang)*

```
struct S4 {
   double d1;  // 8 bytes
   char c1;    // 1 byte
   char c2;    // 1 byte
} __attribute__((packed));  // GCC/Clang-specific directive

int main() {
   cout << sizeof(S4) << endl;
   return 0;
}
```

***Memory Layout**:*

```
d1   [8 bytes]
c1   [1 byte]
c2   [1 byte]
Total = 10 bytes
```

This directive tells the compiler not to add padding, resulting in a compact structure.

**Trade-offs of Compact Representation While reducing padding saves memory, it may lead to:**

- **Performance Degradation**: Misaligned memory access requires multiple CPU cycles.
- **Increased Complexity**: Accessing packed structures may require special handling in some architectures.

**Union in C++**Unions in C++ are a special type of user-defined data type that allow the storage of multiple data members in the same memory location. While syntactically similar to structures, unions differ significantly in terms of internal memory representation and applications.

**What is a Union?**

A **union** is used to store multiple members, but only one member can occupy the memory at a time. This feature makes unions memory-efficient when the stored values are mutually exclusive.

**Defining a Union**

```
union Test {
   int x;
   int y;
};
```

- **Keyword**: Use the union keyword to define unions.
- **Syntax**: Similar to structures (struct) and classes.
- **Access**: Members are accessed using the . operator, or the -> operator if using a pointer.

**Difference Between Union and Structure**

1. **Memory Allocation**:
   - **Structure**: Allocates memory for all members.
   - **Union**: Allocates memory equal to the largest member size.
2. **Member Access**:
   - For unions, modifying one member affects others since they share the same memory location.

```
#include<iostream>
using namespace std;
   union Test {
   int x;
   int y;
};
   int main() {
   Test t;
   t.x = 10;
   cout << t.x << " " << t.y << '\n';  // Output: 10 10
   t.y = 20;
   cout << t.x << " " << t.y << '\n';  // Output: 20 20
   return 0;
}
```

**Explanation**:

- Assigning 10 to t.x updates the shared memory.
- Reassigning 20 to t.y overwrites the same memory.

**Size of a Union:-**The size of a union is determined by the largest member it contains, plus padding for alignment if necessary. **Example:**

```
#include<iostream>
using namespace std;
   union Test {
   int x;      // 4 bytes
   char y;     // 1 byte
   double z;   // 8 bytes
};
   int main() {
   cout << sizeof(Test);  // Output: 8 (size of double)
```

```
    return 0;
}
```

**Applications of Unions**

**1. Type Punning:-**Type punning allows interpreting the binary representation of one data type as another without explicit type casting. Example:

```
#include<iostream>
using namespace std;

union Test1 {
    int x;
    float y;
};
    int main() {
    Test1 t1;
    t1.y = 1.1f;
    cout << t1.x << '\n';  // Output: Binary representation of 1.1 as an int
    return 0;
}
```

**2. Byte-Level Access:-** Unions can be used to access individual bytes of larger data types, which is useful for understanding or manipulating the internal representation of numbers.Example:

```
#include<iostream>
using namespace std;
    union Test2 {
    int x;
    char bin[4];
};
    int main() {
    Test2 t2;
    t2.x = 512;
    cout << (int)t2.bin[0] << " " << (int)t2.bin[1] << " "
        << (int)t2.bin[2] << " " << (int)t2.bin[3];
    return 0;
}
```

**Explanation**:

- t2.x = 512 stores the integer value 512 in memory.
- Accessing bin[0], bin[1], etc., retrieves individual bytes of 512.

**Using Anonymous Unions:-**Anonymous unions allow you to use the union's members without explicitly referencing the union name. This is useful for multi-purpose structures. Example: Node for Binary Tree and Linked List

```
#include<iostream>
using namespace std;
    struct Node {
    int data;
    union {
        struct {
            Node *left, *right;  // For binary tree
        };
```

```
      struct {
         Node *prev, *next;   // For doubly linked list
      };
   };
};

int main() {
   Node n;
   n.data = 10;
   n.left = nullptr;  // Binary tree usage
   n.next = nullptr;  // Doubly linked list usage
   return 0;
}
```

**Explanation**:

- The union allows a single Node structure to be used in different contexts, like a binary tree or a doubly linked list.

**Advantages and Disadvantages**

**Advantages:**

1. **Memory Efficiency**: Requires only enough memory for the largest member.
2. **Versatility**: Useful for type punning and multi-purpose data types.

**Disadvantages:**

1. **Complexity**: Members share memory, leading to potential overwrites.
2. **Limited Use Cases**: Best for mutually exclusive data types.

**Complex Number** :- Addition Using Structure in C++Complex numbers are numbers in the form a+bi, where a is the real part and b is the imaginary part. To perform operations on complex numbers, such as addition, we can use structures in C++ to group the real and imaginary parts of a complex number into a single entity.

**Example Problem  Suppose we are given two complex numbers:**

1. a+bi
2. c+di

The task is to add these two numbers. The result of the addition is calculated as:

- Real part: a+c
- Imaginary part: b+d

For example:

- Input: 1+2i and 3+4i
- Output: 4+6i

**Below is the C++ program that demonstrates how to add two complex numbers using structures.**

```
#include<iostream>
using namespace std;
   struct Complex
```

```
{
   int real, imaginary;
};
int main()
{
  Complex n1, n2, n3;

 cin >> n1.real >> n1.imaginary;
   cin >> n2.real >> n2.imaginary;

   n3.real = n1.real + n2.real;
   n3.imaginary = n1.imaginary + n2.imaginary;

   cout << n3.real << " + " << n3.imaginary << 'i';
   return 0;
}
```

**Explanation**

1. **Structure Definition**:
   - A structure named Complex is defined to represent a complex number.
   - It contains two members: real (for the real part) and imaginary (for the imaginary part).
2. **Input**:
   - Two complex numbers (n1 and n2) are declared.
   - The user provides the real and imaginary parts for each complex number.
3. **Addition**:
   - The real parts of n1 and n2 are added and stored in n3.real.
   - The imaginary parts of n1 and n2 are added and stored in n3.imaginary.
4. **Output**:
   - The program outputs the resulting complex number in the format: n3.real+n3.imaginaryin3.real + n3.imaginary in3.real+n3.imaginaryi.

**Example Execution**

**Input:**

```
5 3
4 6
```

**Output:**

```
9 + 9i
```

**Multidimensional array in CPP** :- In C++, multidimensional arrays allow us to store data in a matrix or tabular form. For example, a 2D array stores data in rows and columns, while a 3D array adds an extra layer of depth. These arrays are commonly used for tasks like working with matrices, grids, or any form of structured data.

The ***general form of declaring N-dimensional arrays*** is: data_type array_name[size1][size2]....[sizeN];

- **data_type**: Type of data to be stored in the array.
- **array_name**: Name of the array
- **size1, size2,… ,sizeN**: Sizes of the dimension

**2D Arrays in C++**

**1. Fixed size Arrays:- A** 2D array **is a matrix with rows and columns. Here's an example of a 2D array:**

```
#include <iostream>
using namespace std;
   int main()
{
   int arr[3][2] = {{10, 20}, {30, 40}, {50, 60}}; // 3 rows, 2 columns

   for (int i = 0; i < 3; i++) // Loop through rows
   {
      for (int j = 0; j < 2; j++) // Loop through columns
      {
         cout << arr[i][j] << " ";
      }
   }
    return 0;
}
```

**Output**: 10 20 30 40 50 60

This array is initialized using curly brackets, where each row's values are enclosed in separate curly brackets.

**Row-Major Order :- C++ stores multidimensional arrays in** row-major order**:**

- Data from the first row is stored first, followed by the second row, and so on.
- Example: For the above array, data is stored as 10, 20, 30, 40, 50, 60.

If the memory starts at address 2000 and an integer takes 4 bytes, the memory layout will be:

- 2000: 10, 2004: 20, 2008: 30, 2012: 40, 2016: 50, 2020: 60.

**Initialization**

1. **With Internal Braces**:
   int arr[3][2] = {{10, 20}, {30, 40}, {50, 60}};
2. **Without Internal Braces** (optional, but less readable):
   int arr[3][2] = {10, 20, 30, 40, 50, 60};

**Variable sized 2D Arrays :- If the size of the array isn't known at compile time, you can dynamically allocate memory for 2D arrays.Code Example**

```
#include <iostream>
using namespace std;
   int main()
{
   int m = 3, n = 2;
   int arr[m][n];    // 2D array
   for (int i = 0; i < m; i++)
   {
      for (int j = 0; j < n; j++)
      {
         arr[i][j] = i + j; // Assigning values
      }
   }
   for (int i = 0; i < m; i++)
   {
      for (int j = 0; j < n; j++)
      {
         cout << arr[i][j] << " ";
```

```
    }}
   return 0;
}
Output: 0 1 1 2 2 3
```

A two – dimensional array can be seen as a table with ‘x’ rows and ‘y’ columns where the row number ranges from 0 to (x-1) and the column number ranges from 0 to (y-1). A two – dimensional array ‘x’ with 3 rows and 3 columns is shown below:

| | Column 0 | Column 1 | Column 2 |
|---|---|---|---|
| Row 0 | x[0][0] | x[0][1] | x[0][2] |
| Row 1 | x[1][0] | x[1][1] | x[1][2] |
| Row 2 | x[2][0] | x[2][1] | x[2][2] |

**Creating 2D Arrays Dynamically**

**1. Using Double Pointers :-** To create a dynamic 2D array of size 3 x 2, we can use a **double pointer** approach. Here’s how it works:

```
#include <iostream>
using namespace std;
   int main()
{
   int m = 3, n = 2;
   int **arr = new int *[m]; // Allocate memory for row pointers

for (int i = 0; i < m; i++)
      arr[i] = new int[n]; // Allocate memory for each row

   for (int i = 0; i < m; i++)
      for (int j = 0; j < n; j++)
         arr[i][j] = 10;

   for (int i = 0; i < m; i++)
      for (int j = 0; j < n; j++)
         cout << arr[i][j] << " ";

   return 0;
}
```

**How the Memory is Structured**

- The double pointer (int **arr) points to an **array of row pointers**.
- Each row pointer points to a dynamically allocated array for that row. *For example:*

```
arr -> [row1_pointer] -> [10, 10]
      [row2_pointer] -> [10, 10]
      [row3_pointer] -> [10, 10]
```

**Output**:

```
10 10 10 10 10 10
```

- **Dynamic Allocation**:
  - Memory is allocated dynamically for both the row pointers and the rows themselves.
  - This approach ensures flexibility in creating 2D arrays at runtime.
- **Jagged Arrays**:
  - Using this technique, you can also create **jagged arrays**, where each row has a different number of columns.
  - For example:
    ```
    for (int i = 0; i < m; i++)
      arr[i] = new int[i + 1]; // Rows with increasing column sizes
    ```
    - In this case, the first row will have 1 column, the second row 2 columns, and so on.
- **Limitations**:
- This approach is **not cache-friendly** because rows are stored in non-contiguous memory locations. As a result, accessing the elements may cause **cache misses**, slowing down performance.

**2. Using Array of Pointers :-** This method creates an array of pointers, each pointing to a dynamically allocated row:

```
#include <iostream>
using namespace std;

int main()
{
   int m = 3, n = 2;
   int *arr[m];

   for (int i = 0; i < m; i++)
      arr[i] = new int[n];

   for (int i = 0; i < m; i++)
      for (int j = 0; j < n; j++)
         arr[i][j] = 10;

   for (int i = 0; i < m; i++)
      for (int j = 0; j < n; j++)
         cout << arr[i][j] << " ";

   return 0;
}
```

**How the Memory is Structured**

- The array arr contains **pointers to rows**, which are dynamically allocated on the heap. *For example:*

```
arr -> [row1_pointer] -> [10, 10]
      [row2_pointer] -> [10, 10]
      [row3_pointer] -> [10, 10]
```

- Here:
  - arr is stored on the **stack**.
  - Each row is dynamically allocated on the **heap**.

**3. Using Array of Vectors** :- This method uses STL vectors to create a 2D array. Specifically, an array of vectors is utilized to dynamically manage rows and their contents.

```
#include <iostream>
#include <vector>
using namespace std;

int main()
{
   int m = 3, n = 2;
   vector<int> arr[m];

   for (int i = 0; i < m; i++)
      for (int j = 0; j < n; j++)
         arr[i].push_back(10); // Push values into the vector

   for (int i = 0; i < m; i++)
      for (int j = 0; j < n; j++)
         cout << arr[i][j] << " ";

   return 0;
}
```

**How Memory is Structured** In this method:

- arr is an array on the stack.
- Each element of arr is a dynamically resizable **vector**, which stores its data on the heap. *For example:*

```
arr -> [vector1] -> [10, 10]
      [vector2] -> [10, 10]
      [vector3] -> [10, 10]
```

***Advantages:***

1. **Dynamic Row Sizes**:
   - Individual rows can have **dynamic sizes**, meaning the number of elements in each row can grow or shrink as needed.
2. **Simplified Memory Management**:
   - STL vectors manage memory internally, eliminating the need for manual new and delete operations.
3. **Ease of Use**:
   - Vectors provide built-in functionality like push_back, size, etc., making operations on 2D arrays more intuitive.

***When to Use This method is ideal when:***

- You need **dynamic row sizes** but a **fixed number of rows**.
- You prefer STL features for easier memory management and operations.

**4. Using Vector of Vectors :-** The fourth method for creating 2D arrays in C++ utilizes a vector of vectors, which is one of the most flexible ways to implement a 2D array. This method not only allows dynamic row sizes but also permits the number of rows to be dynamic, making it highly versatile.

```
#include <iostream>
#include <vector>
using namespace std;

int main()
```

```
{
   int m = 3, n = 2;
   vector<vector<int>> arr;

   for (int i = 0; i < m; i++)
   {
      vector<int> row;
      for (int j = 0; j < n; j++)
         row.push_back(10);
      arr.push_back(row);
   }

   for (int i = 0; i < arr.size(); i++)
      for (int j = 0; j < arr[i].size(); j++)
         cout << arr[i][j] << " ";

   return 0;
}
```

*Output:*

```
10 10 10 10 10 10
```

**When to Use  This method is ideal when:**

- Both the number of rows and the size of each row need to be **dynamically determined** at runtime.
- You require the flexibility of dynamic resizing or deletion of rows or elements.

**Passing 2D arrays as arguments in C++**:- Passing multi-dimensional arrays as arguments in functions is an essential concept in C++ programming. It helps developers handle matrices and 2D arrays for various operations like printing, transposing, and other manipulations.

**Example 1: Fixed-Size Arrays:-**The simplest way to pass a 2D array is by specifying both dimensions explicitly in the function definition. Consider the following example**:**

```
#include <iostream>
using namespace std;

void print(int mat[3][2]) {
   for (int i = 0; i < 3; i++) {
      for (int j = 0; j < 2; j++) {
         cout << mat[i][j] << " ";
      } }  }

int main() {
   int mat[3][2] = {{10, 20}, {30, 40}, {50, 60}};
   print(mat);
   return 0;
}
```

**Output:** 10 20 30 40 50 60

Limitation:

- The print function works only for a 3x2 matrix. If the matrix size changes, this function needs to be rewritten.

**Example 2: Omitted First Dimension :-** C++ allows omitting the first dimension when declaring the function parameter for a 2D array. This makes the function flexible for arrays with the same column size but varying row sizes.

```
#include <iostream>
using namespace std;

void print(int mat[][2], int m) {
   for (int i = 0; i < m; i++) {
      for (int j = 0; j < 2; j++) {
         cout << mat[i][j] << " ";
      } }}

int main() {
   int mat[3][2] = {{10, 20}, {30, 40}, {50, 60}};
   print(mat, 3);
   return 0;
}
```

**Output:** 10 20 30 40 50 60

Advantage:

- The function now works for any matrix with 2 columns and an arbitrary number of rows.

Limitation:

- This approach still requires the column size to be fixed at compile-time.

**Example 3: Double Pointers**:- Using double pointers offers flexibility in handling matrices of any size. Memory is dynamically allocated for rows and columns.

```
#include <iostream>using namespace std;

   void print(int **arr, int m, int n) {
   for (int i = 0; i < m; i++) {
      for (int j = 0; j < n; j++) {
         cout << arr[i][j] << " ";
      }}}
   int main() {
   int m = 3, n = 2;
   int **arr = new int*[m];
   for (int i = 0; i < m; i++)
      arr[i] = new int[n];

   for (int i = 0; i < m; i++) {
      for (int j = 0; j < n; j++) {
         arr[i][j] = i + j;
      }}
   print(arr, m, n);
    return 0;
}
```

**Output:** 0 1 1 2 2 3

Advantages:

- Works for matrices of any size.
- Offers flexibility for dynamic memory allocation.

Disadvantages:

- Manual memory management is required, which can lead to potential memory leaks if not handled correctly.
- Rows may not be stored contiguously in memory, making this method less cache-friendly.

**Example 4: Array of Pointers:-** This approach is similar to using double pointers but uses a static array for row pointers, reducing the need for dynamic memory allocation for row pointers**.**

```
#include <iostream>
#include <cmath>
#include <bits/stdc++.h>
#include <climits>
#include <deque>
using namespace std;

void print(int **arr, int m, int n)
{
   for(int i = 0; i < m; i++)
   {
      for(int j = 0; j < n; j++)
         cout << arr[i][j] << " ";
   } }

int main()
{
   int m = 3, n = 2;
   int *arr[m];
   for(int i = 0; i < m; i++)
   {
      arr[i] = new int[n];
      for(int j = 0; j < n; j++)
      {
         arr[i][j] = i;
         cout << arr[i][j] << " ";
      }  }
   return 0;
}
```

**Explanation:**

- An array of pointers (int* arr[m]) is used instead of a double pointer.
- Each element of the array points to a dynamically allocated row.
- The print function takes this array of pointers along with the dimensions to print the matrix.
- **Advantage:** More straightforward than using double pointers.

**Example 5: Using STL Array of Vectors:-**By leveraging STL vectors, we can simplify the process of passing and managing matrices.

```
#include <iostream>
#include <cmath>
#include <bits/stdc++.h>
#include <climits>
#include <deque>
using namespace std;

void print(vector<int> arr[], int m)
{
   for(int i = 0; i < m; i++)
```

```
    {
        for(int j = 0; j < arr[i].size(); j++)
            cout << arr[i][j] << " ";
    }}
    int main()
{
    int m = 3, n = 2;
    vector<int> arr[m];
    for(int i = 0; i < m; i++)
    {
        for(int j = 0; j < n; j++)
        {
            arr[i].push_back(i);
        }  }

    print(arr, m);
    return 0;
}
```

**Explanation:**

- Instead of traditional arrays, an array of vectors is used.
- Each vector represents a row, and elements are added using the push_back method.
- The print function uses the size method of vectors to iterate through the matrix elements.
- **Advantage:** Allows dynamic resizing of rows.

***Example 6: Using Vector of Vectors*** This is the most flexible and modern approach in C++, allowing both dynamic rows and columns.

```
#include <iostream>
#include <cmath>
#include <bits/stdc++.h>
#include <climits>
#include <deque>
using namespace std;

void print(vector<vector<int>> arr)
{
    for(int i = 0; i < arr.size(); i++)
    {
        for(int j = 0; j < arr[i].size(); j++)
            cout << arr[i][j] << " ";
    }
}

int main()
{
    int m = 3, n = 2;

    vector<vector<int>> arr;

    for(int i = 0; i < m; i++)
    {
        vector<int> v;

        for(int j = 0; j < n; j++)
        {
            v.push_back(i);
        }

        arr.push_back(v);
```

```
    }

    print(arr);
    return 0;
}
```

**Explanation:**

- A vector of vectors is used to represent the matrix.
- Rows and columns are dynamically created and managed using STL vectors.
- The print function uses the size method of vectors to determine the number of rows and columns.

**Advantages:**

- Fully dynamic: Both rows and columns can grow or shrink.
- Easy to pass as a reference to functions, avoiding unnecessary copying.
- **Drawback:** Slightly less cache-friendly due to non-contiguous memory allocation.

*Summary of Methods*

| **Method** | **Flexibility** | **Ease of Use** | **Performance** |
|---|---|---|---|
| Fixed-Size Arrays | Limited to fixed size | Simple | High (cache-friendly) |
| Omitted First Dimension | Flexible column size | Moderate | High (cache-friendly) |
| Double Pointers | Fully dynamic | Complex | Low (less cache-friendly) |
| Array of Pointers | Fully dynamic | Moderate | Low (less cache-friendly) |
| STL Array of Vectors | Fully dynamic rows | Moderate | Moderate |
| STL Vector of Vectors | Fully dynamic rows/columns | Easy | Moderate |

**Transpose of a Matrix**:- The **transpose** of a matrix is obtained by **swapping its rows and columns**. In simpler terms:

- Each row of the original matrix becomes a column in the transposed matrix.
- Each column of the original matrix becomes a row in the transposed matrix.
- In other words, transpose of A[N][M] is obtained by changing A[i][j] to A[j][i].

**Example:**

**Approach to Solve the Problem** To transpose a matrix:

1. Create a **2D vector** to store the input matrix.
2. Create another **2D vector** to store the transposed matrix.
3. Traverse through the input matrix and assign:
   Transpose[i][j] = Original[j][i]

**Implementation** **Below is the implementation of the transpose of a matrix in C++:**

```
#include <bits/stdc++.h>
using namespace std;
  int main() {
  int n;
  cin >> n;

  vector<vector<int>> vec(n, vector<int> (n, 0));
  vector<vector<int>> ans(n, vector<int> (n, 0));

  for(int i = 0; i < n ;i++){
    for(int j = 0; j < n; j++){
      int t;
      cin >> t;
      vec[i][j] = t;
    }
  }

  for(int i = 0; i < n ;i++){
    for(int j = 0; j < n; j++){
      cout << vec[i][j] << " ";
    }
    cout << "\n";
  }

  for(int i = 0; i < n ;i++){
    for(int j = 0; j < n; j++){
      ans[i][j] = vec[j][i];
    }
  }

  cout << "\n\nTranspose: \n\n";

  for(int i = 0; i < n ;i++){
    for(int j = 0; j < n; j++){
      cout << ans[i][j] << " ";
    }
    cout << "\n";
  }

  return 0;
}
```

Input:

```
3
1 2 3
4 5 6
7 8 9
```

Output:

```
Original Matrix:
1 2 3
4 5 6
7 8 9

Transpose of the Matrix:
1 4 7
2 5 8
3 6 9
```

**Input Matrix**

1. We first take the size of the matrix n.
2. A 2D vector vec is created to store the n x n matrix.
3. Elements are taken as input and stored in vec.

**Print the Original Matrix**

- A nested loop is used to print the elements of the input matrix row by row.

**Transpose Logic**

1. We create another 2D vector ans to store the transposed matrix.
2. Using two nested loops, we swap the rows and columns:
3. The element at position (j, i) in the original matrix becomes (i, j) in the transposed matrix.

**Multiplication of Matrices** :- Matrix multiplication is the operation of multiplying two matrices by taking the **dot product of rows of the first matrix with columns of the second matrix**.

For two matrices A and B:

- If A has dimensions **n** × **m** and B has dimensions **m** × **p**, then the resulting matrix C will have dimensions **n** × **p**.
- The value at position (i, j) in the resulting matrix is calculated as:

```
C[i][j] = A[i][0]*B[0][j] + A[i][1]*B[1][j] + ... + A[i][m-1]*B[m-1][j]
```

**Implementation**

```
#include <bits/stdc++.h>
using namespace std;

int main()
{
   int n;
   cin >> n;

   vector<vector<int>> matA(n, vector<int> (n, 0));
   vector<vector<int>> matB(n, vector<int> (n, 0));
   vector<vector<int>> res(n, vector<int> (n, 0));

   for(int i = 0; i < n ; i++){
      for(int j = 0; j < n; j++){
         int t;
         cin >> t;
         matA[i][j] = t;
      }   }

   for(int i = 0; i < n ; i++){
```

```
        for(int j = 0; j < n; j++){
            int t;
            cin >> t;
            matB[i][j] = t;
        }  }
    for(int i = 0; i < n ; i++){
        for(int j = 0; j < n; j++){
            res[i][j] = 0;
            for(int k = 0; k < n; k++){
                res[i][j] = res[i][j] + (matA[i][k] * matB[k][j]);
            }}}

    for(int i = 0; i < n ;i++){
        for(int j = 0; j < n; j++){
            cout<< res[i][j] << " ";
        }
        cout << "\n";
    }

    return 0;
}
```

Explanation of the Code

1. **Input the Size of the Matrix**:
    - Read the value n for the size of square matrices.
2. **Create 2D Vectors**:
    - matA for matrix A.
    - matB for matrix B.
    - res for storing the result of multiplication.
3. **Input Elements for Matrices**:
    - Take user input for matA and matB.
4. **Matrix Multiplication Logic**:
    - For each element (i, j) in the result matrix res:
        - Compute the dot product of the ith row of matA with the jth column of matB:
          res[i][j] = Σ(matA[i][k] * matB[k][j]), where k ranges from 0 to n-1
        - Use three nested loops:
            - Outer loop for rows of matA.
            - Middle loop for columns of matB.
            - Inner loop for the dot product computation.  Example Execution

*Input*

```
2
1 2
3 4
1 2
3 4
```

Output

```
Resultant Matrix:
7 10
15 22
```

**Templates in C++**:- Templates are one of the most powerful features of C++ that allow the creation of generic, reusable, and type-safe code. By enabling developers to write a single piece of code that can work with different data types, templates eliminate redundancy and make programs more flexible.

Templates are expanded at compiler time. This is like macros. The difference is, that the compiler does type

checking before template expansion. The idea is simple, source code contains only function/class, but compiled code may contain multiple copies of the same function/class.

**What Are Templates?**

Templates allow developers to write generic code, where the data type is determined at compile time. The idea is to **write code once and use it for any data type**. Instead of rewriting the same logic for different data types (e.g., int, float, char), templates let the compiler generate a specific type-based implementation when needed.

**Key Characteristics of Templates:**

- **Write Once, Use for Any Data Type**: Templates enable defining a single generic function or class that works for all data types.
- **Compiler-Generated Code**: The compiler automatically generates type-specific code during compilation based on the template.
- **Avoids Redundancy**: Saves effort by reusing the same logic for different data types.
- **Better Than Macros**: Unlike macros, templates perform **type checking** during compilation, making the code safer and less error-prone.

**Types of Templates in C++**

1. **Function Templates**:
   - Used to create generic functions.
   - Examples: Sorting, searching, finding maximum or minimum values in an array, etc.
2. **Class Templates**:
   - Used to create generic classes.
   - Examples: Data structures like stacks, queues, deques, and other containers.

**Function Templates:-**Function templates are a way to write generic functions where the data type is determined at compile time. For example, a function to find the maximum of two numbers can be written as a template to work with int, char, or any user-defined data type. Example: Function Template for Maximum

```
#include<iostream>
using namespace std;
   template <typename T>
T myMax(T x, T y) {
   return (x > y) ? x : y;
}
   int main() {
   cout << myMax<int>(3, 7) << endl;      // Max of two integers
   cout << myMax<char>('c', 'g') << endl; // Max of two characters
   return 0;
}
```

**How It Works:**

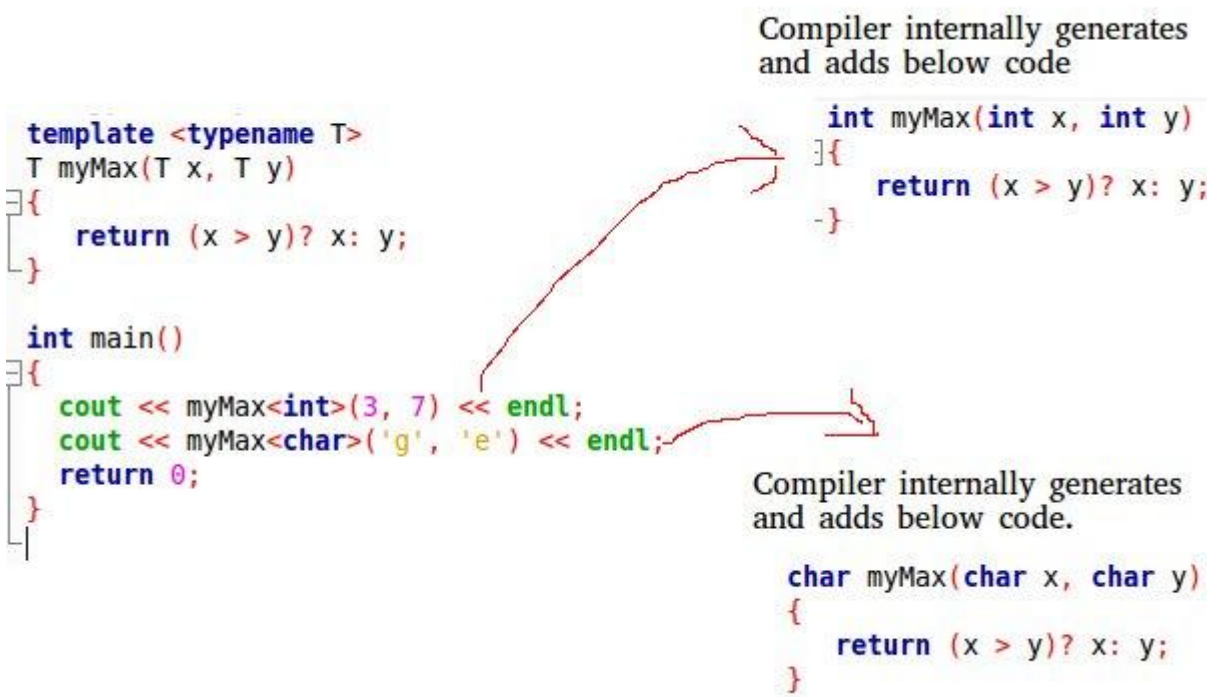
1. **Template Definition**:
   - template <typename T>: Defines a generic type T.
   - The function myMax takes two parameters of type T and returns the larger value.
2. **Usage**:
   - myMax<int>(3, 7): Replaces T with int and finds the maximum of 3 and 7.
   - myMax<char>('c', 'g'): Replaces T with char and finds the maximum of 'c' and 'g'.
3. **Output**:

```
7
g
```

**Internal Process:**

When a function template is called:

- The compiler generates a **type-specific version** of the function, replacing the template parameter (T) with the data type provided.
- For example, for myMax<int>(3, 7), the compiler generates a function where T is replaced with int.




**Difference Between Templates and Macros**

While macros also enable generic code, templates provide a better alternative with more features and safety.Example:

*Macro for Maximum:*

```
#define myMax(X, Y) ((X) > (Y) ? (X) : (Y))
```

*Template for Maximum:*

```
template <typename T>
T myMax(T x, T y) {
   return (x > y) ? x : y;
}
```

**Why Templates Are Better Than Macros:**

1. **Type Checking**: Templates ensure type safety during compilation.
2. **Error-Free**: Unlike macros, templates do not blindly replace text, reducing the chances of errors.
3. **Ease of Debugging**: Errors in templates are easier to debug compared to macros.
4. **Reusable**: Templates enable creating generic code for multiple data types, which macros cannot achieve efficiently.

**Function Templates in C++**:- The idea of function templates is simple: **write a function once, and the compiler generates type-specific versions of the function as needed.** This approach saves time, avoids redundancy, and ensures consistency when implementing the same logic for different data types.

**How Do Function Templates Work?**

- Function templates are ideal for scenarios where the same functionality is required for multiple data types.
- For instance, you can use a single template for tasks like:
  - Sorting arrays
  - Finding the maximum element
  - Searching in a data structure (e.g., linear or binary search)

The **Standard Template Library (STL)** in C++ provides many function templates that can work seamlessly with arrays, vectors, linked lists, and other containers.

**Syntax for Defining Function Templates** A function template in C++ is defined using the following syntax:

```
template <typename T>

T functionName(T parameter1, T parameter2, ...) {

 // code

}
```

- **template**: A keyword that tells the compiler the following function is a template.
- **<typename T>**: Specifies a placeholder for the data type. This T can be replaced by any type during compilation (e.g., int, float, char, etc.).
- You can replace typename with class, as both keywords are interchangeable in this context.
- Multiple placeholders can also be used

**Example Code: Finding the Maximum Element in an Array** Let's see a practical example where a function template is used to find the maximum element in an array of different data types.

```
#include<iostream>
using namespace std;

template <typename T>
T arrMax(T arr[], int n){
   T res = arr[0];
   for(int i = 1; i < n; i++){
      if(arr[i] > res)
         res = arr[i];
   }
   return res;
}

int main()
{
   int arr1[] = {10, 40, 3};
   cout << arrMax<int>(arr1, 3) << " ";

   float arr2[] = {10.5, 3.5, 1.5, 30.5};
   cout << arrMax<float>(arr2, 4) << " ";
   return 0;
}
```

**Explanation of the Example**

1. **Template Definition**:
   - template <typename T> defines a generic data type T that will be used in the function.
   - The function arrMax accepts an array of type T and an integer n (array size).
2. **Logic**:

- Initialize res with the first element of the array.
- Iterate through the array and update res if a larger element is found.

3. **Function Calls**:
   - **arrMax<int>**:
     - The compiler creates a version of arrMax specifically for int data type.
     - The output for the input array {10, 40, 3} will be 40.
   - **arrMax<float>**:
     - The compiler creates a version of arrMax for float data type.
     - The output for the input array {10.5, 3.5, 1.5, 30.5} will be 30.5.

**How Templates Work Internally** When you use a function template:

- The compiler generates **type-specific versions** of the function during compilation.
- For example:
  - For arrMax<int>, the compiler creates a function where all instances of T are replaced with int.
  - For arrMax<float>, the compiler creates a similar function with float replacing T.

**Output of the Code** For the above example, the output will be: 40 30.5

**Advanced Usage: Non-Type Parameters :-** In addition to passing data types to templates, C++ allows you to pass non-type parameters (e.g., constants) to function templates. These parameters are evaluated and processed at compile time, enabling added flexibility while maintaining type safety. Here's an example where a **limit** on the size of the array is enforced using a **non-type parameter**:

```
#include<iostream>
using namespace std;
  template <typename T, int limit>
T arrMax(T arr[], int n) {
  if (n > limit) {
    throw "Array size exceeds limit!";
  }
  T res = arr[0];
  for (int i = 1; i < n; i++) {
    if (arr[i] > res)
      res = arr[i];
  }
  return res;
}
  int main()
{
  try {
    int arr[] = {10, 20, 30};
    cout << arrMax<int, 5>(arr, 3) << endl; // Limit is 5
  }
   catch (const char* msg) {
    cout << msg << endl;
  }
  return 0;
}
```

**Explanation of the Code**

1. **Non-Type Parameter:**
   - The **int limit** is a non-type template parameter, which specifies the maximum allowable size for the array.
   - This parameter is **constant**, meaning its value must be known at compile time.
2. **Function Logic:**
   - The arrMax function takes:
     - An array of type T.

        - The size of the array (n).
    - If the size of the array exceeds the limit, the function throws an exception with the message "Array size exceeds limit!".
    - Otherwise, it calculates and returns the maximum value in the array.
3. **Compile-Time Evaluation:**
    - When calling the function:
      arrMax<int, 5>(arr, 3);
        - The compiler replaces the placeholder T with int.
        - The **limit** parameter is replaced by 5, making it possible to enforce the size check at runtime.
4. **Exception Handling:**
    - If the size of the array exceeds the limit, the program throws an exception, which is caught and displayed in the catch block.

**Output**For the input array {10, 20, 30} and the limit set to 5: 30

## Class Template in C++:- Just like function templates, **class templates** let you write a single implementation that works for all data types. For example:

- You can create a **stack class** that works with int, float, or even user-defined data types like Student.
- A **pair class** can represent a pair of any two data types such as integers, strings, or even objects.

The compiler generates type-specific code at compile time, reducing redundancy and enabling flexibility.

### How Class Templates Work

A **class template** allows you to parameterize the data type. Here's how it works:

1. Define a generic class using the template keyword.
2. Use typename or class to define the data type as a parameter (e.g., T).
3. The compiler generates a type-specific class during instantiation when you specify the data type.

**Class Template Example: Pair Class :-** Let's look at an example of a **Pair class**. The class stores two values of the same type and provides methods to retrieve them.

```
#include<iostream>
using namespace std;
   template <typename T> //Template Declaration
struct Pair{
   T x, y; //Member Variables
   Pair(T i, T j){x = i; y = j;}   // Constructor
   T getFirst(){return x;}       // member functions
   T getSecond(){return y;}      // member functions
};
   int main()
{
   Pair <int> p1(10, 20);          // Create a pair of integers
   cout << p1.getFirst() << " " << p1.getSecond();
   return 0;
}
```

### Explanation

1. **Template Declaration**:
    - T is the generic type that represents the type of the pair's elements.
    - You can replace typename with class—both are interchangeable in templates.

2. **Member Variables**:
   - These variables store the two elements of the pair.
3. **Constructor**:
   - Initializes the pair with the given values.
4. **Member Functions**:
   - getFirst() returns the first element.
   - getSecond() returns the second element.
5. **Usage**:
   - The type int is passed as a parameter, creating a pair of integers.
   - The compiler generates a type-specific class with T replaced by int.

**Output For the code above: 10 20**

**Defining Class Methods Outside the Class :-** In C++, you can define class methods outside the class using the scope resolution operator (::). For **templated classes**, you must include the template declaration when defining methods. **Here's how you can define the getFirst method outside the Pair class:**

```
template <typename T>
T Pair<T>::getFirst() {
   return x;
}
```

By default, struct members are public. Use class for private members and enforce encapsulation.

**Introduction to Object Oriented Programming**:- Object-Oriented Programming (OOP) is a programming paradigm that is widely used to design complex software systems. Unlike functional programming, which is suitable for smaller, simpler tasks, OOP helps manage complexity by organizing software around objects and classes. This article will explain the basic concepts of OOP in Java, how it differs from functional programming, and provide an example of when to use OOP.

**What is Object-Oriented Programming (OOP)?** Object-Oriented Programming (OOP) is a way of structuring your software by organizing it into objects. These objects interact with one another and share common functionality. Two key concepts of OOP are **classes** and **objects**.

- **Class**: A class is essentially a blueprint for creating objects. It defines the attributes (data members) and the behaviors (methods) that the objects of that class will have. In Java, a class can contain both data (variables) and functions (methods) that define the behavior of objects.
- **Object**: An object is an instance of a class. Just as variables are instances of primitive data types (e.g., int, float, char), objects are instances of a class. For example, if you have a Car class, each individual car (object) you create from that class will have specific attributes like color, brand, and model.

**Functional Programming vs. Object-Oriented Programming** In Java, you can program in both functional programming style and object-oriented programming style. Both paradigms are supported by the language, but they are suited for different kinds of tasks.

- **Functional Programming**: This style of programming focuses on writing functions that take input, perform some computations, and return an output. It is great for simple, small tasks such as automation scripts or basic computations. Functional programming works well when there are only a few pieces of logic and little interaction between different parts of the program.
- **Object-Oriented Programming**: OOP becomes particularly useful when building larger, more complex systems. If your program needs to model real-world entities with attributes and behaviors, and these entities interact with each other in a meaningful way, OOP is the preferred approach. For example, in a software system for managing students and teachers, where entities like students,

teachers, and subjects need to interact, OOP is the more efficient and scalable way to organize the system.

**Why OOP is Preferred for Complex Systems** Imagine you are building software for a university. You need to manage students, teachers, and subjects. Each student has attributes such as their name, the subjects they are enrolled in, and their grades. Each teacher has a name and a list of subjects they teach. Each subject has its own name and weightage, which influences the grading of students.

In a functional programming approach, managing these relationships could quickly become complicated. It would involve creating multiple functions that manipulate data structures directly, which can lead to tangled, hard-to-maintain code.

With OOP, you can represent **students**, **teachers**, and **subjects** as objects, each encapsulating their own data and methods for interacting with other objects. For example:

- A **Student** object could have attributes like name, subjects, and grades, and methods like addSubject() and calculateGrade().
- A **Teacher** object could have attributes like name and subjectsTaught, and methods like assignGrade() or teachSubject().
- A **Subject** object could have attributes like name and weightage, and methods like assignGradeToStudent().

This approach allows you to design a system that mirrors real-world entities and their interactions, making the software easier to understand, extend, and maintain.

**Advantages of Object-Oriented Programming**

- **Modularity:** Code is organized into classes, making it easier to understand and maintain.
- **Reusability:** Classes can be reused in other programs or projects.
- **Scalability:** OOP makes it easy to extend software by adding new classes or functionalities.
- **Error Reduction:** Encapsulation and abstraction help prevent invalid states and reduce errors.

**Class and Class Members:-Classes** are the building blocks of Object-Oriented programming. It is a user-defined data-type which contains data members carrying values that describe the characteristics of the entity the class represents. It contains member methods/functions which describe the behavior of the class-entity. Note that the **class is just a blueprint** that has no physical memory associated with it. **When a class is instantiated, an object is created** which is provided a memory/storage location.

**Class Definition and Object Instantiation**

The syntax for class declaration is as:

```
class < class-name > {
< access-modifier >:
   //data-members
   //constructors
   //member-functions
   //destructors
};   //don't forget the semi-colon here !!
```

Each of the above terms (constructors, destructors, access-modifiers etc.) will be explained in detail afterwards.

```
#include <bits/stdc++.h>
using namespace std;
 Employee {  //Class Declaration
 public:
      string id, name;
      int years;  //experience (in years)

      Employee(string id, string name, int years) {
         this->id = id;
         this->name = name;
         this->years = years;
      }
      void work() {
         cout << "Employee: " << this->id << " is working\n";
      }
};
 int main()
{
   //Class Instantiation (Direct)
   Employee emp("GFG123", "John", 3);

   //Class Instantiation (Indirect)
   Employee *emp_ptr = new Employee("GFG456", "James", 4);

   cout << "Employee ID: " << emp.id << endl;
   cout << "Name: " << emp.name << endl;
   cout << "Experience (in years): " << emp.years << endl;

   emp.work();
   cout << endl;

   cout << "Employee ID: " << emp_ptr->id << endl;
   cout << "Name: " << emp_ptr->name << endl;
   cout << "Experience (in years): " << emp_ptr->years << endl;

   emp_ptr->work();
   return 0;
}
```

**Output:**

```
Employee ID: GFG123
Name: John
Experience (in years): 3
Employee: GFG123 is working

Employee ID: GFG456
Name: James
Experience (in years): 4
Employee: GFG456 is working
```

**Data Members** :- Data Members are the identifiers that describe the characteristics and hold important data related to the class. In our Employee class, we have *id, name, years* as the data members. Data members can be anything from primitive data-types (int, char, double, etc.) to collections (arrays, strings, etc.) to user-defined

data-types (structs, unions, even other class objects, etc.). Anything that can hold data value can be a data-member.

**Member Functions:-** Member Functions/Methods are class-functions/methods which describe the behavior of the class. i.e. what kind of operation we can perform with objects of this class. e.g. In our Employee example, *work()* is a member function. Member functions in C++ can be defined inside/outside the class, however, it is mandatory to have prototype declaration inside the class if go for an outside-the-class definition.

```
#include <bits/stdc++.h>

using namespace std;

class Employee {  //Class Declaration
   public:
      string id, name;
      int years;  //experience (in years)

      Employee(string id, string name, int years) {
         this->id = id;
         this->name = name;
         this->years = years;
      }//Prototype Declaration
      void work();
};//Outside-class definition
void Employee::work() {
   cout << "Employee: " << this->id << " is working\n";
}
 int main()
{  //Class Instantiation (Direct)
   Employee emp("GFG123","John",3);
    emp.work();
   return 0;
}
```

**Output:** Employee: GFG123 is working

As we observe, while providing the definition for a class member-function we use the **scope-resolution operator ::** with the **class-name::function-name** format to differentiate it from normal global-scoped functions.

**Structs vs Classes in C++**:- In C++, a structure is the same as a class except for a few differences. The most important of them is security. A Structure is not secure and cannot hide its implementation details from the end user while a class is secure and can hide its programming and designing details. Following are the points that expound on this difference:

1. Members of a class are private by default and members of a struct are public by default. e.g.

```
#include <bits/stdc++.h>
using namespace std;
   struct A {
   int x; // x is public
};
   class B {
   int x; // x is private
};
   int main()
{
   A a;
   B b;
```

```
    cout << a.x << endl;
   cout << b.x << endl;
   return 0;}
```

Error generated by above code:

```
prog.cpp: In function 'int main()':
prog.cpp:10:9: error: 'int B::x' is private
    int x; // x is private
        ^
prog.cpp:19:15: error: within this context
    cout << b.x << endl;
              ^
```

Above code generates compilation-error because we tried to access data-member *x* for instance *b* (which is private).

2. When deriving a **struct from a class/struct**, default access-specifier for a base class/struct is **public**. And when **deriving a class**, the default access specifier is **private**.

```
#include <bits/stdc++.h>
using namespace std;
   class Base
{
 public:
    int x; // x is public
};
   class Derived1 : Base //equivalent to private Base
{
};
   struct Derived2 : Base //equivalent to public Base
{
};
   int main()
{
   Derived1 d1;  //class
   Derived2 d2;  //struct
   cout << d1.x << endl;
   cout << d2.x << endl;
    return 0;
}
```

Error generated by above code:

```
prog.cpp: In function 'int main()':
prog.cpp:8:13: error: 'int Base::x' is inaccessible
        int x; // x is public
            ^
prog.cpp:26:16: error: within this context
    cout << d1.x << endl;
              ^
```

The above code generates compilation error because of the access statement *d1.x*. Since we didn't specify the access-modifier for the Base class, *x* became private in Derived Class.

**Constructors**:- Constructors and destructors are special member functions in C++ used to initialize and clean up objects, respectively. They are vital for efficient resource management, especially when dealing with

dynamic memory allocation or other system resources. In this article, we will discuss constructors and destructors in detail, with examples to demonstrate their functionality and usage.

**What are Constructors?** Constructors are special member functions that have the same name as the class and no return type. They are automatically invoked when an object of the class is created. Constructors can be:

1. **Default Constructor**: Takes no parameters and initializes members with default values.
2. **Parameterized Constructor**: Takes arguments to initialize members with specific values.
3. **Copy Constructor**: Creates a new object as a copy of an existing object.

**Key Properties:**

- Automatically called when an object is created.
- Can be overloaded.
- Cannot return any value.

**Code Example 1: Default and Parameterized Constructors**

```
#include<iostream>
using namespace std;
   class Point {
private:
   int x, y;
public:  // Default Constructor
   Point() {
      x = 0;
      y = 0;
   } // Parameterized Constructor
   Point(int x1, int y1) {
      x = x1;
      y = y1;
   }
   void print() {
      cout << x << " " << y << endl;
   }
};
int main() {
   Point p1, p2(10, 20); // Using default and parameterized constructors
   p1.print();          // Output: 0 0
   p2.print();          // Output: 10 20
   return 0;
}
```

**Explanation:**

- The Point class has two constructors: a default constructor and a parameterized constructor.
- Point() initializes x and y to 0.
- Point(int x1, int y1) initializes x to x1 and y to y1.
- In main(), p1 is created using the default constructor, while p2 is created using the parameterized constructor.
- The print() method displays the values of x and y.

*Code Example 2: Constructors with Initializer List*

```
#include<iostream>
using namespace std;
   class Point {
private:
   int x, y;
public: // Default Constructor with Initializer List
```

```
    Point() : x(0), y(0) {}  // Parameterized Constructor with Initializer List
    Point(int x1, int y1) : x(x1), y(y1) {}
        void print() {
        cout << x << " " << y << endl;
 }};

int main() {
    Point p1, p2(10, 20);
    p1.print(); // Output: 0 0
    p2.print(); // Output: 10 20
    return 0;
}
```

**Explanation:**

- The initializer list (: x(0), y(0)) directly initializes the members x and y.
- This avoids the double initialization that would occur if x and y were first default-initialized and then assigned values in the constructor body.
- This method is particularly useful when initializing const or reference members, as they must be initialized at the time of object creation.

**Member Initialization Order**

*Code Example 3: Constructor Initialization without Initializer List*

```
 #include<iostream>
using namespace std;
    class Test {
public:
    Test() {
        cout << "Default\n";
    }
    Test(int x) {
        cout << "Parameterized\n";
    }
};
    class Main {
    Test t;
public:
    Main() {
        t = Test(10);
    }
};
    int main() {
    Main m;
    return 0;
}
```

**Explanation:**

- The Main class contains an object of Test. In its constructor, t is assigned a new value.
- During this process, the default constructor of Test is called first, followed by the parameterized constructor.
- The output is:

```
Default
Parameterized
```

- **Problem:** The member is initialized twice, leading to performance inefficiencies.

**Code Example 4: Constructor with Initializer List**

```
#include<iostream>
```

```
using namespace std;
   class Test {
public:
   Test(int x) {
     cout << "Parameterized: " << x << "\n";
   }
};
   class Main {
   Test t;
public:
   Main() : t(10) {} // Using initializer list
};
   int main() {
   Main m;
   return 0;
}
```

**Explanation:**

- Here, the Main class uses an initializer list to initialize the Test object t.
- The output is:

```
Parameterized: 10
```

- **Advantage:** The member t is initialized directly, avoiding the overhead of default initialization followed by assignment.

**Copy Constructor** A copy constructor creates a new object by copying an existing object. If not explicitly defined, the compiler generates a default copy constructor, which performs a shallow copy. For dynamic memory, a user-defined copy constructor is necessary to perform a deep copy.

*Code Example 5: Shallow vs. Deep Copy*

```
#include<iostream>
using namespace std;
   class Test {
private:
   int* ptr;
public:// Constructor
   Test(int x) {
      ptr = new int(x);
   }  // Copy Constructor for Deep Copy
   Test(const Test &t) {
      ptr = new int(*(t.ptr));
   }
   void set(int x) {
      *ptr = x;
   }
void print() {
      cout << *ptr << endl;
   }
   ~Test() {
      delete ptr;
   }
};
   int main() {
   Test t1(10);
   Test t2 = t1; // Calls copy constructor

   t2.set(20);
```

```
    t1.print(); // Output: 10
    t2.print(); // Output: 20
    return 0;
}
```

**Explanation:**

- The constructor Test(int x) dynamically allocates memory for an integer and initializes it with x.
- The copy constructor Test(const Test &t) allocates new memory and copies the value from the original object to avoid shared memory issues.
- Without the custom copy constructor, the default copy constructor would copy the pointer, leading to a shallow copy where both objects share the same memory.
- The destructor ~Test() ensures that the dynamically allocated memory is released.

**Destructors:-**Destructors are special member functions that clean up resources when an object is destroyed. They have the same name as the class, preceded by a tilde (~), and cannot take arguments or return a value**.**

*Code Example 6: Destructor*

```
#include<iostream>
using namespace std;
    class Test {
    int x;
    public:
    Test(int i) : x(i) {
        cout << "Cons " << x << endl;
    }
    ~Test() {
        cout << "Dest " << x << endl;
    }
};
    int main() {
    Test t1(10);
    Test t2(20);
    return 0;
}
```

**Explanation:**

- The constructor is called for t1 and t2 during their creation.
- The destructor is called in reverse order of creation when the program exits.
- The output is:

```
Cons 10
Cons 20
Dest 20
Dest 10
```

**'this' pointer** :- In C++, the this pointer plays a crucial role in managing object-oriented programming. When multiple objects of a class are created, each object has its own copy of non-static data members, but the member functions are shared among all instances. The this pointer ensures that when a function is called on an object, it operates only on the members of that specific object.

**What is the this Pointer?**

The this pointer is an implicit pointer available to all non-static member functions of a class. It points to the object on which the member function is invoked. Internally, the compiler uses the this pointer to ensure that each member function accesses the data of the correct object.

**Key Points:**

1. The this pointer refers to the calling object.
2. It is automatically passed to all non-static member functions.
3. The this pointer cannot be modified since it is a constant pointer.

**Example 1: Constructor Initialization Using the this Pointer**

```
#include<iostream>
using namespace std;

class Point {
   int x, y;
public:
   Point(int x, int y) {
      this->x = x;
      this->y = y;
   }
};
   int main() {
   Point p1(10, 20), p2(5, 5);
   return 0;
}
```

**Explanation:In the constructor of the Point class:**

- The parameters (x and y) have the same names as the class members.
- To avoid naming conflicts, the this pointer is used. For example, ***this->x*** refers to the member variable x of the object, while x refers to the constructor parameter.

Without the ***this*** pointer, the parameter would shadow the member variable, making it inaccessible.

**Example 2: Function Chaining Using the this Pointer**

```
#include<iostream>
using namespace std;

class Point {
   int x, y;
public:
   Point(int x, int y) {
      this->x = x;
      this->y = y;
   }
   Point& setX(int x) {
      this->x = x;
      return *this;
   }
   Point& setY(int y) {
      this->y = y;
      return *this;
   }
};

int main() {
   Point p1(10, 10);
   p1.setX(5).setY(5);
   return 0;
```

}

**Explanation**: This example demonstrates **function chaining**, where multiple member function calls can be made on the same object in a single statement.

*Steps:*

1. The ***setX*** and ***setY*** functions return a reference to the calling object by using ***return *this;***.
2. The returned reference allows another member function (setY) to be called immediately on the same object.
3. The this pointer ensures that the operations are performed on the same object.

## Static data members and methods :- In C++, static members and functions are essential for sharing data and behavior among all objects of a class. They are particularly useful when designing systems that require shared data or functionality, such as tracking the number of active instances of a class.

This article explores the concepts of static data members and static functions, with examples illustrating their usage.

**What Are Static Members?**

Static members belong to the class rather than any specific object. They are shared across all objects of the class and retain their values between object creations and destructions.

**Key Characteristics:**

1. **Static Data Members**:
   - Shared among all objects of the class.
   - Defined outside the class.
   - Initialized only once, regardless of the number of objects created.
2. **Static Member Functions**:
   - Can access only static data members and other static functions.
   - Do not have access to the this pointer.
   - Can be called using the class name.

**Example 1: Static Data Members**

```
#include<iostream>
using namespace std;
   class Player {
public:
   static int count; // Declaration of static data member

   Player() { count++; }  // Increment count when a player is created
   ~Player() { count--; } // Decrement count when a player is destroyed
};

int Player::count = 0; // Definition of static data member

int main() {
   Player p1;  // Creating the first player
   cout << Player::count << " "; // Accessing static member using class name

   Player p2;  // Creating the second player
   cout << Player::count << " "; // Accessing static member again
```

```
    return 0;
}
```

**Explanation:**

1. The static data member count keeps track of the number of Player objects.
2. When p1 is created, count is incremented to 1.
3. When p2 is created, count is incremented to 2.
4. The program outputs 1 2, showing the count after each object creation.

**Example 2: Static Member Functions**

```
#include<iostream>
using namespace std;

class Player {
private:
    static int count;

public:
    Player() { count++; }
    ~Player() { count--; }

    static int getCount() { return count; } // Static member function
};

int Player::count = 0;

int main() {
    Player p1, p2;
    cout << Player::getCount() << " "; // Accessing static function using class name

    return 0;
}
```

Explanation:

1. The static function getCount allows safe access to the private static data member count.
2. Since count is shared among all objects, the function correctly returns the value 2 after creating two Player objects.

**Key Points About Static Members**

**1. Sharing Across Objects :-**Static data members are shared across all instances of a class. This makes them ideal for maintaining information common to all objects, such as counters.

**2. Accessing Static Members** Static members can be accessed:

- Using the class name: Player::count
- Using an object: p1.count
    - While this is allowed, using the class name is the recommended approach as it clearly indicates the member is static.

**3. Lifetime of Static Members:-**Static members are created once when the class is loaded and persist until the program ends.

**Key Points About Static Member Functions**

**1. Purpose:-**Static member functions are designed to access or modify static data members. They provide a clean and controlled way to manage shared data**.**

**2. Access Rules**

- Static functions can only access static members.
- They cannot access non-static members or call non-static functions since they do not have a this pointer.

**3. Calling Static** Functions Static functions can be called:

- Using the class name: Player::getCount()
- Using an object: p1.getCount()
    - Similar to static members, the class name is the preferred way to call static functions.

**Inheritance** Inheritance is a fundamental concept in object-oriented programming (OOP), allowing classes to reuse code and extend functionality. In C++, inheritance promotes reusability and organizes shared and unique attributes among related classes. This article discusses the basics of inheritance, access specifiers, and their impact on class hierarchies, with examples to clarify these concepts.

**What is Inheritance?**

Inheritance allows a class (derived or subclass) to acquire properties and behaviors (attributes and methods) of another class (base or superclass). This helps in reducing code redundancy and improving maintainability.

For example, if we are building a college management system, we may have classes like Student and Teacher that share attributes such as ID, Name, and Address. These shared attributes can be moved to a base class called Person, from which Student and Teacher inherit.

**Example: Inheriting Attributes and Methods**

```
#include<bits/stdc++.h>
using namespace std;

class Person {
protected:
   string name;
   int id;

public:
   Person(string n, int i) : name(n), id(i) {}
};

class Student : public Person {
private:
   int marks;
```

```
public:
    Student(string n, int i, int m) : Person(n, i), marks(m) {}

    void print() {
        cout << name << " " << id << " " << marks << endl;
    }
};

int main() {
    Student stu("Rahul", 1001, 88);
    stu.print();
    return 0;
}
```

**Explanation:**

1. **Base Class (Person)**:
   - Contains common attributes like name and id.
   - These attributes are marked as protected, meaning they are accessible within the base class and derived classes but not outside.
2. **Derived Class (Student)**:
   - Inherits name and id from Person.
   - Adds a new attribute marks and a method print() to display details.
3. **Inheritance Syntax**:
   - The Student class inherits from Person using the public access specifier.

**Access Specifiers in Inheritance:-** Access specifiers (public, protected, private) determine the visibility of base class members in the derived class**.**

**1. Public Inheritance:**

- **Effect**:
  - public members of the base class remain public in the derived class.
  - protected members of the base class remain protected in the derived class.
- **Usage**: Suitable when the derived class represents an "is-a" relationship with the base class.

**2. Protected Inheritance:**

- **Effect**:
  - public and protected members of the base class become protected in the derived class.
- **Usage**: Used when the base class functionality should only be accessible within the derived class or its further subclasses.

**3. Private Inheritance:**

- **Effect**:
  - public and protected members of the base class become private in the derived class.
- **Usage**: Used when the derived class wants to hide the base class functionality from external access.

**The protected Access Specifier:-**The protected specifier is particularly useful in inheritance. Members marked as protected are:

- Accessible within the base class.
- Accessible within derived classes.

- Not accessible outside the class hierarchy.

In the above example, name and id are marked as protected in Person, allowing Student to access them directly.

**Inheritance ExampleExample 1: Base and Derived Class Constructor**

```
#include <iostream>
using namespace std;

class Base {
protected:
   int x;
public:
   Base(int a) : x(a) {
      cout << "Base \n";
   }
};

class Derived : public Base {
private:
   int y;
public:
   Derived(int a, int b) : Base(a), y(b) {
      cout << "Derived \n";
   }
   void print() {
      cout << x << " " << y << '\n';
   }
};

int main() {
   Derived d(10, 20);
   d.print();
   return 0;
}
```

***Explanation***

**1. Base Class Constructor:**

- The Base class contains a parameterized constructor that initializes the member x and prints "Base".

**2. Derived Class Constructor:**

- The Derived class calls the Base constructor using an initializer list to initialize x. It also initializes its own member y.

**3. Program Output:**

```
Base
Derived
10 20
```

**4. Key Point:**

- The base class constructor is always executed first, followed by the derived class constructor.
- If the base class does not have a default constructor and no explicit call to a parameterized constructor is made, the program results in a compiler error.

**Example 2: Access Modifiers and Inheritance**

Case 1: Protected to Private

```
#include <iostream>
using namespace std;

class Base {
private:
   int x;
public:
   Base(int a) : x(a) {
      cout << "Base \n";
   }
};

class Derived : public Base {
private:
   int y;
public:
   Derived(int a, int b) : Base(a), y(b) {
      cout << "Derived \n";
   }
   void print() {
       cout << x << " " << y << '\n'; // Compiler Error
   }
};

int main() {
   Derived d(10, 20);
   d.print();
   return 0;
}
```

**Explanation:**

- x is private in the Base class and cannot be accessed in the Derived class.
- The program results in a **compiler error** when trying to access x.

**Case 2: Making x Public**

```
#include <iostream>
using namespace std;

class Base {
public:
   int x;
   Base(int a) : x(a) {
      cout << "Base \n";
   }
};

class Derived : public Base {
private:
   int y;
public:
   Derived(int a, int b) : Base(a), y(b) {
      cout << "Derived \n";
   }
```

```
    void print() {
        cout << x << " " << y << '\n';
    }
};

int main() {
    Derived d(10, 20);
    d.print();
    cout << d.x << '\n';
    return 0;
}
```

**Explanation:**

- x is declared as public in the Base class and remains accessible in the Derived class and outside it.
- The program compiles successfully and outputs:

```
Base
Derived
10 20
10
```

Case 3: Protected Access in Inheritance

```
#include <iostream>
using namespace std;

class Base {
protected:
    int x;
public:
    Base(int a) : x(a) {
        cout << "Base \n";
    }
};

class Derived : protected Base {
private:
    int y;
public:
    Derived(int a, int b) : Base(a), y(b) {
        cout << "Derived \n";
    }
    void print() {
        cout << x << " " << y << '\n';
    }
};

int main() {
    Derived d(10, 20);
    d.print();
     cout << d.x; // Compiler Error
    return 0;
}
```

**Explanation:**

- When Base is inherited as protected, the x member becomes protected in Derived.
- While x is accessible within Derived, it is not accessible outside the class using a Derived object.
- Attempting to access d.x results in a **compiler error**.

**Key Takeaways**

1. **Constructors in Inheritance:**
   - The base class constructor is called before the derived class constructor.
   - Explicitly calling a parameterized constructor of the base class is necessary if there is no default constructor in the base class.
2. **Access Modifiers:**
   - Private members of a base class are not accessible in the derived class.
   - Public members remain accessible as per the inheritance type (public, protected, or private).
   - Protected members are accessible within the derived class but not outside it when using objects.

## Multiple Inheritance and Diamond Problem

In this article, we will explore the concept of **Multiple Inheritance** in C++ in detail. We will also cover the associated **Diamond Problem** and its resolution using the virtual keyword. All explanations will be accompanied by code examples for clarity.

**What is Multiple Inheritance?:- Multiple Inheritance** occurs when a class inherits from two or more base classes. This means that the derived class gains access to all the members (attributes and methods) of the base classes. However, it can lead to certain issues like the **Diamond Problem**, which we will address shortly.

*Example Code for Multiple Inheritance:*

```
class A {
   // Attributes and methods of A
};

class B {
   // Attributes and methods of B
};

class C : public A, public B {
   // C inherits from both A and B
};

int main() {
   C obj; // Object of class C
   return 0;
}
```

Here, class C inherits from both class A and class B. When an object of C is created, it will have access to the members of both A and B.

**Constructor Call Order in Multiple Inheritance:-**When an object of a derived class is created, the constructors of its base classes are called in the order in which they are specified in the inheritance list. For example:

```
#include <iostream>
class A {
public:
   A() { std::cout << "A's Constructor called\n"; }
};

class B {
public:
   B() { std::cout << "B's Constructor called\n"; }
```

```
};

class C : public A, public B {
public:
   C() { std::cout << "C's Constructor called\n"; }
};

int main() {
   C obj;
   return 0;
}
```

**Output:** A's Constructor called
B's Constructor called
C's Constructor called

If we change the order in the inheritance list (e.g., class C : public B, public A), the constructors will be called in that new order.

**The Diamond Problem:-** The **Diamond Problem** arises in multiple inheritance when a class inherits from two classes that both inherit from a common base class. This leads to ambiguity because the members of the common base class are inherited twice.

*Example Code Demonstrating the Diamond Problem:*

```
#include <iostream>
class A {
public:
   int x;
   A() { x = 10; }
};

class B : public A {};
class C : public A {};

class D : public B, public C {};

int main() {
   D obj;
   std::cout << obj.x; // Ambiguity: Which x?
   return 0;
}
```

Here, class D inherits from both class B and class C, and both of these inherit from class A. This means that class D contains two copies of A's members (e.g., x). When we try to access x in main(), the compiler throws an ambiguity error because it cannot determine which x to use: the one inherited through B or the one inherited through C.

**Resolving the Diamond Problem :-** To resolve the **Diamond Problem**, C++ provides the virtual keyword. When a class is inherited virtually, only one copy of the base class's members is shared across all derived classes.
Corrected Code Using virtual Keyword:

```
#include <iostream>
class A {
public:
   int x;
```

```
    A() { x = 10; }
};

class B : virtual public A {};
class C : virtual public A {};

class D : public B, public C {};

int main() {
    D obj;
    std::cout << obj.x; // No ambiguity
    return 0;
}
```

In this code, the virtual keyword ensures that there is only one copy of A's members in class D. When we access obj.x, it is clear which x is being referred to, and the ambiguity is resolved. **Output:**10

Additionally, when an object of class D is created, A's constructor is called only once, even though A is inherited through both B and C.

**Summary**

1. **Multiple Inheritance** allows a class to inherit from two or more base classes.
2. It introduces the **Diamond Problem**, where ambiguity arises due to multiple copies of the same base class members.
3. The **virtual** keyword resolves this problem by ensuring that only one copy of the base class's members is shared.

**Virtual Functions & Polymorphism:-** In object-oriented programming, **virtual functions** play a crucial role in achieving runtime polymorphism. This article explains the concept of virtual functions in C++, how they work, and their significance. It also delves into related coding examples to clarify their usage.

**What Are Virtual Functions?** Virtual functions allow derived classes to override methods in the base class. The key idea is to ensure that the function call is determined by the **type of the object being pointed to** rather than the **type of the pointer or reference**.

This concept is tightly linked to inheritance, where a base class type can refer to a derived class object. Without virtual functions, the method call is resolved based on the pointer's type, not the object it points to.

**Key Fact About Pointers and References** In C++, a **base class pointer** or **reference** can refer to a derived class object. Here's an example:

```
Base *b = new Derived(); // Base pointer to Derived object
Derived d;
Base &bRef = d; // Base reference to Derived object
```

Both statements are valid and compile successfully. However, the behavior of function calls depends on whether the base class function is declared virtual.

**Why Use Virtual Functions?** Consider a real-world scenario with a base class Employee and derived classes like SoftwareDeveloper and SalesExecutive. If you need to perform an operation, such as printing financial data for all employees, you'd want to iterate over an array of Employee pointers, where each pointer can point to different derived class objects.

Without virtual functions, the base class function is always called, regardless of the derived class object being pointed to. This limitation is resolved by marking the base class function as virtual.

**Example 1: Without Virtual Functions**

```
#include <iostream>
using namespace std;
class Base {
public:
   void print() {
      cout << "In Base\n";
   }
};
   class Derived : public Base {
public:
   void print() {
      cout << "In Derived\n";
   }
};
   int main() {
   Base b;
   Derived d;

   b.print(); // Output: In Base
   d.print(); // Output: In Derived

   Base *bp = &d;
   bp->print(); // Output: In Base
}
```

**Explanation:**

1. b.print() calls the print function of the Base class.
2. d.print() calls the overridden print function in Derived.
3. However, bp->print() calls the Base class function because bp is a Base pointer, even though it points to a Derived object. This happens because the print function in the base class is not virtual.

**Example 2: With Virtual Functions**

```
#include <iostream>
using namespace std;
class Base {
public:
   virtual void print() {
      cout << "In Base\n";
   }
};
   class Derived : public Base {
public:
   void print() override {
      cout << "In Derived\n";
   }
};
```

```
int main() {
   Base b;
   Derived d;

   b.print(); // Output: In Base
   d.print(); // Output: In Derived

   Base *bp = &d;
   bp->print(); // Output: In Derived
}
```

**Explanation:**

1. By declaring print() in the base class as virtual, the compiler ensures that the derived class function is called when bp->print() is invoked.
2. The keyword override (introduced in C++11) ensures that Derived::print is correctly overriding a virtual function in the base class.

**How Virtual Functions Work Internally** Virtual functions rely on the **VTable (Virtual Table)** and **VPTR (Virtual Pointer)**:

1. **VPTR:** A hidden pointer maintained by each object of a class with virtual functions. It points to the VTable.
2. **VTable:** A table maintained per class that stores addresses of all virtual functions.

When a virtual function is called, the VPTR is used to find the VTable of the object, and then the appropriate function's address is fetched from the table.

**Using the override Keyword** The override keyword, introduced in C++11, improves code readability and prevents errors. For example:

```
class Base {
public:
   virtual void print() {}
};

class Derived : public Base {
public:
   void Print() override {} // Error: 'Print' does not override any base class function
};
```

The override keyword ensures that a derived class function matches the signature of a virtual function in the base class.

## Friend function in C++:- Friend functions and friend classes in C++ are unique features that provide access to private and protected members of a class to designated functions or classes. While this capability goes against the principles of encapsulation, it is a tool that can simplify certain designs and improve code readability when used judiciously.

**What Are Friend Functions?** A **friend function** is a function that is not a member of a class but is granted access to the class's private and protected members. The function must be explicitly declared as a friend within the class using the friend keyword.

**What Are Friend Classes?** A **friend class** is a class that is granted access to another class's private and protected members. It allows all the member functions of the friend class to access the private and protected members of the class that declared the friendship**.**

**Key Points About Friendship in C++**

1. **Friendship is Granted, Not Taken:**
   - A class specifies its friends using the friend keyword.
   - The class being granted friendship has no control over this relationship.
2. **Friendship is Not Mutual:**
   - If ClassA is a friend of ClassB, it does not imply that ClassB is a friend of ClassA.
3. **Friendship is Not Transitive:**
   - If ClassA is a friend of ClassB, and ClassB is a friend of ClassC, ClassA is not automatically a friend of ClassC.
4. **Friendship is Not Inherited:**
   - A derived class does not inherit the friendship of its base class.

*Example 1: Friend Function***Code**

***:#include <iostream>***

```
using namespace std;

class Employee; // Forward declaration

class Printer {
public:
   void printEmp(const Employee &e); // Declaration
};

class Employee {
private:
   int id;
   string name;

   // Granting friendship to printEmp function
   friend void Printer::printEmp(const Employee &e);

public:
   Employee(int i, string n) : id(i), name(n) {}
};

void Printer::printEmp(const Employee &e) {
   cout << e.id << " " << e.name << '\n'; // Accessing private members
}

int main() {
   Printer p;
   Employee e(101, "ABC");
   p.printEmp(e); // Output: 101 ABC
   return 0;
}
```

Explanation:

1. The printEmp function is declared as a friend of the Employee class.
2. Being a friend, it can access private members (id and name) of Employee.
3. The function prints the private data of an Employee object when invoked.

**Example 2: Friend Class Code:**

```
#include <iostream>
using namespace std;

class Printer; // Forward declaration

class Employee {
private:
   int id;
   string name;

   // Granting friendship to the Printer class
   friend class Printer;

public:
   Employee(int i, string n) : id(i), name(n) {}
};

class Printer {
public:
   void printEmp(const Employee &e) {
      cout << e.id << " " << e.name << '\n'; // Accessing private members
   }
};

int main() {
   Printer p;
   Employee e(101, "ABC");
   p.printEmp(e); // Output: 101 ABC
   return 0;
}
```

Explanation:

1. The Printer class is declared as a friend of the Employee class.
2. All member functions of Printer have access to private members of Employee.
3. The printEmp function prints the private data of an Employee object.

**Operator Overloading in C++ :-** Operator overloading is a feature in C++ that allows developers to redefine the way operators work for user-defined types. This capability is not present in many programming languages, such as Java, making it a unique aspect of C++. Operator overloading can significantly enhance code readability and usability, especially when working with mathematical or custom data types.

**What is Operator Overloading? :-** Operators in C++ (e.g., +, -, *) are designed to work with primitive data types like int, float, and char. However, C++ allows these operators to be redefined for user-defined data types, such as classes. This process is known as **operator overloading**.

For example, when dealing with a Complex number class, you can redefine the + operator to add two complex numbers just like you would add integers.

**Example: Overloading the + Operator for a Complex Number Class**

```
#include <iostream>
using namespace std;

class Complex {
private:
   int r, i; // Real and imaginary parts

public:
   // Constructor with default arguments
   Complex(int r = 0, int i = 0) {
      this->r = r;
 this->i = i;
   }// Overloading the + operator
   Complex operator+(const Complex &c) {
      Complex ans;
      ans.r = this->r + c.r;
      ans.i = this->i + c.i;
      return ans;
   }   // Print function to display the complex number
   void print() {
      cout << this->r << " + " << this->i << "i\n";
   }
};

int main() {
   Complex c1(2, 3), c2(3, 4); // Create two Complex objects
   auto sum = c1 + c2;         // Use overloaded + operator
   sum.print();                // Output: 5 + 7i
   return 0;
}
```

**Explanation of the Code**

1. **Class Definition:**
   - The Complex class contains two private members: r (real part) and i (imaginary part).
2. **Constructor:**
   - The constructor initializes the real and imaginary parts. Default arguments are used to allow initialization with no arguments.
3. **Operator Overloading:**
   - The operator+ function is defined as a member function of the Complex class.
   - It takes a const reference to another Complex object as a parameter.
   - Inside the function, a new Complex object (ans) is created. Its real and imaginary parts are calculated by adding the corresponding parts of the current object (this->r, this->i) and the passed object (c.r, c.i).
   - The result is returned as a Complex object.
4. **Print Function:**
   - A print function is defined to display the complex number in the format real + imaginary i.
5. **Main Function:**
   - Two Complex objects, c1 and c2, are created and initialized.
   - The + operator is used to add c1 and c2, which internally calls the operator+ function.
   - The result (sum) is printed using the print function.

**Key Concepts in Operator Overloading**

1. **Internal Function Call:**
   - The expression c1 + c2 is equivalent to c1.operator+(c2) when operator+ is defined as a member function.
2. **Readability:**

- Operator overloading makes mathematical operations on user-defined types more intuitive and readable. Instead of writing c1.add(c2), you can write c1 + c2.

3. **Global vs. Member Function:**
   - Operators can also be overloaded using global functions, but member functions are preferred for operators that modify or access the private members of a class.

**Advantages of Operator Overloading**

1. **Enhanced Code Readability:**
   - Simplifies mathematical expressions involving user-defined types.
2. **Reusability:**
   - Customizes operators for specific use cases, making the code more reusable.
3. **Integration with Existing Syntax:**
   - Allows user-defined types to work seamlessly with operators, maintaining consistency with built-in types.

**Limitations of Operator Overloading**

1. **Not All Operators Can Be Overloaded:**
   - Certain operators, such as the . (member access), :: (scope resolution), sizeof, and ?: (ternary operator), cannot be overloaded.
2. **Maintain Expected Behavior:**
   - Overloaded operators should mimic the behavior expected of their built-in counterparts. For instance, overloading the + operator for addition should not perform subtraction.
3. **Complexity:**
   - Overuse of operator overloading can lead to less maintainable code.

**Exception Handling in C++**:- Exception handling is a crucial part of robust programming in C++. It helps manage errors or unexpected behavior that might arise during runtime, allowing the program to handle them gracefully instead of terminating abruptly.

**What Are Exceptions?** Exceptions are runtime anomalies or issues that disrupt the normal flow of execution. Common examples include:

1. **Divide By Zero:** When a program attempts to divide a number by zero, it results in a mathematical error that crashes the program. Handle such scenarios using exception handling or conditional checks to avoid performing division when the denominator is zero.
2. **No Heap Memory Available:** This occurs when the program tries to allocate memory dynamically using new or malloc, but the system doesn't have enough memory to fulfill the request.
3. **Accessing Array Elements Outside Allowed Range:** This happens when the program tries to access an index of an array that doesn't exist, leading to undefined behavior.
4. **Pop Operation on an Empty Stack:** Attempting to remove an element from an empty stack or queue results in an exception or undefined behavior.

**Example Problem: Calculating Average** Let's consider a problem where we compute the average of numbers in an array. A critical issue arises when the size of the array (n) is zero, leading to a **divide-by-zero error**.

```
#include<iostream>
using namespace std ;
   int average(int arr[], int n)
{
   int sum = 0;
   for(int i = 0; i < n; i++)
```

```
        sum = sum + arr[i];
    int res = sum / n; // Division operation
    return res;
}
    int main()
{
    int n;
    cin >> n;
    int arr[n];
    for(int i = 0; i < n; i++)
        cin >> arr[i];
    cout << average(arr, n);
    return 0;
}
```

***Explanation of the Code***

1. The program calculates the average of integers in the array.
2. The user provides the size of the array (n) and the array elements as input.
3. The function average performs the division to compute the average.

Issue Highlight

- If the input n = 0, the division sum / n results in a **divide-by-zero error**, causing the program to crash.

## Try Throw and Catch in C++

Exception handling in C++ provides mechanisms to manage runtime errors, ensuring that programs can handle unexpected conditions gracefully without crashing. The three main components of exception handling in C++ are:

1. **Try Block**: Encapsulates the code that might throw an exception.
2. **Throw Statement**: Used to signal that an exception has occurred.
3. **Catch Block**: Captures and handles the exception.

Let's explore these concepts with practical examples.

**1. Try Block** The try block contains the code that might generate exceptions.

**Syntax**:

```
try {
    // Code that may throw an exception
}
```

**2. Throw Statement** The throw statement is used to signal an exception when an error occurs. Syntax**:**

```
throw exceptionValue;
```

**3. Catch Block:-**The catch block handles exceptions thrown by the try block. You can have multiple catch blocks for different exception types. Syntax**:**

```
catch (exceptionType identifier) {
    // Code to handle the exception
}
```

**Basic Example: Division by Zero**

```
#include<iostream>
using namespace std;

int main() {
   int x, y;
   cin >> x >> y;
   try {
      if (y == 0)
         throw 0;
      cout << "Result is " << x / y;
   }
   catch (int x) {
      cout << "Division with 0";
   }
   return 0;
}
```

**Explanation**:

- The program takes two integers x and y as input.
- Inside the try block, it checks if y is zero. If true, it throws an exception (throw 0).
- The catch block captures this exception and outputs a meaningful message, "Division with 0".
- If no exception occurs, the division result is printed.

*Input* ***10 0***

*Output* ***Division with 0***

**Advanced Example: Multiple Exceptions**

```
#include<iostream>
using namespace std;
   int main() {
   double x, y;
   cin >> x >> y;
   try {
      if (x == 0.0)
         throw 0;
      if (y == 0)
         throw string("Y is 0");
      if (x + y < 0.0)
         throw (x + y);
   }
   catch (int e1) {
      cout << e1;
   }
   catch (string &e2) {
      cout << e2;
   }
   catch (...) {
      cout << "x+y < 0";
   }
   cout << "\nBye\n";
   return 0;
}
```

**Explanation**:

- **Multiple throw statements**:
  - Throws an integer if x == 0.0.

- Throws a string if y == 0.
- Throws a double if the sum of x + y is negative.

- **Multiple catch blocks**:
  - Each catch block handles a specific type of exception.
  - The catch(...) block is a catch-all block, which handles any exception type not explicitly handled earlier.
- The program gracefully handles each error scenario and continues execution.

**Input**: 0 1

**Output**:0

Bye

**Input**:2 0 **Output**:Y is 0 Bye

**Input**:-2 -3 **Output**: x+y < 0 Bye

**Exception Handling with Functions**

```
#include<iostream>
using namespace std;

int average(int arr[], int n) {
   if (n == 0)
      throw string("Array size is 0");
   int sum = 0;
   for (int i = 0; i < n; i++)
      sum += arr[i];
   return sum / n;
}
   int main() {
   int n;
   cin >> n;
   int arr[n];
   for (int i = 0; i < n; i++)
      cin >> arr[i];

   try {
      int res = average(arr, n);
      cout << res;
   }
   catch (string &e) {
      cout << e;
   }
   cout << "\nBye\n";
   return 0;
}
```

**Explanation**:

- **Throw from Function**:
  - The average function checks if n == 0. If true, it throws a string exception.
  - If the input array size is valid, it calculates and returns the average.
- The main function handles the exception using a try-catch block. If an exception is thrown, the message "Array size is 0" is displayed.

**Input**:3m 4 5 6 **Output**:5 Bye

**Input**:0 **Output**: Array size is 0 Bye

**Using Exception Specification**

```
#include<iostream>
using namespace std;
   int average(int arr[], int n) throw (string) {
   if (n == 0)
      throw string("Array size is 0");
   int sum = 0;
   for (int i = 0; i < n; i++)
      sum += arr[i];
   return sum / n;
}

int main() {
   int n;
   cin >> n;
   int arr[n];
   for (int i = 0; i < n; i++)
      cin >> arr[i];

   try {
      int res = average(arr, n);
      cout << res;
   }
   catch (string &e) {
      cout << e;
   }
   cout << "\nBye\n";
   return 0;
}
```

**Explanation**:

- The throw(string) specifies that the average function may throw an exception of type string.
- This helps document potential exceptions for developers using the function.

**Input**:4 10 20 30 40 **Output**:25 Bye

**Input**: 0 **Output**: Array size is 0 Bye

**Stack Unwinding in C++**:- Stack unwinding is a critical concept in C++ Exception Handling that deals with the process of cleaning up the function call stack when an exception is thrown**.** The local objects are destroyed in reverse order in which they were constructed.

**What is Stack Unwinding?:-**When an exception is thrown in C++, the program starts unwinding the stack to find a matching catch block. During this process:

- Functions in the call stack are **popped off** one by one.
- The destructors of objects in these functions are executed to release resources.
- If no matching catch block is found, the program terminates.

**Code Example 1: Stack Unwinding with Two Functions**

```
#include<iostream>
using namespace std;
   void f1() throw (int)
{
   cout << "F1 begins \n";
   throw 100; // Exception is thrown here
   cout << "F1 ends \n"; // This line will not be executed
}

void f2() throw (int)
{
   cout << "F2 begins \n";
   f1(); // Call to f1
   cout << "F2 ends \n"; // This line will not be executed
}

int main()
{
   try
   {
      f2(); // Call to f2
   }
   catch (int x)
   {
      cout << "Caught Exception \n"; // Exception is caught here
   }
   cout << "Bye ... \n";
   return 0;
}
```

**Explanation with Stack Diagram**

1. **Function Calls:**
   - main() calls f2().
   - f2() calls f1().
2. **Execution Flow:**
   - f1() begins and throws an exception (throw 100).
   - Since no catch block exists in f1() or f2(), the program starts stack unwinding.
3. **Stack Unwinding:**
   - The function f1() exits without completing execution (the line F1 ends is skipped).
   - The function f2() also exits without completing (the line F2 ends is skipped).
   - The exception is caught in the catch block of main().

**Explanation:**

- When f1() throws exception, its entry is removed from the function call stack, because f1() doesn't contain exception handler for the thrown exception, then next entry in call stack is looked for exception handler.
- The next entry is f2(). Since f2() also doesn't have a handler, its entry is also removed from the function call stack.
- The next entry in the function call stack is f3(). Since f3() contains an exception handler, the catch block inside f3() is executed, and finally, the code after the catch block is executed.

*Output*

```
F2 begins
F1 begins
Caught Exception
Bye ...
```

**Code Example 2: Nested Try-Catch with Multiple Functions**

```
#include<iostream>
using namespace std;
   void f1() throw (int)
{
   cout << "F1 begins \n";
   throw 100; // Exception is thrown here
   cout << "F1 ends \n";
}
   void f2() throw (int)
{
   cout << "F2 begins \n";
   f1(); // Call to f1
   cout << "F2 ends \n";
}
   void f3() throw (int)
{
   cout << "F3 begins \n";
   try
   {
      f2(); // Call to f2
   }
   catch (int x)
   {
      cout << "Caught Exception \n"; // Exception is caught here
   }
   cout << "F3 ends \n";
}
   int main()
{
   f3(); // Call to f3
   cout << "Bye ... \n";
   return 0;
}
```

**Explanation with Stack Diagram**

1. **Function Calls:**
   - main() calls f3().
   - f3() calls f2(), which calls f1().
2. **Execution Flow:**
   - f1() begins and throws an exception (throw 100).
   - Stack unwinding begins. The control goes back to f2() and then f3().
   - The catch block in f3() catches the exception and handles it.
3. **Stack Unwinding Details:**
   - The function f1() exits, skipping the line F1 ends.
   - The function f2() exits, skipping the line F2 ends.
   - In f3(), the catch block handles the exception, and execution continues.

*Output:*

```
F3 begins
F2 begins
F1 begins
Caught Exception
F3 ends
Bye ...
```

**User Defined Exceptions in C++:-** In C++, we can throw exceptions using either **primitive data types** (like int, double, string, etc.) or **user-defined data types** (like classes). User-defined exceptions allow us to create meaningful and descriptive error messages for specific scenarios, making it easier to debug and handle errors effectively. Let us explore **user-defined exceptions** with examples and best practices.

**Example 1: Handling Array Size Errors with User-Defined Exceptions**

Problem Statement::- **We want to calculate the average of an array. However, if:**

- The array size is **zero**, an exception for "Array Size Zero" must be thrown.
- The array size is **negative**, an exception for "Array Size Negative" must be thrown.

We can implement this using two user-defined exception classes.

```
#include<iostream>
using namespace std ;
   class arraySizeZero {};
class arraySizeNegative {};

int average(int arr[], int n)
{
   if(n == 0)
      throw arraySizeZero();
   if(n < 0)
      throw arraySizeNegative();
   int sum = 0;
   for(int i = 0; i < n; i++)
      sum = sum + arr[i];
   int res = sum/n;
   return res;
}
int main()
{
   int n;
   cin >> n;
   int arr[n];
   for(int i = 0; i < n; i++)
      cin >> arr[i];
   try
   {
      int res = average(arr, n);
      cout << res;
   }
   catch (arraySizeNegative &e1)
   {
      cout << "Array Size Negative"; // Handle negative size exception
   }
   catch (arraySizeZero &e2)
   {
      cout << "Array Size Zero";   // Handle zero size exception
   }
   cout << "\nBye\n";
    return 0 ;
}
```

*Explanation:*

1. **Classes for Exceptions**:

- ArraySizeZero is used for the zero-size exception.
- ArraySizeNegative is used for the negative-size exception.

2. **Throwing Exceptions**:
   - If the size n == 0, we throw an ArraySizeZero object.
   - If n < 0, we throw an ArraySizeNegative object.
3. **Catching Exceptions**:
   - catch(ArraySizeNegative &e1) handles negative-size exceptions.
   - catch(ArraySizeZero &e2) handles zero-size exceptions.

**Input 1:**3 4 5 6 **Output 1:** Average: 5 Bye!

**Input 2:** 0 **Output 2:** Array Size Zero Bye!

**Input 3:** -3 **Output 3:** Array Size Negative Bye!

Example 2: Creating a Custom Exception Using the exception Class

*Problem Statement:**We want to create a custom exception class by inheriting from the standard exception class. This allows us to override the what() function to provide a custom error message.***

```
#include <iostream>
#include <exception>
using namespace std;

// Custom Exception Class
class MyException : public exception {
   virtual const char *what() const throw() {
      return "Exception Occurred";  // Custom error message
   }
};
   int main() {
   try {
      throw MyException();  // Throwing an object of MyException
   }
   catch (exception &e) {
      cout << e.what() << endl;  // Catching and printing the error message
   }
   return 0;
}
```

*Explanation:*

1. **Inheriting from exception**:
   - The MyException class inherits from the standard exception class.
   - The what() function is overridden to return a custom message.
2. **Throwing and Catching**:
   - We throw an object of MyException.
   - Since what() is a virtual function, it prints the custom message: "Exception Occurred".

*Input: **No input Required***

*Output: **Exception Occurred***

Class Template in C++ STL

In C++, a **class template** also follows the trend of writing code once and using it for various data types. For example, a Stack class can be created that might be used as an integer stack, float class, etc. It can also be useful for classes like LinkedList, BinaryTree, Stack, Queue, Array, etc. We can either use a structure or a class for templates.

**Example:**

```
#include <iostream>
using namespace std;

template <typename T>
struct Pair
{
   T x, y;
   Pair(T i, T j) {x = i; y = j;}
   T getFirst() { return x; }
   T getSecond() { return y; }
};

int main() {
   Pair<int> p1(10, 20);
   cout << p1.getFirst() << " " << p1.getSecond();
   return 0;
}
```

### *Output*

```
10 10
```

**Explanation:** In this example, we create a template structure that can hold two values of the same data type. The template allows you to create a pair of integers, pair of floats, or pairs of any other data types. The constructor **Pair** is used to initialize these values when you create a **Pair** Object. The functions **getFirst**() and **getSecond**() are provided to retrieve the values of **x** and **y** respectively.

## Function Outside of Class Template

The function definition of class templates can also be written outside the class. You must specify the class name along with the template type. For example, we can define the **getFirst** function outside the **Pair** class (which is templated with type T).

```
#include <iostream>
using namespace std;

template <typename T>
struct Pair
{
   T x, y;
   Pair(T i, T j) { x = i; y =  j;}
   T getFirst();
   T getSecond() { return y; }
};

template <typename T>
T Pair<T>::getFirst()
{
   return x;
}
```

```
int main() {
   Pair<int> p1(10, 20);
   cout << p1.getFirst() << " " << p1.getSecond();
   return 0;
}
```

## Output

```
10 20
```

**Explanation:** In this example, we can initialize the **getFirst**() function outside of the structure.